

Rechtliche Hinweise

Warnhinweiskonzept

Dieses Handbuch enthält Hinweise, die Sie zu Ihrer persönlichen Sicherheit sowie zur Vermeidung von Sachschäden beachten müssen. Die Hinweise zu Ihrer persönlichen Sicherheit sind durch ein Warndreieck hervorgehoben, Hinweise zu alleinigen Sachschäden stehen ohne Warndreieck. Je nach Gefährdungsstufe werden die Warnhinweise in abnehmender Reihenfolge wie folgt dargestellt.

 GEFAHR
bedeutet, dass Tod oder schwere Körperverletzung eintreten wird , wenn die entsprechenden Vorsichtsmaßnahmen nicht getroffen werden.
 WARNUNG
bedeutet, dass Tod oder schwere Körperverletzung eintreten kann , wenn die entsprechenden Vorsichtsmaßnahmen nicht getroffen werden.
 VORSICHT
bedeutet, dass eine leichte Körperverletzung eintreten kann, wenn die entsprechenden Vorsichtsmaßnahmen nicht getroffen werden.
ACHTUNG
bedeutet, dass Sachschaden eintreten kann, wenn die entsprechenden Vorsichtsmaßnahmen nicht getroffen werden.

Beim Auftreten mehrerer Gefährdungsstufen wird immer der Warnhinweis zur jeweils höchsten Stufe verwendet. Wenn in einem Warnhinweis mit dem Warndreieck vor Personenschäden gewarnt wird, dann kann im selben Warnhinweis zusätzlich eine Warnung vor Sachschäden angefügt sein.

Qualifiziertes Personal

Das zu dieser Dokumentation zugehörige Produkt/System darf nur von für die jeweilige Aufgabenstellung **qualifiziertem Personal** gehandhabt werden unter Beachtung der für die jeweilige Aufgabenstellung zugehörigen Dokumentation, insbesondere der darin enthaltenen Sicherheits- und Warnhinweise. Qualifiziertes Personal ist auf Grund seiner Ausbildung und Erfahrung befähigt, im Umgang mit diesen Produkten/Systemen Risiken zu erkennen und mögliche Gefährdungen zu vermeiden.

Bestimmungsgemäßer Gebrauch von Siemens-Produkten

Beachten Sie Folgendes:

 WARNUNG
Siemens-Produkte dürfen nur für die im Katalog und in der zugehörigen technischen Dokumentation vorgesehenen Einsatzfälle verwendet werden. Falls Fremdprodukte und -komponenten zum Einsatz kommen, müssen diese von Siemens empfohlen bzw. zugelassen sein. Der einwandfreie und sichere Betrieb der Produkte setzt sachgemäßen Transport, sachgemäße Lagerung, Aufstellung, Montage, Installation, Inbetriebnahme, Bedienung und Instandhaltung voraus. Die zulässigen Umgebungsbedingungen müssen eingehalten werden. Hinweise in den zugehörigen Dokumentationen müssen beachtet werden.

Marken

Alle mit dem Schutzrechtsvermerk ® gekennzeichneten Bezeichnungen sind eingetragene Marken der Siemens Aktiengesellschaft. Die übrigen Bezeichnungen in dieser Schrift können Marken sein, deren Benutzung durch Dritte für deren Zwecke die Rechte der Inhaber verletzen kann.

Haftungsausschluss

Wir haben den Inhalt der Druckschrift auf Übereinstimmung mit der beschriebenen Hard- und Software geprüft. Dennoch können Abweichungen nicht ausgeschlossen werden, so dass wir für die vollständige Übereinstimmung keine Gewähr übernehmen. Die Angaben in dieser Druckschrift werden regelmäßig überprüft, notwendige Korrekturen sind in den nachfolgenden Auflagen enthalten.

Inhaltsverzeichnis

1	Einleitung.....	8
1.1	Was ist S7-PLCSIM Advanced?	8
1.2	Nutzung der Dokumentation für S7-PLCSIM Advanced.....	9
1.3	Zugriff auf Instanzen.....	11
1.4	Schnittstellen der Simulation Runtime.....	12
1.5	Übersicht Anwenderschnittstellen für Native C++.....	13
1.6	Übersicht Schnittstellen für Managed Code	17
1.7	Einschränkungen bei Dateipfaden.....	20
1.8	Übersicht Datentypen für Native C++.....	21
1.9	Übersicht Datentypen für Managed Code.....	22
1.10	Cybersecurity-Hinweise.....	23
2	API initialisieren	24
2.1	API-Bibliothek laden.....	24
2.2	Native C++	25
2.2.1	InitializeApi()	25
2.2.2	RuntimeApiEntry_Initialize.....	26
2.3	.NET (C#).....	28
2.3.1	Initialize	28
3	API herunterfahren.....	29
3.1	Native C++.....	29
3.1.1	DestroyInterface().....	30
3.1.2	RuntimeApiEntry_DestroyInterface.....	30
3.1.3	FreeApi().....	31
3.1.4	ShutdownAndFreeApi().....	32
3.2	.NET (C#).....	32
3.2.1	API herunterfahren	32
4	Globale Funktionen (Native C++).....	33
5	IBaseRuntimeManager.....	38
5.1	Simulations-Laufzeitinstanzen.....	38
5.2	Schnittstellen für IBaseRuntimeManager.....	43
6	API ISimulationRuntimeManager.....	51
6.1	Remote-Verbindungen.....	51

6.1.1	RunAutodiscover().....	54
6.2	Ereignisse für ISimulationRuntimeManager.....	56
6.2.1	Ereignisse OnSoftwareConfigurationChanged.....	56
6.2.2	Ereignisse OnRuntimeManagerLost.....	58
6.2.3	Ereignisse OnAutodiscoverData.....	61
7	API Instance.....	64
7.1	Schnittstellen für Instances.....	64
7.2	Controller - Informationen und Einstellungen.....	71
7.3	Betriebszustand	80
7.4	Simulation anhalten.....	91
7.5	Variablentabelle.....	92
7.6	IO-Zugriff.....	97
7.6.1	Synchronisieren von Eingängen und Ausgängen.....	97
7.6.2	I/O-Zugriff über Adresse - Lesen.....	98
7.6.3	I/O-Zugriff über Adresse - Schreiben.....	103
7.6.4	I/O-Zugriff über Variablenname - Lesen.....	107
7.6.5	I/O-Zugriff über Variablenname - Schreiben.....	126
7.8	Zykluskontrolle.....	148
7.9	Virtuelle und reale Zeit.....	152
7.10	Überwachung Überlauf	155
7.11	Azyklische Dienste.....	156
7.11.1	Übersicht.....	156
7.11.2	ReadRecordDone/WriteRecordDone.....	158
7.11.3	AlarmNotification.....	160
7.11.4	ProcessEvent.....	162
7.11.5	PullOrPlugEvent.....	163
7.11.6	StatusEvent.....	165
7.11.7	ProfileEvent.....	166
7.11.8	UpdateEvent.....	167
7.11.9	GetConfiguredProcessEvent.....	168
7.11.10	RackOrStationFaultEvent.....	168
7.12	Ereignisse für Instances.....	170
7.12.1	Ereignisse für Betriebszustand und Zykluskontrolle.....	170
7.12.1.1	Simulations-Partner synchronisieren.....	171
7.12.1.2	Ereignisse OnOperatingStateChanged.....	175
7.12.1.3	Ereignisse OnLedChanged.....	178
7.12.1.4	Ereignisse OnSoftwareConfigurationChanged.....	181
7.12.1.5	Ereignisse OnIPAddressChanged.....	183
7.12.1.6	Ereignisse OnSyncPointReached.....	184
7.12.1.7	Ereignisse OnHardwareConfigChanged	187
7.12.2	Ereignisse für Azyklische Dienste.....	189
7.12.2.1	Ereignisse OnAlarmNotification.....	190
7.12.2.2	Ereignisse OnProcessEvent.....	191

7.12.2.3	Ereignisse OnPullOrPlugEvent.....	192
7.12.2.4	Ereignisse OnStatusEvent.....	193
7.12.2.5	Ereignisse OnProfileEvent.....	194
7.12.2.6	Ereignisse OnUpdateEvent.....	195
7.12.2.7	Ereignisse RackOrStationFault.....	196
8	API IRemoteRuntimeManager.....	198
8.1	Schnittstellen - Information und Einstellungen.....	198
8.2	Ereignisse für IRemoteRuntimeManager.....	200
8.2.1	Ereignisse OnConnectionLost.....	200
9	Datentypen.....	203
9.1	DLL-Importfunktionen (Native C++).....	204
9.1.1	ApiEntry_Initialize.....	204
9.1.2	ApiEntry_DestroyInterface.....	204
9.2	Event Callback-Funktionen (Native C++).....	205
9.2.1	EventCallback_VOID.....	205
9.2.2	EventCallback_SRCC_UINT32_UINT32_INT32.....	205
9.2.3	EventCallback_SRRSI_AD.....	206
9.2.4	EventCallback_IRRTM.....	206
9.2.5	EventCallback_II_SREC_ST_SROS_SROS.....	207
9.2.6	EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32.....	207
9.2.7	EventCallback_II_SREC_ST.....	208
9.2.8	EventCallback_II_SREC_ST_SRICC_UINT32_UINT32_UINT32_UINT32.....	208
9.2.9	EventCallback_II_SREC_ST_SRLT_SRLM.....	209
9.2.10	EventCallback_II_SREC_ST_SDRI.....	209
9.2.11	EventCallback_II_SREC_ST_SDRI_BYTE.....	210
9.2.12	EventCallback_II_SREC_ST_UINT32_UINT32.....	210
9.2.13	EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32.....	211
9.2.14	EventCallback_II_SREC_ST_UINT32_EPPET_UINT32.....	211
9.2.15	EventCallback_II_SREC_ST_UINT32_ERSFET.....	212
9.2.16	EventCallback_II_SREC_ST_UINT32.....	212
9.3	Delegat Definitionen (Managed Code).....	213
9.3.1	Delegate_Void.....	213
9.3.2	Delegate_SRCC_UINT32_UINT32_INT32.....	213
9.3.3	Delegate_SRRSI_AD.....	214
9.3.4	Delegate_II_EREC_DT.....	214
9.3.5	Delegate_II_EREC_DT_EOS_EOS.....	215
9.3.6	Delegate_II_EREC_DT_ELT_ELM.....	215
9.3.7	Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32.....	216
9.3.8	Delegate_IRRTM.....	216
9.3.9	Delegate_II_EREC_DT_SRICC_UINT32_UINT32_UINT32_UINT32.....	217
9.3.10	Delegate_II_EREC_DT_SDRI.....	217
9.3.11	Delegate_II_EREC_DT_SDR.....	218
9.3.12	Delegate_SREC_ST_UINT32_EPPET_UINT32.....	218
9.3.13	Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32.....	219
9.3.14	Delegate_SREC_ST_UINT32.....	219
9.3.15	Delegate_SREC_ST_UINT32_UINT32.....	220
9.3.16	Delegate_SREC_ST_UINT32_ERSFET.....	220

9.4	Definitionen und Konstanten.....	221
9.5	Unions (Native C++).....	222
9.5.1	UIP.....	222
9.5.2	UDataValue.....	222
9.6	Strukturen.....	223
9.6.1	SDataValue.....	224
9.6.2	SDVBNI.....	225
9.6.3	SDataValueByAddress.....	225
9.6.4	SDataValueByAddressWithCheck.....	226
9.6.5	SDataValueByName.....	227
9.6.6	SDataValueByNameWithCheck.....	228
9.6.7	SConnectionInfo.....	228
9.6.8	SInstanceInfo.....	229
9.6.9	SDimension.....	229
9.6.10	STagInfo.....	230
9.6.11	SIP.....	231
9.6.12	SIPSuite4.....	232
9.6.13	SOnSyncPointReachedResult.....	232
9.6.14	SDataRecordInfo.....	234
9.6.15	SDataRecord.....	235
9.6.16	SConfiguredProcessEvents.....	235
9.6.17	SDiagExtChannelDescription.....	236
9.6.18	SAutodiscoverData.....	238
9.6.19	SMACAddress.....	238
9.6.20	SNetInterfaceInfo.....	238
9.7	Aufzählungen.....	239
9.7.1	ERuntimeErrorCode.....	239
9.7.2	EArea.....	242
9.7.3	EOperatingState.....	242
9.7.4	EOperatingMode.....	243
9.7.5	ECPUType.....	244
9.7.6	ECommunicationInterface.....	246
9.7.7	ENetworkMode.....	246
9.7.8	EPLCInterface.....	247
9.7.9	ELEDType.....	247
9.7.10	ELEDMode.....	248
9.7.11	EPrimitiveDataType.....	248
9.7.12	EDataType.....	250
9.7.13	ETagListDetails.....	253
9.7.14	ERuntimeConfigChanged.....	254
9.7.15	EInstanceConfigChanged.....	254
9.7.16	EPullOrPlugEventType.....	255
9.7.17	EProcessEvent.....	255
9.7.18	ESoftwareConfigChanged.....	256
9.7.19	EDirection.....	256
9.7.20	EDiagProperty.....	256
9.7.21	EDiagSeverity.....	257
9.7.22	ERackOrStationFaultType.....	257

9.7.23	ECycleTimeMonitoringMode.....	258
9.7.24	EAutodiscoverType.....	258
10	Beispiele/Anwendungsfälle.....	259
10.1	Beispiel: Kommunikation über TCP/IP im "Single Adapter"-Netzwerkmodus (Promiscuous-Modus)	259
10.2	Beispiel: Kommunikation über TCP/IP im Multi-Adapter Network Mode (Non-Promiscuous-Modus)	260

Einleitung

1.1 Was ist S7-PLCSIM Advanced?

Mit S7-PLCSIM Advanced simulieren Sie Ihre CPU-Programme auf einer simulierten PLC-Instanz. Für diese Simulation benötigen Sie keine realen Controller. Sie konfigurieren Ihre CPU mit STEP 7 im TIA Portal, programmieren Ihre Anwendungslogik und laden dann die Hardware-Konfiguration und das Programm in die simulierte CPU. Von STEP 7 aus führen Sie Ihre Programmlogik aus, beobachten die Auswirkungen der simulierten Eingänge und Ausgänge und passen Ihre Programme an.

Neben der Kommunikation über Softbus, der die Kommunikation auf einen lokalen PC oder eine virtuelle Maschine beschränkt, bietet S7-PLCSIM Advanced einen vollwertigen Ethernet-Anschluss und unterstützt somit auch dezentrale Peripheriesysteme.

S7-PLCSIM Advanced umfasst außerdem eine Anwenderschnittstelle (API), die die Interaktion mit eigenen C++/C#-Programmen oder mit Simulationssoftware ermöglicht. Die API erweitert die Simulationsmöglichkeiten und stellt Funktionen wie die folgenden zur Verfügung:

- Simulation von Prozessalarmen
- Stoppen einer Simulation
- Synchronisierung mit Simulationspartnern

Weitere Informationen über diese und andere API-Funktionen finden Sie im Handbuch für die S7-PLCSIM Advanced-API, das mit ihrer Version ausgeliefert wurde.

Anwendungsgebiete

Typische Anwendungsgebiete von S7-PLCSIM Advanced sind:

- Verifikation des Anwenderprogramms (TIA Portal)
- Testen des STEP 7-Programms
- Software-in-the-Loop-Simulation zur virtuellen Inbetriebnahme von Werkzeug-/Produktionsmaschinen, Produktionszellen und Produktionslinien in einer Anlage
- Bedienerschulungen durch Anbindung einer realen HMI

Vorteile

Der Einsatz von S7-PLCSIM Advanced bietet zahlreiche Vorteile:

- Qualität der Automatisierungsprojekte durch frühzeitige Fehlererkennung verbessern
- Kosten für Hardware in Simulationsumgebungen vermeiden
- Inbetriebnahmezeit verkürzen
- Risiken bei der Inbetriebnahme reduzieren
- Bediener frühzeitiger schulen
- Effizienz der Fertigung durch Optimierung von Programmteilen steigern
- Effizienz beim Tausch von Maschinenkomponenten steigern
- Effizienz bei der Erweiterung existierender Anlagen steigern

1.2 Nutzung der Dokumentation für S7-PLCSIM Advanced

Das Funktionshandbuch und das API-Handbuch für S7-PLCSIM Advanced sind HTML5-Webseiten. HTML5-Webseiten bieten Vorteile für die Bildschirmanzeige auf einer Vielzahl von Geräten.

Navigation

Wenn Sie eine HTML5-Webseite öffnen, klicken Sie in der Startseite auf "Begin".

Das Inhaltsverzeichnis wird links angezeigt. Jedes Kapitel wird eingeklappt, wenn Sie zu einem anderen Kapitel wechseln, sodass der Platz auf dem Bildschirm für Geräte wie Mobiltelefone und Tablets optimal genutzt werden kann. Mit einem Klick auf die Schaltfläche  können Sie das Inhaltsverzeichnis ausblenden. Zum Wiederherstellen klicken Sie auf .

Mit den Pfeiltasten nach links und rechts können Sie sich durch die zuvor angesehenen Themen bewegen:  .

Mit den Schaltflächen Zurück und Weiter können Sie sich schrittweise durch die Themen und Kapitel in der Reihenfolge des Inhaltsverzeichnisses bewegen:  .

Suchen

Klicken Sie auf die Schaltfläche , um eine HTML5-Webseite zu suchen. Wenn Sie Ihren Suchtext eingeben, wird eine Liste von Themen angezeigt, die den Suchtext enthalten. Klicken Sie auf ein beliebiges Thema, um dieses anzuzeigen. Auf der Seite sind die Stellen markiert, an denen der Suchtext erscheint.

Um zu sehen, wo das Thema im Inhaltsverzeichnis erscheint, klicken Sie auf die Schaltfläche , um von der Suchansicht in das Inhaltsverzeichnis umzuschalten. Um zurück zur Suchansicht zu gelangen, klicken Sie erneut auf . Die Suche funktioniert weitgehend so wie jede Suche in einem Webbrowser. Mit Strg+F können Sie innerhalb des angezeigten Themas suchen.

Sprache ändern

Wenn eine HTML5-Webseite mehrere Sprachen enthält, klicken Sie auf die Schaltfläche , um auf eine andere Sprache umzuschalten.

Online auf die Dokumentation in anderen Formaten zugreifen

Die Dokumentation für dieses Produkt steht im Siemens Industry Online Support (<https://support.industry.siemens.com/>) zur Verfügung. Für Anzeige oder Download sind die folgenden Formate zulässig:

- PDF-Dokument
- Multimedia-Dokument (HTML5-Webseite)
- Webseite, nach Kapiteln strukturiert (nur Anzeige, nützlich für die Bereitstellung von Links zu bestimmten Themen für andere Personen)

Suchen Sie auf der Webseite nach Ihrem Produktnamen. Filtern Sie die Ergebnisse nach Handbüchern, wenn Sie nur eine Dokumentation suchen. Wählen Sie die Ausgabe, die der Version Ihres Produkts entspricht. Sie können die Anzeigesprache der Seite und Dokumente im Menü umschalten.

Die verschiedenen Dokumentformate bieten Navigation und Suchfunktionen abhängig vom jeweiligen Format.

Komponenten der Simulation Runtime

Für den Umgang mit der Simulation Runtime der S7-PLCSIM Advanced sind folgende Komponenten relevant:

Tabelle 1-1 Komponenten der Simulation Runtime

Komponenten	Beschreibung
"Siemens.Simatic.Simulation.Runtime.Manager.exe"	Ein Windows Prozess, der im Hintergrund abläuft. Hauptkomponente der Runtime, die alle weiteren Runtime-Komponenten verwaltet. Der Prozess wird automatisch gestartet, sobald eine Anwendung versucht, die Runtime API zu initialisieren. Er wird automatisch beendet, sobald keine Anwendung mehr läuft, die die Runtime API initialisiert hat.
"Siemens.Simatic.Simulation.Runtime.Instance.exe"	Der Prozess der Instanz, der eine DLL einer simulierten PLC-Instanz lädt. Jede simulierte PLC-Instanz erzeugt jeweils ihren eigenen Prozess.
"Siemens.Simatic.Simulation.Runtime.Api.x86.dll" "Siemens.Simatic.Simulation.Runtime.Api.x64.dll"	API-Bibliotheken, die eine Anwendung laden muss, um die Simulation Runtime zu verwenden. Die Bibliotheken enthalten Schnittstellen für Native und Managed Code. Die "Runtime.Api.x86.dll" wird ausschließlich von 32 Bit-Anwendungen geladen, die Runtime.Api.x64.dll von 64 Bit-Anwendungen.
"SimulationRuntimeApi.h"	Header-Datei, die alle Datentypen beschreibt, die eine Native C++ Anwendung benötigt, um die API-Bibliothek zu verwenden.
"Siemens.Simatic.PlcSim.Vplc1500.ODKClient.so.exe"	ODK-Client-Prozess für eine CPU-Funktionsbibliothek (original Shared Object)
"Siemens.Simatic.PlcSim.Vplc1500.ODKClient.x86.exe"	ODK-Client-Prozess für eine 32 Bit-Anwendung
"Siemens.Simatic.PlcSim.Vplc1500.ODKClient.x64.exe"	ODK-Client-Prozess für eine 64 Bit-Anwendung

Externe Anwendungen und Simulation Runtime

Die folgende Abbildung zeigt schematisch den Zugriff externer Anwendungen über die Runtime API auf die Simulation Runtime. Der Simulation Runtime Manager verwaltet die Runtime-Instanzen. Diese laden die Bibliotheken der simulierten PLC-Instanzen. Eine externe Anwendung kann z. B. eine weitere Simulations-Software oder eine grafische Oberfläche (GUI) sein.

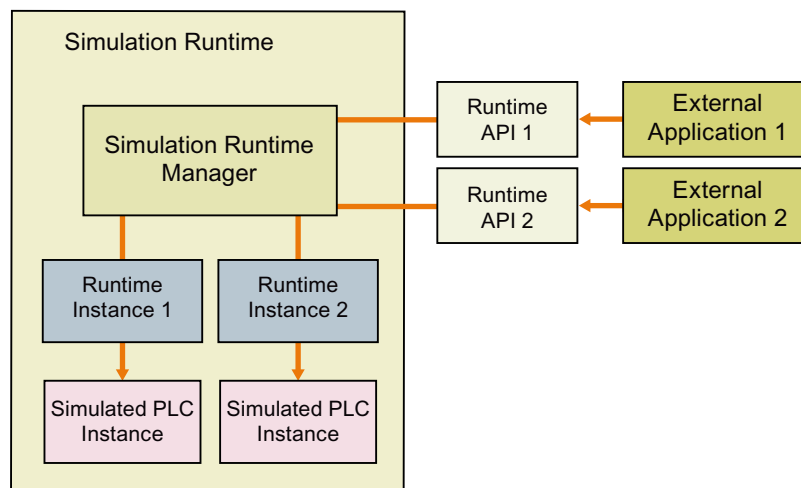
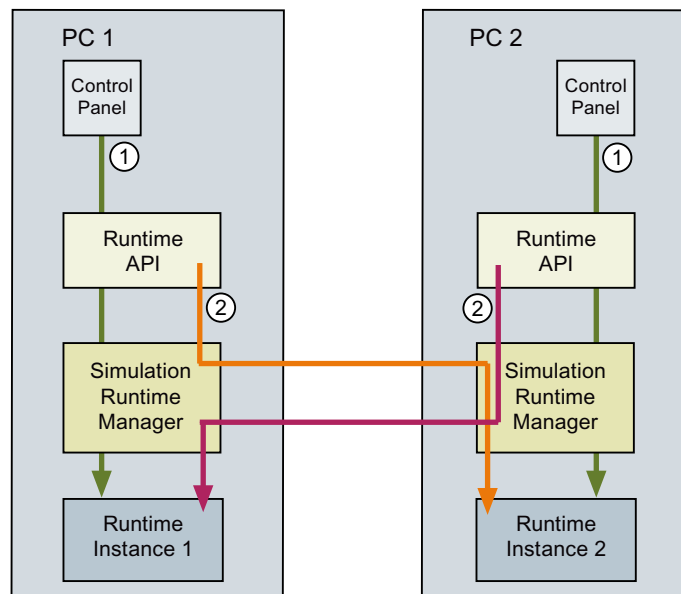


Bild 1-1 Externe Anwendungen und Simulation Runtime

1.3 Zugriff auf Instanzen

Zugriff über das Control Panel und die API

Über das Control Panel können Sie auf eine lokale Instanz auf dem PC zugreifen. Es spielt keine Rolle, auf welchem der PCs eine Instanz erstellt und gestartet wurde. Bei der verteilten Kommunikation greift die Runtime API über den Simulation Runtime Manager auf die Instanz des anderen PCs zu.



- ① Zugriff auf eine lokale Instanz über das Control Panel
- ② Zugriff auf eine Remote-Instanz über die Runtime API

Bild 1-2 Zugriff auf Instanzen bei verteilter Kommunikation

API-Funktionen

Die folgenden Schnittstellen stehen für Native C++ (Seite 13) und Managed Code (Seite 17) zur Verfügung:

- Übersicht Anwenderschnittstellen für Native C++ (Seite 13)
- Übersicht Schnittstellen für Managed Code (Seite 17)

1.4 Schnittstellen der Simulation Runtime

Unter den Benutzeroberflächen der Simulation Runtime stehen Funktionen zur Verfügung, mit denen sich u. a. folgende Aufgaben erledigen lassen:

- Instanzen anlegen
- Betriebszustand einer simulierten PLC-Instanz wechseln
- E/A-Daten austauschen

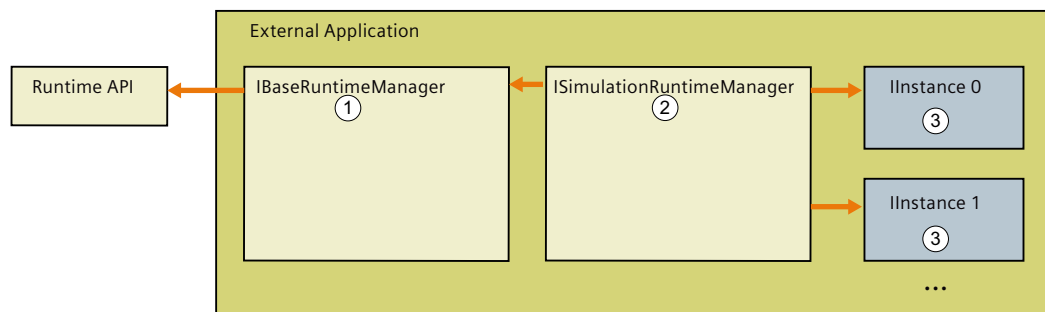
Die Simulation Runtime enthält folgende Anwenderschnittstellen:

- IBaseRuntimeManager
- ISimulationRuntimeManager
- IInstances
- IRemoteRuntimeManager

ISimulationRuntimeManager und IRemoteRuntimeManager sind vom IBaseRuntimeManager abgeleitete Produkte.

API und externe Anwendungen

Die Runtime API stellt einer externen Anwendung die Schnittstellen zur Verfügung.



- ① **IBaseRuntimeManager**
Basisschnittstelle des Runtime Managers. Sie wird benutzt, um neue Runtime-Instanzen zu registrieren, bereits bestehende zu durchsuchen und eine Schnittstelle einer registrierten Instanz zu erhalten. In einem Runtime Manager können bis zu 16 Instanzen registriert werden.
- ② **ISimulationRuntimeManager**
Abgeleitet von IBaseRuntimeManager. Dient zur Behandlung von Ereignissen, zum Verwalten von Verbindungen mit anderen Runtime Managern sowie zum Überprüfen des API- und Runtime Manager-Status.
- ③ **IInstances**
Schnittstelle einer Runtime-Instanz. Dient dazu, den Betriebszustand einer simulierten PLC-Instanz zu ändern und E/A-Daten auszutauschen. Jede Instanz hat einen eindeutigen Namen und eine ID.

Bild 1-3 API und externe Anwendungen

Zugriff auf API-Funktionen und Datentypen

Erforderliche Funktionen und Datentypen stehen für Native C++ und .NET (C#) zur Verfügung.

- Übersicht Anwenderschnittstellen für Native C++ [\(Seite 13\)](#)
- Übersicht Datentypen für Native C++ [\(Seite 21\)](#)
- Übersicht Schnittstellen für Managed Code [\(Seite 17\)](#)
- Übersicht Datentypen für Managed Code [\(Seite 22\)](#)

HINWEIS

Einen direkten Zugriff auf die Beschreibung der einzelnen Funktionen und Datentypen erhalten Sie über das Tabellenverzeichnis in diesem Handbuch.

Übergabeparameter für API-Funktionen

Alle API-Funktionen, die einen Wert über die Funktionsparameter zurückgeben, erwarten als Übergabeparameter einen vom Benutzer zugewiesenen Speicherbereich. Null-Zeiger sind nicht zulässig. Eine Ausnahme hiervon bilden die Funktionen, die eine Schnittstelle einer simulierten PLC-Instanz zurückgeben:

- eine IBaseRuntimeManager-Schnittstelle
- eine ISimulationRuntimeManager-Schnittstelle
- eine IRemoteRuntimeManager-Schnittstelle
- eine IInstance-Schnittstelle

1.5 Übersicht Anwenderschnittstellen für Native C++

API initialisieren und herunterfahren

Tabelle 1-2 Übersicht API initialisieren und herunterfahren - Native C++

Aktionen	Funktionen
API initialisieren (Seite 25)	InitializeApi RuntimeApiEntry_Initialize
API herunterfahren (Seite 29)	DestroyInterface RuntimeApiEntry_DestroyInterface
API-Bibliothek abmelden (Seite 31)	FreeApi ShutdownAndFreeApi

Globale Funktionen

Tabelle 1-3 Übersicht Globale Funktionen - Native C++

Aktionen	Funktionen
Globale Funktionen (Seite 33)	GetNameOfAreaSection() GetNameOfCPUType() GetNameOfCommunicationInterface() GetNameOfDataType() GetNameOfLEDMode() GetNameOfLEDType() GetNameOfOperatingMode() GetNameOfOperatingState() GetNameOfPrimitiveDataType() GetNameOfTagListDetails() GetNameOfErrorCode() GetNameOfRuntimeConfigChanged() GetNameOfInstanceConfigChanged() GetNameOfDiagSeverity() GetNameOfDirection() GetNameOfRackOrStationFaultType() GetNameOfProcessEventTypes() GetNameOfPullOrPlugEventType() GetNameOfCycleTimeMonitoringMode() GetNameOfDiagProperty() GetNameOfAutodiscoverType() GetNameOfSoftwareConfigChanged()

API IRemoteRuntimeManager

Tabelle 1-4 Übersicht IBaseRuntimeManager-Funktionen - Native C++

Einstellungen	Funktionen
Schnittstelle (Seite 43)	GetVersion() GetPort() GetStrictMotionTiming() SetStrictMotionTiming() GetNetworkMode() SetNetworkMode() GetNetInterfaces() SetNetInterfaceBindings() ResetNetInterfaceBindings() GetDefaultStoragePath() SetDefaultStoragePath()
Simulation Runtime-Instanzen (Seite 38)	GetRegisteredInstancesCount() GetRegisteredInstanceInfoAt() RegisterInstance() RegisterCustomInstance() CreateInterface()

API ISimulationRuntimeManager

Tabelle 1-5 Übersicht API ISimulationRuntimeManager-Funktionen - Native C++

Einstellungen	Funktionen
Schnittstelle (Seite 61)	IsInitialized() IsRuntimeManagerAvailable() Shutdown()
Remote-Verbindungen (Seite 51)	OpenPort() ClosePort() GetRemoteConnectionsCount() GetRemoteConnectionInfoAt() RemoteConnect() RunAutodiscover()

Tabelle 1-6 Übersicht API ISimulationRuntimeManager-Ereignisse - Native C++

Ereignisse	Funktionen
OnSoftwareConfigurationChanged (Seite 181)	RegisterOnSoftwareConfigurationChangedCallback() UnregisterOnSoftwareConfigurationChangedCallback() RegisterOnSoftwareConfigurationChangedEvent() UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()
OnRuntimeManagerLost (Seite 58)	RegisterOnRuntimeManagerLostCallback() UnregisterOnRuntimeManagerLostCallback() RegisterOnRuntimeManagerLostEvent() UnregisterOnRuntimeManagerLostEvent() WaitForOnRuntimeManagerLostEvent()
OnAutodiscover (Seite 61)	RegisterOnAutodiscoverCallback() UnregisterOnAutodiscoverCallback()

API IInstance

Tabelle 1-7 Übersicht IInstance-Funktionen - Native C++

Einstellungen	Funktionen
Schnittstelle (Seite 64)	GetID() GetName() GetCPUType() SetCPUType() GetCommunicationInterface() SetCommunicationInterface() GetInfo() UnregisterInstance() GetStrictMotionTiming() SetStrictMotionTiming()
Controller (Seite 71)	GetControllerName() GetControllerShortDesignation() GetControllerIPCount() GetControllerIP() GetControllerIPSuite4() SetIPSuite() GetStoragePath() SetStoragePath() ArchiveStorage() RetrieveStorage() CleanupStoragePath()
Betriebszustand (Seite 80)	PowerOn() PowerOff() Run() Stop() GetOperatingState() MemoryReset()

Einstellungen	Funktionen
Variablentabelle (Seite 92)	UpdateTagList() GetTagListStatus() GetTagInfoCount() GetTagInfos() CreateConfigurationFile()
I/O-Zugriff über Adresse - Lesen (Seite 98)	GetAreaSize() ReadBit() ReadByte() ReadBytes() ReadSignals()
I/O-Zugriff über Adresse - Schreiben (Seite 103)	WriteBit() WriteByte() WriteBytes() WriteSignals()
I/O-Zugriff über Variablenname - Lesen (Seite 107)	Read() ReadBool() ReadChar(), ReadWChar() ReadDouble() ReadFloat() ReadInt8(), ReadInt16(), ReadInt32(), ReadInt64() ReadUInt8(), ReadUInt16(), ReadUInt32(), ReadUInt64() ReadSignals() ReadString() ReadWString()
I/O-Zugriff über Variablenname - Schreiben (Seite 126)	Write() WriteBool() WriteChar(), WriteWChar() WriteDouble() WriteFloat() WriteInt8(), WriteInt16(), WriteInt32(), WriteInt64(), WriteUInt8(), WriteUInt16(), WriteUInt32(), WriteUInt64() WriteSignals() WriteString() WriteWString()
Virtuelle Zeit (Seite 145)	GetSystemTime() SetSystemTime() GetScaleFactor() SetScaleFactor()
Zykluskontrolle (Seite 148)	GetOperatingMode() SetOperatingMode() SetSendSyncEventInDefaultModeEnabled() IsSendSyncEventInDefaultModeEnabled() GetOverwrittenMinimalCycleTime_ns() SetOverwrittenMinimalCycleTime_ns() RunToNextSyncPoint() StartProcessing() SetCycleTimeMonitoringMode() GetCycleTimeMonitoringMode()
Azyklische Dienste (Seite 156)	

Tabelle 1-8 Übersicht Instance-Ereignisse - Native C++

Ereignisse	Funktionen
OnOperatingStateChanged (Seite 175)	RegisterOnOperatingStateChangedCallback() UnregisterOnOperatingStateChangedCallback() RegisterOnOperatingStateChangedEvent() UnregisterOnOperatingStateChangedEvent() WaitForOnOperatingStateChangedEvent()
OnLedChanged (Seite 178)	RegisterOnLedChangedCallback() UnregisterOnLedChangedCallback() RegisterOnLedChangedEvent() UnregisterOnLedChangedEvent() WaitForOnLedChangedEvent()

Ereignisse	Funktionen
OnSoftwareConfigurationChanged (Seite 181)	RegisterOnSoftwareConfigurationChangedCallback() UnregisterOnSoftwareConfigurationChangedCallback() RegisterOnSoftwareConfigurationChangedEvent() UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()
OnSyncPointReached (Seite 184)	RegisterOnSyncPointReachedCallback() UnregisterOnSyncPointReachedCallback() RegisterOnSyncPointReachedEvent() UnregisterOnSyncPointReachedEvent() WaitForOnSyncPointReachedEvent()
OnHardwareConfigChanged (Seite 187)	RegisterOnHardwareConfigChangedCallback() UnregisterOnHardwareConfigChangedCallback() RegisterOnHardwareConfigChangedEvent() UnregisterOnHardwareConfigChangedEvent() WaitForOnHardwareConfigChangedEvent()

API IRemoteRuntimeManager

Tabelle 1-9 Übersicht IRemoteRuntimeManager-Funktionen - Native C++

Einstellungen	Funktionen
Schnittstelle (Seite 198)	GetIP() GetRemoteComputerName() Disconnect()

Tabelle 1-10 Übersicht IRemoteRuntimeManager-Ereignisse - Native C++

Ereignisse	Funktionen
OnConnectionLost (Seite 200)	RegisterOnConnectionLostCallback() UnregisterOnConnectionLostCallback() RegisterOnConnectionLostEvent() UnregisterOnConnectionLostEvent() WaitForOnConnectionLostEvent()

1.6 Übersicht Schnittstellen für Managed Code

API initialisieren und herunterfahren

Tabelle 1-11 Übersicht API initialisieren und herunterfahren - .NET (C#)

Aktionen	Funktionen
API initialisieren (Seite 28)	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeManager
API herunterfahren (Seite 32)	

API IRemoteRuntimeManager

Tabelle 1-12 Übersicht IBaseRuntimeManager-Funktionen - .NET (C#)

Einstellungen	Funktionen
Schnittstelle (Seite 43)	<pre>Version { get; } Port { get; } StrictMotionTiming { get; set; } NetworkMode { get; set; } NetInterfaces { get; } SetNetInterfaceBindings(); ResetNetInterfaceBindings(); DefaultStoragePath { get; set; }</pre>
Simulation Runtime-Instanzen (Seite 38)	<pre>RegisteredInstanceInfo { get; } RegisterInstance(); RegisterCustomInstance(); CreateInterface();</pre>

API ISimulationRuntimeManager

Tabelle 1-13 Übersicht ISimulationRuntimeManager-Funktionen - .NET (C#)

Einstellungen	Funktionen
Schnittstelle (Seite 61)	<pre>IsInitialized { get; } IsRuntimeManagerAvailable { get; } Shutdown() StrictMotionTiming { get; }</pre>
Remote-Verbindungen (Seite 51)	<pre>OpenPort() ClosePort() Port { get; } RemoteConnectionInfo { get; } RemoteConnect() RunAutodiscover()</pre>

Tabelle 1-14 Übersicht ISimulationRuntimeManager-Ereignisse - .NET (C#)

Ereignisse	Funktionen
OnSoftwareConfigurationChanged (Seite 56)	<pre>UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()</pre>
OnRuntimeManagerLost (Seite 58)	<pre>OnRuntimeManagerLost() RegisterOnRuntimeManagerLostEvent() UnregisterOnRuntimeManagerLostEvent() WaitForOnRuntimeManagerLostEvent()</pre>
OnAutodiscover (Seite 61)	<pre>OnAutodiscoverData()</pre>

API IInstance

Tabelle 1-15 Übersicht IInstance-Funktionen - .NET (C#)

Einstellungen	Funktionen
Schnittstelle (Seite 64)	<pre>Dispose() ID { get; } Name { get; } CommunicationInterface { get; } CPUType { get; set; } Info { get; } UnregisterInstance() StrictMotionTiming { get; set; }</pre>

Einstellungen	Funktionen
Controller - Informationen und Einstellungen (Seite 71)	ControllerName { get; } ControllerShortDesignation { get; } ControllerIPSuite4 { get; } SetIPSuite() StoragePath { get; set; } ArchiveStorage() RetrieveStorage() CleanupStoragePath()
Betriebszustand (Seite 80)	PowerOn() PowerOff() Run() Stop() OperatingState { get; } MemoryReset()
Variablentabelle (Seite 92)	UpdateTagList() GetTagListStatus() TagInfos { get; } CreateConfigurationFile()
I/O-Zugriff über Adresse - Lesen (Seite 98)	InputArea MarkerArea OutputArea { get; } AreaSize { get; } ReadBit() ReadByte() ReadBytes() ReadSignals()
I/O-Zugriff über Adresse - Schreiben (Seite 103)	WriteBit() WriteByte() WriteBytes() WriteSignals()
I/O-Zugriff über Variablenname - Lesen (Seite 107)	Read() ReadBool() ReadChar(), ReadWChar() ReadDouble() ReadFloat() ReadInt8(), ReadInt16(), ReadInt32(), ReadInt64() ReadUInt8(), ReadUInt16(), ReadUInt32(), ReadUInt64() ReadSignals() ReadString() ReadWString()
I/O-Zugriff über Variablenname - Schreiben (Seite 126)	Write() WriteBool() WriteChar(), WriteWChar() WriteDouble() WriteFloat() WriteInt8(), WriteInt16(), WriteInt32(), WriteInt64(), WriteUInt8(), WriteUInt16(), WriteUInt32(), WriteUInt64() WriteSignals() WriteString() WriteWString()
Virtuelle Zeit (Seite 145)	SystemTime { get; set; } ScaleFactor { get; set; }
Zykluskontrolle (Seite 148)	OperatingMode { get; set; } IsSendSyncEventInDefaultModeEnabled { get; set; } OverwrittenMinimalCycleTime_ns { get; set; } RunToNextSyncPoint() StartProcessing() SetCycleTimeMonitoringMode() GetCycleTimeMonitoringMode()
Azyklische Dienste (Seite 156)	

Tabelle 1-16 Übersicht IInstance-Ereignisse - .NET (C#)

Ereignisse	Funktionen
OnOperatingStateChanged (Seite 175)	OnOperatingStateChanged() RegisterOnOperatingStateChangedEvent() UnregisterOnOperatingStateChangedEvent() WaitForOnOperatingStateChangedEvent()
OnLedChanged (Seite 178)	OnLedChanged() RegisterOnLedChangedEvent() UnregisterOnLedChangedEvent() WaitForOnLedChangedEvent()
OnSoftwareConfigurationChanged (Seite 56)	UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()
OnSyncPointReached (Seite 184)	OnSyncPointReached() RegisterOnSyncPointReachedEvent() UnregisterOnSyncPointReachedEvent() WaitForOnSyncPointReachedEvent()
OnHardwareConfigChanged (Seite 187)	OnHardwareConfigChanged() RegisterOnHardwareConfigChangedEvent() UnregisterOnHardwareConfigChangedEvent() WaitForOnHardwareConfigChangedEvent()

API IRemoteRuntimeManager

Tabelle 1-17 Übersicht IRemoteRuntimeManager-Funktionen - .NET (C#)

Einstellungen	Funktionen
Schnittstelle (Seite 198)	Dispose() IP { get; } RemoteComputerName { get; } Disconnect() StrictMotionTiming { get; }

Tabelle 1-18 Übersicht IRemoteRuntimeManager-Ereignisse - .NET (C#)

Ereignisse	Funktionen
OnConnectionLost() (Seite 200)	OnConnectionLost() RegisterOnConnectionLostEvent() UnregisterOnConnectionLostEvent() WaitForOnConnectionLostEvent()

1.7 Einschränkungen bei Dateipfaden

Für Anwenderschnittstellen, die als Übergabeparameter einen Pfad oder einen vollständigen Dateinamen erwarten, gelten folgende Einschränkungen:

Einschränkungen bei lokalen Pfaden	Schreibrechte auf systemkritische Ordner, wie der Windows-Ordner (%Windows%) oder die Programmordner (%Program Files%, %Program Files (x86)%) sind nicht erlaubt. Die C++ Anwenderschnittstelle liefert in diesem Fall den Fehlercode SREC_WRONG_ARGUMENT zurück. Die managed Anwenderschnittstelle gibt in diesem Fall eine Ausnahme mit dem Fehlercode <code>RuntimeErrorCode.WrongArgument</code> zurück.
Einschränkungen bei Netzwerkpfaden	Um Netzwerkpfade nutzen zu können, müssen Sie diese als Netzlaufwerk einbinden. Ansonsten liefert die C++ Anwenderschnittstelle den Fehlercode SREC_WRONG_ARGUMENT zurück. Die managed Anwenderschnittstelle gibt eine Ausnahme mit dem Fehlercode <code>RuntimeErrorCode.WrongArgument</code> zurück.

1.8 Übersicht Datentypen für Native C++

Die folgende Tabelle zeigt, welche Datentypen für die Simulation im Runtime Manager vorhanden sind.

Tabelle 1-19 Übersicht Datentypen - Native C++

Datentyp	
DLL-Importfunktionen (Seite 204)	ApiEntry_Initialize ApiEntry_DestroyInterface
Event Callback- Funktionen (Seite 205)	EventCallback_VOID EventCallback_SRCC_UINT32_UINT32_INT32 EventCallback_SRRSI_AD EventCallback_IRRTM EventCallback_II_SREC_ST_SROS_SROS EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 EventCallback_II_SREC_ST EventCallback_II_SREC_ST_SRICC_UINT32_UINT32_UINT32_UINT32 EventCallback_II_SREC_ST_SRLT_SRLM EventCallback_II_SREC_ST_SDRI EventCallback_II_SREC_ST_SDRI_BYTE EventCallback_II_SREC_ST_UINT32_UINT32 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 EventCallback_II_SREC_ST_UINT32_ERSFET EventCallback_II_SREC_ST_UINT32 EventCallback_II_SSCCP
Definitionen und Konstanten (Seite 221)	
Unions (Seite 222)	UIP UDataValue
Strukturen (Seite 223)	SDataValue SDVBNI SDataValueByAddress SDataValueByName SConnectionInfo SInstanceInfo SDimension STagInfo SIP SIPSuite4 SOnSyncPointReachedResult SDataValueByAddressWithCheck SDataValueByNameWithCheck SDataRecordInfo SDataRecord SConfiguredProcessEvents SDiagExtChannelDescription SAutodiscoverData SOnSoftwareConfigChangedParameter
Aufzählungen (Seite 239)	ERuntimeErrorCode EArea ECommunicationInterface EOperatingState EOperatingMode ECPUType ENetworkMode EPLCInterface ELEDType ELEDMode EPrimitiveDataTypes EDataType ETagListDetails ERuntimeConfigChanged EInstanceConfigChanged EPullOrPlugEventType EProcessEventTypes EDirection

Datentyp	
	EDiagProperty EDiagSeverity ERackOrStationFaultType ECycleTimeMonitoringMode EAutodiscoverType

1.9 Übersicht Datentypen für Managed Code

Die folgende Tabelle zeigt, welche Datentypen für die Simulation im Runtime Manager vorhanden sind.

Tabelle 1-20 Übersicht Datentypen - .NET (C#)

Datentyp	
Delegate-Definitionen (Seite 213) - Event Handler-Methoden	Delegate_Void Delegate_SRCC_UINT32_UINT32_INT32 Delegate_SRRSI_AD Delegate_IRRTM Delegate_II_EREC_DT_EOS_EOS Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 Delegate_II_EREC_DT Delegate_II_EREC_DT_SRICC_UINT32_UINT32_UINT32_UINT32 Delegate_II_EREC_DT_ELT_ELM Delegate_II_EREC_DT_SDR_I Delegate_II_EREC_DT_SDR Delegate_SREC_ST_UINT32_UINT32 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 Delegate_SREC_ST_UINT32_EPPET_UINT32 Delegate_SREC_ST_UINT32_ERSFET Delegate_SREC_ST_UINT32
Definitionen und Konstanten (Seite 221)	
Strukturen (Seite 223)	SDataValue SDVBNI SDataValueByAddress SDataValueByName SConnectionInfo SInstanceInfo SDimension STagInfo SIP SIPSuite4 SOnSyncPointReachedResult SDataValueByAddressWithCheck SDataValueByNameWithCheck SDataRecordInfo SDataRecord SConfiguredProcessEvents SDiagExtChannelDescription SAutodiscoverData
Aufzählungen (Seite 239)	ERuntimeErrorCode EArea ECommunicationInterface EOperatingState EOperatingMode ECPUType ELEDType ELEDMode ENetworkMode EPrimitiveDataTypes EDataType ETagListDetails ERuntimeConfigChanged EInstanceConfigChanged EPullOrPlugEventType

Datentyp	
	EProcessEventType EDirection EDiagProperty EDiagSeverity ERackOrStationFaultType ECycleTimeMonitoringMode EAutodiscoverType

1.10 Cybersecurity-Hinweise

Siemens bietet Produkte und Lösungen mit Industrial Cybersecurity-Funktionen an, die den sicheren Betrieb von Anlagen, Systemen, Maschinen und Netzwerken unterstützen.

Um Anlagen, Systeme, Maschinen und Netzwerke gegen Cyber-Bedrohungen zu sichern, ist es erforderlich, ein ganzheitliches Industrial Cybersecurity-Konzept zu implementieren (und kontinuierlich aufrechtzuerhalten), das dem aktuellen Stand der Technik entspricht. Die Produkte und Lösungen von Siemens formen einen Bestandteil eines solchen Konzepts.

Die Kunden sind dafür verantwortlich, unbefugten Zugriff auf ihre Anlagen, Systeme, Maschinen und Netzwerke zu verhindern. Diese Systeme, Maschinen und Komponenten sollten nur mit dem Unternehmensnetzwerk oder dem Internet verbunden werden, wenn und soweit dies notwendig ist und nur wenn entsprechende Schutzmaßnahmen (z. B. Firewalls und/oder Netzwerksegmentierung) ergriffen wurden.

Weiterführende Informationen zu möglichen Schutzmaßnahmen im Bereich Industrial Cybersecurity finden Sie unter

(<https://www.siemens.com/global/en/products/automation/topic-areas/industrial-cybersecurity.html>).

Die Produkte und Lösungen von Siemens werden ständig weiterentwickelt, um sie noch sicherer zu machen. Siemens empfiehlt ausdrücklich, Produkt-Updates anzuwenden, sobald sie zur Verfügung stehen und immer nur die aktuellen Produktversionen zu verwenden. Die Verwendung veralteter oder nicht mehr unterstützter Versionen kann das Risiko von Cyber-Bedrohungen erhöhen.

Um stets über Produkt-Updates informiert zu sein, abonnieren Sie den Siemens Industrial Cybersecurity RSS Feed unter

(<https://new.siemens.com/global/en/products/services/cert.html>).

API initialisieren

2.1 API-Bibliothek laden

Beschreibung

Bei S7-PLCSIM Advanced sind die Schnittstellen der API V6.0 nicht kompatibel mit den Schnittstellen früherer API-Versionen. Der Runtime Manager von S7-PLCSIM Advanced V6.0 ist aber kompatibel mit der API von früheren S7-PLCSIM Advanced-Versionen.

Beim Installieren von S7-PLCSIM Advanced V6.0 werden auch frühere Versionen der API installiert.

Der voreingestellte Pfad lautet:

- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\1.0
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\2.0
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\2.1
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\3.0
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\4.0
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\4.1
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\5.0
- C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API\6.0

HINWEIS

API-Versionen

Um die neuesten Features und Bugfixes aufrufen und verwenden zu können, benutzen Sie die neueste API-Version.

Den Installationspfad von S7-PLCSIM Advanced entnehmen Sie der Registry:

- Schlüssel: "HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Siemens\Shared Tools\PLCSIMADV_SimRT"
- Wert: "Path"

Um den Pfad zur API zu erhalten, fügen Sie an das Ende der Zeichenfolge folgendes Unterverzeichnis hinzu: "API\<API version>" (z. B. "API\6.0").

Wenn Sie diesen Pfad nutzen, dann wird die API-Bibliothek (DLL) direkt aus dem Installationsverzeichnis geladen.

Verweis

Weitere Informationen erhalten Sie:

- Für Native C++ im Kapitel `InitializeApi()` ([Seite 25](#)).
- Für .NET über den Aufruf der Funktion "`System.Reflection.Assembly.LoadFile(string)`" in der Online-Dokumentation zu MSDN.

2.2 Native C++

2.2.1 InitializeApi()

Beschreibung

Die Funktion `InitializeApi` lädt die API-Bibliothek (DLL) und initialisiert die API. Die Funktion lädt dabei die Version der DLL, die mit der Architektur Ihrer Anwendung und mit der Header-Datei der API kompatibel ist ("SimulationRuntimeApi.h").

Um die DLL zu laden, sucht die Funktion `InitializeApi` der Reihe nach in folgenden Ordnern:

- Ordner, zu dem der Parameter der Funktion führt (`in_SimulationRuntimeApiDllPath`)
- Ordner, in dem auch Ihre Anwendung liegt, die diese Funktion aufruft
- Installationsverzeichnis von S7-PLCSIM Advanced

Wenn keine DLL vorliegt, greift die Funktion auf den nächsten Ordner zu.

Die Funktion gibt an den Simulation Runtime Manager eine Schnittstelle zurück. Nutzen Sie diese Schnittstelle, um eine neue Instanz einer simulierten PLC zu erzeugen oder um Zugriff auf eine bereits bestehende Instanz zu erhalten.

Tabelle 2-1 InitializeApi() - Native C++

Syntax	<pre>ERuntimeErrorCode InitializeApi(ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface); ERuntimeErrorCode InitializeApi(WCHAR* in_SimulationRuntimeApiDllPath, ISimulationRuntimeManager** inout_SimulationRuntimeManagerInterface);</pre>	
Parameter	• <code>ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface</code>: Zeiger auf einen Runtime Manager-Schnittstellenzeiger. Der Zeiger muss mit <code>NULL</code> initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt. Siehe Datentypen (Seite 203). • <code>WCHAR* in_SimulationRuntimeApiDllPath</code>: Der Pfad zur Runtime API-Bibliothek.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_WRONG_ARGUMENT</code>	Der Zeiger auf die Runtime Manager-Schnittstelle ist ungleich <code>NULL</code> .
	<code>SREC_WRONG_VERSION</code>	• Die angeforderte Version der Schnittstelle ist nicht kompatibel mit der Version, mit der die API übersetzt wurde. • Die Version der API ist nicht kompatibel mit der Runtime. Siehe hierzu das Thema Kompatibilität im Funktionshandbuch von S7-PLCSIM Advanced.
	<code>SREC_CONNECTION_ERROR</code>	Zum Runtime Manager kann keine Verbindung hergestellt werden.
	<code>SREC_ERROR_LOADING_DLL</code>	Die API-Bibliothek kann nicht geladen werden.

Rückgabewerte	SREC_RUNTIME_NOT_AVAILABLE	In dieser Windows-Benutzersitzung läuft kein Runtime Manager.
	SREC_CONFIG_FILE_ERROR	Operation bezüglich der Konfigurationsdatei "UserInterfaceConfiguration.xml" ist fehlgeschlagen, z. B. Erzeugen, Lesen, Schreiben.
Beispiel C++	<pre>// Einschließen der Header-Datei der API #include "SimulationRuntimeApi.h" // Vorbereiten der Variablen ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; ISimulationRuntimeManager* api = NULL; // Initialisieren der API und Abrufen der RuntimeManager-Schnittstelle result = InitializeApi(&api);</pre>	

HINWEIS

Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle. Siehe DestroyInterface() ([Seite 30](#)).

2.2.2 RuntimeApiEntry_Initialize

Beschreibung

Nutzen Sie die Funktion `RuntimeApiEntry_Initialize` nur, wenn die API-Bibliothek (DLL) von einem anderen Ordner geladen werden soll als dem Ordner, in dem auch Ihre Anwendung liegt, die diese Funktion aufruft.

Beim Initialisieren der API wird zuerst die API-Bibliothek geladen und dann die Funktion `Initialize` importiert und aufgerufen.

Die Funktion gibt an den Simulation Runtime Manager eine Schnittstelle zurück. Nutzen Sie diese Schnittstelle, um eine neue Instanz einer simulierten PLC zu erzeugen oder um Zugriff auf eine bereits bestehende Instanz zu erhalten.

Tabelle 2-2 RuntimeApiEntry_Initialize - Native C++

Syntax	<pre>__declspec(dllexport) ERuntimeErrorCode RuntimeApiEntry_Initialize(ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface, UINT32 in_InterfaceVersion);</pre>	
Parameter	<code>ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface</code>: Zeiger auf einen Runtime Manager-Schnittstellenzeiger. Der Zeiger muss mit <code>NULL</code> initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt. Siehe Datentypen (Seite 203). <code>UINT32 in_InterfaceVersion</code>: Die Version der API-Schnittstelle, die geladen werden soll: <code>DAPI_DLL_INTERFACE_VERSION</code>.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_WRONG_ARGUMENT	Der Zeiger auf die Runtime Manager-Schnittstelle ist ungleich <code>NULL</code> .

Rückgabewerte	SREC_WRONG_VERSION	- Die angeforderte Version der Schnittstelle ist nicht kompatibel mit der Version, mit der die API übersetzt wurde. - Die Version der API ist nicht kompatibel mit der Runtime. Siehe die Hinweise zur Kompatibilität im Funktionshandbuch von S7-PLCSIM Advanced.
	SREC_CONNECTION_ERROR	Zum Runtime Manager kann keine Verbindung hergestellt werden.
	SREC_RUNTIME_NOT_AVAILABLE	In dieser Windows-Benutzersitzung läuft kein Runtime Manager.
	SREC_CONFIG_FILE_ERROR	Operation bezüglich der Konfigurationsdatei "UserInterfaceConfiguration.xml" ist fehlgeschlagen, z. B. Erzeugen, Lesen, Schreiben.
Beispiel C++	<pre>// Einschließen der Header-Datei der API #include "SimulationRuntimeApi.h" // Vorbereiten der Variablen ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; HMODULE dllHandle = NULL; ApiEntry_Initialize Initialize = NULL; ISimulationRuntimeManager* api = NULL; // Laden der DLL und Importieren der Funktion "Initialize" (über die // Windows-API) dllHandle = LoadLibrary(DAPI_DLL_NAME_X86); if (dllHandle != NULL) { Initialize = (ApiEntry_Initialize)GetProcAddress(dllHandle, DAPI_ENTRY_INITIALIZE); } // Initialisieren der API und Abrufen der RuntimeManager-Schnittstelle if (Initialize != NULL) { result = Initialize(&api, DAPI_DLL_INTERFACE_VERSION); }</pre>	

HINWEIS

Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle.

Siehe `DestroyInterface()` ([Seite 30](#)).

2.3 .NET (C#)

2.3.1 Initialize

Beschreibung

Der Eintrittspunkt zur API ist die statische Klasse
Siemens.Simatic.Simulation.Runtime.SimulationRuntimeManager.
Die API wird initialisiert, wenn eine Funktion dieser Klasse erstmalig genutzt wird.

Tabelle 2-3 Initialize - .NET (C#)

Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationInitializationException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.ConnectionError	Zum Runtime Manager kann keine Verbindung hergestellt werden.
	ERuntimeErrorCode.WrongVersion	Die Version der API ist nicht kompatibel mit der Runtime. Siehe die Hinweise zur Kompatibilität im Funktionshandbuch von S7-PLCSIM Advanced.
	ERuntimeErrorCode.RuntimeNotAvailable	In dieser Windows-Benutzersitzung läuft kein Runtime Manager.
	ERuntimeErrorCode.ConfigFileError	Operation bezüglich der Konfigurationsdatei "UserInterface-Configuration.xml" ist fehlgeschlagen, z. B. Erzeugen, Lesen, Schreiben.

API herunterfahren

3.1 Native C++

Prinzipielles Vorgehen beim Löschen der Anwenderschnittstellen

Um alle Anwenderschnittstellen zu löschen, gehen Sie prinzipiell so vor:

1. Löschen Sie die Schnittstellen `IInstances` und `IRemoteRuntimeManager`.
2. Rufen Sie die Funktion `Shutdown()` der Schnittstelle `ISimulationRuntimeManager` auf.
3. Löschen Sie die Schnittstelle `ISimulationRuntimeManager`.
4. Entladen Sie die API-Bibliothek (DLL) mit der Windows API-Funktion `FreeLibrary()`.

Löschen der Anwenderschnittstellen über Funktionen

Das Löschen der Anwenderschnittstellen ist auch über Funktionen möglich.

Wenn die API über die Funktion `InitializeApi()` initialisiert wurde, dann löschen Sie die Anwenderschnittstellen über folgende Funktionen:

- `FreeApi()` (Seite 31)
- `ShutdownAndFreeApi()` (Seite 32)

3.1.1 DestroyInterface()

Beschreibung

Ein Funktionszeiger auf die Funktion `RuntimeApiEntry_DestroyInterface`. Der Funktionszeiger `DestroyInterface()` wird nur gültig, wenn die Funktion `InitializeApi` erfolgreich aufgerufen wurde.

Die Funktion entlädt den Speicher einer `ISimulationRuntimeManager`, `IRemoteRuntimeManager` oder `IInstance` Schnittstelle.

Tabelle 3-1 DestroyInterface() - Native C++

Syntax	<code>ERuntimeErrorCode DestroyInterface(IBaseInterface* in_Interface);</code>	
Parameter	<code>IBaseInterface* in_Interface</code>: Die Schnittstelle, die gelöscht werden soll.	
Rückgabewerte	Runtime Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_WRONG_ARGUMENT</code>	Der Zeiger auf die Schnittstelle ist NULL.
Beispiel C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the DLL and create an instance result = InitializeApi(&api); result = api->RegisterInstance(&instance); // Destroy Instance Interfaces result = DestroyInterface(instance); instance = NULL;</pre>	

3.1.2 RuntimeApiEntry_DestroyInterface

Beschreibung

Nutzen Sie die Funktion `RuntimeApiEntry_DestroyInterface` nur, wenn die API-Bibliothek (DLL) von einem anderen Ordner geladen werden soll als dem Startup-Ordner der Anwendung, die diese Funktion aufruft.

Wenn die API über die Funktion `InitializeApi` initialisiert wurde, dann wählen Sie die Funktion `DestroyInterface()` ([Seite 30](#)).

Die Funktion entlädt den Speicher einer `ISimulationRuntimeManager`, `IRemoteRuntimeManager` oder `IInstance` Schnittstelle.

Tabelle 3-2 RuntimeApiEntry_DestroyInterface() - Native C++

Syntax	<code>_declspec(dllexport) ERuntimeErrorCode RuntimeApiEntry_DestroyInterface(IBaseInterface* in_Interface);</code>	
Parameter	<code>IBaseInterface* in_Interface</code>: Die Schnittstelle, die gelöscht werden soll.	
Rückgabewerte	Runtime Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_WRONG_ARGUMENT	Der Zeiger auf die Schnittstelle ist NULL.
Beispiel C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // Prepare The Variables ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; HMODULE dllHandle = NULL; ApiEntry_DestroyInterface Destroy = NULL; IInstance* instance = NULL; // Load The DLL And Import The "DestroyInterface" Function (using the Windows API) dllHandle = LoadLibraryA(DAPI_DLL_NAME_X86); if (dllHandle != NULL) { Destroy = (ApiEntry_DestroyInterface)GetProcAddress(dllHandle, DAPI_ENTRY_DESTROY_INTERFACE); } ... // Frees the memory of an IInstance interface result = Destroy(instance);</pre>	

3.1.3 FreeApi()

Beschreibung

Die Funktion `FreeApi()` entlädt die Bibliothek der Runtime API.

Diese Funktion kann nur aufgerufen werden nach dem erfolgreichen Aufruf der Funktion `InitializeApi`. Wenn die Funktion `InitializeApi` nicht aufgerufen wurde, muss die Bibliothek über die Windows API-Funktion `FreeLibrary()` entladen werden.

Tabelle 3-3 FreeApi() - Native C++

Syntax	<code>ERuntimeErrorCode FreeApi();</code>	
Parameter	Keine	
Rückgabewerte	Runtime Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_API_NOT_INITIALIZED	Die Funktion <code>InitializeApi</code> wurde nicht erfolgreich aufgerufen.
Beispiel C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the API result = InitializeApi(&api); ... // Shutdown The API api->Shutdown(); result = DestroyInterface(api); api = NULL; result = FreeApi();</pre>	

3.1.4 ShutdownAndFreeApi()

Beschreibung

Die Funktion `ShutdownAndFreeApi()` fährt die Runtime API herunter, löscht die `IRuntimeManager` Schnittstelle und entlädt die Bibliothek der Runtime API. Diese Funktion kann nur aufgerufen werden nach dem erfolgreichen Aufruf der Funktion `InitializeApi`. Wenn die Funktion `InitializeApi` nicht aufgerufen wurde, muss die Bibliothek über die Windows API-Funktion `FreeLibrary()` entladen werden.

Tabelle 3-4 ShutdownAndFreeApi() - Native C++

Syntax	<code>ERuntimeErrorCode ShutdownAndFreeApi(ISimulationRuntimeManager* in_SimulationRuntimeManagerInterface);</code>	
Parameter	<code>ISimulationRuntimeManager* in_SimulationRuntimeManagerInterface:</code> Die Schnittstelle des Runtime Managers, die gelöscht werden soll.	
Rückgabewerte	Runtime Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_API_NOT_INITIALIZED</code>	Die Funktion <code>InitializeApi</code> wurde nicht erfolgreich aufgerufen.
	<code>SREC_WRONG_ARGUMENT</code>	Der Zeiger auf die Schnittstelle ist <code>NULL</code> .
Beispiel C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the API result = InitializeApi(&api); ... // Shutdown The API result = ShutdownAndFreeApi(api); api = NULL;</pre>	

3.2 .NET (C#)

3.2.1 API herunterfahren

Die .NET-Komponenten der API können Sie für die `IInstance` und `IRemoteRuntimeManager` Schnittstellen über den Aufruf der Funktion `Dispose` (Seite 64) beenden. Sie können diese Schnittstellen auch automatisch mit dem .NET Garbage Collector bereinigen.

API manuell bereinigen

- Um die API manuell zu bereinigen, gehen Sie so vor:
- Löschen Sie alle Schnittstellen.
Weitere Informationen hierzu finden Sie unter Schnittstellen für `IInstances` (Seite 64).
 - Rufen Sie die Funktion `Shutdown()` (Seite 61) der `ISimulationRuntimeManager` Schnittstelle auf.

Globale Funktionen (Native C++)

Die globalen Funktionen `GetNameOf...` liefern den Namen des Aufzählungseintrags zurück (`const WCHAR*`).

GetNameOfAreaSection()

Tabelle 4-1 `GetNameOfAreaSection()` - Native C++

Syntax	<code>const WCHAR* GetNameOfAreaSection(EArea in_AreaSection);</code>
Parameter	<code>EArea in_AreaSection</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfCPUType()

Tabelle 4-2 `GetNameOfCPUType()` - Native C++

Syntax	<code>const WCHAR* GetNameOfCPUType(ECPUType in_CPUType);</code>
Parameter	<code>ECPUType in_CPUType</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfCommunicationInterface()

Tabelle 4-3 `GetNameOfCommunicationInterface()` - Native C++

Syntax	<code>const WCHAR* GetNameOfCommunicationInterface(CommunicationInterface in_CommunicationInterface);</code>
Parameter	<code>CommunicationInterface in_CommunicationInterface</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfDataType()

Tabelle 4-4 `GetNameOfDataType()` - Native C++

Syntax	<code>const WCHAR* GetNameOfDataType(EDDataType in_DataType);</code>
Parameter	<code>EDDataType in_DataType</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfErrorCode()

Tabelle 4-5 GetNameOfErrorCode() - Native C++

Syntax	<code>const WCHAR* GetNameOfErrorCode(ERuntimeErrorCode in_ErrorCode);</code>
Parameter	<code>ERuntimeErrorCode in_ErrorCode</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfLEDMode()

Tabelle 4-6 GetNameOfLEDMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfLEDMode(ELEDMode in_LEDMode);</code>
Parameter	<code>ELEDMode in_LEDMode</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfLEDType()

Tabelle 4-7 GetNameOfLEDType() - Native C++

Syntax	<code>const WCHAR* GetNameOfLEDType(ELEDType in_LEDType);</code>
Parameter	<code>ELEDType in_LEDType</code> : Aufzählungseintrag.
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfOperatingMode()

Tabelle 4-8 GetNameOfOperatingMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfOperatingMode(EOperatingMode in_OperatingMode);</code>
Parameter	<code>EOperatingMode in_OperatingMode</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfErrorCode()

Tabelle 4-9 GetNameOfErrorCode() - Native C++

Syntax	<code>const WCHAR* GetNameOfErrorCode(ERuntimeErrorCode in_ErrorCode);</code>
Parameter	<code>ERuntimeErrorCode in_ErrorCode</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfOperatingState

Tabelle 4-10 GetNameOfOperatingState() - Native C++

Syntax	<code>const WCHAR* GetNameOfOperatingState(EOperatingState in_OperatingState);</code>
Parameter	EOperatingState in_OperatingState: Aufzählungseintrag
Rückgabewerte	const WCHAR*: Name des Aufzählungseintrags

GetNameOfPrimitiveDataType

Tabelle 4-11 GetNameOfPrimitiveDataType() - Native C++

Syntax	<code>const WCHAR* GetNameOfPrimitiveDataType(EPrimitiveDataType in_DataType);</code>
Parameter	EPrimitiveDataType in_DataType: Aufzählungseintrag
Rückgabewerte	const WCHAR*: Name des Aufzählungseintrags

GetNameOfTagListDetails

Tabelle 4-12 GetNameOfTagListDetails() - Native C++

Syntax	<code>const WCHAR* GetNameOfTagListDetails(ETagListDetails in_TagListDetails);</code>
Parameter	ETagListDetails in_TagListDetails: Aufzählungseintrag
Rückgabewerte	const WCHAR*: Name des Aufzählungseintrags

GetNameOfRuntimeConfigChanged()

Tabelle 4-13 GetNameOfRuntimeConfigChanged() - Native C++

Syntax	<code>const WCHAR* GetNameOfRuntimeConfigChanged(ERuntimeConfigChanged in_RuntimeConfigChanged);</code>
Parameter	ERuntimeConfigChanged in_RuntimeConfigChanged: Aufzählungseintrag
Rückgabewerte	const WCHAR*: Name des Aufzählungseintrags

GetNameOfInstanceConfigChanged()

Tabelle 4-14 GetNameOfInstanceConfigChanged() - Native C++

Syntax	<code>const WCHAR* GetNameOfInstanceConfigChanged(EInstanceConfigChanged in_InstanceConfigChanged);</code>
Parameter	EInstanceConfigChanged in_InstanceConfigChanged: Aufzählungseintrag
Rückgabewerte	const WCHAR*: Name des Aufzählungseintrags

GetNameOfDirection()

Tabelle 4-15 GetNameOfDirection() - Native C++

Syntax	<code>const WCHAR* GetNameOfDirection(EDirection in_Direction);</code>
Parameter	<code>EDirection in_Direction</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfDiagSeverity()

Tabelle 4-16 GetNameOfDiagSeverity() - Native C++

Syntax	<code>const WCHAR* GetNameOfDiagSeverity(EDiagSeverity in_DiagSeverity);</code>
Parameter	<code>EDiagSeverity in_DiagSeverity</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfRackOrStationFaultType()

Tabelle 4-17 GetNameOfRackOrStationFaultType() - Native C++

Syntax	<code>const WCHAR* GetNameOfRackOrStationFaultType(ERackOrStationFaultType in_RackOrStationFaultType);</code>
Parameter	<code>ERackOrStationFaultType in_RackOrStationFaultType</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfProcessEventTypes()

Tabelle 4-18 GetNameOfProcessEventTypes() - Native C++

Syntax	<code>const WCHAR* (GetNameOfProcessEventTypes(EProcessEventTypes in_ProcessEventTypes);</code>
Parameter	<code>EProcessEventTypes in_ProcessEventTypes</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfPullOrPlugEventType()

Tabelle 4-19 GetNameOfPullOrPlugEventType() - Native C++

Syntax	<code>const WCHAR* GetNameOfPullOrPlugEventType(EPullOrPlugEventType in_PullOrPlugEventType);</code>
Parameter	<code>EPullOrPlugEventType in_PullOrPlugEventType</code> : Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*</code> : Name des Aufzählungseintrags

GetNameOfCycleTimeMonitoringMode()

Tabelle 4-20 GetNameOfCycleTimeMonitoringMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode);</code>
Parameter	<code>ECycleTimeMonitoringMode in_CycleTimeMonitoringMode:</code> Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*:</code> Name des Aufzählungseintrags

GetNameOfDiagProperty()

Tabelle 4-21 GetNameOfDiagProperty() - Native C++

Syntax	<code>const WCHAR* GetNameOfDiagProperty(EDiagProperty in_DiagProperty);</code>
Parameter	<code>EDiagProperty in_DiagProperty:</code> Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*:</code> Name des Aufzählungseintrags

GetNameOfAutodiscoverType()

Tabelle 4-22 GetNameOfAutodiscoverType() - Native C++

Syntax	<code>const WCHAR* GetNameOfAutodiscoverType(EAutodiscoverType in_AutodiscoverType);</code>
Parameter	<code>EAutodiscoverType in_AutodiscoverType:</code> Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*:</code> Name des Aufzählungseintrags

GetNameOfSoftwareConfigChanged()

Tabelle 4-23 GetNameOfSoftwareConfigChanged() - Native C++

Syntax	<code>const WCHAR* GetNameOfSoftwareConfigChanged(ESoftwareConfigChanged in_SoftwareConfigChanged);</code>
Parameter	<code>ESoftwareConfigChanged in_SoftwareConfigChanged:</code> Aufzählungseintrag
Rückgabewerte	<code>const WCHAR*:</code> Name des Aufzählungseintrags

IBaseRuntimeManager

5.1 Simulations-Laufzeitanstzen

CreateInterface()

Erzeugt und gibt eine Schnittstelle einer bereits registrierten Instanz einer simulierten PLC zurück.

Die Instanz kann über die Anwendung registriert worden sein oder über eine andere Anwendung, die die Simulation Runtime API nutzt.

Tabelle 5-1 CreateInterface() - Native C++

Syntax	<pre>ERuntimeErrorCode CreateInterface(WCHAR* in_InstanceName, IInstance** out_InstanceInterface); ERuntimeErrorCode CreateInterface(INT32 in_InstanceID, IInstance** out_InstanceInterface);</pre>	
Parameter	- INT32 in_InstanceID: Die ID der registrierten Instanz, von der Sie die Schnittstelle empfangen möchten. - WCHAR* in_InstanceName: Der Name der registrierten Instanz, von der Sie die Schnittstelle empfangen möchten. - IInstance** out_InstanceInterface: Zeiger auf einen Simulation Runtime-Schnittstellenzeiger. Der Zeiger muss mit NULL initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der Name, die ID oder der IInstance- Zeiger ist ungültig.
	SREC_DOES_NOT_EXIST	Die Instanz ist im Runtime Manager nicht registriert.
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa1 = NULL; IInstance* psa2 = NULL; if (result == SREC_OK) { result = api->CreateInterface(0, &psa1); result = api->CreateInterface(0, &psa2); // psa2 ist dasselbe wie psa1 }</pre>	
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->CreateInterface(L"My SimulationRuntime Instance", &psa); }</pre>	

HINWEIS**Native C++**

Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle.

Siehe DestroyInterface() (Seite 30)

Tabelle 5-2 CreateInterface() - .NET (C#)

Syntax	<pre>IInstance CreateInterface(string in_InstanceName); IInstance CreateInterface(INT32 in_InstanceID);</pre>	
Parameter	INT32 in_InstanceID: Die ID der registrierten Instanz, von der Sie die Schnittstelle empfangen möchten. string in_InstanceName: Der Name der registrierten Instanz, von der Sie die Schnittstelle empfangen möchten.	
Rückgabewerte	Wenn die Funktion erfolgreich ist, eine Schnittstelle einer simulierten PLC-Instanz, ansonsten ein Null-Zeiger.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	Der Name oder die ID ist ungültig.
	ERuntimeErrorCode.DoesNotExists	Die Instanz ist im Runtime Manager nicht registriert.

GetRegisteredInstanceInfoAt()

Gibt die Information über eine bereits registrierte Instanz zurück.

Sie können die ID oder den Namen verwenden, um eine Schnittstelle dieser Instanz zu erzeugen (siehe CreateInterface()).

Tabelle 5-3 GetRegisteredInstanceInfoAt() - Native C++

Syntax	<pre>ERuntimeErrorCode GetRegisteredInstanceInfoAt(UINT32 in_Index, SInstanceInfo* out_InstanceInfo);</pre>	
Parameter	UINT32 in_Index: Index der erzeugten Instanz, von der Sie die Information empfangen möchten. Der Index muss kleiner sein als der Wert, den Sie empfangen, wenn Sie GetRegisteredInstanceCount() aufrufen. SInstanceInfo* out_InstanceInfo: Die Information mit dem Namen und der ID der Instanz.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Es gibt keine Instanz-Information zu diesem Index.

RegisteredInstanceInfo { get; }

Gibt die Information über eine bereits registrierte Instanz zurück. Sie können die ID oder den Namen dieser Instanz verwenden, um eine Schnittstelle von dieser Instanz zu erzeugen; siehe `CreateInterface()`.

Tabelle 5-4 RegisterInstanceInfo { get; } - .NET (C#)

Syntax	SInstanceInfo[] RegisteredInstanceInfo { get; }	
Parameter	Keine	
Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

GetRegisteredInstancesCount()

Gibt die Anzahl der Instanzen zurück, die im Runtime Manager registriert sind. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.

Tabelle 5-5 GetRegisteredInstancesCount() - Native C++

Syntax	UINT32 GetRegisteredInstancesCount();
Parameter	Keine
Rückgabewerte	UINT32: Anzahl der verfügbaren Instanzen.

RegisterInstance()

Registriert im Runtime Manager eine neue Instanz einer simulierten PLC-Instanz. Erzeugt und gibt eine Schnittstelle dieser Instanz zurück.

Tabelle 5-6 RegisterInstance() - Native C++

Syntax	ERuntimeErrorCode RegisterInstance(IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(WCHAR* in_InstanceName, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(ECPUType in_CPUType, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(ECPUType in_CPUType, WCHAR* in_InstanceName, IInstance** out_InstanceInterface);
Parameter	- ECPUType in_CPUType: Definiert, welcher CPU-Typ beim Start der Instanz simuliert wird. Die Voreinstellung ist "SRCT_1500_Unspecified". Wenn über STEP 7 oder von der Virtual SIMATIC Memory Card ein anderer CPU-Typ geladen wird, dann gilt dieser CPU-Typ. - WCHAR* in_InstanceName: Name, den die Instanz erhalten soll. Jede Instanz muss einen eindeutigen Namen erhalten. Wenn beim Registrieren einer neuen Instanz kein Name vergeben wird, dann erhält die Instanz den Namen "Instance_#" (# ist die ID der Instanz). Wenn dieser Name bereits existiert, wird der Name "Instance_#. #" verwendet, wobei das zweite # ein Zähler ist, der so lange erhöht wird, bis der Name eindeutig ist. Die Länge des Namens muss kürzer sein als DINSTANCE_NAME_LENGTH und darf keine Unicode-Zeichen enthalten. Siehe Datentypen (Seite 203).

	<code>IInstance** out_InstanceInterface</code>: Zeiger auf einen Simulation Runtime-Schnittstellenzeiger. Der Zeiger muss mit <code>NULL</code> initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_CONFIG_FILE_ERROR</code>	Fehler in Konfigurationsdatei festgestellt.
	<code>SREC_INTERFACE_REMOVED</code>	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_WRONG_ARGUMENT</code>	Der Name oder der <code>IInstance</code> -Zeiger ist ungültig.
	<code>SREC_LIMIT_REACHED</code>	Es sind bereits 16 Instanzen im Runtime Manager registriert.
	<code>SREC_ALREADY_EXISTS</code>	Eine Instanz mit diesem Namen existiert bereits.
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); // Beispiel: Schritte zum Anlegen und Registrieren einer Instanz // und für den gleichzeitigen Abruf der Instanz IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); }</pre>	

HINWEIS**Native C++**

Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle.

Siehe `DestroyInterface()` ([Seite 30](#)).

Tabelle 5-7 RegisterInstance() - .NET (C#)

Syntax	<pre>IInstance RegisterInstance(); IInstance RegisterInstance(string in_InstanceName); IInstance RegisterInstance(ECPUType in_CPUType); IInstance RegisterInstance(ECPUType in_CPUType string in_InstanceName);</pre>	
Parameter	<code>ECPUType in_CPUType</code>: Definiert, welcher CPU-Typ beim Start der Instanz simuliert wird. Die Voreinstellung ist <code>"ECPUType.Unspecified"</code>. Wenn über STEP 7 oder von der Virtual SIMATIC Memory Card ein anderer CPU-Typ geladen wird, dann gilt dieser CPU-Typ. <code>string in_InstanceName</code>: Name, den die Instanz erhalten soll. Jede Instanz muss einen eindeutigen Namen erhalten. Wenn beim Registrieren einer neuen Instanz kein Name vergeben wird, dann erhält die Instanz den Namen <code>"Instance_#"</code> (# ist die ID der Instanz). Wenn dieser Name bereits existiert, wird der Name <code>"Instance_#. #"</code> verwendet, wobei das zweite # ein Zähler ist, der so lange erhöht wird, bis der Name eindeutig ist. Die Länge des Namens muss kürzer sein als <code>DINSTANCE_NAME_LENGTH</code>. Siehe Datentypen (Seite 203).	
Rückgabewerte	Wenn die Funktion erfolgreich ist, eine Schnittstelle einer simulierten PLC-Instanz, ansonsten ein Null-Zeiger.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.ConfigFileError</code>	Fehler in Konfigurationsdatei festgestellt.

5.1 Simulations-Laufzeitinstanzen

Ausnahmen	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	Der Name ist ungültig.
	ERuntimeErrorCode.LimitReached	Es sind bereits 16 Instanzen im Runtime Manager registriert.
	ERuntimeErrorCode.AlreadyExists	Eine Instanz mit diesem Namen existiert bereits.

RegisterCustomInstance()

Registriert im Runtime Manager eine neue simulierte PLC-Instanz. Erzeugt und gibt eine Schnittstelle dieser Instanz zurück.

Tabelle 5-8 RegisterCustomInstance() - Native C++

Syntax	<pre>ERuntimeErrorCode RegisterCustomInstance(WCHAR* in_VplcDll, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterCustomInstance(WCHAR* in_VplcDll, WCHAR* in_InstanceName, IInstance** out_InstanceInterface);</pre>	
Parameter	- WCHAR* in_VplcDll: Der vollständige Pfad zur DLL der simulierten PLC-Instanz, die "Siemens.Simatic.Simulation.Runtime.Instance.exe" bei PowerOn laden wird. - WCHAR* in_InstanceName: Name, den die Instanz erhalten soll. Jede Instanz muss einen eindeutigen Namen erhalten. Wenn beim Registrieren einer neuen Instanz kein Name vergeben wird, dann erhält die Instanz den Namen "Instance_#" (# ist die ID der Instanz). Wenn dieser Name bereits existiert, wird der Name "Instance_#. #" verwendet, wobei das zweite # ein Zähler ist, der so lange erhöht wird, bis der Name eindeutig ist. Die Länge des Namens muss kürzer sein als DINSTANCE_NAME_LENGTH. Siehe Datentypen (Seite 203). - IInstance** out_InstanceInterface: Zeiger auf einen Simulation Runtime-Schnittstellenzeiger. Der Zeiger muss mit NULL initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der DLL-Name, der Instanzname oder der IInstance-Zeiger ist ungültig.
	SREC_LIMIT_REACHED	Es sind bereits 16 Instanzen im Runtime Manager registriert.
	SREC_ALREADY_EXISTS	Eine Instanz mit diesem Namen existiert bereits.
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); // Beispiel: Schritte zum Anlegen und Registrieren einer Instanz // und für den gleichzeitigen Abruf der Instanz IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterCustomInstance(L"C:\\Temp\\vplc.dll", &psa); }</pre>	

HINWEIS

Native C++

Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle.

Siehe DestroyInterface() (Seite 30).

Tabelle 5-9 RegisterCustomInstance() - .NET (C#)

Syntax	<pre>IInstance RegisterCustomInstance(string in_VplcDll); IInstance RegisterCustomInstance(string in_VplcDll, string in_InstanceName);</pre>	
Parameter	string in_VplcDll: Der vollständige Pfad zur DLL der simulierten PLC-Instanz, die "Siemens.Simatic.Simulation.Runtime.Instance.exe" bei PowerOn laden wird. string in_InstanceName: Name, den die Instanz erhalten soll. Jede Instanz muss einen eindeutigen Namen erhalten. Wenn beim Registrieren einer neuen Instanz kein Name vergeben wird, dann erhält die Instanz den Namen "Instance_#" (# ist die ID der Instanz). Wenn dieser Name bereits existiert, wird der Name "Instance_#. #" verwendet, wobei das zweite # ein Zähler ist, der so lange erhöht wird, bis der Name eindeutig ist. Die Länge des Namens muss kürzer sein als <code>DINSTANCE_NAME_LENGTH</code>. Siehe Datentypen (Seite 203).	
Rückgabewerte	Wenn die Funktion erfolgreich ist, eine Schnittstelle einer simulierten PLC-Instanz, ansonsten ein Null-Zeiger.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	Der Name oder die ID ist ungültig.
	ERuntimeErrorCode.LimitReached	Es sind bereits 16 Instanzen im Runtime Manager registriert.
	ERuntimeErrorCode.AlreadyExists	Eine Instanz mit diesem Namen existiert bereits.

5.2 Schnittstellen für IBaseRuntimeManager

GetPort() / Port { get; }

Gibt den offenen Port des PC zurück, auf dem der Runtime Manager läuft. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.

Tabelle 5-10 GetPort() - Native C++

Syntax	<code>UINT16 GetPort();</code>
Parameter	Keine
Rückgabewerte	<code>UINT16</code> : Offener Port des PC, auf dem der Remote Runtime Manager läuft.

Tabelle 5-11 Port { get; } - .NET (C#)

Syntax	<code>UInt16 Port { get; }</code>
Parameter	Keine
Rückgabewerte	<code>UInt16</code> : Offener Port des PC, auf dem der Remote Runtime Manager läuft.

GetVersion() / Version { get; }

Gibt die Version des Runtime Managers zurück. Wenn die Funktion fehlschlägt, wird die Version 0.0 zurückgegeben.

Tabelle 5-12 GetVersion() - Native C++

Syntax	UINT32 GetVersion();
Parameter	Keine
Rückgabewerte	UINT32: Remote Runtime Manager Version (HIWORD = Major, LOWORD = Minor)

Tabelle 5-13 Version { get; } - .NET (C#)

Syntax	UInt32 Version { get; }
Parameter	Keine
Rückgabewerte	UInt32: Remote Runtime Manager Version (HIWORD = Major, LOWORD = Minor)

GetStrictMotionTiming() / StrictMotionTiming { get; }

Gibt die aktuelle globale Einstellung für das "Strict Motion Timing" Feature zurück, die sich auf neu erzeugte Instanzen auswirkt. Das Feature "Strict Motion Timing" befindet sich auf dem Control Panel von S7-PLCSIM Advanced.

Tabelle 5-14 GetStrictMotionTiming() - Native C++

Syntax	ERuntimeErrorCode GetStrictMotionTiming(bool* enabled);	
Parameter	bool* enabled: Empfängt die aktuelle Einstellung. true: Aktiv false: Inaktiv	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_CONFIG_FILE_ERROR	Die Einstellung konnte nicht aus der Konfigurationsdatei UserInterfaceConfiguration.xml gelesen werden.

Tabelle 5-15 StrictMotionTiming { get; } - .NET (C#)

Syntax	bool StrictMotionTiming { get; }	
Parameter	Keine	
Rückgabewerte	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.ConfigFileError	Die Einstellung konnte nicht aus der Konfigurationsdatei UserInterfaceConfiguration.xml gelesen werden.

SetStrictMotionTiming() / StrictMotionTiming { set; }

Setzt die globale Einstellung für das "Strict Motion Timing" Feature, die sich auf neu erzeugte Instanzen auswirkt. Das Feature "Strict Motion Timing" befindet sich auf dem Control Panel von S7-PLCSIM Advanced.

Tabelle 5-16 SetStrictMotionTiming() - Native C++

Syntax	<code>ERuntimeErrorCode SetStrictMotionTiming(bool enable);</code>	
Parameter	bool enable: Der einzustellende Wert. true: Aktiv false: Inaktiv	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_ALREADY_EXISTS	Eine Instanz ist registriert. Es darf keine Instanz registriert sein, mit der die Einstellung geändert wird.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_ACCESS_DENIED	Schreibrechte fehlen für die Konfigurationsdatei.
	SREC_CONFIG_FILE_ERROR	Die Einstellung konnte nicht in die Konfigurationsdatei UserInterfaceConfiguration.xml geschrieben werden.

Tabelle 5-17 StrictMotionTiming { set; } - .NET (C#)

Syntax	<code>bool StrictMotionTiming { set; }</code>	
Parameter	Keine	
Rückgabewerte	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.AlreadyExists	Eine Instanz ist registriert. Es darf keine Instanz registriert sein, mit der die Einstellung geändert wird.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.AccessDenied	Schreibrechte fehlen für die Konfigurationsdatei.
	ERuntimeErrorCode.ConfigFileError	Die Einstellung konnte nicht in die Konfigurationsdatei UserInterfaceConfiguration.xml geschrieben werden.

GetNetworkMode

Gibt die aktuellen Einstellungen des Netzwerkmodus zurück, die im Netzwerkadapter für verteilte Kommunikation verwendet werden.

Tabelle 5-18 GetNetworkMode() - Native C++

Syntax	<code>ERuntimeErrorCode IBaseRuntimeManager::GetNetworkMode(ENetworkMode* out_Mode);</code>	
Parameter	ENetworkMode* out_Mode Empfängt den Netzwerkmodus als #ENetworkMode, der aktuell im Treiber des S7-PLCSIM Advanced Virtual Switch eingestellt ist	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Der Funktionsaufruf ist erfolgreich.

SetNetworkMode()

Stellt den Netzwerkmodus ein, wie er im Netzadapter für verteilte Kommunikation verwendet wird.

Überprüfen Sie, ob das Binding auf den Netzwerkschnittstellen korrekt ist.

Tabelle 5-19 SetNetworkMode() - Native C++

Syntax	<code>ERuntimeErrorCode SetNetworkMode(const ENetworkMode in_mode);</code>	
Parameter	<code>ENetworkMode in_Mode</code> Art des einzustellenden Netzwerk-Kommunikationsmodus	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Der Funktionsaufruf ist erfolgreich.
	<code>SREC_VIRTUAL_SWITCH_MISCONFIGURED</code>	Der Treiber des S7-PLCSIM Advanced Virtual Switch läuft nicht.
	<code>SREC_INSTANCE_ALREADY_RUNNING</code>	Der Netzwerkmodus kann nicht geändert werden, weil bereits eine laufende Instanz vorhanden ist.
	<code>SREC_PCAP_DRIVER_NOT_RUNNING</code>	Der Nmap Packet Capture Driver (Npcap) ist auf dem System nicht aktiv. Abhilfe: 1. Starten Sie die Kommandozeile im Administrator-Modus. 2. Führen Sie folgenden Befehl aus: <code>net start npcap</code>

NetworkMode { get; set; };

Gibt den Netzwerkmodus zurück oder setzt den Netzwerkmodus, der auf dem Netzwerkadapter für verteilte Kommunikation verwendet wird.

Überprüfen Sie, ob das Binding auf den Netzwerkschnittstellen korrekt ist.

Tabelle 5-20 NetworkMode { get; } - .NET (C#)

Syntax	<code>ENetworkMode NetworkMode { get; };</code>
Rückgabewert	<code>ENetworkMode</code> Empfängt den Netzwerkmodus als <code>#ENetworkMode</code> , der aktuell im Treiber des S7-PLCSIM Advanced Virtual Switch eingestellt ist
Ausnahmen	Keine

Tabelle 5-21 NetworkMode { set; } - .NET (C#)

Syntax	<code>ENetworkMode NetworkMode { set; };</code>	
Parameter	Keine	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.VirtualSwitchMisconfigured</code>	Der S7-PLCSIM Advanced Virtual Switch wird nicht ausgeführt.
	<code>ERuntimeErrorCode.InstanceAlreadyRunning</code>	Der Netzwerkmodus kann nicht geändert werden, weil bereits eine laufende Instanz vorhanden ist.
	<code>ERuntimeErrorCode.PCAPDriverNotRunning</code>	Der Nmap Packet Capture Driver (Npcap) ist auf dem System nicht aktiv. Abhilfe: 1. Starten Sie die Kommandozeile im Administrator-Modus. 2. Führen Sie folgenden Befehl aus: <code>net start npcap</code>

GetNetInterfaces() / NetInterfaces { get; }

Liefert Informationen zu den PC-Schnittstellen, die Sie für SetNetInterfaceMapping nutzen können.

GetNetInterfaces() gibt keine Drahtlosadapter zurück.

Tabelle 5-22 GetNetInterfaces() - Native C++

Syntax	ERuntimeErrorCode GetNetInterfaces(SNetInterfaceInfo* inout_InterfaceInfo, UINT* inout_InterfaceCount);	
Parameter	SNetInterfaceInfo* inout_InterfaceInfo: Adresse des ersten Items des SNetInterfaceInfo-Arrays, das Informationen über PC-Netzwerkadapter enthält. Wenn der Wert NULL ist, wurde nur die Anzahl der Netzwerkadapter angefordert. UINT* inout_InterfaceCount: Bei der Eingabe: Anzahl der SNetInterfaceInfo-Items, die vom Benutzer zugewiesen wurden Bei der Ausgabe: Anzahl der Netzwerkadapter (Array Items)	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Der Funktionsaufruf ist erfolgreich.
	SREC_INDEX_OUT_OF_RANGE	Die Anzahl der Array-Einheiten ist niedriger als die Anzahl der Netzwerkadapter.
	SREC_INTERNAL_ERROR	Der Systemaufruf hat einen internen Fehler verursacht.
Beispiel C++	<pre>#include <iostream> #include "SimulationRuntimeApi.h" ISimulationRuntimeManager* rtm = nullptr; ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; //Initialisieren der Verbindung InitializeApi(&rtm); //Anzahl der Netzwerkschnittstellen abrufen UINT count; result = rtm->GetNetInterfaces(nullptr, &count); //Reale Netzwerkschnittstellen abrufen std::unique_ptr<SNetInterfaceInfo[]> netInterfaces = std::make_unique<SNetInterfaceInfo[]>(count); result = rtm->GetNetInterfaces(netInterfaces.get(), &count); ShutdownAndFreeApi(rtm);</pre>	

Tabelle 5-23 NetInterfaces { get; } - .NET (C#)

Syntax	SNetInterfaceInfo[] NetInterfaces { get; }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SNetInterfaceInfo[]	Informationen über die PC-Netzwerkschnittstellen
Ausnahmen	ERuntimeErrorCode.InternalError	Der Systemaufruf hat einen internen Fehler verursacht.

SetNetInterfaceBindings()

Setzt das Binding eines S7-PLCSIM Advanced Virtual Switch auf PC-Schnittstellen gemäß dem Netzwerkmodus und der Mapping-Konfiguration der Netzwerkschnittstellen für die Instanz.

5.2 Schnittstellen für IBaseRuntimeManager

Sie müssen die Funktion von einer übergeordneten Session (Administratormodus) aus aufrufen, da Sie andernfalls keinen Zugriff darauf haben und die Meldung `SREC_ACCESS_DENIED` erscheint. Diese Funktion ist keine gültige Funktion für den `NetworkMode Softbus`.

Tabelle 5-24 SetNetInterfaceBindings() - Native C++

Syntax	ERuntimeErrorCode SetNetInterfaceBindings(DWORD in_PCInterfaceIndex = 0);	
Parameter	DWORD in_PCInterfaceIndex = 0: • Wenn der Single Adapter Network Mode (Promiscuous-Modus) eingestellt ist, können Sie den Index der PC-Schnittstellen für die Netzwerkkommunikation angeben. Um nur eine lokale PC-Kommunikation zu ermöglichen, verwenden Sie den Wert (0). • Wenn der Multi-Adapter Network Mode (Non-Promiscuous-Modus) eingestellt ist, wird dieser Parameter nicht verwendet und das Binding wird gemäß dem konfigurierten Mapping der Schnittstellen gesetzt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Der Funktionsaufruf ist erfolgreich.
	SREC_INVALID_CONFIGURATION	Die Konfiguration ist ungültig, weil einer Instanz ein ungültiger Adapter zugeordnet ist. Beispielsweise wird in einer Datei "sim_hwdb.ini" einer registrierten Instanz die MAC-Adresse eines ungültigen Adapters verwendet, wie z. B. die eines PLCSIM-vAdapters, eines deaktivierten Adapters, eines Drahtlosadapters oder eines Adapters, der den PLCSIM vSwitch nicht unterstützt.
	SREC_ACCESS_DENIED	Sie haben keine Berechtigung, das Binding auf die PC-Netzwerkschnittstellen durchzuführen.
	SREC_WRONG_COMMUNICATION_INTERFACE	Die PC-Schnittstelle existiert nicht oder ist deaktiviert.
	SREC_INTERNAL_ERROR	Der Systemaufruf hat einen internen Fehler verursacht. Aktivieren Sie die Traces, um Detailinformationen zu erhalten.
	SREC_INSTANCE_ALREADY_RUNNING	Im Single Adapter Network Mode (Promiscuous-Modus) können die Bindings nicht geändert werden, während eine Instanz gerade läuft.
	SREC_VIRTUAL_SWITCH_MISCONFIGURED	Die Konfiguration eines S7-PLCSIM Advanced Virtual Switch ist nicht korrekt oder der Treiber funktioniert nicht einwandfrei.
	SREC_NOT_SUPPORTED	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

Tabelle 5-25 SetNetInterfaceBindings() - .NET (C#)

Syntax	void SetNetInterfaceBindings(); void SetNetInterfaceBindings(DWORD in_PCInterfaceIndex);	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode. InvalidConfiguration	Die Konfiguration ist ungültig, weil z. B. ein nicht vorhandener Adapter verwendet wurde.
	ERuntimeErrorCode.AccessDenied	Sie haben keine Berechtigung, das Binding auf die PC-Netzwerkschnittstellen durchzuführen.

Ausnahmen	ERuntimeErrorCode.WrongCommunicationInterface	Die PC-Schnittstelle existiert nicht oder ist deaktiviert.
	ERuntimeErrorCode.InternalError	Der Systemaufruf hat einen internen Fehler verursacht. Aktivieren Sie die Traces, um Detailinformationen zu erhalten.
	ERuntimeErrorCode.InstanceAlreadyRunning	Im Single Adapter Network Mode (Promiscuous-Modus) können die Bindings nicht geändert werden, während eine Instanz gerade läuft.
	ERuntimeErrorCode.VirtualSwitchMisconfigured	Die Konfiguration eines S7-PLCSIM Advanced Virtual Switch ist nicht korrekt oder der Treiber funktioniert nicht einwandfrei.
	ERuntimeErrorCode.NotSupported	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

ResetNetInterfaceBindings()

Deaktiviert das Binding eines S7-PLCSIM Advanced Virtual Switch auf PC-Schnittstellen. Das Binding auf dem Siemens PLCSIM Virtual Ethernet Adapter bleibt bestehen.

Sie müssen die Funktion von einer übergeordneten Session (Administratormodus) aus aufrufen, da Sie andernfalls keinen Zugriff darauf haben und die Meldung SREC_ACCESS_DENIED erscheint. Bei einem lokalen Runtime Manager muss sich die Anwendung, die die API verwendet, in einer übergeordneten Sitzung befinden; bei Remote-Zugriff muss sich der remote zugreifende Runtime Manager in der übergeordneten Sitzung befinden. Diese Funktion ist keine gültige Funktion für den NetworkMode Softbus.

Tabelle 5-26 ResetNetInterfaceBindings() - Native C++

Syntax	ERuntimeErrorCode ResetNetInterfaceBindings();	
Parameter	Keine	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Der Funktionsaufruf ist erfolgreich.
	SREC_ACCESS_DENIED	Sie haben keine Berechtigung, das Binding auf die PC-Netzwerkschnittstellen durchzuführen.
	SREC_INSTANCE_ALREADY_RUNNING	Im Single Adapter Network Mode (Promiscuous-Modus) können Bindings nicht geändert werden, während eine Instanz gerade läuft.
	SREC_INTERNAL_ERROR	Der Systemaufruf hat einen internen Fehler verursacht. Aktivieren Sie die Traces, um Detailinformationen zu erhalten.
	SREC_NOT_SUPPORTED	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

ResetNetInterfaceBindings() - .NET (C#)

Syntax	void ResetNetInterfaceBindings();	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.AccessDenied	Sie haben keine Berechtigung, das Binding auf die PC-Netzwerkschnittstellen durchzuführen.

Ausnahmen	<code>ERuntimeErrorCode.InternalError</code>	Der Systemaufruf hat einen internen Fehler verursacht. Aktivieren Sie die Traces, um Detailinformationen zu erhalten.
	<code>ERuntimeErrorCode.InstanceAlreadyRunning</code>	Im Single Adapter Network Mode (Promiscuous-Modus) können Bindings nicht geändert werden, während eine Instanz gerade läuft.
	<code>ERuntimeErrorCode.NotSupported</code>	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

GetDefaultStoragePath()

Gibt den vollständigen Pfad des Default-Ordners zurück, in dem die Instanz ihre Daten speichert.

Tabelle 5-27 GetDefaultStoragePath() - Native C++

Syntax	<code>ERuntimeErrorCode GetDefaultStoragePath(WCHAR* out_DefaultStoragePath);</code>	
Parameter	<code>WCHAR* out_DefaultStoragePath</code> Ausgabeparameter, der mit dem Pfad des Default-Ordners befüllt wird.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.

SetDefaultStoragePath()

Setzt den vollständigen Pfad des Default-Ordners, in dem die Instanz ihre Daten speichert.

Tabelle 5-28 SetDefaultStoragePath() - Native C++

Syntax	<code>ERuntimeErrorCode SetDefaultStoragePath(const WCHAR* in_DefaultStoragePath);</code>	
Parameter	<code>const WCHAR* in_DefaultStoragePath</code> Eingabeparameter, der den Pfad des Default-Ordners angibt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_WRONG_ARGUMENT</code>	Der Pfad enthält ungültige Zeichen.
	<code>SREC_CONFIG_FILE_ERROR</code>	Die Konfigurationsdatei <code>UserInterfaceConfiguration.xml</code> konnte nicht gelesen werden.

DefaultStoragePath { get; set; }

Setzt den vollständigen Pfad des Default-Ordners oder gibt diesen Pfad zurück, in dem die Instanz ihre Daten speichert.

Tabelle 5-29 DefaultStoragePath { get; set; } - .NET (C#)

Syntax	<code>string DefaultStoragePath { get; set; }</code>	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>OK</code>	Die Funktion ist erfolgreich.
	<code>WrongArgument</code>	Der Pfad enthält ungültige Zeichen.
	<code>ConfigFileError</code>	Die Konfigurationsdatei <code>UserInterfaceConfiguration.xml</code> konnte nicht gelesen werden.

API ISimulationRuntimeManager

6.1 Remote-Verbindungen

OpenPort()

Öffnet einen Port, mit dem sich ein weiterer Runtime Manager verbinden kann.

Tabelle 6-1 OpenPort() - Native C++

Syntax	<code>ERuntimeErrorCode OpenPort(UINT16 in_Port);</code>	
Parameter	<code>UINT16 in_Port</code>: Der Port. Der Wert muss größer als 1024 sein.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_ALREADY_EXISTS</code>	Es ist bereits ein Port offen.
	<code>SREC_WRONG_ARGUMENT</code>	Der Port ist ungültig.
	<code>SREC_CONNECTION_ERROR</code>	Der Port kann nicht geöffnet werden.

Tabelle 6-2 OpenPort() - .NET (C#)

Syntax	<code>void OpenPort(UInt16 in_Port);</code>	
Parameter	<code>UInt16 in_Port</code>: Der Port. Der Wert muss größer als 1024 sein.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.AlreadyExists</code>	Es ist bereits ein Port offen.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Port ist ungültig.
	<code>ERuntimeErrorCode.ConnectionError</code>	Der Port kann nicht geöffnet werden.

ClosePort()

Schließt einen offenen Port und alle offenen Verbindungen, die ein weiterer Runtime Manager zu diesem offenen Port erstellt hat.

Tabelle 6-3 ClosePort() - Native C++

Syntax	<code>ERuntimeErrorCode ClosePort();</code>	
Parameter	Keine	
Rückgabewerte	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung

6.1 Remote-Verbindungen

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WARNING_INVALID_CALL	Es ist kein Port offen.

Tabelle 6-4 ClosePort() - .NET (C#)

Syntax	void ClosePort();	
Parameter	Keine	
Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

GetRemoteConnectionsCount()

Liefert die Anzahl der offenen Remote-Verbindungen.

Tabelle 6-5 GetRemoteConnectionsCount() - Native C++

Syntax	UINT32 GetRemoteConnectionsCount();
Parameter	Keine
Rückgabewerte	UINT32: Anzahl der offenen Remote-Verbindungen.

GetRemoteConnectionInfoAt()

Gibt die Information zu einer offenen Verbindung zurück.

Tabelle 6-6 GetRemoteConnectionInfoAt() - Native C++

Syntax	ERuntimeErrorCode GetRemoteConnectionInfoAt(UINT32 in_Index, SConnectionInfo* out_ConnectionInfo);	
Parameter	- UINT32 in_Index: Index der Verbindungsinformationen, die erwartet werden. - SConnectionInfo* out_ConnectionInfo: Die Verbindungs-Information für diesen Index.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Verbindungsinformationen zu diesem Index existieren nicht.

RemoteConnectionInfo { get; }

Gibt ein Array von Informationen zu allen offenen Verbindungen zurück.

Tabelle 6-7 RemoteConnectionInfo { get; } - .NET (C#)

Syntax	SConnectionInfo[] RemoteConnectionInfo { get; }
Parameter	Keine
Rückgabewerte	SConnectionInfo[] : Ein Array von Informationen über alle offenen Verbindungen.

Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

RemoteConnect()

Erstellt eine neue Verbindung zu einem Remote Runtime Manager oder nutzt eine bereits existierende Verbindung, um eine IRemoteRuntimeManager-Schnittstelle zu erstellen.

Tabelle 6-8 RemoteConnect() - Native C++

Syntax	<pre>ERuntimeErrorCode RemoteConnect(UINT8 in_IP3, UINT8 in_IP2, UINT8 in_IP1, UINT8 in_IP0, UINT16 in_Port, IRemoteRuntimeManager**out_RemoteRuntimeManagerInterface); ERuntimeErrorCode RemoteConnect(UIP in_IP, UINT16 in_Port, IRemoteRuntimeManager** out_RunTimeManagerInterface);</pre>	
Parameter	- UINT8 in_IP3: Erster Teil der IP-Adresse des Remote-PC. - UINT8 in_IP2: Zweiter Teil der IP-Adresse des Remote-PC. - UINT8 in_IP1: Dritter Teil der IP-Adresse des Remote-PC. - UINT8 in_IP0: Letzter Teil der IP-Adresse des Remote-PC. - UIP in_IP: IP-Adresse des Remote-PC. - UINT16 in_Port: Der Port, der auf dem Remote-PC geöffnet ist. - IRemoteRuntimeManager** out_RemoteRuntimeManagerInterface: Zeiger auf einen Remote Runtime Manager-Schnittstellenzeiger. Der Zeiger muss mit NULL initialisiert werden. Die Schnittstelle wird in der Funktion erzeugt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_CONNECTION_ERROR	Die Verbindung zum Remote Runtime Manager kann nicht hergestellt werden.
	SREC_WRONG_ARGUMENT	IP, Port oder Instance-Zeiger ist ungültig.
	SREC_WRONG_VERSION	Die Version der API ist nicht kompatibel mit der Runtime. Siehe die Hinweise zur Kompatibilität im Funktionshandbuch von S7-PLCSIM Advanced.
Beispiel C++	<pre>ISimulationRuntimeManager* api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IRemoteRuntimeManager * client = NULL; if (result == SREC_OK) { result = api->RemoteConnect(192,203,145,144, 4444, &client); }</pre>	

HINWEIS
Native C++
Wenn Sie die Schnittstelle nicht mehr benötigen, löschen Sie die Schnittstelle.
Siehe DestroyInterface() (Seite 30).

Tabelle 6-9 RemoteConnect() - .NET (C#)

Syntax	<pre>IRemoteRuntimeManager RemoteConnect(string in_ConnectionString); IRemoteRuntimeManager RemoteConnect(SIP in_IP, UInt16 in_Port); IRemoteRuntimeManager RemoteConnect(Byte in_IP3, Byte in_IP2, Byte in_IP1, Byte in_IP0, UInt16 in_Port);</pre>	
Parameter	- Byte in_IP3: Erster Teil der IP-Adresse des Remote-PC. - Byte in_IP2: Zweiter Teil der IP-Adresse des Remote-PC. - Byte in_IP1: Dritter Teil der IP-Adresse des Remote-PC. - Byte in_IP0: Letzter Teil der IP-Adresse des Remote-PC. - string in_ConnectionString: Ein String in Form von "<IP3>.<IP2>.<IP1>.<IP0>:<Port>" Beispiel: "182.203.145.144:4444". - SIP in_IP: IP-Adresse des Remote-PC. - UInt16 in_Port: Der Port, der auf dem Remote-PC geöffnet ist.	
Rückgabewerte	IRemoteRuntimeManager: Schnittstelle zum Remote Runtime Manager.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.ConnectionError	Verbindung zum Remote Runtime Manager kann nicht hergestellt werden.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	IP oder Port ist ungültig.
	ERuntimeErrorCode.WrongVersion	Die Version der API ist nicht kompatibel mit der Run-time. Siehe die Hinweise zur Kompatibilität im Funktionshandbuch von S7-PLCSIM Advanced.

6.1.1 RunAutodiscover()

Beschreibung

Mit dieser Funktion werden alle Runtime Manager identifiziert, die sich im Netzwerk befinden und bereit sind, eine Remote-Verbindung herzustellen.

HINWEIS

Die Funktion identifiziert Runtime Manager ab S7-PLCSIM Advanced V5.0.

Voraussetzungen

- Der Runtime Manager muss ausgeführt werden und Remote-Verbindungen zulassen.
- Die Firewall des Remote-PC darf den Datenverkehr am ausgewählten UDP-Port nicht blockieren.
- Geräte im lokalen Netzwerk (z. B. Router, Switches, Firewalls) dürfen Multicast-Pakete der ausgewählten Klasse nicht blockieren.

RunAutodiscover()

Die Funktion startet im Netzwerk die Identifizierung der Runtime Manager.

Tabelle 6-10 RunAutodiscover() - Native C++

Syntax	<code>ERuntimeErrorCode RunAutodiscover(UINT32 in_Timeout = 2000);</code>	
Parameter	• <code>UINT32 in_Timeout</code> Ein Timeout-Wert in Millisekunden, der definiert, wie lange der lokale Runtime Manager auf Antworten vom Remote Manager wartet. Gültig ist ein Wert zwischen 500 ms und 30000 ms. Voreinstellung: 2000 ms.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_WRONG_ARGUMENT</code>	Der Timeout-Wert ist außerhalb des zulässigen Bereichs.
	<code>SREC_AUTODISCOVER_ALREADY_RUNNING</code>	Ein Aufruf <code>RunAutodiscover()</code> läuft bereits im Hintergrund. Warten Sie auf die Meldung <code>SRRSI_DISCOVER_FINISHED</code> in der Callback-Funktion. Siehe <code>EAutodiscoverType</code> (Seite 258).
	<code>SREC_TIMEOUT</code>	Kommunikationsfehler im lokalen Runtime Manager.

Tabelle 6-11 RunAutodiscover() - .NET (C#)

Syntax	<code>void RunAutodiscover(UInt32 in_Timeout = 2000);</code>	
Parameter	• <code>UInt32 in_Timeout</code> Ein Timeout-Wert in Millisekunden, der definiert, wie lange der lokale Runtime Manager auf Antworten vom Remote Manager wartet. Gültig ist ein Wert zwischen 500 ms und 30000 ms. Voreinstellung: 2000 ms.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Timeout-Wert ist außerhalb des zulässigen Bereichs.
	<code>ERuntimeErrorCode.AutodiscoverAlreadyRunning</code>	Ein Aufruf <code>RunAutodiscover()</code> läuft bereits im Hintergrund. Warten Sie auf die Meldung <code>AutodiscoverFinished</code> in der Callback-Funktion. Siehe <code>EAutodiscoverType</code> (Seite 258).
	<code>ERuntimeErrorCode.Timeout</code>	Kommunikationsfehler im lokalen Runtime Manager.

6.2 Ereignisse für ISimulationRuntimeManager

Ereignisse für Runtime Instanzen und Remote-Verbindungen

Für die Schnittstelle ISimulationRuntimeManager werden folgende Ereignisse ausgelöst:

Tabelle 6-12 Ereignisse für ISimulationRuntimeManager

Ereignis	Ursache
Ereignisse OnSoftwareConfigurationChanged (Seite 56)	Die Runtime Manager Konfiguration hat sich geändert: • Eine neue Instanz wird registriert. • Eine Instanz wird entfernt. • Eine Verbindung zu einem Client wird aufgebaut. Das Control Panel nutzt ein solches Ereignis, um die Liste der verfügbaren Instanzen zu aktualisieren.
OnRuntimeManagerLost (Seite 58)	Die Verbindung zum Runtime Manager ist unterbrochen.
Ereignisse OnAutodiscoverData (Seite 61)	Im Netzwerk wird nach Runtime Managern gesucht, die bereit sind, eine Remote-Verbindung herzustellen.

6.2.1 Ereignisse OnSoftwareConfigurationChanged

OnSoftwareConfigurationChanged

Meldet Software-Änderungen in den Betriebszuständen STOP und RUN.

Tabelle 6-13 OnSoftwareConfigurationChanged

Syntax	<code>event Delegate_SRSCC OnSoftwareConfigurationChanged();</code>
Parameter	<code>IInstance, ERuntimeErrorCode, SYSTEMTIME, ESoftwareConfigChanged</code>
Rückgabewerte	Keine
Ausnahmen	Keine
Bemerkung	Die Event-Handler-Methode läuft in einem separaten Thread.

RegisterOnSoftwareConfigurationChangedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es kann nur eine Callback-Funktion für das Ereignis registriert sein. Die Registrierung einer neuen Callback-Funktion führt zum Löschen der vorhergehenden.

Tabelle 6-14 RegisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedCallback();</code>
Parameter	<code>CallbackFunction</code>
Rückgabewerte	Keine
Bemerkung	Die Event-Handler-Methode läuft in einem separaten Thread.

RegisterOnSoftwareConfigurationChangedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Die Registrierung eines neuen Event-Objekts führt zum Löschen des vorhergehenden Event-Objekts.

Tabelle 6-15 RegisterOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	• <code>None</code> Ein internes Event-Objekt wird registriert. • <code>HANDLE*</code> Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

UnregisterOnSoftwareConfigurationChangedCallback()

Hebt die Registrierung der Callback-Funktion auf. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 6-16 UnregisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedCallback();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnSoftwareConfigurationChangedEvent()

Hebt die Registrierung des Event-Objekts auf.

Tabelle 6-17 UnregisterOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 6-18 UnregisterOnSoftwareConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnSoftwareConfigurationChangedEvent()

Die Funktion blockiert das Programm so lange, bis das registrierte Event-Objekt in den signalisierten Zustand gesetzt ist oder das Timeout-Intervall überschritten wird.

Tabelle 6-19 WaitForOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>bool WaitForOnSoftwareConfigurationChangedEvent();</code> <code>bool WaitForOnSoftwareConfigurationChangedEvent(uint in_Time_ms);</code>
Parameter	• None: Das Zeitlimit ist auf INFINITE gesetzt. • UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.
Bemerkung	Die Funktion blockiert das CoSimulation-Programm so lange, bis ein Software-Download in den signalisierten Zustand gesetzt ist oder das Timeout-Intervall überschritten wird.

Tabelle 6-20 WaitForOnSoftwareConfigurationChangedEvent() - .NET (C#)

Syntax	<code>bool WaitForOnSoftwareConfigurationChangedEvent();</code> <code>bool WaitForOnSoftwareConfigurationChangedEvent(uint in_Time_ms);</code>
Parameter	• None: Das Zeitlimit ist auf INFINITE gesetzt. • UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.
Bemerkung	Die Funktion blockiert das CoSimulation-Programm so lange, bis ein Software-Download in den signalisierten Zustand gesetzt ist oder das Timeout-Intervall überschritten wird.

6.2.2 Ereignisse OnRuntimeManagerLost**OnRuntimeManagerLost**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 6-21 OnRuntimeManagerLost - .NET (C#)

Syntax	<code>event Delegate_Void OnRuntimeManagerLost;</code>
Parameter	Keine. Siehe Delegate_Void (Seite 213).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnRuntimeManagerLostCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 6-22 RegisterOnRuntimeManagerLostCallback() - Native C++

Syntax	<code>void RegisterOnRuntimeManagerLostCallback(EventCallback_VOID in_CallbackFunction);</code>
Parameter	EventCallback_VOID in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_VOID (Seite 205).
Rückgabewerte	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnRuntimeManagerLostEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Die Registrierung eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 6-23 RegisterOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>void RegisterOnRuntimeManagerLostEvent();</code> <code>void RegisterOnRuntimeManagerLostEvent(HANDLE* in_Event);</code>
Parameter	- None: Ein interner Event-Handle wird registriert. - HANDLE* in_Event: Ein anwenderspezifischer Event-Handle wird registriert.
Rückgabewerte	Keine

Tabelle 6-24 RegisterOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>void RegisterOnRuntimeManagerLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnRuntimeManagerLostCallback()

Meldet die Callback-Funktion ab. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 6-25 UnregisterOnRuntimeManagerLostCallback() - Native C++

Syntax	<code>void UnregisterOnRuntimeManagerLostCallback();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

UnregisterOnRuntimeManagerLostEvent()

Meldet das Event-Objekt ab.

Tabelle 6-26 UnregisterOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>void UnregisterOnRuntimeManagerLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 6-27 UnregisterOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>void UnregisterOnRuntimeManagerLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnRuntimeManagerLostEvent()

Die Funktion wird das Programm solange blockieren, bis das registrierte Event-Objekt in den signalisierten Zustand gesetzt ist oder bis das Timeout-Intervall überschritten wird.

Tabelle 6-28 WaitForOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>bool WaitForOnRuntimeManagerLostEvent();</code> <code>bool WaitForOnRuntimeManagerLostEvent(UINT32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Timeout-Intervalls kein Ereignis empfangen wurde.

Tabelle 6-29 WaitForOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>bool WaitForOnRuntimeManagerLostEvent();</code> <code>bool WaitForOnRuntimeManagerLostEvent(UInt32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UInt32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Timeout-Intervalls kein Ereignis empfangen wurde.

6.2.3 Ereignisse OnAutodiscoverData

OnAutodiscoverData

Meldet eine Event-Handler Methode an oder ab.

Tabelle 6-30 OnAutodiscoverData - .NET (C#)

Syntax	<code>event Delegate_SRRSI_AD OnAutodiscoverData</code>
Parameter	Keine. Siehe Delegate_SRRSI_AD (Seite 214).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnAutodiscoverCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 6-31 RegisterOnAutodiscoverCallback() - Native C++

Syntax	<code>void RegisterOnAutodiscoverCallback(EventCallback_SRRSI_AD in_CallbackFunction);</code>
Parameter	EventCallback_SRRSI_AD in_CallbackFunction: Zeiger auf eine anwenderdefinierte Callback-Funktion. Siehe EventCallback_SRRSI_AD (Seite 206).
Rückgabewerte	Keine

UnregisterOnAutodiscoverCallback()

Meldet die Callback-Funktion ab.

Tabelle 6-32 UnregisterOnAutodiscoverCallback() - Native C++

Syntax	<code>void UnregisterOnAutodiscoverCallback();</code>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

IsInitialized() / IsInitialized { get; }

Gibt einen Wert zurück, der anzeigt, ob die API erfolgreich initialisiert wurde.

Tabelle 6-33 IsInitialized() - Native C++

Syntax	<code>bool IsInitialized();</code>
Parameter	Keine
Rückgabewerte	<code>false</code>: Wenn die API nicht initialisiert wurde. <code>true</code>: Wenn die API initialisiert wurde.

Tabelle 6-34 IsInitialized { get; } - .NET (C#)

Syntax	<code>bool IsInitialized { get; }</code>
Parameter	Keine
Rückgabewerte	<code>false</code>: Wenn die API nicht initialisiert wurde. <code>true</code>: Wenn die API initialisiert wurde.

IsRuntimeManagerAvailable() / IsRuntimeManagerAvailable { get; }

Die Funktion gibt `false` zurück, wenn die Verbindung zum Runtime Manager unterbrochen ist. Dies passiert nur, wenn der Prozess des Runtime Managers geschlossen ist.

Abonnieren Sie das Ereignis `OnRuntimeManagerLost()`, um zu erfahren, ob die Verbindung unterbrochen ist. Siehe Ereignisse ([Seite 56](#)).

Tabelle 6-35 IsRuntimeManagerAvailable() - Native C++

Syntax	<code>bool IsRuntimeManagerAvailable();</code>
Parameter	Keine
Rückgabewerte	<code>false</code>: Wenn die Verbindung unterbrochen ist. <code>true</code>: Wenn die Verbindung aktiv ist.

Tabelle 6-36 IsRuntimeManagerAvailable { get; } - .NET (C#)

Syntax	<code>bool IsRuntimeManagerAvailable{ get; }</code>
Parameter	Keine
Rückgabewerte	<code>false</code>: Wenn die Verbindung unterbrochen ist. <code>true</code>: Wenn die Verbindung aktiv ist.

Shutdown()

Beendet die Kommunikation mit dem Runtime Manager und bereinigt die Schnittstellen.

Rufen Sie diese Funktion in folgenden Fällen auf:

- Unmittelbar bevor die API-Bibliothek (DLL) abgemeldet wird (Native C++).
- Wenn Ihre Anwendung den Runtime Manager nicht mehr nutzt.

Tabelle 6-37 Shutdown() - Native C++

Syntax	<code>ERuntimeErrorCode Shutdown();</code>	
Parameter	Keine	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Der Funktionsaufruf ist erfolgreich.

Tabelle 6-38 Shutdown() - .NET (C#)

Syntax	<code>void Shutdown()</code>
Parameter	Keine
Rückgabewerte	Keine

API Instance

7.1 Schnittstellen für Instances

Dispose()

Löscht die managed-Schnittstelle und entlädt die nativen Komponenten der Anwenderschnittstellen.

Tabelle 7-1 Dispose() - .NET (C#)

Syntax	<code>void Dispose()</code>
Parameter	Keine
Rückgabewerte	Keine

GetID() / ID { get; }

Gibt die Instanz-ID zurück. Die ID wird vom Runtime Manager zugewiesen, wenn die Instanz registriert wird.

Tabelle 7-2 GetID() - Native C++

Syntax	<code>INT32 GetID();</code>
Parameter	Keine
Rückgabewerte	INT32: Instanz-ID

Tabelle 7-3 ID { get; } - .NET (C#)

Syntax	<code>UInt32 ID { get; }</code>
Parameter	Keine
Rückgabewerte	UInt32: Instanz-ID
Ausnahmen	Keine

GetName() / Name { get; }

Gibt den Namen der Instanz zurück.

Tabelle 7-4 GetName() - Native C++

Syntax	<code>ERuntimeErrorCode GetName(WCHAR inout_Name[], UINT32 in_ArrayLength);</code>	
Parameter	<code>WCHAR inout_Name[]</code>: Ein vom Benutzer zugewiesener Speicher für den Namen der Instanz. Die Feldlänge soll mindestens so groß sein wie <code>DINSTANCE_NAME_MAX_LENGTH</code>. Siehe Definitionen und Konstanten (Seite 221). <code>UINT32 in_ArrayLength</code>: Feldlänge (Wide-Character)	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.

Rückgabewerte	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_ARGUMENT	Der Name passt nicht in den Speicher.
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } WCHAR name[DINSTANCE_NAME_MAX_LENGTH]; if (result == SREC_OK) { result = psa->GetName(name, DINSTANCE_NAME_MAX_LENGTH); }</pre>	

Tabelle 7-5 Name { get; } - .NET (C#)

Syntax	string Name { get; }	
Parameter	Keine	
Rückgabewerte	Name der Instanz.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.

GetCPUType()

Gibt den CPU-Typ der simulierten PLC-Instanz zurück.

Tabelle 7-6 GetCPUType() - Native C++

Syntax	ECPUType GetCPUType();
Parameter	Keine
Rückgabewerte	Ein Aufzählungselement, das den CPU-Typ definiert. Siehe ECPUType (Seite 244).

SetCPUType()

Legt den CPU-Typ der simulierten PLC-Instanz fest. Ein Wechsel des CPU-Typs erfolgt nur bei einem Neustart des Controllers.

Tabelle 7-7 SetCPUType() - Native C++

Syntax	void SetCPUType(ECPUType in_Value);
Parameter	ECPUType in_Value: Definiert, welcher CPU-Typ beim Start der Instanz simuliert wird. Wenn über STEP 7 oder von der Virtual Memory Card ein anderer CPU-Typ geladen wird, dann gilt dieser CPU-Typ.
Rückgabewerte	Keine

CPUType { get; set; }

Gibt den CPU-Typ der simulierten PLC-Instanz zurück oder legt ihn fest. Ein Wechsel des CPU-Typs erfolgt nur bei einem Neustart des Controllers.

Wenn über STEP 7 oder von der Virtual Memory Card ein anderer CPU-Typ geladen wird, dann gilt dieser CPU-Typ.

Tabelle 7-8 CPUType { get; set; } - .NET (C#)

Syntax	<code>ECPUType CPUType { get; set; }</code>
Parameter	Keine
Rückgabewerte	Ein Aufzählungselement, das den CPU-Typ definiert.
Ausnahmen	Keine

GetCommunicationInterface()

Die Eigenschaft ist nun schreibgeschützt. Informationen zum Einstellen des Netzwerkmodus finden Sie im Kapitel "NetworkMode".

Gibt die Kommunikationsschnittstelle des simulierten Controllers zurück: Lokale Kommunikation (Softbus) oder TCPIP.

Tabelle 7-9 GetCommunicationInterface() - Native C++

Syntax	<code>ECommunicationInterface GetCommunicationInterface();</code>
Parameter	Keine
Rückgabewerte	• <code>SRCI_NONE</code> Kann nicht ausgewählt werden. Wird zurückgegeben, wenn die Instanz-Schnittstelle nicht mehr gültig ist. • <code>SRCI_SOFTBUS</code> Wird zurückgegeben, wenn der simulierte Controller den Softbus nutzt. • <code>SRCI_TCPIP</code> Wird zurückgegeben, wenn der simulierte Controller über den virtuellen Adapter kommuniziert.

CommunicationInterface { get; }

Die Eigenschaft ist nun schreibgeschützt. Informationen zum Einstellen des Netzwerkmodus finden Sie im Kapitel [\(Seite 61\)](#) "NetworkMode".

Gibt die Kommunikationsschnittstelle des simulierten Controllers zurück: Lokale Kommunikation (Softbus) oder TCPIP.

Tabelle 7-10 CommunicationInterface { get; } - .NET (C#)

Syntax	<code>ECommunicationInterface CommunicationInterface { get; }</code>
Parameter	Keine
Rückgabewerte	• <code>ECommunicationInterface.None</code> Kann nicht ausgewählt werden. Wird zurückgegeben, wenn die Instanz-Schnittstelle nicht mehr gültig ist. • <code>ECommunicationInterface.Softbus</code> Wird zurückgegeben, wenn der simulierte Controller den Softbus nutzt. • <code>ECommunicationInterface.TCPIP</code> Wird zurückgegeben, wenn der simulierte Controller über den virtuellen Adapter kommuniziert.
Ausnahmen	Keine

GetInfo() / Info { get; }

Gibt eine Struktur zurück, die Informationen über die Instanz bereitstellt.

Tabelle 7-11 GetInfo() - Native C++

Syntax	<code>SInstanceInfo GetInfo();</code>
Parameter	Keine
Rückgabewerte	<code>SInstanceInfo</code> : Eine Struktur, die Informationen über die Instanz bereitstellt. Siehe <code>SInstanceInfo</code> (Seite 229).

Tabelle 7-12 Info { get; } - .NET (C#)

Syntax	<code>SInstanceInfo Info { get; }</code>
Parameter	Keine
Rückgabewerte	<code>SInstanceInfo</code> : Eine Struktur, die Informationen über die Instanz bereitstellt.
Ausnahmen	Keine

GetNetInterfaceMapping()

Gibt einen Schnittstellen-Index der PC-Schnittstelle zurück, auf die eine bestimmte CPU-Schnittstelle gemappt ist. Verwenden Sie die Funktion `GetNetInterfaces()`, um eine Liste von PC-Schnittstellen mit Schnittstellen-Index zu erhalten. Der Wert 0 bedeutet, dass keine Zuordnung (Mapping) eingestellt wurde.

Beachten Sie, dass `GetNetInterfaceMapping()` nur eine Mapping-Tabelle zurückgibt und nicht die eigentliche Adresse bei laufender Instanz.

Tabelle 7-13 GetNetInterfaceMapping() - Native C++ und .NET (C#)

Syntax C++	<code>INT32 GetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID);</code>	
Syntax .NET (C#)	<code>int GetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID);</code>	
Parameter	<code>EPLCInterface in_PLCInterfaceID</code> : ID der PLC-Schnittstelle, für die Sie das Mapping erhalten wollen.	
Rückgabewerte	Wert	Beschreibung
	Größer als 0	Index der gemappten (zugeordneten) Computer-Schnittstelle
	0	Für die PLC-Schnittstelle ist kein Mapping definiert oder der <code>NetworkMode</code> ist <code>Softbus</code> .
	-1	Das Mapping kann nicht abgerufen werden.

SetNetInterfaceMapping()

Setzt Schnittstellen-Mappings zwischen CPU-Schnittstelle und PC-Schnittstelle. Ethernet-Pakete, die eine bestimmte PC-Schnittstelle empfängt, werden an die zugeordnete CPU-Schnittstelle weitergeleitet. Verwenden Sie die Funktion `GetNetInterfaces()`, um eine Liste an PC-Schnittstellen zu erhalten.

Das Mapping von PLCSIM Virtual-Ethernet-Adaptern oder -Drahtlosadapttern ist nicht möglich.
Die Funktion ist keine gültige Funktion für den NetworkMode Softbus.

Tabelle 7-14 SetNetInterfaceMapping() - Native C++

Syntax	<pre> ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, DWORD in_PCInterfaceIndex); ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const WCHAR* in_PCInterfaceName); ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const SMACAddress* in_MACAddressParam); </pre>	
Parameter	EPLCInterface in_PLCInterfaceID: ID der CPU-Schnittstelle, für die eine MAC-Adresse gesetzt werden soll. DWORD* in_PCInterfaceIndex: Index der PC-Schnittstelle; Wenn dieser Wert 0 ist, wird die Zuordnung (Mapping) gelöscht. const WCHAR* in_PCInterfaceName: Name der PC-Schnittstelle const UMACAddress* in_MACAddressParam: Hardware-MAC-Adresse der PC-Schnittstelle in der Form FF-FF-FF-FF-FF-FF und mit "-" als Oktett-Trennzeichen. Sie erhalten diese Werte mit der Funktion GetNetInterfaces().	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Der Funktionsaufruf ist erfolgreich.
	SREC_WRONG_COMMUNICATION_INTERFACE	Der Fehler wird in den TCP/IP-Modi zurückgegeben, falls die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle deaktiviert ist oder falls der PLCSIM Virtual Adapter die Schnittstelle ist.
	SREC_ALREADY_EXISTS	Die PC-Schnittstelle wurde bereits einer anderen CPU-Schnittstelle zugeordnet
	SREC_CONFIG_FILE_ERROR	Bei Schreiben der Konfiguration in die Datei "sim_hwdb.ini" ist ein Fehler aufgetreten.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_INSTANCE_ALREADY_RUNNING	Bindings können nicht geändert werden, während die Instanz gerade läuft.
	SREC_NOT_SUPPORTED	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

Tabelle 7-15 SetNetInterfaceMapping() - .NET (C#)

Syntax	<pre> SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, uint in_PCInterfaceIndex); SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const String in_PCInterfaceName); SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const SMACAddress in_MACAddressParam); </pre>	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode. WrongCommunicationInterface	Der Fehler wird in den TCP/IP-Modi zurückgegeben, falls die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle deaktiviert ist oder falls der PLCSIM Virtual Adapter die Schnittstelle ist.
	ERuntimeErrorCode.AlreadyExists	Die PC-Schnittstelle wurde bereits einer anderen CPU-Schnittstelle zugeordnet
	ERuntimeErrorCode.ConfigFileError	Bei Schreiben der Konfiguration in die Datei "sim_hwdb.ini" ist ein Fehler aufgetreten.

Ausnahmen	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.InstanceAlreadyRunning</code>	Bindings können nicht geändert werden, während die Instanz gerade läuft.
	<code>ERuntimeErrorCode.NotSupported</code>	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

CheckNetInterfaceMapping()

Die Funktion prüft die Gültigkeit der Schnittstellenzuordnung (Interface-Mapping), um die Instanz starten zu können. Alle bereits laufenden PC-Instanzen werden auf mögliche doppelte Zuordnungen (Mappings) hin überprüft. Außerdem überprüft die Funktion, ob das Binding an die PC-Netzwerkschnittstellen gültig ist. Die Funktion ist keine gültige Funktion für den NetworkMode Softbus.

Tabelle 7-16 CheckNetInterfaceMapping() - Native C++

Syntax	<code>ERuntimeErrorCode CheckNetInterfaceMapping();</code>	
Parameter	Keine	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Der Funktionsaufruf ist erfolgreich.
	<code>SREC_INVALID_CONFIGURATION</code>	Die Konfiguration ist ungültig. Beispiele: - Die aktuelle Instanz hat eine zugeordnete PC-Netzwerkschnittstelle, die bereits von einer anderen laufenden Instanz verwendet wird. - Es existiert ein zugeordneter PLCSIM Virtual Ethernet Adapter.
	<code>SREC_VIRTUAL_SWITCH_MISCONFIGURED</code>	Das Binding eines S7-PLCSIM Advanced Virtual Switch auf einer PC-Netzwerkschnittstelle ist nicht korrekt oder der Treiber eines S7-PLCSIM Advanced Virtual Switch funktioniert nicht einwandfrei. Falls das Binding nicht korrekt ist, rufen Sie die Funktion <code>SetNetInterfaceBindings()</code> auf oder stellen Sie das Binding manuell ein, bevor Sie die Instanz starten.
	<code>SREC_WARNING_NO_EXTERNAL_COMMUNICATION</code>	Sie haben den Multi-Adapter Network Mode (Non-Promiscuous-Modus) ohne Schnittstellenzuordnung (Interface-Mapping) oder Softbus als Kommunikationsschnittstelle eingestellt.
	<code>SREC_INTERNAL_ERROR</code>	Der Systemaufruf hat einen internen Fehler verursacht.
	<code>SREC_WRONG_COMMUNICATION_INTERFACE</code>	Der Fehler wird in den TCP/IP-Modi zurückgegeben, falls die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle deaktiviert ist oder falls der PLCSIM Virtual Adapter die Schnittstelle ist.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_PCAP_DRIVER_NOT_RUNNING</code>	Der PCAP-Treiber wird nicht ausgeführt.
	<code>SREC_NOT_SUPPORTED</code>	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

Tabelle 7-17 CheckNetInterfaceMapping() - .NET (C#)

Syntax	<code>ERuntimeErrorCode CheckNetInterfaceMapping();</code>	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.OK</code>	Der Funktionsaufruf ist erfolgreich.
	<code>ERuntimeErrorCode.InvalidConfiguration</code>	Die Konfiguration ist ungültig. Beispiele: - Die aktuelle Instanz hat eine zugeordnete PC-Netzwerkschnittstelle, die bereits von einer anderen laufenden Instanz verwendet wird. - Es existiert ein zugeordneter PLCSIM Virtual Ethernet Adapter.
	<code>ERuntimeErrorCode.VirtualSwitchMisconfigured</code>	Das Binding eines S7-PLCSIM Advanced Virtual Switch auf einer PC-Netzwerkschnittstelle ist nicht korrekt oder der Treiber eines S7-PLCSIM Advanced Virtual Switch funktioniert nicht einwandfrei. Falls das Binding nicht korrekt ist, rufen Sie die Funktion <code>SetNetInterfaceBindings()</code> auf oder stellen Sie das Binding manuell ein, bevor Sie die Instanz starten.
	<code>ERuntimeErrorCode.WarningNoExternalCommunication</code>	Sie haben den Multi-Adapter Network Mode (Non-Promiscuous-Modus) ohne Schnittstellenzuordnung (Interface-Mapping) oder Softbus als Kommunikationschnittstelle eingestellt.
	<code>ERuntimeErrorCode.InternalError</code>	Der Systemaufruf hat einen internen Fehler verursacht.
	<code>ERuntimeErrorCode.WrongCommunicationInterface</code>	Der Fehler wird in den TCP/IP-Modi zurückgegeben, falls die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle deaktiviert ist oder falls der PLCSIM Virtual Adapter die Schnittstelle ist.
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.PCAPDriverNotRunning</code>	Der PCAP-Treiber wird nicht ausgeführt.
	<code>ERuntimeErrorCode.NotSupported</code>	Sie können die Funktion im NetworkMode "Softbus" nicht aufrufen.

UnregisterInstance()

Meldet diese Instanz vom Runtime Manager ab.

HINWEIS**Verlust der Schnittstellen**

Andere Anwendungen, die mit dieser Instanz verbunden sind, werden ihre Schnittstelle zu dieser Instanz verlieren.

HINWEIS

Heben Sie die Registrierung einer Instanz nicht auf und geben Sie die API nicht frei, während entweder das Bedienpanel oder die API eine Instanz startet. Warten Sie, bis die Aktion PowerOn für diese Instanz abgeschlossen ist.

Tabelle 7-18 UnregisterInstance() - Native C++

Syntax	ERuntimeErrorCode UnregisterInstance();	
Parameter	Keine	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-19 UnregisterInstance() - .NET (C#)

Syntax	void UnregisterInstance();	
Parameter	Keine	
Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

7.2 Controller - Informationen und Einstellungen

GetControllerName() / ControllerName { get; }

Gibt den heruntergeladenen Namen der simulierten PLC-Instanz zurück.

Tabelle 7-20 GetControllerName() - Native C++

Syntax	ERuntimeErrorCode GetControllerName(WCHAR inout_Name[], UINT32 in_ArrayLength);	
Parameter	- WCHAR inout_Name[]: Ein vom Benutzer zugewiesener Speicher für den Namen. - UINT32 in_ArrayLength: Die Länge des Speichers.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Der Name passt nicht in den Speicher.

Tabelle 7-21 ControllerName { get; } - .NET (C#)

Syntax	string ControllerName { get; }	
Parameter	Keine	
Rückgabewerte	string: Der heruntergeladene Name der simulierten PLC-Instanz.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

GetControllerShortDesignation() / ControllerShortDesignation { get; }

Gibt die heruntergeladene Kurzbezeichnung der simulierten PLC-Instanz zurück.

Tabelle 7-22 GetControllerShortDesignation() - Native C++

Syntax	ERuntimeErrorCode GetControllerShortDesignation(WCHAR inout_ShortDesignation[], UINT32 in_ArrayLength);	
Parameter	- WCHAR inout_ShortDesignation[]: Ein vom Benutzer zugewiesener Speicher für die Kurzbezeichnung. - UINT32 in_ArrayLength: Die Länge des Speichers.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Der Name passt nicht in den Speicher.

Tabelle 7-23 ControllerShortDesignation { get; } - .NET (C#)

Syntax	string ControllerShortDesignation { get; }	
Parameter	Keine	
Rückgabewerte	string: Die heruntergeladene Kurzbezeichnung der simulierten PLC-Instanz.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

GetControllerIPCount()

Gibt die Anzahl der konfigurierten IP-Adressen der simulierten PLC-Instanz zurück. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.

Tabelle 7-24 GetControllerIPCount() - Native C++

Syntax	UINT32 GetControllerIPCount();
Parameter	Keine
Rückgabewerte	INT32: Anzahl der konfigurierten IP-Adressen der simulierten PLC-Instanz.

GetControllerIP() / ControllerIP { get; }

Gibt eine konfigurierte IP-Adresse der Instanz zurück.

Tabelle 7-25 GetControllerIP() - Native C++

Syntax	UIP GetControllerIP(); UIP GetControllerIP(UINT32 in_Index);	
Parameter	WCHAR in_Index: Der Index der IP-Adresse, die Sie erhalten möchten. Der Index muss kleiner sein als der Wert, den Sie von GetControllerIPCount() erhalten. Die Voreinstellung ist 0.	
Rückgabewerte	UIP: IP-Adresse der simulierten PLC-Instanz. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.	

Tabelle 7-26 ControllerIP { get; } - .NET (C#)

Syntax	<code>string[] ControllerIP { get; }</code>
Parameter	Keine
Rückgabewerte	<code>string</code> : Alle heruntergeladenen IP-Adressen der simulierten PLC-Instanz. Wenn die Funktion fehlschlägt, ist das Feld leer.
Ausnahmen	Keine

GetControllerIPSuite4() / ControllerIPSuite4 { get; }

Gibt die IP-Suite der Instanz zurück. Wenn die Kommunikationsschnittstelle "Softbus" genutzt wird, sind Subnetzmaske und Standard-Gateway 0.

Tabelle 7-27 GetControllerIPSuite4() Native C++

Syntax	<code>SIPSuite4 GetControllerIPSuite4();</code> <code>SIPSuite4 GetControllerIPSuite4(UINT32 in_Index);</code>
Parameter	<code>WCHAR in_Index</code>: Der Index der IP-Adresse, die Sie erhalten möchten. Der Index muss kleiner sein als der Wert, den Sie von <code>GetControllerIPCount()</code> erhalten. Die Voreinstellung ist 0.
Rückgabewerte	<code>SIPSuite4</code> : Die IP-Suite der simulierten PLC-Instanz. Wenn die Funktion fehlschlägt, sind die Rückgabewerte 0.

Tabelle 7-28 ControllerIPSuite4 { get; } - .NET (#)

Syntax	<code>SIPSuite4[] ControllerIPSuite4 { get; };</code>
Parameter	Keine
Rückgabewerte	<code>SIPSuite4[]</code> : Alle heruntergeladenen IP-Suiten der simulierten PLC-Instanz. Wenn die Funktion fehlschlägt, ist das Feld leer.
Ausnahmen	Keine

SetIPSuite()

Legt die IP-Suite der Netzwerkschnittstelle einer simulierten PLC-Instanz fest.

Tabelle 7-29 SetIPSuite() - Native C++

Syntax	<code>ERuntimeErrorCode SetIPSuite(UINT32 in_InterfaceID, SIPSuite4 in_IPSuite, bool in_IsRemanent);</code>	
Parameter	<code>UINT32 in_InterfaceID</code>: Die ID der Netzwerk-Schnittstelle. <code>SIPSuite4 in_IPSuite</code>: Die IP-Suite, die der Netzwerk-Schnittstelle zugewiesen werden soll. Die IP-Suite enthält die IP-Adresse, die Subnetzmaske und das Standard-Gateway. Wenn die Kommunikationsschnittstelle "Softbus" ist, werden Subnetzmaske und Standard-Gateway ignoriert. <code>bool in_IsRemanent</code>: Wenn <code>true</code>, dann wird die IP-Suite nach dem Neustart der simulierten PLC-Instanz gespeichert. Wenn die Kommunikationsschnittstelle "Softbus" ist, dann wird dieses Flag ignoriert.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.2 Controller - Informationen und Einstellungen

Rückgabewerte	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_DOES_NOT_EXIST	Es gibt keine Netzwerk-Schnittstelle mit dieser ID.
	SREC_INVALID_OPERATING_STATE	Die simulierte PLC-Instanz hat den Boot-Prozess noch nicht beendet oder befindet sich bereits in der Shutdown-Phase.

Tabelle 7-30 SetIPSuite() - .NET (C#)

Syntax	void SetIPSuite(UInt32 in_InterfaceID, SIPSuite4 in_IPSuite, bool in_IsRemanent);	
Parameter	- UInt32 in_InterfaceID: Die ID der Netzwerk-Schnittstelle. - SIPSuite4 in_IPSuite: Wenn die Kommunikationsschnittstelle "Softbus" ist, werden Subnetzmaske und Standard-Gateway ignoriert. - bool in_IsRemanent: Wenn true, dann wird die IP-Suite nach dem Neustart der simulierten PLC-Instanz gespeichert. Wenn die Kommunikationsschnittstelle "Softbus" ist, dann wird dieses Flag ignoriert.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.DoesNotExist	Es gibt keine Netzwerk-Schnittstelle mit dieser ID.
	ERuntimeErrorCode.InvalidOperatingState	Die simulierte PLC-Instanz hat den Boot-Prozess noch nicht beendet oder befindet sich bereits in der Shutdown-Phase.

GetStoragePath()

Gibt den vollständigen Pfad des Ordners zurück, in dem die Instanz ihre Daten speichert.

Tabelle 7-31 GetStoragePath() - Native C++

Syntax	ERuntimeErrorCode GetStoragePath(WCHAR inout_StoragePath[], UINT32 in_ArrayLength);	
Parameter	- WCHAR inout_StoragePath[]: Ein vom Benutzer zugewiesener Speicher für den Speicherpfad. Die Länge des Arrays soll mindestens so groß sein wie DSTORAGE_PATH_MAX_LENGTH. Siehe Datentypen (Seite 203). - UINT32 in_ArrayLength: Länge des Arrays (Wide-Character)	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Der Pfad passt nicht in den Speicher.

SetStoragePath()

Setzt den vollständigen Pfad des Ordners, in dem die Instanz ihre Daten speichert. Dies kann auch eine Netzwerkfreigabe sein.

Setzen Sie den Pfad, bevor Sie die Instanz starten. Eine Änderung des Pfads wirkt sich erst bei einem Neustart des Controllers aus.

Wenn kein Pfad gesetzt ist, gilt die Voreinstellung:

<Eigene Dokumente>\Siemens\Simatic\Simulation\Runtime\Persistence\<Instance Name>.

Tabelle 7-32 SetStoragePath() - Native C++

Syntax	ERuntimeErrorCode SetStoragePath(WCHAR* in_StoragePath);	
Parameter	WCHAR* in_StoragePath: Vollständiger Name des Speicherpfads. Die Länge des Namens muss kürzer sein als <code>DSTORAGE_PATH_MAX_LENGTH</code> und darf keine Unicode-Zeichen enthalten. Siehe Datentypen (Seite 203).	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INDEX_OUT_OF_RANGE	Die Länge des Pfads überschreitet das Limit.
	SREC_WRONG_ARGUMENT	Der Pfad enthält ungültige Zeichen.

StoragePath { get; set; }

Gibt den vollständigen Pfad des Ordners zurück oder setzt diesen Pfad, in dem die Instanz ihre remanenten Daten speichert. Dies kann auch eine Netzwerkfreigabe sein.

Setzen Sie den Pfad, bevor Sie die Instanz starten. Eine Änderung des Pfads wirkt sich erst bei einem Neustart des Controllers aus.

Wenn kein Pfad gesetzt ist, gilt die Voreinstellung:

<Eigene Dokumente>\Siemens\Simatic\Simulation\Runtime\Persistence\<Instance Name>.

Tabelle 7-33 StoragePath { get; set; } - .NET (C#)

Syntax	string StoragePath { get; set; }	
Parameter	Keine	
Rückgabewerte	string: Der konfigurierte Speicherpfad.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.IndexOutOfRangeException	Die Länge des Pfads überschreitet das Limit.
	ERuntimeErrorCode.WrongArgument	Der Pfad enthält ungültige Zeichen.

ArchiveStorage()

Das Anwenderprogramm, die Hardware-Konfiguration und die remanenten Daten werden in einer Datei gespeichert. Der Virtual SIMATIC Memory Card. `ArchiveStorage()` speichert diese Datei als ZIP-Datei. Die simulierte PLC-Instanz muss dazu im Betriebszustand OFF sein.

Um Fehler zu vermeiden, müssen Sie die Funktion `PowerOn()` mindestens einmal aufrufen.

Tabelle 7-34 ArchiveStorage() - Native C++

Syntax	ERuntimeErrorCode ArchiveStorage(WCHAR* in_FullFileName);	
Parameter	WCHAR in_FullFileName: Der vollständige Pfad zur ZIP-Datei. Der Pfad bezieht sich auf die Ordner auf dem Computer, auf dem die API aufgerufen wird.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INVALID_OPERATING_STATE	Die Instanz ist nicht im Betriebszustand OFF.
	SREC_INVALID_ARCHIVE_PATH	Der Archiv-Pfad ist ungültig.
	SREC_CREATE_DIRECTORIES_FAILED	Der Ordner für die ZIP-Datei konnte nicht erstellt werden.
	SREC_ARCHIVE_STORAGE_FAILED	Die ZIP-Datei konnte nicht erstellt werden.
	SREC_STORAGE_TRANSFER_ERROR	Fehler beim Netzwerk-Datentransfer. Speicherdaten zwischen Client- und Server-Computern stimmen nicht überein.
	SREC_NO_STORAGE_PATH_SET	Es ist kein Speicherpfad gesetzt. Sie müssen zuvor die Funktion SetStoragePfad() ausführen.
	SREC_ARCHIVE_STORAGE_NOT_CREATED	Die Speicherdatei konnte nicht erstellt werden. Um Fehler zu vermeiden, müssen Sie die Funktion PowerOn() mindestens einmal aufrufen.

Tabelle 7-35 ArchiveStorage() - .NET (C#)

Syntax	void ArchiveStorage(string in_FullFileName);	
Parameter	string in_FullFileName: Der vollständige Pfad zur ZIP-Datei. Der Pfad bezieht sich auf Ordner des Computers, auf dem die API aufgerufen wird.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InvalidOperatingState	Die Instanz ist nicht im Betriebszustand OFF.
	ERuntimeErrorCode.InvalidArchivePath	Der Archiv-Pfad ist ungültig.
	ERuntimeErrorCode.CreateDirectoriesFailed	Der Ordner für die ZIP-Datei konnte nicht erstellt werden.
	ERuntimeErrorCode.ArchiveStorageNotCreated	Die ZIP-Datei konnte nicht erstellt werden.
	ERuntimeErrorCode.StorageTransferError	Fehler beim Netzwerk-Datentransfer. Speicherdaten zwischen Client- und Server-Computern stimmen nicht überein.

RetrieveStorage()

RetrieveStorage() stellt aus der archivierten ZIP-Datei wieder eine Virtual SIMATIC Memory Card her. Die simulierte PLC-Instanz muss dazu im Betriebszustand OFF sein.

Tabelle 7-36 RetrieveStorage() - Native C++

Syntax	ERuntimeErrorCode RetrieveStorage(WCHAR* in_FullFileName);	
Parameter	WCHAR* in_FullFileName: Der vollständige Pfad zur ZIP-Datei. Der Pfad bezieht sich auf Ordner des Computers, auf dem die API aufgerufen wird.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INVALID_OPERATING_STATE	Die Instanz ist nicht im Betriebszustand OFF.
	SREC_INVALID_ARCHIVE_PATH	Der Archiv-Pfad ist ungültig.
	SREC_DELETE_EXISTING_STORAGE_FAILED	Der alte Speicher kann nicht gelöscht werden.
	SREC_RETRIEVE_STORAGE_FAILURE	Das ZIP-Datei kann nicht entpackt werden.
	SREC_STORAGE_TRANSFER_ERROR	Fehler beim Netzwerk-Datentransfer. Speicherdaten zwischen Client- und Server-Computern stimmen nicht überein.
	SREC_NO_STORAGE_PATH_SET	Es ist kein Speicherpfad gesetzt. Der Anwender muss die Funktion SetStoragePfad() vorher ausführen.

Tabelle 7-37 RetrieveStorage() - .NET (C#)

Syntax	void RetrieveStorage(string in_FullFileName);	
Parameter	string in_FullFileName: Der vollständige Pfad zur ZIP-Datei. Der Pfad bezieht sich auf Ordner des Computers, auf dem die API aufgerufen wird.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InvalidOperatingState	Die Instanz ist nicht im Betriebszustand OFF.
	ERuntimeErrorCode.InvalidArchivePath	Der Archiv-Pfad ist ungültig.
	ERuntimeErrorCode.DeleteExistingStorageFailed	Der alte Speicher kann nicht gelöscht werden.
	ERuntimeErrorCode.RetrieveStorageFailure	Das ZIP-Datei kann nicht entpackt werden.
	ERuntimeErrorCode.StorageTransferError	Fehler beim Netzwerk-Datentransfer. Speicherdaten zwischen Client- und Server-Computern stimmen nicht überein.

HINWEIS
Bearbeitung einer mit S7-PLCSIM Advanced erzeugten Instanz mit Hilfe von S7-PLCSIM Advanced API V3.0 mit den Funktionen ArchiveStorage() und RetrieveStorage()

Wenn Sie eine mit S7-PLCSIM Advanced V4.0 oder höher erzeugte Instanz mit den API-Funktionen ArchiveStorage() und RetrieveStorage() der V3.0 bearbeiten, dann wird mit der API V3.0 nicht der gesamte Karteninhalt der virtuellen Speicherkarte gespeichert. Die Dateien retain.pms und sim_hwdb.ini fehlen.

Das bedeutet, dass das Passwort zum Schutz vertraulicher Konfigurationsdaten nicht gespeichert wird und verloren geht.

Abhilfe:

Sie haben mehrere Optionen:

- Das Passwort zum Schutz der vertraulichen Konfigurationsdaten vor jeder Simulation erneut setzen, und zwar mit Hilfe von STEP 7, des SIMATIC Automation Tools oder TIA Portal Openness.
- Eine Remote-Verbindung der Instanz über den Runtime Manager herstellen, auch wenn die Instanz lokal kommuniziert. In diesem Fall wird das Passwort auf der virtuellen Speicherkarte gesichert.
- Kein Passwort zum Schutz vertraulicher Konfigurationsdaten vergeben.

CleanupStoragePath()

Die Funktion löscht den Ordner mit der Virtual SIMATIC Memory Card einer lokalen Instanz oder einer Remote-Instanz. Dazu prüft die Funktion, ob erforderliche und unzulässige Dateien vorhanden sind. Auch wenn der Ordner fehlt, gilt die Funktion als erfolgreich.

Um sicherzustellen, dass der korrekte Ordner gelöscht wird, prüft die Funktion, ob die Dateien vorhanden sind, die in der Virtual SIMATIC Memory Card vorhanden sein müssen:

- ".\sim_hwdb.ini"
- ".\SIMATIC_MC\SIMATIC.S7S"
- ".\SIMATIC_MC\RDdata"

Um den Ordner endgültig zu löschen, sind außerdem nur die folgenden Ordner mit Dateien erlaubt:

- ".\CrashDump"
- ".\Traces"
- ".\retain.pms"

Die Instanz muss im Betriebszustand OFF sein ("PowerOff").

Tabelle 7-38 CleanupStoragePath() - Native C++

Syntax	ERuntimeErrorCode CleanupStoragePath();	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INVALID_OPERATING_STATE	Die Instanz ist nicht im Betriebszustand OFF.
	SREC_DELETE_EXISTING_STORAGE_FAILED	Der Ordner mit dem Speicher kann nicht gelöscht werden.
	SREC_INVALID_STORAGE	Der Speicher ist ungültig. Er enthält Dateien oder Ordner, die nicht erlaubt sind.

Tabelle 7-39 CleanupStoragePath() - .NET (C#)

Syntax	void CleanupStoragePath();	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

Ausnahmen	ERuntimeErrorCode.InvalidOperatingState	Die Instanz ist nicht im Betriebszustand OFF.
	ERuntimeErrorCode.DeleteExistingStorageFailed	Der Ordner mit dem Speicher kann nicht gelöscht werden.
	SREC_INVALID_STORAGE	Der Speicher ist ungültig. Er enthält Dateien oder Ordner, die nicht erlaubt sind.

GetStrictMotionTiming() / StrictMotionTiming { get; }

Gibt die aktuelle Instanzeinstellung für das "Strict Motion Timing"-Feature zurück. Das Feature "Strict Motion Timing" befindet sich auf dem Control Panel von S7-PLCSIM Advanced.

Tabelle 7-40 GetStrictMotionTiming() - Native C++

Syntax	ERuntimeErrorCode GetStrictMotionTiming (bool* enabled);	
Parameter	bool* enabled: Empfängt die aktuelle Einstellung. true: Aktiv false: Inaktiv	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-41 StrictMotionTiming { get; } - .NET (C#)

Syntax	bool StrictMotionTiming { get; }	
Parameter	Keine	
Rückgabewerte	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

SetStrictMotionTiming() / StrictMotionTiming { set; }

Setzt die Instanzeinstellung für das "Strict Motion Timing"-Feature. Das Feature "Strict Motion Timing" befindet sich auf dem Control Panel von S7-PLCSIM Advanced.

Tabelle 7-42 SetStrictMotionTiming() - Native C++

Syntax	ERuntimeErrorCode GetStrictMotionTiming (bool* enabled);	
Parameter	bool enabled: Der einzustellende Wert. true: Aktiv false: Inaktiv	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INVALID_OPERATING_STATE	Die Instanz ist in Power On. Die Instanz darf nicht in Power On sein, um die Einstellung zu ändern.

Tabelle 7-43 StrictMotionTiming { set; } - .NET (C#)

Syntax	bool StrictMotionTiming { set; }	
Parameter	Keine	

Rückgabewerte	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InvalidOperatingState	Die Instanz ist in Power On. Die Instanz darf nicht in Power On sein, um die Einstellung zu ändern.

7.3 Betriebszustand

PowerOn()

Die Funktion erzeugt den Prozess für die Simulation Runtime-Instanz und startet die Firmware der simulierten PLC-Instanz.

Ohne einen Timeout-Wert ist der Zustand PowerOn() erreicht:

1. Sobald eine Instanz angelegt wurde
2. Bevor das Anwenderprogramm in der CPU gestartet wurde

Tabelle 7-44 PowerOn() - Native C++

Syntax	<pre>ERuntimeErrorCode PowerOn(); ERuntimeErrorCode PowerOn(UINT32 in_Timeout_ms);</pre>	
Parameter	• UInt32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion wieder, sobald eine Instanz angelegt und bevor das Anwenderprogramm in der CPU gestartet wurde. Abonnieren Sie das Ereignis OnOperatingStateChanged(), um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 8500 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. – Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: { SROS_STOP oder SROS_RUN, SROS_FREEZE, SROS_HOLD }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	SREC_ERROR_LOADING_DLL	Die "Siemens.Simatic.Simulation.Runtime.Instance.exe" kann die "Siemens.Simatic.Plc-Sim.Vplc1500.dll" nicht laden.
	SREC_STORAGE_PATH_ALREADY_IN_USE	Der ausgewählte Pfad für diese Instanz wird bereits von einer anderen Instanz genutzt.
	SREC_NO_STORAGE_PATH_SET	Es ist kein Speicherpfad gesetzt. Der Anwender muss die Funktion SetStoragePfad() vorher ausführen.
	SREC_WARNING_ALREADY_EXISTS	Warnung: Die Instanz ist gestartet.
	SREC_VIRTUAL_SWITCH_MISCONFIGURED	• Der S7-PLCSIM Advanced Virtual Switch ist falsch konfiguriert.

Rückgabewerte		Das Binding eines S7-PLCSIM Advanced Virtual Switch auf einer PC-Netzwerkschnittstelle ist nicht korrekt oder der Treiber eines S7-PLCSIM Advanced Virtual Switch funktioniert nicht einwandfrei. Falls das Binding nicht korrekt ist, rufen Sie die Funktion <code>SetNetInterfaceBindings()</code> auf oder stellen Sie das Binding manuell ein, bevor Sie die Instanz starten.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht mehr.
	SREC_UNSUPPORTED_PCAP_DRIVER	Der verwendete PCAP-Treiber wird nicht unterstützt. S7-PLCSIM Advanced unterstützt Npcap ab V0.9995. Die TCP/IP-Kommunikation ist nicht möglich.
	SREC_WARNING_TRIAL_MODE_ACTIVE	Warnung: Keine Lizenz verfügbar. Sie können die Instanz ohne Einschränkung mit der Trial License nutzen. Danach wird die Instanz abgeschaltet.
	SREC_WARNING_RUNNING_ON_TIA_PORTAL_TEST_SUITE	Warnung: Es ist keine gültige Lizenz vorhanden, jedoch eine "TIA Portal Test Suite"-Lizenz. S7-PLCSIM Advanced startet mit dieser Lizenz. Ein Download aus dem TIA Portal ist möglich, aber die Instanz wird ohne Feedback beendet, wenn der Download nicht über die TIA Portal Test Suite vorgenommen wurde.
	SREC_NOT_EMPTY	Warnung: Es ist keine gültige Lizenz für S7-PLCSIM Advanced vorhanden, jedoch eine "TIA Portal Test Suite"-Lizenz. Ist dies der Fall, wird der Anlauf über die Virtual SIMATIC Memory Card nicht unterstützt.
	SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE	Bei lokaler Kommunikation über Softbus S7-PLCSIM Advanced kann sich nicht mit dem Softbus verbinden. Abhilfe - Versuchen Sie erneut, die Verbindung herzustellen. - Beenden Sie S7-PLCSIM Advanced und das TIA Portal und starten Sie die Anwendungen erneut. - Starten Sie den PC erneut. - Reparieren Sie die Installation von S7-PLCSIM Advanced. Bei TCP/IP-Kommunikation Auf Ihrem PC ist noch eine Anwendung mit dem Softbus verbunden. Abhilfe - Beenden Sie alle SIMATIC Anwendungen, z. B. TIA Portal, WinCC, PLCSIM. - Starten Sie den PC erneut. - Reparieren Sie die Installation von S7-PLCSIM Advanced.
	SREC_ACCESS_DENIED	Keine Schreibrechte im Storage Directory.

Rückgabewerte	SREC_PCAP_DRIVER_NOT_RUNNING	Der Nmap Packet Capture Driver (Npcap) ist auf dem System nicht aktiv. Abhilfe 1. Starten Sie die Kommandozeile im Administrator-Modus. 2. Führen Sie den Befehl "net start npcap" aus.
	SREC_WRONG_COMMUNICATION_INTERFACE	- Sie haben versucht, die zweite Instanz mit der Kommunikationsschnittstelle zu versorgen, die sich von der bereits eingeschalteten ersten Instanz unterscheidet. Beispiel: - PowerOn() der ersten Instanz mit Softbus ist erfolgreich. - PowerOn() der zweiten Instanz mit TCP/IP; der Fehlercode wird zurückgegeben. - Die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle ist deaktiviert oder die Schnittstelle ist der PLCSIM Virtual Adapter.
	SREC_WARNING_PASSWORD_PROTECTION_ERROR	Wegen des Passwortschutzes war es nicht möglich, den Speicher zu öffnen. Sehen Sie sich die Einschränkungen bei aktiviertem Passwortschutz an.
	SREC_INVALID_CONFIGURATION	Die Konfiguration ist ungültig, weil z. B. die aktuelle Instanz eine zugeordnete (gemappte) PC-Netzwerkschnittstelle hat, die bereits von einer anderen laufenden Instanz verwendet wird.
	SREC_WARNING_NO_EXTERNAL_COMMUNICATION	Sie haben den Multi-Adapter Network Mode (Non-Promiscuous-Modus) ohne Schnittstellenzuordnung (Interface-Mapping) oder Softbus als Kommunikationsschnittstelle eingestellt.
	SREC_INTERNAL_ERROR	Der Systemaufruf hat einen internen Fehler verursacht.
	SREC_LICENSE_NOT_FOUND	Dieser Fehler tritt unter einer der folgenden Bedingungen auf: - Im Automation License Manager ist keine gültige Lizenz verfügbar. - Der Automation License Manager ist beschädigt.

Tabelle 7-45 PowerOn() - .NET (C#)

Syntax	<pre>ERuntimeErrorCode PowerOn(); ERuntimeErrorCode PowerOn(UInt32 in_Timeout_ms);</pre>	
Parameter	- UInt32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. - Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion wieder, sobald eine Instanz angelegt und bevor das Anwenderprogramm in der CPU gestartet wurde. Abonnieren Sie das Ereignis OnOperatingStateChanged(), um zu erfahren, wann die Operation durchgeführt wurde. - Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 8500 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. - Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: { EOperatingState.Run oder EOperatingState.Stop, EOperatingState.Freeze, EOperatingState.Hold }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	ERuntimeErrorCode.OK	Die Funktion ist erfolgreich.
	ERuntimeErrorCode.WarningAlreadyExists	Warnung: Die Instanz ist gestartet.
	ERuntimeErrorCode.UnsupportedPcapDriver	Der verwendete PCAP-Treiber wird nicht unterstützt. S7-PLCSIM Advanced unterstützt Npcap ab V0.9995. Die TCP/IP-Kommunikation ist nicht möglich.
	ERuntimeErrorCode.WarningTrialModeActive	Warnung: Keine Lizenz verfügbar. Sie können die Instanz ohne Einschränkung mit der Trial License nutzen. Danach wird die Instanz abgeschaltet.
	ERuntimeErrorCode.WarningRunningOnTiaPortalTestSuite	Warnung: Es ist keine gültige Lizenz vorhanden, jedoch eine "TIA Portal Test Suite"-Lizenz. S7-PLCSIM Advanced startet mit dieser Lizenz. Ein Download aus dem TIA Portal ist möglich, aber die Instanz wird ohne Feedback beendet, wenn der Download nicht über die TIA Portal Test Suite vorgenommen wurde.
	ERuntimeErrorCode.NotEmpty	Warnung: Es ist keine gültige Lizenz für S7-PLCSIM Advanced vorhanden, jedoch eine "TIA Portal Test Suite"-Lizenz. Ist dies der Fall, wird der Anlauf über die Virtual SIMATIC Memory Card nicht unterstützt.
	ERuntimeErrorCode.CommunicationInterfaceNotAvailable	Bei lokaler Kommunikation über Softbus S7-PLCSIM Advanced kann sich nicht mit dem Softbus verbinden. Abhilfe • Versuchen Sie erneut, die Verbindung herzustellen. • Beenden Sie S7-PLCSIM Advanced und das TIA Portal und starten Sie die Anwendungen erneut. • Starten Sie den PC erneut. • Reparieren Sie die Installation von S7-PLCSIM Advanced. Bei TCP/IP-Kommunikation Auf Ihrem PC ist noch eine Anwendung mit dem Softbus verbunden. Abhilfe • Beenden Sie alle SIMATIC Anwendungen, z. B. TIA Portal, WinCC, PLCSIM. • Starten Sie den PC erneut. • Reparieren Sie die Installation von S7-PLCSIM Advanced.
	ERuntimeErrorCode.AccessDenied	Keine Schreibrechte im Storage Directory.
	ERuntimeErrorCode.PCAPDriverNotRunning	Der Nmap Packet Capture Driver (Npcap) ist auf dem System nicht aktiv. Abhilfe 1. Starten Sie die Kommandozeile im Administrator-Modus. 2. Führen Sie den Befehl "net start npcap" aus.

Rückgabewerte	<code>ERuntimeErrorCode.WrongCommunicationInterface</code>	- Sie haben versucht, die zweite Instanz mit der Kommunikationsschnittstelle zu versorgen, die sich von der bereits eingeschalteten ersten Instanz unterscheidet. - Beispiel: <code>PowerOn()</code> der ersten Instanz mit Softbus ist erfolgreich. <code>PowerOn()</code> der zweiten Instanz mit TCP/IP; der Fehlercode wird zurückgegeben. - Die in der Datei "sim_hwdb.ini" angegebene/zugeordnete Computerschnittstelle ist deaktiviert oder die Schnittstelle ist der PLCSIM Virtual Adapter.
	<code>ERuntimeErrorCode.WarningPasswordForProtectionError</code>	Wegen des Passwortschutzes war es nicht möglich, den Speicher zu öffnen. Sehen Sie sich die Einschränkungen bei aktiviertem Passwortschutz an.
	<code>ERuntimeErrorCode.WarningNoExternalCommunication</code>	Sie haben den Multi-Adapter Network Mode (Non-Promiscuous-Modus) ohne Schnittstellenzuordnung (Interface-Mapping) oder Softbus als Kommunikationsschnittstelle eingestellt.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>ERuntimeErrorCode.ErrorLoadingDll</code>	Die "Siemens.Simatic.Simulation.Runtime.Instance.exe" kann die "Siemens.Simatic.Plc-Sim.Vplc1500.dll" nicht laden.
	<code>ERuntimeErrorCode.StoragePathAlreadyInUse</code>	Der ausgewählte Pfad für diese Instanz wird bereits von einer anderen Instanz genutzt.
	<code>ERuntimeErrorCode.NoStoragePathSet</code>	Der Pfad konnte nicht erstellt werden. Eventuell wird die Länge der <code>DSTORAGE_PATH_MAX_LENGTH</code> Zeichen überschritten.
	<code>ERuntimeErrorCode.VirtualSwitchMisconfigured</code>	- Der S7-PLCSIM Advanced Virtual Switch ist falsch konfiguriert. - Das Binding eines S7-PLCSIM Advanced Virtual Switch auf einer PC-Netzwerkschnittstelle ist nicht korrekt oder der Treiber eines S7-PLCSIM Advanced Virtual Switch funktioniert nicht einwandfrei. Falls das Binding nicht korrekt ist, rufen Sie die Funktion <code>SetNetInterfaceBindings()</code> auf oder stellen Sie das Binding manuell ein, bevor Sie die Instanz starten.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht mehr.

Ausnahmen	<code>ERuntimeErrorCode.InvalidConfiguration</code>	Die Konfiguration ist ungültig, weil z. B. die aktuelle Instanz eine zugeordnete (gemappte) PC-Netzwerkschnittstelle hat, die bereits von einer anderen laufenden Instanz verwendet wird.
	<code>ERuntimeErrorCode.InternalError</code>	Der Systemaufruf hat einen internen Fehler verursacht.
	<code>ERuntimeErrorCode.LicenseNotFound</code>	Dieser Fehler tritt unter einer der folgenden Bedingungen auf: - Im Automation License Manager ist keine gültige Lizenz verfügbar. - Der Automation License Manager ist beschädigt.

PowerOff()

Führt die Simulation Runtime herunter und schließt deren Prozess.

Tabelle 7-46 PowerOff() - Native C++

Syntax	<pre>ERuntimeErrorCode PowerOff(); ERuntimeErrorCode PowerOff(UINT32 in_Timeout_ms);</pre>	
Parameter	<code>UINT32 in_Timeout_ms</code>: Ein Timeout-Wert in Millisekunden. - Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. - Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 800 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwarteter Betriebszustand, wenn diese Funktion erfolgreich ist: <code>{ SROS_OFF }</code>	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Tabelle 7-47 PowerOff() - .NET (C#)

Syntax	<pre>void PowerOff(); void PowerOff(UInt32 in_Timeout_ms);</pre>	
Parameter	<code>UInt32 in_Timeout_ms</code>: Ein Timeout-Wert in Millisekunden. - Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. - Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 800 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwarteter Betriebszustand, wenn diese Funktion erfolgreich ist: <code>{ EOperatingState.Off }</code>	
Rückgabewerte	Keine	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung

Ausnahmen	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Run()

Schaltet die simulierte PLC-Instanz in den Betriebszustand RUN.

Tabelle 7-48 Run() - Native C++

Syntax	<pre>ERuntimeErrorCode Run(); ERuntimeErrorCode Run(UINT32 in_Timeout_ms);</pre>	
Parameter	UINT32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. - Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. - Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 20 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: { <code>SROS_STOP</code> , <code>SROS_RUN</code> }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_IS_EMPTY</code>	Eine der beiden folgenden Bedingungen trifft zu: Bedingung: Nach dem Erstellen und Starten einer Instanz haben Sie versucht, die simulierte PLC-Instanz in den Betriebszustand RUN zu schalten, ohne Download eines STEP 7-Projekts. Bedingung: Nach dem Erstellen und Einschalten einer Instanz mit Download eines STEP 7-Projekts mit aktiviertem Passwort zum Schutz vertraulicher Konfigurationsdaten haben Sie die virtuelle Speicherkarte von einem System auf ein anderes verschoben. Sie haben versucht, die Instanz auf dem neuen System zu starten. Nach dem Start wird der folgende Fehlercode zurückgegeben: "SREC_WARNING_PASSWORD_PROTECTION_ERROR" Danach haben Sie versucht, die simulierte PLC-Instanz in den Betriebszustand RUN zu schalten.

Tabelle 7-49 Run() - .NET (C#)

Syntax	<pre>void Run(); void Run(UInt32 in_Timeout_ms);</pre>	
Parameter	• <code>UInt32 in_Timeout_ms</code>: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 20 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: <pre>{ EOperatingState.Run }</pre>	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.IsEmpty</code>	Eine der beiden folgenden Bedingungen trifft zu: Bedingung: Nach dem Erstellen und Starten einer Instanz haben Sie versucht, die simulierte PLC-Instanz in den Betriebszustand RUN zu schalten, ohne Download eines STEP 7-Projekts. Bedingung: Nach dem Erstellen und Einschalten einer Instanz mit Download eines STEP 7-Projekts mit aktiviertem Passwort zum Schutz vertraulicher Konfigurationsdaten haben Sie die virtuelle Speicherkarte von einem System auf ein anderes verschoben. Sie haben versucht, die Instanz auf dem neuen System zu starten. Nach dem Start wird der folgende Fehlercode zurückgegeben: "SREC_WARNING_PASSWORD_PROTECTION_ERROR" Danach haben Sie versucht, die simulierte PLC-Instanz in den Betriebszustand RUN zu schalten.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Betriebszustand RUN nach dem Einschalten einer Instanz

Wenn Sie nach dem Einschalten einer Instanz den Betriebszustand RUN erwarten, überprüfen Sie, ob sich die Instanz bereits im RUN befindet, indem Sie das Ereignis `OnOperatingStateChanged()` abonnieren. Als Alternative rufen Sie direkt nach dem Einschalten die API-Funktion `Run()` auf, um die Instanz in den Betriebszustand RUN zu schalten.

Stop()

Schaltet die simulierte PLC-Instanz in den Betriebszustand STOP.

Tabelle 7-50 Stop() - Native C++

Syntax	<pre>ERuntimeErrorCode Stop(); ERuntimeErrorCode Stop(UINT32 in_Timeout_ms);</pre>	
Parameter	• UINT32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 200 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwarteter Betriebszustand, wenn diese Funktion erfolgreich ist: { SROS_STOP }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.

Tabelle 7-51 Stop() - .NET (C#)

Syntax	<pre>void Stop(); void Stop(UInt32 in_Timeout_ms);</pre>	
Parameter	• UInt32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Timeout-Wert von mindestens 200 ms), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwarteter Betriebszustand, wenn diese Funktion erfolgreich ist: { <code>EOperatingState.Stop</code> }	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

GetOperatingState() / OperatingState { get; }

Gibt den Betriebszustand der simulierten PLC-Instanz zurück. Wenn sich der Betriebszustand ändert, wird das Ereignis `OnOperatingStateChanged()` (Seite 175) ausgelöst. Details zum Betriebszustand siehe unter Datentypen (Seite 242).

Tabelle 7-52 GetOperatingState() - Native C++

Syntax	<code>EOperatingState GetOperatingState();</code>
Parameter	Keine
Rückgabewerte	• <code>SROS_INVALID_OPERATING_STATE</code>: wenn die Funktion fehlschlägt. • <code>SROS_OFF</code>: wenn die Simulation Runtime-Instanz nicht läuft. • <code>SROS_BOOTING</code>: wenn in diesem Zustand <code>PowerOn()</code> aufgerufen wurde, aber die simulierte PLC-Instanz noch nicht bereit ist, das Anwenderprogramm zu starten. • <code>SROS_STOP</code>: wenn die simulierte PLC-Instanz in STOP ist. • <code>SROS_STARTUP</code>: wenn das Anwenderprogramm gerade von STOP nach RUN wechselt. • <code>SROS_RUN</code>: wenn das Anwenderprogramm läuft. • <code>SROS_FREEZE</code>: wenn das Anwenderprogramm angehalten wird (Freeze-Status). • <code>SROS_HOLD</code>: wenn das Anwenderprogramm beim Erreichen des Haltepunkts in HALT versetzt wird. • <code>SROS_SHUTTING_DOWN</code>: wenn <code>PowerOff()</code> aufgerufen wurde, aber sich die simulierte PLC-Instanz noch in der Shutdown-Phase befindet.

Tabelle 7-53 OperatingState { get; } - .NET (C#)

Syntax	<code>EOperatingState OperatingState { get; }</code>
Parameter	Keine
Rückgabewerte	• <code>EOperatingState.InvalidOperatingState</code>: wenn die Funktion fehlschlägt. • <code>EOperatingState.Off</code>: wenn die Simulation Runtime Instanz nicht läuft. • <code>EOperatingState.Bootng</code>: wenn in diesem Zustand <code>PowerOn()</code> aufgerufen wurde, aber die simulierte PLC-Instanz noch nicht bereit ist, das Anwenderprogramm zu starten. • <code>EOperatingState.Stop</code>: wenn die simulierte PLC-Instanz in STOP ist. • <code>EOperatingState.Startup</code>: wenn das Anwenderprogramm gerade von STOP nach RUN wechselt. • <code>EOperatingState.Run</code>: wenn das Anwenderprogramm läuft. • <code>EOperatingState.Freeze</code>: wenn das Anwenderprogramm angehalten wird (Freeze-Status). • <code>EOperatingState.Hold</code>: wenn das Anwenderprogramm beim Erreichen des Haltepunkts in HALT versetzt wird. • <code>EOperatingState.ShuttingDown</code>: wenn <code>PowerOff()</code> aufgerufen wurde, aber sich die simulierte PLC-Instanz noch in der Shutdown-Phase befindet.

MemoryReset()

Führt die simulierte PLC-Instanz herunter, schließt deren Prozesse und führt einen Neustart durch.

Durch das Schließen der Prozesse wird automatisch ein Urlöschen durchgeführt.

Tabelle 7-54 MemoryReset() - Native C++

Syntax	<pre>ERuntimeErrorCode MemoryReset(); ERuntimeErrorCode MemoryReset(UINT32 in_Timeout_ms);</pre>	
Parameter	• UINT32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Wert von 60000), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: { <code>SROS_STOP</code>, <code>SROS_RUN</code> }	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Tabelle 7-55 MemoryReset() - .NET (C#)

Syntax	<pre>void MemoryReset(); void MemoryReset(UInt32 in_Timeout_ms);</pre>	
Parameter	• UInt32 in_Timeout_ms: Ein Timeout-Wert in Millisekunden. – Wenn kein Timeout-Wert gesetzt ist, kehrt die Funktion sofort wieder. Abonnieren Sie das Ereignis <code>OnOperatingStateChanged()</code>, um zu erfahren, wann die Operation durchgeführt wurde. – Wenn der Wert größer als 0 ist (empfohlen ist ein Wert von 60000), kehrt die Funktion wieder, wenn die Operation durchgeführt wurde oder nach einem Timeout. Erwartete Betriebszustände, wenn diese Funktion erfolgreich ist: { <code>EOperatingState.Run</code>, <code>EOperatingState.Stop</code> }	
Rückgabewerte	Keine	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Der erwartete Betriebszustand tritt nicht rechtzeitig ein.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

7.4 Simulation anhalten

Freeze-Zustand der simulierten PLC-Instanz

Um eine Simulation anzuhalten und Simulations-Partner zu synchronisieren, können Sie eine simulierte PLC-Instanz über die API in einen Freeze-Zustand versetzen. Wenn die Instanz einen Synchronisationspunkt erreicht hat, sendet sie das Ereignis `OnSyncPointReached` an die API-Clients.

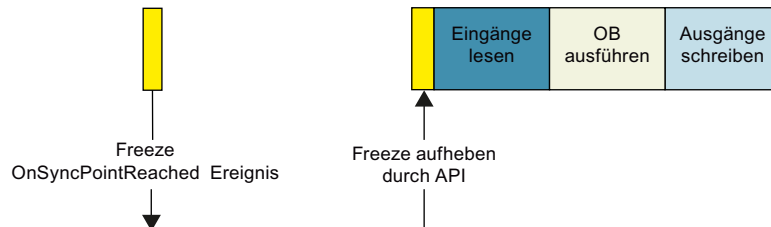


Bild 7-1 Freeze-Zustand der simulierten PLC-Instanz

Im Freeze-Zustand tritt Folgendes ein:

- Die virtuelle Zeit wird angehalten.
- Die aktuell ausgeführten OBs werden angehalten.
Das Vorgehen entspricht dem Verhalten eines Programms, das einen Haltepunkt erreicht hat.
- Das Anwenderprogramm wird nicht weiter ausgeführt.
- Die simulierte PLC-Instanz ist vom TIA Portal aus noch erreichbar.

HINWEIS

Freeze-Zustand beim Download

Um einen Download im Freeze-Zustand abzuschließen, muss die simulierte PLC-Instanz einen aufgehobenen oder beendeten Freeze-Zustand durchlaufen.

HINWEIS

Freeze-Zustand ≠ Betriebszustand

Der Freeze-Zustand ist ein interner Betriebszustand der simulierten PLC-Instanz. Er entspricht nicht dem Betriebszustand RUN/STOP einer CPU. Im Freeze-Zustand behält die Instanz den letzten Betriebszustand bei.

- Die LED-Anzeige auf dem Control Panel und auf dem Webserver zeigt für die Instanz entsprechend RUN oder STOP an.
- Die Instanz zeigt den Betriebszustand `SROS_FREEZE` / Freeze an; siehe `EOperatingState` (Seite 242).

HINWEIS

Reaktion einer S7-PLCSIM Advanced-API auf einen Freeze-Zustand

Wenn Sie eine SingleStep-Betriebsart verwenden, müssen Sie eine Callback-Funktion für das Ereignis `OnSyncPointReached` registrieren. Durch den Aufruf der betriebsartspezifischen Methode `RunToNextSyncPoint` ist sichergestellt, dass der Freeze-Zustand der simulierten PLC-Instanz wieder verlassen wird.

Synchronisationspunkte

Ein Synchronisationspunkt liegt immer dann vor, **bevor** Eingänge eingelesen werden, z. B. am Zykluskontrollpunkt oder am Anfang eines zyklischen OBs.

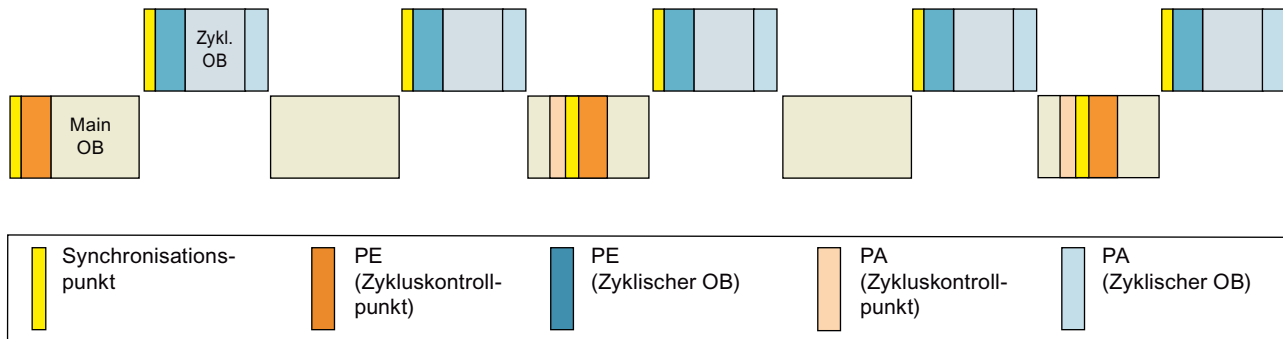


Bild 7-2 Übersicht zu den Synchronisationspunkten

Freeze-Zustand auslösen

Um den Freeze-Zustand auszulösen, stehen für die simulierte PLC-Instanz folgende Betriebsarten zur Verfügung:

- SingleStep-Betriebsarten
Siehe Simulations-Partner zyklusgesteuert synchronisieren ([Seite 171](#)).
- TimespanSynchronized-Betriebsarten
Siehe Simulations-Partner zeitgesteuert synchronisieren ([Seite 174](#)).

In der Betriebsart Default geht die simulierte PLC-Instanz nicht in einen Freeze-Zustand.

API-Funktionen

- Einstellungen für die Zykluskontrolle ([Seite 148](#))
- GetOperatingMode() ([Seite 148](#))
- SetOperatingMode() ([Seite 148](#))
- OperatingMode { get; set; } ([Seite 148](#))
- EOperatingMode ([Seite 243](#))

7.5 Variablentabelle

HINWEIS

Elemente mit Datentypen, die der API nicht bekannt sind (`EDataType.Unknown`), werden nicht mit in die Variablentabelle aufgenommen.

UpdateTagList()

Die Funktion liest die Variablen aus der simulierten PLC-Instanz und schreibt sie anhand des Namens geordnet in den gemeinsamen Speicher.

Wenn die Variable ein Feld oder eine Struktur ist, gibt es mehrere Einträge.

Bei einer Struktur gibt es einen Eintrag für die Struktur selbst und einen zusätzlichen Eintrag für jedes Strukturelement

Entry_1: "StructName"

Entry_2: "StructName.ElementName_1"

..

Entry_N: "StructName.ElementName_n"

Bei einem Array, in diesem Beispiel ein zweidimensionales Array, gibt es einen Eintrag für das Array selbst und einen zusätzlichen Eintrag für jedes Arrayelement.

Entry_1: "ArrayName"

Entry_2: "ArrayName[a,b]" ({a} und {b} entsprechen dem ersten Index der jeweiligen Dimension)

..

Entry_N: "ArrayName[x,y]" ({x} und {y} entsprechen dem letzten Index der jeweiligen Dimension)

Für die Liste ist Speicher für bis zu 500000 Einträge (nicht PLC-Variablen) reserviert. Wenn die Liste zu groß wird, liefert die Funktion den Fehler / die Ausnahme "NOT_ENOUGH_MEMORY" zurück.

Wenn es Probleme mit der maximalen Anzahl der Einträge gibt, aber nicht alle Variablen benötigt werden, können beim Aktualisieren der Variablen-tabelle zwei Filter genutzt werden.

Tabelle 7-56 UpdateTagList() - Native C++

Syntax	<pre>ERuntimeErrorCode UpdateTagList(); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly, WCHAR* in_DataBlockFilterList);</pre>	
Parameter	- ETagListDetails in_TagListDetails: Jede Kombination der folgenden vier Bereiche: IO: Eingänge und Ausgänge M: Merker CT: Zähler und Zeiten DB: Datenbausteine Die Voreinstellung ist IOMCTDB. Beispiel: IOM liest nur die Variablen aus dem Bereich Eingänge / Ausgänge und Merker. - bool in_IsHMIVisibleOnly: Wenn true, werden nur Variablen gelesen, die mit "HMI Visible" markiert sind. Die Voreinstellung ist true. - WCHAR* in_DataBlockFilterList: Ein String, der die Namen aller Datenbausteine enthält, die im Variablen-tabellen-Speicher verfügbar sein sollen. Der String muss in Anführungszeichen stehen. Beispiel: <code>"\"DB_1\", \"DB_2\", \"DB_3\", \"DB_4\", \"DB_5\""</code> Alle Zeichen innerhalb der Anführungszeichen werden als DB-Name interpretiert. Wenn der Datenbaustein im PLC-Programm nicht existiert, wird er dem Variablen-tabellen-Speicher nicht hinzugefügt. Dabei wird kein Fehler ausgelöst. Damit diese Liste berücksichtigt wird, muss in_DataBlockFilterList ungleich NULL sein und in_TagListDetails muss "DB" enthalten.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_NOT_ENOUGH_MEMORY	Es werden mehr als 500000 Einträge angefordert.

Rückgabewerte	SREC_WARNING_ALREADY_EXISTS	Die Variablentabelle ist aktuell.
	SREC_WRONG_ARGUMENT	Die Syntax von <code>in_DataBlockFilterList</code> ist ungültig. Die Liste muss 3 Zeichen lang sein, das erste und letzte Zeichen muss jeweils ein Anführungszeichen sein.
Beispiel C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } if (result == SREC_OK) { // Beispiel: Schritte zum Filtern von Datenbausteinen // DB-Name: TEST_DB, DB-Nummer: 2 result = psa->UpdateTagList(SRTLD_MDB, false, L"\"TEST_DB\""); }</pre>	

Tabelle 7-57 UpdateTagList() - .NET (C#)

Syntax	<pre>void UpdateTagList(); void UpdateTagList(ETagListDetails in_TagListDetails); void UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly, string in_DataBlockFilterList);</pre>
Parameter	<code>ETagListDetails in_TagListDetails</code>: Jede Kombination der folgenden vier Bereiche: IO: Eingänge und Ausgänge M: Merker CT: Zähler und Zeiten DB: Datenbausteine Die Voreinstellung ist IOMCTDB. Beispiel: IOM liest nur die Variablen aus dem Bereich Eingänge / Ausgänge und Merker. <code>bool in_IsHMIVisibleOnly</code>: Wenn <code>true</code>, werden nur Variablen gelesen, die mit "HMI Visible" markiert sind. Die Voreinstellung ist <code>true</code>. <code>string in_DataBlockFilterList</code>: Ein String, der die Namen aller Datenbausteine enthält, die im Variablentabellen-Speicher verfügbar sein sollen. Der String muss in Anführungszeichen stehen. Beispiel: <code>"\"DB_1\", \"DB_2\", \"DB_3\", \"DB_4\", \"DB_5\""</code> Alle Zeichen innerhalb der Anführungszeichen werden als DB-Name interpretiert. Wenn der Datenbaustein im PLC-Programm nicht existiert, wird er dem Variablentabellen-Speicher nicht hinzugefügt. Dabei wird kein Fehler ausgelöst. Damit diese Liste berücksichtigt wird, muss <code>in_DataBlockFilterList</code> ungleich <code>NULL</code> sein und <code>in_TagListDetails</code> muss "DB" enthalten.
Rückgabewerte	Keine
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>

Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Es werden mehr als 500000 Einträge angefordert.
	<code>ERuntimeErrorCode.WrongArgument</code>	Die Syntax von <code>in_DataBlockFilterList</code> ist ungültig. Die Liste muss 3 Zeichen lang sein, das erste und letzte Zeichen muss jeweils ein Anführungszeichen sein.

GetTagListStatus()

Gibt den aktuellen Update-Status des Variablen-tabellen-Speichers zurück.

"`inout_TagListDetails`" ist `NONE`, wenn die Tabelle aktualisiert werden muss.

Tabelle 7-58 GetTagListStatus() - Native C++

Syntax	<code>ERuntimeErrorCode GetTagListStatus(ETagListDetails* out_TagListDetails, bool* out_IsHMIVisibleOnly);</code>	
Parameter	<code>ETagListDetails out_TagListDetails</code>: Status der Variablen-tabellen-Details. <code>SRTLD_NONE</code>, wenn ein Update der Tabelle erforderlich ist. <code>bool out_IsHMIVisibleOnly</code>: Wenn <code>true</code>, sind nur Variablen in der Tabelle verfügbar, die mit "HMI Visible" markiert sind.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-59 GetTagListStatus() - .NET (C#)

Syntax	<code>void GetTagListStatus(out ETagListDetails out_TagListDetails, out bool out_IsHMIVisibleOnly);</code>	
Parameter	<code>out ETagListDetails out_TagListDetails</code>: Status der Variablen-tabellen-Details. <code>ETagListDetails.None</code>, wenn ein Update der Tabelle erforderlich ist. <code>out bool out_IsHMIVisibleOnly</code>: Wenn <code>true</code>, sind nur Variablen in der Tabelle verfügbar, die mit "HMI Visible" markiert sind.	
Rückgabewerte	Keine	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

GetTagInfoCount()

Gibt die Anzahl der Einträge im Variablentabellen-Speicher zurück. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.

Tabelle 7-60 GetTagInfoCount() - Native C++

Syntax	UINT32 GetTagInfoCount();
Parameter	Keine
Rückgabewerte	Anzahl der Einträge im Variablentabellen-Speicher.

GetTagInfos() / TagInfos { get; }

Gibt eine Liste aller Variablen zurück.

Tabelle 7-61 GetTagInfos() - Native C++

Syntax	ERuntimeErrorCode GetTagInfos(UINT32 in_BufferLength, STagInfo* inout_TagInfos, UINT32* out_TagCount);	
Parameter	- UINT32 in_BufferLength: Die Anzahl der Elemente, die der Speicher aufnehmen kann. - STagInfo* inout_TagInfos: Der vom Benutzer zugewiesene Speicher, der die Variablen aufnimmt. - UINT32* out_TagCount: Gibt die Anzahl der Variablen zurück, die in den Speicher geschrieben wurden.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Die Elemente passen nicht in den Speicher.

Tabelle 7-62 TagInfos { get; } - .NET (C#)

Syntax	STagInfo[] TagInfos { get; }	
Parameter	Keine	
Rückgabewerte	Ein Array, das alle verfügbaren Einträge des Speichers enthält.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.

CreateConfigurationFile()

Schreibt alle Einträge aus der Variablentabelle in eine XML-Datei.

Tabelle 7-63 CreateConfigurationFile() - Native C++

Syntax	ERuntimeErrorCode CreateConfigurationFile(WCHAR* in_FullFileName);	
Parameter	WCHAR* in_FullFileName: Vollständiger Dateiname der XML-Datei: <Pfad> + <Dateiname> + <Dateiendung>.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der Dateiname ist ungültig.

Tabelle 7-64 CreateConfigurationFile() - .NET (C#)

Syntax	<code>void CreateConfigurationFile(string in_FullFileName);</code>	
Parameter	Keine	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Dateiname ist ungültig.

7.6 IO-Zugriff

7.6.1 Synchronisieren von Eingängen und Ausgängen

Beschreibung

In S7-PLCSIM Advanced kann der komplette Umfang des Eingangs- und Ausgangsbereichs genutzt werden (siehe [GetAreaSize/AreaSize \(Seite 98\)](#)). Das ist auch dann möglich, wenn kein E/A-Modul konfiguriert ist.

Eingänge und Ausgänge, die über konfigurierte E/A-Module definiert sind, werden zur definierten Aktualisierung des Teilprozessabbilds (TPA) synchronisiert.

Eingänge und Ausgänge, die keinem E/A-Modul zugeordnet sind, werden am Zykluskontrollpunkt synchronisiert.

Beachten Sie beim Synchronisieren dieser Eingänge und Ausgänge Folgendes:

- Eingänge können nur als Eingänge genutzt werden.
Werte können über die API geschrieben werden. Jedoch sind Werte, die über STEP 7 oder über Kommunikation mit dem Eingabebereich geschrieben werden, in der API nicht sichtbar. Wenn Sie beispielsweise im TIA Portal online gehen und Daten in den Eingabebereich schreiben, sind deren Datenwerte in der API nicht sichtbar. Mit Methoden wie `ReadBytes()` können nur Werte gelesen werden, die von der API aus geschrieben wurden.
- Ausgänge können sowohl als Ausgang als auch als Eingang genutzt werden.
Sie können Werte über die API und über die CPU / das STEP 7-Anwenderprogramm schreiben. Wenn API und Anwenderprogramm in den gleichen Bereich schreiben, überschreiben die Werte aus der API die Werte aus dem Anwenderprogramm.

HINWEIS

Bedingungen für die Synchronisierung der fehlersicheren Peripherie

Fehlersichere Ein- und Ausgänge werden nur synchronisiert, wenn ein fehlersicheres Modul gesteckt ist und das fehlersichere Programm mindestens ein Bit des Moduls nutzt.

7.6.2 I/O-Zugriff über Adresse - Lesen

InputArea { get; }, MarkerArea { get; }, OutputArea { get; }

Gibt eine Schnittstelle zurück, die Sie nutzen, um die .NET-Funktionen in diesem Kapitel aufzurufen.

Tabelle 7-65 InputArea { get; } MarkerArea { get; } OutputArea { get; } - .NET (C#)

Syntax	<code>IIOArea InputArea { get; } IIOArea MarkerArea { get; } IIOArea OutputArea { get; }</code>
Parameter	Keine
Rückgabewerte	IIOArea: Die Schnittstelle, die genutzt wird, um die Funktionen "E/A-Zugriff über Adresse" aufzurufen.

GetAreaSize() / AreaSize { get; }

Gibt die Größe des Bereichs in Byte zurück.

Tabelle 7-66 GetAreaSize() - Native C++

Syntax	<code>UINT32 GetAreaSize(EArea in_Area);</code>
Parameter	EArea in_Area: Der Bereich, dessen Größe Sie erhalten möchten. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242).
Rückgabewerte	UINT32: Größe des Bereichs in Byte. Wenn die Funktion erfolgreich war, ist der Wert ungleich 0.

Tabelle 7-67 AreaSize { get; } - .NET (C#)

Syntax	<code>UInt32 InputArea.AreaSize { get; } UInt32 MarkerArea.AreaSize { get; } UInt32 OutputArea.AreaSize { get; }</code>
Parameter	Keine
Rückgabewerte	UInt32: Größe des Bereichs in Byte. Wenn die Funktion erfolgreich war, ist der Wert ungleich 0.

ReadBit()

Liest ein einzelnes Byte aus dem Bereich.

HINWEIS

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz. Nutzen Sie daher den Zugriff über den Variablennamen (Seite 107) und nicht über die Adressbereiche.

Tabelle 7-68 ReadBit() - Native C++

Syntax	<code>ERuntimeErrorCode ReadBit(EArea in_Area, UINT32 in_Offset, UINT8 in_Bit, bool* out_Value);</code>
Parameter	- EArea in_Area: Der Bereich, aus dem gelesen werden soll. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den GetAreaSize() zurückgibt.

	• <code>UINT8 in_Bit</code>: Der Bit-Offset innerhalb des Bytes. Der Wert muss zwischen 0 und 7 sein. • <code>bool* out_Value</code>: Gibt den Bitwert zurück.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Offset oder Bits sind ungültig.
	<code>SREC_WRONG_ARGUMENT</code>	Der Bereich ist ungültig.

Tabelle 7-69 ReadBit() - .NET (C#)

Syntax	<pre>bool InputArea.ReadBit(UInt32 in_Offset, Byte in_Bit); bool MarkerArea.ReadBit(UInt32 in_Offset, Byte in_Bit); bool OutputArea.ReadBit(UInt32 in_Offset, Byte in_Bit);</pre>	
Parameter	• <code>UInt32 in_Offset</code>: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den <code>AreaSize</code> zurückgibt. • <code>Byte in_Bit</code>: Der Bit-Offset innerhalb des Bytes. Der Wert muss zwischen 0 und 7 sein.	
Rückgabewerte	<code>bool</code> : Bitwert	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Offset oder Bits sind ungültig.

ReadByte()

Liest ein einzelnes Byte aus dem Bereich.

HINWEIS

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz. Nutzen Sie daher den Zugriff über den Variablennamen und nicht über die Adressbereiche.

Tabelle 7-70 ReadByte() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadByte(EArea in_Area, UINT32 in_Offset, BYTE* out_Value);</pre>	
Parameter	• <code>EArea in_Area</code>: Der Bereich, aus dem Sie lesen möchten. Zulässige Werte: <code>{SRA_INPUT, SRA_MARKER, SRA_OUTPUT}</code>. Siehe <code>EArea</code> (Seite 242). • <code>UINT32 in_Offset</code>: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den <code>GetAreaSize()</code> zurückgibt. • <code>BYTE* out_Value</code>: Gibt den Bytewert zurück.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Offset ist ungültig.
	SREC_WRONG_ARGUMENT	Der Bereich ist ungültig.

Tabelle 7-71 ReadByte() - .NET (C#)

Syntax	<pre>Byte InputArea.ReadByte(UInt32 in_Offset); Byte MarkerArea.ReadByte(UInt32 in_Offset); Byte OutputArea.ReadByte(UInt32 in_Offset);</pre>	
Parameter	UInt32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den AreaSize zurückgibt.	
Rückgabewerte	Byte: Bytewert.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Offset ist ungültig.

ReadBytes()

Liest ein Byte-Array aus dem Bereich.

HINWEIS

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz. Nutzen Sie daher den Zugriff über den Variablennamen und nicht über die Adressbereiche.

Tabelle 7-72 ReadByte() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadBytes(EArea in_Area, UINT32 in_Offset, UINT32 in_BytesToRead, UINT32* out_BytesRead, BYTE inout_Values[]);</pre>	
Parameter	- EArea in_Area: Der Bereich, aus dem Sie lesen möchten. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den GetAreaSize() zurückgibt. - UINT32 in_BytesToRead: Enthält die Größe des Wertespeichers. - UINT32* out_BytesRead: Gibt die Anzahl der Bytes zurück, die gerade in den Wertespeicher geschrieben wurden. - BYTE inout_Values[]: Der Speicher für die Bytes, die aus dem Bereich gelesen werden.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.

Rückgabewerte	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Byte konnte gelesen werden.
	SREC_WRONG_ARGUMENT	Der Bereich ist ungültig.

Tabelle 7-73 ReadBytes() - .NET (C#)

Syntax	<pre>Byte[] InputArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead); Byte[] MarkerArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead); Byte[] OutputArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead);</pre>	
Parameter	- UInt32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den AreaSize zurückgibt. - UInt32 in_BytesToRead: Die Anzahl der Byte, die gelesen werden.	
Rückgabewerte	Byte[]: Die gelesenen Bytes.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Byte konnte gelesen werden.

ReadSignals()

Strukturen und Felder können durch Signallisten nachgebildet und dann über ReadSignals() gelesen werden.

Die Funktion berücksichtigt auch die Byte-Reihenfolge (Endianness).

Es werden nur primitive Datentyp-Signale unterstützt, aber die Funktion ist nicht typsicher.

HINWEIS

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz.

Nutzen Sie daher den Zugriff über den Variablennamen (Seite 107) und nicht über die Adressbereiche.

Tabelle 7-74 ReadSignals() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadSignals(EArea in_Area, SDataValueByAddress* inout_Signals, UINT32 in_SignalCount, bool validateAddressTag = false); ERuntimeErrorCode ReadSignals(EArea in_Area, SDataValueByAddressWithCheck* inout_Signals, UINT32 in_SignalCount, bool* out_SignalsHaveChanged, bool validateAddressTag = false);</pre>	
Parameter	- EArea in_Area Der Bereich, aus dem Sie lesen möchten. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - SDataValueByAddress* inout_Signals Die Signalliste, die gelesen wird. Das Ergebnis wird in der Struktur gespeichert. - SDataValueByAddressWithCheck* inout_Signals Die Signalliste, die gelesen wird. Das Ergebnis wird in der Struktur gespeichert. "ValueHasChanged" wird auf true gesetzt, wenn sich der Wert des Signals seit dem vorhergehenden Aufruf geändert hat.	

	• <code>UINT32 in_SignalCount</code> Anzahl der Signale in der Liste. • <code>bool* out_SignalsHaveChanged</code> Gibt <code>true</code> zurück, wenn sich der Wert von mindestens einem Signal seit dem vorhergehenden Aufruf geändert hat. • <code>bool validateAddressTag</code> Boolescher Wert, mit dem die Prüfung der Verknüpfung von Speicheradressen und Variablen aktiviert oder deaktiviert wird.	
Signalfehler	Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Signaloperation ist erfolgreich.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_WRONG_ARGUMENT</code>	Der Bereich ist ungültig.
	<code>SREC_SIGNAL_CONFIGURATION_ERROR</code>	Mindestens ein Signalfehler ist in der Liste.

Tabelle 7-75 ReadSignals() - .NET (C#)

Syntax	<pre>void ReadSignals(ref SDataValueByAddress[] inout_Signals, bool validateAddressTag); void ReadSignals(ref SDataValueByAddressWithCheck[] inout_Signals out bool out_SignalsHaveChanged, bool validateAddressTag);</pre>	
Parameter	• <code>ref SDataValueByAddress[] inout_Signals</code> Die Signalliste, die gelesen wird. • <code>ref SDataValueByAddressWithCheck[] inout_Signals</code> Die Signalliste, die gelesen wird. Das Ergebnis wird in der Struktur gespeichert. "ValueHasChanged" wird auf <code>true</code> gesetzt, wenn sich der Wert des Signals seit dem vorhergehenden Aufruf geändert hat. • <code>out bool out_SignalsHaveChanged</code> Gibt <code>true</code> zurück, wenn sich der Wert von mindestens einem Signal seit dem vorhergehenden Aufruf geändert hat. • <code>bool validateAddressTag</code> Optionaler boolescher Wert, mit dem die Prüfung der Verknüpfung von Speicheradressen und Variablen aktiviert oder deaktiviert wird.	
Signalfehler	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.OK</code>	Die Signaloperation ist erfolgreich.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Offset oder Bits sind ungültig.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.

Ausnahmen	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.SignalConfigurationError	Mindestens ein Signalfehler ist in der Liste.

7.6.3 I/O-Zugriff über Adresse - Schreiben

WriteBit()

Schreibt ein einzelnes Bit in den Bereich.

HINWEIS

Daten können überschrieben werden

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz.

Nutzen Sie daher den Zugriff über den Variablennamen ([Seite 126](#)) und nicht über die Adressbereiche.

Tabelle 7-76 WriteBit() - Native C++

Syntax	ERuntimeErrorCode WriteBit(EArea in_Area, UINT32 in_Offset, UINT8 in_Bit, bool in_Value);	
Parameter	- EArea in_Area: Der Bereich, in den geschrieben werden soll. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den GetAreaSize() zurückgibt. - UINT8 in_Bit: Der Bit-Offset innerhalb des Bytes. Der Wert muss zwischen 0 und 7 sein. - bool in_Value: Bitwert.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Offset oder Bits sind ungültig.
	SREC_WRONG_ARGUMENT	Bereich ist ungültig.

Tabelle 7-77 WriteBit() - .NET (C#)

Syntax	void InputArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value); void MarkerArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value); void OutputArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value);	
Parameter	- UInt32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den AreaSize zurückgibt. - Byte in_Bit: Der Bit-Offset innerhalb des Bytes. Der Wert muss zwischen 0 und 7 sein. - bool in_Value: Bitwert.	

Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Offset oder Bits sind ungültig.

WriteByte()

Schreibt ein einzelnes Byte in den Bereich.

HINWEIS

Daten können überschrieben werden

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz. Nutzen Sie daher den Zugriff über den Variablennamen (Seite 126) und nicht über die Adressbereiche.

Tabelle 7-78 WriteByte() - Native C++

Syntax	ERuntimeErrorCode WriteByte(EArea in_Area, UINT32 in_Offset, BYTE in_Value);	
Parameter	- EArea in_Area: Der Bereich, in den geschrieben werden soll. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den GetAreaSize() zurückgibt. - BYTE in_Value: Bytewert.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Offset ist ungültig.
	SREC_WRONG_ARGUMENT	Bereich ist ungültig.

Tabelle 7-79 WriteByte() - .NET (C#)

Syntax	void InputArea.WriteByte(UInt32 in_Offset, Byte in_Value); void MarkerArea.WriteByte(UInt32 in_Offset, Byte in_Value); void OutputArea.WriteByte(UInt32 in_Offset, Byte in_Value);	
Parameter	- UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den AreaSize zurückgibt. - BYTE in_Value: Bytewert.	
Rückgabewerte	Keine	

Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Offset ist ungültig.

WriteBytes()

Schreibt ein Byte-Array in den Bereich.

HINWEIS

Daten können überschrieben werden

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz. Achten Sie besonders darauf, nicht auf Bytes zu schreiben, die zu anderen Anwendungen gehören oder die interne Daten enthalten, z. B. Qualifier-Bits für fehlersichere Peripheriemodule.

Nutzen Sie daher den Zugriff über den Variablennamen [\(Seite 126\)](#) und nicht über die Adressbereiche.

Tabelle 7-80 WriteBytes() - Native C++

Syntax	ERuntimeErrorCode WriteBytes(EArea in_Area, UINT32 in_Offset, UINT32 in_BytesToWrite, UINT32* out_BytesWritten, BYTE in_Values[]);	
Parameter	- EArea in_Area: Der Bereich, in den geschrieben werden soll. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). - UINT32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den GetAreaSize() zurückgibt. - UINT32 in_BytesToWrite: Enthält die Größe des Arraywerts, der geschrieben werden soll. - UINT32* out_BytesWritten: Enthält die Anzahl der Bytes, die gerade geschrieben wurden. - BYTE in_Values[]: Byte-Array, das in den Bereich geschrieben werden soll.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Byte konnte geschrieben werden.
	SREC_WRONG_ARGUMENT	Der Bereich ist ungültig.

Tabelle 7-81 WriteBytes() - .NET (C#)

Syntax	<pre> UInt32 InputArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 InputArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); UInt32 MarkerArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 MarkerArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); UInt32 OutputArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 OutputArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); </pre>	
Parameter	• UInt32 in_Offset: Der Byte-Offset innerhalb des Bereichs. Der Wert muss zwischen 0 und dem Wert sein, den AreaSize zurückgibt. • UInt32 in_BytesToWrite: Enthält die Anzahl der Bytes, die geschrieben werden sollen. Der Wert muss zwischen 1 und der Größe des Arraywerts liegen. • Byte[] in_Value: Bytewert.	
Rückgabewerte	UInt32: Enthält die Anzahl der Bytes, die gerade geschrieben wurden.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRangeException	Der Offset liegt außerhalb der Bereichsgröße. Kein Byte konnte geschrieben werden.

WriteSignals()

Schreibt mehrere Signale innerhalb eines API-Aufrufs.

Die Funktion berücksichtigt auch die Byte-Reihenfolge (Endianness).

Die Funktion unterstützt nur primitiv Datentyp-Signale, sie ist aber nicht typsicher.

HINWEIS

Daten können überschrieben werden

Die Funktion erlaubt Zugriff auf den gesamten Speicherbereich der simulierten PLC-Instanz.

Nutzen Sie daher den Zugriff über den Variablennamen [\(Seite 126\)](#) und nicht über die Adressbereiche.

Tabelle 7-82 WriteSignals() - Native C++

Syntax	<pre> ERuntimeErrorCode WriteSignals(EArea in_Area, SDataValueByAddress* in_Signals, UINT32 in_SignalCount, bool_validateAddressTag = false); </pre>	
Parameter	• EArea in_Area Der Bereich, in den geschrieben werden soll. Zulässige Werte: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. Siehe EArea (Seite 242). • SDataValueByAddress* inout_Signals Die Signalliste, die geschrieben werden soll. • UINT32 in_SignalCount Anzahl der Signale in der Liste. • bool validateAddressTag Boolescher Wert, mit dem die Prüfung der Verknüpfung von Speicheradressen und Variablen aktiviert oder deaktiviert wird.	

Signalfehler	Fehlercode	Beschreibung
	SREC_OK	Die Signaloperation ist erfolgreich.
	SREC_INDEX_OUT_OF_RANGE	Offset oder Bits sind ungültig.
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_SIGNAL_CONFIGURATION_ERROR	Mindestens ein Signalfehler ist in der Liste.
	SREC_WRONG_ARGUMENT	Der Bereich ist ungültig.

Tabelle 7-83 WriteSignals() - .NET (C#)

Syntax	void WriteSignals(SDataValueByAddress[] in_Signals, bool validateAddressTag);	
Parameter	- SDataValueByAddress[] in_Signals Die Signalliste, die geschrieben werden soll. - bool validateAddressTag Optionaler boolescher Wert, mit dem die Prüfung der Verknüpfung von Speicheradressen und Variablen aktiviert oder deaktiviert wird.	
Signalfehler	Fehlercode	Beschreibung
	ERuntimeErrorCode.OK	Die Signaloperation ist erfolgreich.
	ERuntimeError-Code.IndexOutOfRange	Offset oder Bits sind ungültig.
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode. SignalConfigurationError	Mindestens ein Signalfehler ist in der Liste.

7.6.4 I/O-Zugriff über Variablenname - Lesen

Einzelzugriffe auf E/A-Daten sind dazu geeignet, Werte, die nicht regelmäßig aktualisiert werden, auf einer grafischen Oberfläche (GUI) anzuzeigen und zu schreiben.

HINWEIS

Um einen regelmäßigen Signalaustausch zu simulieren, erstellen Sie für jeden Satz von Signalen eine Signalliste. Nutzen Sie diese Signalliste für alle weiteren Zugriffe. Erstellen Sie eine neue Liste, sobald sich der Satz von Signalen ändert.

Verwenden Sie für die Signallisten die Funktionen `ReadSignals()` und `WriteSignals()`.

Read()

Liest den Wert einer PLC-Variablen.

Tabelle 7-84 Read() - Native C++

Syntax	<code>ERuntimeErrorCode Read(WCHAR* in_Tag, SDataValue* inout_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>SDataValue* inout_Value</code>: Enthält den Wert und den erwarteten Typ der PLC-Variablen. Wenn der erwartete Typ <code>UNSPECIFIC</code> ist, wird er auf den gespeicherten Typ gesetzt, wenn die Funktion erfolgreich war. Der Typ <code>STRUCT</code> wird nicht unterstützt. Strukturen und Felder können durch Signallisten nachgebildet und dann über <code>ReadSignals()</code> gelesen werden.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-85 Read() - .NET (C#)

Syntax	<code>SDataValue Read(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>SDataValue</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.

Ausnahmen	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 249).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadBool()

Liest den Wert einer PLC-Variablen.

Tabelle 7-86 ReadBool() - Native C++

Syntax	<code>ERuntimeErrorCode ReadBool(WCHAR* in_Tag, bool* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>bool* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-87 ReadBool() - .NET (C#)

Syntax	<code>bool ReadBool(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>bool</code> : Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRangeException</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.

Ausnahmen	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadInt8()

Liest den Wert einer PLC-Variablen.

Tabelle 7-88 ReadInt8() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt8(WCHAR* in_Tag, INT8* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>INT8* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-89 ReadInt8() - .NET (C#)

Syntax	<code>Int8 ReadInt8(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>Int8</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Ausnahmen	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

ReadInt16()

Liest den Wert einer PLC-Variablen.

Tabelle 7-90 ReadInt16() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt16(WCHAR* in_Tag, INT16* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>INT16* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-91 ReadInt16() - .NET (C#)

Syntax	<code>Int16 ReadInt16(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>Int16</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Ausnahmen	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadInt32()

Liest den Wert einer PLC-Variablen.

Tabelle 7-92 WechslerReadInt32() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt32 (WCHAR* in_Tag, INT32* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>INT32* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-93 ReadInt32() - .NET (C#)

Syntax	<code>Int32 ReadInt32 (string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>Int32</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.

Ausnahmen	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadInt64()

Liest den Wert einer PLC-Variablen.

Tabelle 7-94 ReadInt64() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt64(WCHAR* in_Tag, INT64* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>INT64* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-95 ReadInt64() - .NET (C#)

Syntax	<code>Int64 ReadInt64(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>Int64</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung

Ausnahmen	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

ReadUInt8()

Liest den Wert einer PLC-Variablen.

Tabelle 7-96 ReadUInt8() - Native C++

Syntax	<code>ERuntimeErrorCode ReadUInt8(WCHAR* in_Tag, UINT8* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>UINT8* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-97 ReadUInt8() - .NET (C#)

Syntax	<code>UInt8 ReadUInt8(string in_Tag)</code>
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.
Rückgabewerte	<code>UInt8</code> : Enthält den Wert und den Typ der PLC-Variablen.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>

Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadUInt16()

Liest den Wert einer PLC-Variablen.

Tabelle 7-98 ReadUInt16() - Native C++

Syntax	<code>ERuntimeErrorCode ReadUInt16(WCHAR* in_Tag, UINT16* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>UINT16* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-99 ReadUInt16() - .NET (C#)

Syntax	<code>UInt16 ReadUInt16(string in_Tag)</code>
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.
Rückgabewerte	<code>UInt16</code> : Enthält den Wert und den Typ der PLC-Variablen.

Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadUInt32()

Liest den Wert einer PLC-Variablen.

Tabelle 7-100 ReadUInt32() - Native C++

Syntax	ERuntimeErrorCode ReadUInt32(WCHAR* in_Tag, UINT32* out_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die gelesen werden soll. - UINT32* out_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-101 ReadUInt32() - .NET (C#)

Syntax	UInt32 ReadUInt32(string in_Tag)	
Parameter	string in_Tag: Der Name der PLC-Variablen, die gelesen werden soll.	

Rückgabewerte	UInt32: Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadUInt64()

Liest den Wert einer PLC-Variablen.

Tabelle 7-102 ReadInt64() - Native C++

Syntax	ERuntimeErrorCode ReadUInt64(WCHAR* in_Tag, UINT64* out_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die gelesen werden soll. - UINT64* out_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-103 ReadUInt64() - .NET (C#)

Syntax	UInt64 ReadUInt64(string in_Tag)	
Parameter	string in_Tag: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	UInt64: Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadFloat()

Liest den Wert einer PLC-Variablen.

Tabelle 7-104 ReadFloat() - Native C++

Syntax	ERuntimeErrorCode ReadFloat(WCHAR* in_Tag, float* out_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die gelesen werden soll. - float* out_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-105 ReadFloat() - .NET (C#)

Syntax	<code>float ReadFloat(string in_Tag)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>float</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadDouble()

Liest den Wert einer PLC-Variablen.

Tabelle 7-106 ReadDouble() - Native C++

Syntax	<code>ERuntimeErrorCode ReadDouble(WCHAR* in_Tag, double* out_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>double* out_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-107 ReadDouble() - .NET (C#)

Syntax	double ReadDouble(string in_Tag)	
Parameter	string in_Tag: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	double: Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadChar()

Liest den Wert einer PLC-Variablen.

Tabelle 7-108 ReadChar() - Native C++

Syntax	ERuntimeErrorCode ReadChar(WCHAR* in_Tag, char* out_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die gelesen werden soll. - char* out_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-109 ReadChar() - .NET (C#)

Syntax	sbyte ReadChar(string in_Tag)	
Parameter	string in_Tag: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	sbyte: Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadWChar()

Liest den Wert einer PLC-Variablen.

Tabelle 7-110 ReadWChar() - Native C++

Syntax	ERuntimeErrorCode ReadWChar(WCHAR* in_Tag, WCHAR* out_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die gelesen werden soll. - WCHAR* out_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-111 ReadWChar() - .NET (C#)

Syntax	char ReadWChar(string in_Tag)	
Parameter	string in_Tag: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	char: Enthält den Wert und den Typ der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

ReadSignals()

Liest mehrere Signale innerhalb eines API-Aufrufs.

Wenn die Funktion das erste Mal aufgerufen wird, speichert sie in den Strukturen SDataValueByName* interne Informationen, um die Leistung der anschließenden Aufrufe zu verbessern.

HINWEIS

Um einen regelmäßigen Signalaustausch zu simulieren, erstellen Sie für jeden Satz von Signalen eine Signalliste. Nutzen Sie diese Signalliste für alle weiteren Zugriffe. Erstellen Sie eine neue Liste, sobald sich der Satz von Signalen ändert.

Tabelle 7-112 ReadSignals() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadSignals(SDataValueByName* inout_Signals, UINT32 in_SignalCount); ERuntimeErrorCode ReadSignals(SDataValueByNameWithCheck* inout_Signals, UINT32 in_SignalCount bool* out_SignalsHaveChanged);</pre>
Parameter	- SDataValueByName* inout_Signals: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Wenn der erwartete Typ UNSPECIFIC ist, wird er auf den gespeicherten Typ gesetzt, wenn die Funktion erfolgreich war. Der Typ STRUCT wird nicht unterstützt. - SDataValueByNameWithCheck* inout_Signals: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Wenn der erwartete Typ UNSPECIFIC ist, wird er auf den gespeicherten Typ gesetzt, wenn die Funktion erfolgreich war. Der Typ STRUCT wird nicht unterstützt. "ValueHasChanged" wird auf true gesetzt, wenn sich der Wert des Signals seit dem vorhergehenden Aufruf geändert hat.

	• <code>UINT32 in SignalCount</code>: Die Anzahl der Signale, die gelesen werden sollen. • <code>bool* out_SignalsHaveChanged</code>: Gibt <code>true</code> zurück, wenn sich der Wert von mindestens einem Signal seit dem vorhergehenden Aufruf geändert hat.	
Signalfehler	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Signaloperation ist erfolgreich.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Offset oder Bits sind ungültig.
Rückgabewerte	Runtime-Fehlercode	Bedingung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>SREC_SIGNAL_CONFIGURATION_ERROR</code>	Mindestens ein Signalfehler ist in der Liste.

Tabelle 7-113 ReadSignals() - .NET (C#)

Syntax	<pre>void ReadSignals(ref SDataValueByName[] inout_Signals) void ReadSignals(ref SDataValueByNameWithCheck[] inout_Signals out bool out_SignalsHaveChanged);</pre>	
Parameter	• <code>ref SDataValueByName[] inout_Signals</code>: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Wenn der erwartete Typ <code>UNSPECIFIC</code> ist, wird er auf den gespeicherten Typ gesetzt, wenn die Funktion erfolgreich war. Der Typ <code>STRUCT</code> wird nicht unterstützt. • <code>ref SDataValueByNameWithCheck[] inout_Signals</code>: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Wenn der erwartete Typ <code>UNSPECIFIC</code> ist, wird er auf den gespeicherten Typ gesetzt, wenn die Funktion erfolgreich war. Der Typ <code>STRUCT</code> wird nicht unterstützt. "ValueHasChanged" wird auf <code>true</code> gesetzt, wenn sich der Wert des Signals seit dem vorhergehenden Aufruf geändert hat. • <code>out bool out_SignalsHaveChanged</code>: Gibt <code>true</code> zurück, wenn sich der Wert von mindestens einem Signal seit dem vorhergehenden Aufruf geändert hat.	
Rückgabewerte	<code>SDataValue</code> : Enthält den Wert und den Typ der PLC-Variablen.	
Signalfehler	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.OK</code>	Die Signaloperation ist erfolgreich.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).

Signalfehler	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Offset oder Bits sind ungültig.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

ReadString()

Liest den Wert einer PLC-Variablen.

Tabelle 7-114 ReadString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::ReadString(const WCHAR* in_Tag, CHAR* out_Value, const UINT32 in_BufferSize, UINT32* out_CharsRead);</pre>	
Parameter	<code>const WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>CHAR* out_Value</code>: Enthält den Wert der PLC-Variablen <code>const UINT32 in_BufferSize</code>: Größe des bereitgestellten Puffers <code>UINT32* out_CharsRead</code>: Anzahl der gelesenen Zeichen	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.
	<code>SREC_BUFFER_OVERFLOW</code>	Der bereitgestellte Puffer ist zu klein.

Tabelle 7-115 ReadString() - .NET (C#)

Syntax	<code>string ReadString(string in_Tag);</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>string</code> : Enthält den Wert der PLC-Variablen.	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>ERuntimeErrorCode.BufferOverflow</code>	Der bereitgestellte Puffer ist zu klein.

ReadWString()

Liest den Wert einer PLC-Variablen.

Tabelle 7-116 ReadWString() - Native C++

Syntax	<code>ERuntimeErrorCode IInstance::ReadWString(const WCHAR* in_Tag, WCHAR* out_Value, const UINT32 in_BufferSize, UINT32* out_CharsRead);</code>	
Parameter	<code>const WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll. <code>WCHAR* out_Value</code>: Enthält den Wert der PLC-Variablen <code>const UINT32 in_BufferSize</code>: Größe des bereitgestellten Puffers <code>UINT32* out_CharsRead</code>: Anzahl der gelesenen Zeichen	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.

Rückgabewerte	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	SREC_BUFFER_OVERFLOW	Der bereitgestellte Puffer ist zu klein.

Tabelle 7-117 ReadWString() - .NET (C#)

Syntax	<code>wstring ReadWString(wstring in_Tag);</code>	
Parameter	<code>wstring in_Tag</code>: Der Name der PLC-Variablen, die gelesen werden soll.	
Rückgabewerte	<code>wstring</code> : Enthält den Wert der PLC-Variablen.	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte gelesen werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>ERuntimeErrorCode.BufferOverflow</code>	Der bereitgestellte Puffer ist zu klein.

7.6.5 I/O-Zugriff über Variablenname - Schreiben

Einzelzugriffe auf E/A-Daten sind dazu geeignet, Werte, die nicht regelmäßig aktualisiert werden, auf einer grafischen Oberfläche (GUI) anzuzeigen und zu schreiben.

HINWEIS

Um einen regelmäßigen Signalaustausch zu simulieren, erstellen Sie für jeden Satz von Signalen eine Signalliste. Nutzen Sie diese Signalliste für alle weiteren Zugriffe. Erstellen Sie eine neue Liste, sobald sich der Satz von Signalen ändert.

Verwenden Sie für die Signallisten die Funktionen `ReadSignals()` und `WriteSignals()`.

Write()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-118 Write() - Native C++

Syntax	<code>ERuntimeErrorCode Write(WCHAR* in_Tag, SDataValue* in_Value);</code>	
Parameter	WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. SDataValue* in_Value: Enthält den Wert und den erwarteten Typ der PLC-Variablen. Die Typen <code>UNSPECIFIC</code> und <code>STRUCT</code> werden nicht unterstützt. Strukturen und Felder können durch Signallisten nachgebildet und dann über <code>ReadSignals()</code> gelesen und über <code>WriteSignals()</code> geschrieben werden.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>SREC_WRONG_ARGUMENT</code>	Der erwartete Typ ist <code>UNSPECIFIC</code> .

Tabelle 7-119 Write() - .NET (C#)

Syntax	<code>void Write(string in_Tag SDataValue in_Value)</code>	
Parameter	string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. SDataValue in_Value: Enthält den Wert und den erwarteten Typ der PLC-Variablen. Die Typen <code>UNSPECIFIC</code> und <code>STRUCT</code> werden nicht unterstützt. Strukturen und Felder können durch Signallisten nachgebildet und dann über <code>WriteSignals()</code> geschrieben werden.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.

Ausnahmen	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der erwartete Typ ist UNSPECIFIC.

WriteBool()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-120 WriteBool() - Native C++

Syntax	<code>ERuntimeErrorCode WriteBool(WCHAR* in_Tag, bool in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>bool in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-121 WriteBool() - .NET (C#)

Syntax	<code>void WriteBool(string in_Tag bool in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>bool in_Value</code>: Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Ausnahmen	<code>ERuntimeErrorCode.IndexOutOfRangeException</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 249).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

WriteInt8()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-122 WriteInt8() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt8(WCHAR* in_Tag, INT8 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>INT8 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-123 WriteInt8() - .NET (C#)

Syntax	<code>void WriteInt8(string in_Tag Int8 in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>Int8 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Ausnahmen	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

WriteInt16()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-124 WriteInt16() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt16(WCHAR* in_Tag, INT16 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>INT16 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-125 WriteInt16() - .NET (C#)

Syntax	<code>void WriteInt16(string in_Tag Int16 in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>Int16 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.

Ausnahmen	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteInt32()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-126 WriteInt32() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt32(WCHAR* in_Tag, INT32 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>INT32 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-127 WriteInt32() - .NET (C#)

Syntax	<code>void WriteInt32(string in_Tag Int32 in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>Int32 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung

Ausnahmen	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

WriteInt64()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-128 WriteInt64() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt64(WCHAR* in_Tag, INT64 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>INT64 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-129 WriteInt64() - .NET (C#)

Syntax	<code>void WriteInt64(string in_Tag Int64 in_Value)</code>
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>Int64 in_Value</code>: Enthält den Wert der PLC-Variablen.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>

Ausnahmen	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteUInt8()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-130 WriteUInt8() - Native C++

Syntax	<code>ERuntimeErrorCode WriteUInt8(WCHAR* in_Tag, UINT8 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>UINT8 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-131 WriteUInt8() - .NET (C#)

Syntax	<code>void WriteUInt8(string in_Tag UInt8 in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>UInt8 in_Value</code>: Enthält den Wert der PLC-Variablen.	

Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteUInt16()

Liest den Wert einer PLC-Variablen.

Tabelle 7-132 WriteUInt16() - Native C++

Syntax	ERuntimeErrorCode WriteUInt16(WCHAR* in_Tag, UINT16 in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - UINT16 in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-133 WriteUInt16() - .NET (C#)

Syntax	<code>void WriteUInt16(string in_Tag UInt16 in_Value)</code>	
Parameter	<code>string in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>UInt16 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteUInt32()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-134 WriteUInt32() - Native C++

Syntax	<code>ERuntimeErrorCode WriteUInt32(WCHAR* in_Tag, UINT32 in_Value);</code>	
Parameter	<code>WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>UINT32 in_Value</code>: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.

Rückgabewerte	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-135 WriteUInt32() - .NET (C#)

Syntax	void WriteUInt32(string in_Tag UInt32 in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - UInt32 in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablentabelle muss aktualisiert werden.

WriteUInt64()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-136 WriteUInt64() - Native C++

Syntax	ERuntimeErrorCode WriteUInt64(WCHAR* in_Tag, UINT64 in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - UINT64 in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.

Rückgabewerte	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-137 WriteUInt64() - .NET (C#)

Syntax	void WriteUInt64(string in_Tag UInt64 in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - UInt64 in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteFloat()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-138 WriteFloat() - Native C++

Syntax	ERuntimeErrorCode WriteFloat(WCHAR* in_Tag, float in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - float in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.

Rückgabewerte	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablentabelle muss aktualisiert werden.

Tabelle 7-139 WriteFloat() - .NET (C#)

Syntax	void WriteFloat(string in_Tag float in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - float in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablentabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablentabelle muss aktualisiert werden.

WriteDouble()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-140 WriteDouble() - Native C++

Syntax	ERuntimeErrorCode WriteDouble(WCHAR* in_Tag, double in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - double in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Rückgabewerte	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-141 WriteDouble() - .NET (C#)

Syntax	void WriteDouble(string in_Tag double in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - double in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteChar()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-142 WriteChar() - Native C++

Syntax	ERuntimeErrorCode WriteChar(WCHAR* in_Tag, char in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - char in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.

Rückgabewerte	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-143 WriteChar() - .NET (C#)

Syntax	void WriteChar(string in_Tag sbyte in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - sbyte in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteWChar()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-144 WriteWChar() - Native C++

Syntax	ERuntimeErrorCode WriteWChar(WCHAR* in_Tag, WCHAR in_Value);	
Parameter	- WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - WCHAR in_Value: Enthält den Wert der PLC-Variablen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.

Rückgabewerte	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

Tabelle 7-145 WriteWChar() - .NET (C#)

Syntax	void WriteWChar(string in_Tag char in_Value)	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - char in_Value: Enthält den Wert der PLC-Variablen.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.

WriteSignals()

Schreibt mehrere Signale innerhalb eines API-Aufrufs.

Wenn die Funktion das erste Mal aufgerufen wird, speichert sie in den Strukturen `SDataValueByName*` interne Informationen, um die Leistung der anschließenden Aufrufe zu verbessern.

HINWEIS

Um einen regelmäßigen Signalaustausch zu simulieren, erstellen Sie für jeden Satz von Signalen eine Signalliste. Nutzen Sie diese Signalliste für alle weiteren Zugriffe. Erstellen Sie eine neue Liste, sobald sich der Satz von Signalen ändert.

Tabelle 7-146 WriteSignals() - Native C++

Syntax	<code>ERuntimeErrorCode WriteSignals(SDataValueByName* inout_Signals, UINT32 in_SignalCount);</code>	
Parameter	<code>SDataValueByName* inout_Signals</code>: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Die Typen <code>UNSPECIFIC</code> und <code>STRUCT</code> werden nicht unterstützt. <code>UINT32 in_SignalCount</code>: Anzahl der Signale.	
Signalfehler	Fehlercode	Beschreibung
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Offset oder Bits sind ungültig.
Rückgabewerte	Runtime-Fehlercode	Bedingung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>SREC_WRONG_ARGUMENT</code>	Der erwartete Typ ist <code>UNSPECIFIC</code> .

Tabelle 7-147 WriteSignals() - .NET (C#)

Syntax	<code>void WriteSignals(SDataValueByName[] in_Signals)</code>	
Parameter	<code>SDataValueByName</code>: Enthält den Namen, den Wert und den erwarteten Typ der PLC-Variablen. Die Typen <code>UNSPECIFIC</code> und <code>STRUCT</code> werden nicht unterstützt.	
Signalfehler	Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.IndexOutOfRangeException</code>	Offset oder Bits sind ungültig.
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.

Ausnahmen	<code>ERuntimeErrorCode.DoesNotExist</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>ERuntimeErrorCode.NotSupported</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>ERuntimeErrorCode.TypeMismatch</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 249).
	<code>ERuntimeErrorCode.NotUpToDate</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der erwartete Typ ist UNSPECIFIC.

WriteString()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-148 WriteString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::WriteString(const WCHAR* in_Tag, const CHAR* in_Value, const UINT32 in_CharsToWrite);</pre>	
Parameter	<code>const WCHAR* in_Tag</code>: Der Name der PLC-Variablen, die geschrieben werden soll. <code>CHAR* in_Value</code>: Enthält den Wert der PLC-Variablen <code>const UINT32 in_CharsToWrite</code>: Anzahl der zu schreibenden Zeichen	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	<code>SREC_DOES_NOT_EXIST</code>	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	<code>SREC_NOT_SUPPORTED</code>	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	<code>SREC_TYPE_MISMATCH</code>	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	<code>SREC_NOT_UP_TO_DATE</code>	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	<code>SREC_BUFFER_OVERFLOW</code>	Die zu schreibenden Zeichen sind größer als der maximale Wert der Variablen.

Tabelle 7-149 WriteString() - .NET (C#)

Syntax	<pre>void WriteString(string in_Tag, string in_Value);</pre>	
Parameter	- string in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - string in_Value: Der Wert, der in die PLC-Variable geschrieben werden soll.	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	ERuntimeErrorCode.BufferOverflow	Die zu schreibenden Zeichen sind größer als der maximale Wert der Variablen.

WriteWString()

Schreibt den Wert einer PLC-Variablen.

Tabelle 7-150 WriteWString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::WriteWString(const WCHAR* in_Tag, const WCHAR* in_Value, const UINT32 in_CharsToWrite,);</pre>	
Parameter	- const WCHAR* in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - const WCHAR* in_Value: Enthält den Wert der PLC-Variablen - const UINT32 in_CharsToWrite: Anzahl der Zeichen, die geschrieben werden sollen.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_INDEX_OUT_OF_RANGE	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.

Rückgabewerte	SREC_DOES_NOT_EXIST	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	SREC_NOT_SUPPORTED	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	SREC_TYPE_MISMATCH	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	SREC_NOT_UP_TO_DATE	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	SREC_BUFFER_OVERFLOW	Die zu schreibenden Zeichen sind größer als der maximale Wert der Variablen.

Tabelle 7-151 WriteWString() - .NET (C#)

Syntax	<pre>void WriteWString(wstring in_Tag, wstring in_Value);</pre>	
Parameter	- wstring in_Tag: Der Name der PLC-Variablen, die geschrieben werden soll. - wstring in_Value: Der Wert, der in die PLC-Variable geschrieben werden soll.	
Ausnahmen	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.InstanceNotRunning	Der Prozess der simulierten PLC-Instanz läuft nicht.
	ERuntimeErrorCode.IndexOutOfRange	Der Offset liegt außerhalb der Bereichsgröße. Kein Wert konnte geschrieben werden.
	ERuntimeErrorCode.DoesNotExist	Der Eintrag existiert nicht in der gespeicherten Variablen-tabelle.
	ERuntimeErrorCode.NotSupported	Der Zugriff auf ganze Strukturen oder Arrays wird nicht unterstützt.
	ERuntimeErrorCode.TypeMismatch	Der erwartete Typ passt nicht zum gespeicherten Typ. Siehe Kompatible primitive Datentypen (Seite 248).
	ERuntimeErrorCode.NotUpToDate	Die gespeicherte Variablen-tabelle muss aktualisiert werden.
	ERuntimeErrorCode.BufferOverflow	Die zu schreibenden Zeichen sind größer als der maximale Wert der Variablen.

GetSystemTime()

Gibt die virtuelle Systemzeit der simulierten PLC-Instanz zurück. Gibt eine leere Struktur zurück, wenn die Funktion fehlschlägt.

Tabelle 7-152 GetSystemTime() - Native C++

Syntax	SYSTEMTIME GetSystemTime();
Parameter	Keine
Rückgabewerte	SYSTEMTIME: Systemzeit der simulierten PLC-Instanz.

SetSystemTime()

Legt die virtuelle Systemzeit der simulierten PLC-Instanz fest. Gültig ist eine Systemzeit zwischen "Jan 1 1970 00:00:00:000" und "Dec 31 2200 23:59:59:999".

Tabelle 7-153 SetSystemTime() - Native C++

Syntax	<code>ERuntimeErrorCode SetSystemTime(SYSTEMTIME in_SystemTime);</code>	
Parameter	SYSTEMTIME in_SystemTime: Systemzeit, die für die simulierte PLC-Instanz festgelegt werden soll.	
Rückgabewerte	Runtime-Fehlercode	Bedingung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der Wert befindet sich außerhalb der Grenzen.

SystemTime { get; set; }

Legt die virtuelle Systemzeit der simulierten PLC-Instanz fest oder gibt sie zurück. Gültig ist eine Systemzeit zwischen "Jan 1 1970 00:00:00:000" und "Dec 31 2200 23:59:59:999".

Tabelle 7-154 SystemTime { get; set; } - .NET (C#)

Syntax	<code>DateTime SystemTime { get; set; }</code>	
Parameter	Keine	
Rückgabewerte	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Bedingung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	Der Wert befindet sich außerhalb der Grenzen.

GetScaleFactor()

Gibt den Skalierfaktor zurück, mit dem die virtuelle Zeit fortschreitet.

Tabelle 7-155 GetScaleFactor() - Native C++

Syntax	<code>double GetScaleFactor();</code>
Parameter	Keine
Rückgabewerte	double: Skalierfaktor der virtuellen Zeit.

SetScaleFactor()

Setzt den Skalierfaktor, mit dem die virtuelle Zeit fortschreitet.

Siemens empfiehlt, zunächst mit einem kleinen Skalierfaktor zu beginnen und sich schrittweise an einen Skalierfaktor heranzutasten, bei dem die simulierte PLC-Instanz in RUN bleibt.

Gültig ist ein Wert zwischen 0,01 und 100. Die Voreinstellung ist 1.

- Wenn der Wert kleiner als 1 ist, läuft die virtuelle Zeit der simulierten PLC-Instanz x-mal langsamer als die reale Zeit.
- Wenn der Wert größer als 1 ist, läuft die virtuelle Zeit der simulierten PLC-Instanz x-mal schneller als die reale Zeit.

Eine Änderung des Werts zur Laufzeit wird erst am Zykluskontrollpunkt wirksam.

Tabelle 7-156 SetScaleFactor() - Native C++

Syntax	<code>ERuntimeErrorCode SetScaleFactor (double in_Value);</code>	
Parameter	• <code>double in_Value</code>: Skalierfaktor der virtuellen Zeit.	
Rückgabewerte	Runtime-Fehlercode	Bedingung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>SREC_WRONG_ARGUMENT</code>	Der Wert befindet sich außerhalb der Grenzen.

ScaleFactor { get; set; }

Setzt den Skalierfaktor oder gibt den Skalierfaktor zurück, mit dem die virtuelle Zeit fortschreitet.

Beginnen Sie mit einem kleinen Skalierfaktor und tasten Sie sich schrittweise an einen Skalierfaktor heran, bei dem die simulierte PLC-Instanz in RUN bleibt.

Gültig ist ein Wert zwischen 0,01 und 100. Die Voreinstellung ist 1.

- Wenn der Wert kleiner als 1 ist, läuft die virtuelle Zeit der simulierten PLC-Instanz x-mal langsamer als die reale Zeit.
- Wenn der Wert größer als 1 ist, läuft die virtuelle Zeit der simulierten PLC-Instanz x-mal schneller als die reale Zeit.

Eine Änderung des Werts zur Laufzeit wird erst am Zykluskontrollpunkt wirksam.

Tabelle 7-157 ScaleFactor { get; set; } - .NET (C#)

Syntax	<code>double ScaleFactor { get; set; }</code>	
Parameter	Keine	
Rückgabewerte	<code>double</code> : Skalierfaktor der virtuellen Zeit.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Bedingung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Wert befindet sich außerhalb der Grenzen.

7.8 Zykluskontrolle

GetOperatingMode()

Gibt die Betriebsart (Seite 243) der simulierten PLC-Instanz an.

Tabelle 7-158 GetOperatingMode() - Native C++

Syntax	<code>EOperatingMode GetOperatingMode();</code>
Parameter	Keine
Rückgabewerte	<code>EOperatingMode</code> : Betriebsart der simulierten PLC-Instanz

SetOperatingMode()

Legt die Betriebsart der simulierten PLC-Instanz fest.

Eine Änderung des Werts zur Laufzeit wird erst am Synchronisationspunkt wirksam.

Tabelle 7-159 SetOperatingMode() - Native C++

Syntax	<code>void SetOperatingMode(EOperatingMode in_OperatingMode);</code>
Parameter	<code>EOperatingMode in_OperatingMode</code>: Betriebsart der simulierten PLC-Instanz

OperatingMode { get; set; }

Gibt die Betriebsart der simulierten PLC-Instanz zurück oder legt sie fest.

Eine Änderung des Werts zur Laufzeit wird erst am Synchronisationspunkt wirksam.

Tabelle 7-160 OperatingMode { get; set; } - .NET (C#)

Syntax	<code>EOperatingMode OperatingMode { get; set; }</code>	
Parameter	Keine	
Rückgabewerte	<code>EOperatingMode</code> : Betriebsart der simulierten PLC-Instanz	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

SetSendSyncEventInDefaultModeEnabled()

Setzt den Modus `SendSyncEventInDefault`. In diesem Modus wird in der Betriebsart Default nach jedem Zyklusende das Ereignis `OnSyncPointReached` ausgelöst. Siehe `OnSyncPointReached` (Seite 184).

Tabelle 7-161 SetSendSyncEventInDefaultModeEnabled() - Native C++

Syntax	<code>ERuntimeErrorCode SetSendSyncEventInDefaultModeEnabled(bool in_Enable);</code>	
Parameter	<code>bool in_Enable</code>: Wenn <code>true</code>, dann wird in der Betriebsart Default nach jedem Zyklus das Ereignis <code>OnSyncPointReached</code> ausgelöst.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung

Rückgabewerte	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

IsSendSyncEventInDefaultModeEnabled()

Gibt den Modus `SendSyncEventInDefaultMode` zurück. Wenn die Funktion fehlschlägt, ist der Rückgabewert `false`.

Tabelle 7-162 IsSendSyncEventInDefaultModeEnabled() - Native C++

Syntax	<code>bool IsSendSyncEventInDefaultModeEnabled();</code>
Parameter	Keine
Rückgabewerte	<code>false</code>: Das Ereignis wird nicht ausgelöst (außer der Modus Sync-Freeze ist aktiv). <code>true</code>: Das Ereignis wird nach jedem Zyklus ausgelöst.

IsSendSyncEventInDefaultModeEnabled { get; set; }

Gibt den Modus `SendSyncEventInDefaultMode` zurück oder setzt ihn. In diesem Modus wird für jede Betriebsart nach jedem Zyklusende das Ereignis `OnSyncPointReached` ausgelöst. Wenn das Ereignis auch in der Betriebsart `Default` empfangen werden soll, setzen Sie den Rückgabewert auf `true`. Siehe `OnSyncPointReached` ([Seite 184](#)).

Tabelle 7-163 IsSendSyncEventInDefaultModeEnabled { get; set; } - .NET (C#)

Syntax	<code>bool IsSendSyncEventInDefaultModeEnabled { get; set; }</code>	
Parameter	Keine	
Rückgabewerte	<code>false</code>: Das Ereignis wird nicht ausgelöst (außer der Modus Sync-Freeze ist aktiv). <code>true</code>: Das Ereignis wird nach jedem Zyklus ausgelöst.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

GetOverwrittenMinimalCycleTime_ns()

Gibt die überschriebene minimale Zykluszeit (in Nanosekunden) zurück, die in den Betriebsarten `SingleStep_CT` und `SingleStep_CPT` verwendet wird.

Tabelle 7-164 GetOverwrittenMinimalCycleTime_ns() - Native C++

Syntax	<code>INT64 GetOverwrittenMinimalCycleTime_ns();</code>
Parameter	Keine
Rückgabewerte	<code>INT64</code> : Die überschriebene minimale Zykluszeit in Nanosekunden.

SetOverwrittenMinimalCycleTime_ns()

Setzt die überschriebene minimale Zykluszeit (in Nanosekunden), die in den Betriebsarten `SingleStep_CT` und `SingleStep_CPT` verwendet wird.

Gültig ist ein Wert zwischen 0 und 6000000000. Die Voreinstellung ist 100 ms.

Eine Änderung des Werts zur Laufzeit wird erst am Zykluskontrollpunkt wirksam.

Tabelle 7-165 SetOverwrittenMinimalCycleTime_ns() - Native C++

Syntax	ERuntimeErrorCode SetOverwrittenMinimalCycleTime_ns(INT64 in_CycleTime_ns);	
Parameter	INT64 in_CycleTime_ns: Die überschriebene minimale Zykluszeit in Nanosekunden.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der Wert befindet sich außerhalb der Grenzen.

OverwrittenMinimalCycleTime_ns { get; set; }

Gibt die überschriebene minimale Zykluszeit in Nanosekunden zurück oder setzt diese Zykluszeit, die in den Betriebsarten SingleStep_CT und SingleStep_CPT verwendet wird.

Gültig ist ein Wert zwischen 0 und 6000000000. Die Voreinstellung ist 100 ms.

Eine Änderung des Werts zur Laufzeit wird erst am Zykluskontrollpunkt wirksam.

Tabelle 7-166 OverwrittenMinimalCycleTime_ns { get; set; } - .NET (C#)

Syntax	Int64 OverwrittenMinimalCycleTime_ns { get; set; }	
Parameter	Keine	
Rückgabewerte	Int64: Die überschriebene minimale Zykluszeit in Nanosekunden.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.
	ERuntimeErrorCode.WrongArgument	Der Wert befindet sich außerhalb der Grenzen.

RunToNextSyncPoint()

Wenn die simulierte PLC-Instanz in einer SingleStep-Betriebsart läuft, wird sie am Synchronisationspunkt angehalten (Freeze-Zustand). Die Funktion RunToNextSyncPoint() hebt den Freeze-Zustand auf. Die simulierte PLC-Instanz läuft bis zum nächsten Synchronisationspunkt weiter.

Tabelle 7-167 RunToNextSyncPoint() - Native C++

Syntax	ERuntimeErrorCode RunToNextSyncPoint(); ERuntimeErrorCode RunToNextSyncPoint(UINT32 in_IoSystemId, UINT64 in_BusTimeStamp_ns);	
Parameter	Der Aufruf mit Parametern ist nur für die Betriebsart SingleStep_Bus gültig. - UINT32 in_IoSystemId: Nummer des Peripheriesystems (PROFIBUS 1..32, PROFINET 100..115), das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet werden soll. - UINT64 in_BusTimeStamp_ns: Zeitstempel in Nanosekunden des jeweiligen Peripheriesystems zum jeweiligen Zyklusbeginn.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.

Rückgabewerte	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INVALID_OPERATING_STATE	Die Instanz ist in einem Betriebszustand, der für die Verarbeitung des Aufrufs nicht gültig ist (BOOTING, SHUTTING_DOWN, OFF)
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_WRONG_ARGUMENT	Für die Betriebsart <code>SingleStep_Bus</code> : Die ID des Peripheriesystems ist ungültig. Für alle anderen Betriebsarten gilt: Der Funktionsaufruf mit Parametern ist ungültig.

Tabelle 7-168 RunToNextSyncPoint() - .NET (C#)

Syntax	<pre>void RunToNextSyncPoint(); void RunToNextSyncPoint(UINT32 in_IoSystemId, UINT64 in_BusTimeStamp_ns);</pre>	
Parameter	Der Aufruf mit Parametern ist nur für die Betriebsart <code>SingleStep_Bus</code> gültig. <code>UINT32 in_IoSystemId</code>: Nummer des Peripheriesystems (PROFIBUS 1..32, PROFINET 100..115), das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet werden soll. <code>UINT64 in_BusTimeStamp_ns</code>: Zeitstempel in Nanosekunden des jeweiligen Peripheriesystems zum jeweiligen Zyklusbeginn.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InvalidOperatingState</code>	Die Instanz ist in einem Betriebszustand, der für die Verarbeitung des Aufrufs nicht gültig ist (BOOTING, SHUTTING_DOWN, OFF)
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.WrongArgument</code>	Für die Betriebsart <code>SingleStep_Bus</code> : Die ID des Peripheriesystems ist ungültig. Für alle anderen Betriebsarten gilt: Der Funktionsaufruf mit Parametern ist ungültig.

StartProcessing()

Wenn die simulierte PLC-Instanz in einer `TimespanSynchronized`-Betriebsart läuft, wird sie am Synchronisationspunkt angehalten (Freeze-Zustand). Die Funktion `StartProcessing()` hebt den Freeze-Zustand auf. Die simulierte PLC-Instanz läuft nun mindestens so lange wie angefordert, bevor sie beim nächsten Synchronisationspunkt wieder in den Freeze-Zustand wechselt.

Tabelle 7-169 StartProcessing() - Native C++

Syntax	<code>ERuntimeErrorCode StartProcessing(INT64 in_MinimalTimeToRun_ns);</code>	
Parameter	<code>INT64 in_MinimalTimeToRun_ns</code>: Die minimale virtuelle Zeit (in Nanosekunden), die die simulierte PLC-Instanz läuft, bevor sie in den Freeze-Zustand wechselt.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.

Rückgabewerte	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_INSTANCE_NOT_RUNNING	Der Prozess der simulierten PLC-Instanz läuft nicht.
	SREC_WRONG_ARGUMENT	Der Wert ist kleiner als 0.

Tabelle 7-170 StartProcessing() - .NET (C#)

Syntax	<code>void StartProcessing(Int64 in_MinimalTimeToRun_ns);</code>	
Parameter	<code>Int64 in_MinimalTimeToRun_ns</code>: Die minimale virtuelle Zeit (in Nanosekunden), die die simulierte PLC-Instanz läuft, bevor sie in den Freeze-Zustand wechselt.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	Der Prozess der simulierten PLC-Instanz läuft nicht.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Wert ist kleiner als 0.

Weitere Informationen

Weitere Informationen siehe Kapitel Virtuelle und reale Zeit ([Seite 152](#)), Simulation anhalten ([Seite 91](#)).

7.9 Virtuelle und reale Zeit

Die simulierte PLC-Instanz verwendet für die Simulation zwei Arten interner Zeitgeber: Eine virtuelle und eine reale Uhr. Basis für das Anwenderprogramm ist immer die virtuelle Uhr. Sie wird von Komponenten genutzt, die für den Ablauf des STEP 7-Anwenderprogramms relevant sind, wie zyklische OBs, Zykluszeitüberwachung, minimale Zykluszeit, virtuelle Systemzeit und Zeitberechnungen. Auch die Dauer zwischen zwei Zykluskontrollpunkten oder Einzelschritt-Kontrollpunkten wird in virtueller Uhrzeit gemessen.

Die virtuelle Uhrzeit kann zu Simulations- und Optimierungszwecken beschleunigt oder verlangsamt werden.

Die reale Uhr läuft immer unverändert. Sie wird von Komponenten genutzt, die nicht von Steuerungsprozessen abhängig sind, z. B. die Kommunikation mit STEP 7.

Unterbrechung des Prozesses

Da S7-PLCSIM Advanced in einer Windows-Umgebung läuft, kann es vorkommen, dass Windows den Prozess der simulierten PLC-Instanz vorübergehend unterbricht. In diesem Fall werden sowohl die virtuelle als auch die reale Uhr in der simulierten PLC-Instanz angehalten. Sie laufen erst dann weiter, wenn Windows die Bearbeitung des Prozesses wieder fortsetzt.

Virtuelle Systemzeit

Beim Starten von S7-PLCSIM Advanced startet die virtuelle Systemzeit der Instanz mit der Systemzeit von Windows.

Die virtuelle Systemzeit basiert auf der virtuellen Uhr. Wenn ein Skalierfaktor verwendet wird, dann läuft die Systemzeit entsprechend schneller oder langsamer.

Alle Ereignisse aus der simulierten PLC-Instanz weisen einen Zeitstempel auf, der auf der Systemzeit basiert.

HINWEIS

Unterschied Systemzeit und Lokalzeit

- Systemzeit: UTC $\pm$ 0 ohne Sommerzeit / Winterzeit
 - Lokalzeit: UTC $\pm$ Zeitzone mit Sommerzeit / Normalzeit
-

Zeitversatz

HINWEIS

Beachten Sie, dass die Uhrzeitangaben von virtueller Systemzeit und realer Lokalzeit sich um den Zeitversatz unterscheiden, der sich zusätzlich zum gewählten Skalierfaktor aus dem Zeitzeonen-Versatz und dem Sommer-/Winterzeit-Versatz bildet.

Skalierfaktor

Mit einem Skalierungsfaktor können Sie die virtuelle Uhr der simulierten PLC-Instanz für Simulationen beschleunigen oder verlangsamen.

Um den gewünschten Skalierfaktor einzustellen, aktivieren Sie das ausgegraute Symbol  auf dem Control Panel rechts neben Ihrer S7-PLCSIM Advanced-Instanz. Das Symbol wird daraufhin aktiv  und Sie können die Skalierfunktion verwenden.

- Die Voreinstellung ist 1, damit entspricht der Verlauf der virtuellen Zeit dem Verlauf der realen Zeit.
- **Zeitraffer:** Ein Skalierfaktor größer 1 beschleunigt die virtuelle Uhr.
Beispiel: Skalierfaktor 2.0 → Die virtuelle Zeit läuft zweimal so schnell.
- **Zeitlupe:** Ein Skalierfaktor kleiner 1 verlangsamt die virtuelle Uhr.
Beispiel: Skalierfaktor 0,5 → Der Fortschritt der virtuellen Zeit verlangsamt sich auf 50 %.

SetCycleTimeMonitoringMode()

Mit dieser Funktion können Sie die Quelle für den Timer zur maximalen Zykluszeitüberwachung ändern.

Tabelle 7-171 SetCycleTimeMonitoringMode() - Native C++

Syntax	<pre>ERuntimeErrorCode SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode) ERuntimeErrorCode SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode, INT64 in_MaxCycleTime_ns)</pre>
--------	--

Parameter	- ECycleTimeMonitoringMode in CycleTimeMonitoringMode: Wählen Sie eine der folgenden Optionen für die maximale Zykluszeitüberwachung: - SRCTMM_DOWNLOADED: Als maximale Zykluszeitüberwachung wird die maximale Zykluszeit aus dem Projekt verwendet, das aus STEP 7 heruntergeladen wurde. - SRCTMM_IGNORED (Voreinstellung): Als maximale Zykluszeitüberwachung wird ein Timer-Wert von einer Minute verwendet, um einen möglichen Fehler beim Überlauf zyklischer Ereignisse zu verhindern. Siehe Überwachung Überlauf (Seite 155). - SRCTMM_SPECIFIED: Als maximale Zykluszeitüberwachung wird ein Wert verwendet, der über den Parameter in_MaxCycleTime_ns angegeben werden kann. Voreinstellung: 150 ms. - INT64 in_MaxCycleTime_ns: Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung. Gültig ist ein Wert zwischen 1000000 und 60000000000 ns (1 Millisekunde bis 1 Minute). Wenn in der API kein Wert angegeben wird, gilt die Voreinstellung von 150 ms.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.
	SREC_WRONG_ARGUMENT	Der Modus für die Zykluszeitüberwachung ist ungültig.
	SREC_INDEX_OUT_OF_RANGE	Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung befindet sich außerhalb der Grenzen.

Tabelle 7-172 SetCycleTimeMonitoringMode() - .NET (C#)

Syntax	<pre>void SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in CycleTimeMonitoringMode) void SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in CycleTimeMonitoringMode, Int64 in_MaxCycleTime_ns)</pre>	
Parameter	- ECycleTimeMonitoringMode in_CycleTimeMonitoringMode: Wählen Sie eine der folgenden Optionen für die maximale Zykluszeitüberwachung: - ECycleTimeMonitoringMode.Downloaded: Als maximale Zykluszeitüberwachung wird die maximale Zykluszeit aus dem Projekt verwendet, das aus STEP 7 heruntergeladen wurde. - ECycleTimeMonitoringMode.Ignored (Voreinstellung): Als maximale Zykluszeitüberwachung wird ein Timer-Wert von einer Minute verwendet, um einen möglichen Fehler beim Überlauf zyklischer Ereignisse zu verhindern. Siehe Überwachung Überlauf (Seite 155). - ECycleTimeMonitoringMode.Specified: Als maximale Zykluszeitüberwachung wird ein Wert verwendet, der über den Parameter in_MaxCycleTime_ns angegeben werden kann. Voreinstellung: 150 ms. - Int64 in_MaxCycleTime_ns: Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung. Gültig ist ein Wert zwischen 1000000 und 60000000000 ns (1 Millisekunde bis 1 Minute). Wenn in der API kein Wert angegeben wird, gilt die Voreinstellung von 150 ms.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.

Ausnahmen	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Modus für die Zykluszeitüberwachung ist ungültig.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung befindet sich außerhalb der Grenzen.

GetCycleTimeMonitoringMode()

Diese Funktion gibt eine Information über die Quelle für den Timer zur maximalen Zykluszeitüberwachung zurück.

Tabelle 7-173 GetCycleTimeMonitoringMode() - Native C++

Syntax	<code>ERuntimeErrorCode GetCycleTimeMonitoringMode(ECycleTimeMonitoringMode* out_CycleTimeMonitoringMode, INT64* out_MaxCycleTime_ns)</code>	
Parameter	<code>ECycleTimeMonitoringMode* out_CycleTimeMonitoringMode</code>: Der konfigurierte Modus für die Zykluszeitüberwachung. Die Voreinstellung ist <code>SRCTM_IGNORED</code>. <code>INT64 in_MaxCycleTime_ns</code>: Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung. Wenn in der API kein Wert angegeben ist, wird der voreingestellte Wert 150 ms zurückgegeben.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-174 GetCycleTimeMonitoringMode() - .NET (C#)

Syntax	<code>void GetCycleTimeMonitoringMode(out ECycleTimeMonitoringMode out_CycleTimeMonitoringMode, out Int64 out_MaxCycleTime_ns)</code>	
Parameter	<code>ECycleTimeMonitoringMode out_CycleTimeMonitoringMode</code>: Der konfigurierte Modus für die Zykluszeitüberwachung. Die Voreinstellung ist <code>ECycleTimeMonitoringMode.Ignored</code>. <code>Int64 in_MaxCycleTime_ns</code>: Der anwenderspezifische Wert für die maximale Zykluszeitüberwachung. Wenn in der API kein Wert angegeben ist, wird der voreingestellte Wert 150 ms zurückgegeben.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.10 Überwachung Überlauf

Überwachung Hauptzyklus

Die maximale Zykluszeitüberwachung beträgt für S7-PLCSIM Advanced eine Minute.

Überwachung zyklischer Ereignisse

Wenn Ihre Simulation Weckalarme enthält, kann die Warteschlange von S7-PLCSIM Advanced für zyklische Ereignisse überlaufen. Aufgrund der Ablaufgeschwindigkeit von

S7-PLCSIM Advanced im Vergleich zu realer Hardware ist die benötigte Zeit zur Erstellung des Diagnosepuffereintrags möglicherweise länger als die Zeit bis zum nächsten Weckalarm. Wenn dies der Fall ist, wird ein zusätzlicher Eintrag in die Warteschlange gestellt, der einen weiteren Überlauf verursacht. Bei einem Überlauf gibt S7-PLCSIM Advanced Diagnosepuffermeldungen aus und zeigt ein rotes Fehlersymbol im Projektbaum in STEP 7.

7.11 Azyklische Dienste

7.11.1 Übersicht

- Die azyklischen Dienste umfassen bei S7-PLCSIM Advanced:
- Lese- und Schreibvorgänge von Parameter- und Status-Daten vom Anwenderprogramm der PLC in die Peripheriemodule
 - Alarm- und Ereignismeldungen, die die Peripheriemodule an die CPU senden.

Lese- und Schreibvorgänge

Vom STEP 7-Anwenderprogramm getriggerte Ereignisse, die sich für die Benachrichtigung angemeldet haben:

Tabelle 7-175 Ereignisse: Lese- und Schreibvorgänge

SFB	Name	API-Methode (Meldung)	API-Ereignis zum Triggern des SFB
52	RDREC	ReadRecordDone (Seite 158)	OnDataRecordRead (Seite 189)
53	WRREC	WriteRecordDone (Seite 158)	OnDataRecordWrite (Seite 189)

Ablaufschema

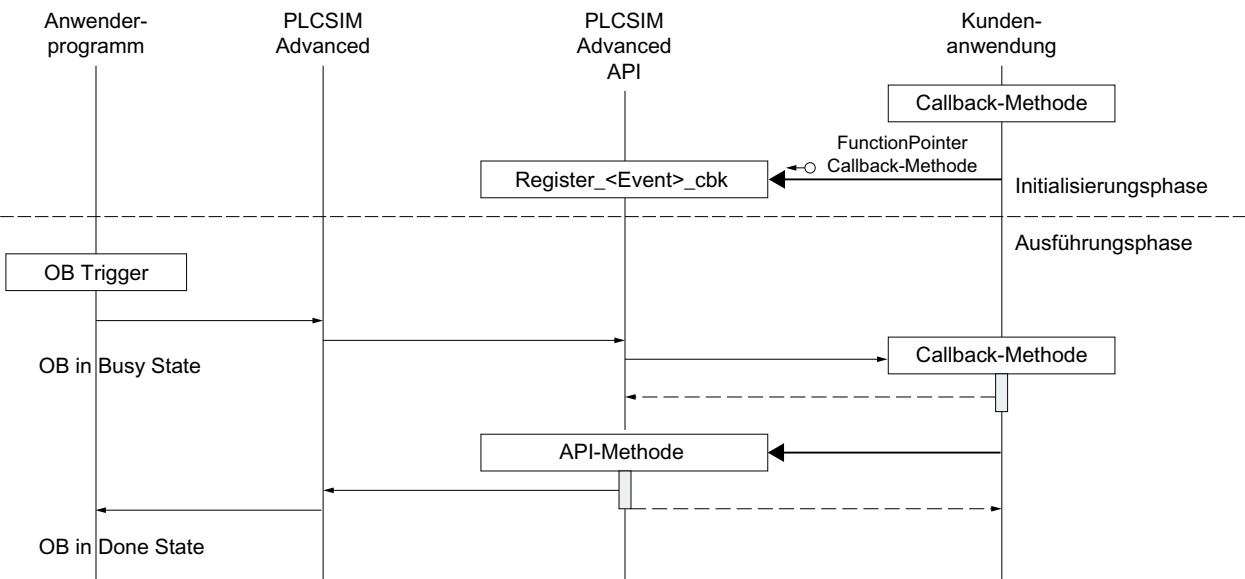


Bild 7-3 Ablaufschema Lese- und Schreibvorgänge

API-Methoden und zugehörige Ereignisse

Ereignisse, die von Peripheriemodulen ausgelöst werden, und zugehörige API-Methoden:

Tabelle 7-176 API-Methoden und zugehörige Ereignisse

OB	Name	API-Methode zum Triggern des OB (Anfrage)	API-Ereignis nach OB-Ausführung (Meldung)
82	Diagnostic error Interrupt	AlarmNotification (Seite 160)	OnAlarmNotificationDone (Seite 190)
4x	Hardware Interrupt	ProcessEvent (Seite 162)	OnProcessEventDone (Seite 191)
83	Pull or Plug of module	PullOrPlugEvent (Seite 163)	OnPullOrPlugEventDone (Seite 192)
55	Status	StatusEvent (Seite 165)	OnStatusEventDone (Seite 193)
57	Profile	ProfileEvent (Seite 166)	OnProfileEventDone (Seite 194)
56	Update	UpdateEvent (Seite 167)	OnUpdateEventDone (Seite 195)
86	Rack or station failure	RackOrStationFaultEvent (Seite 168)	OnRackOrStationFaultEventDone (Seite 196)

Ablaufschema

Ablaufschema bei der Simulation von Ereignissen, die von Peripheriemodulen ausgelöst werden.

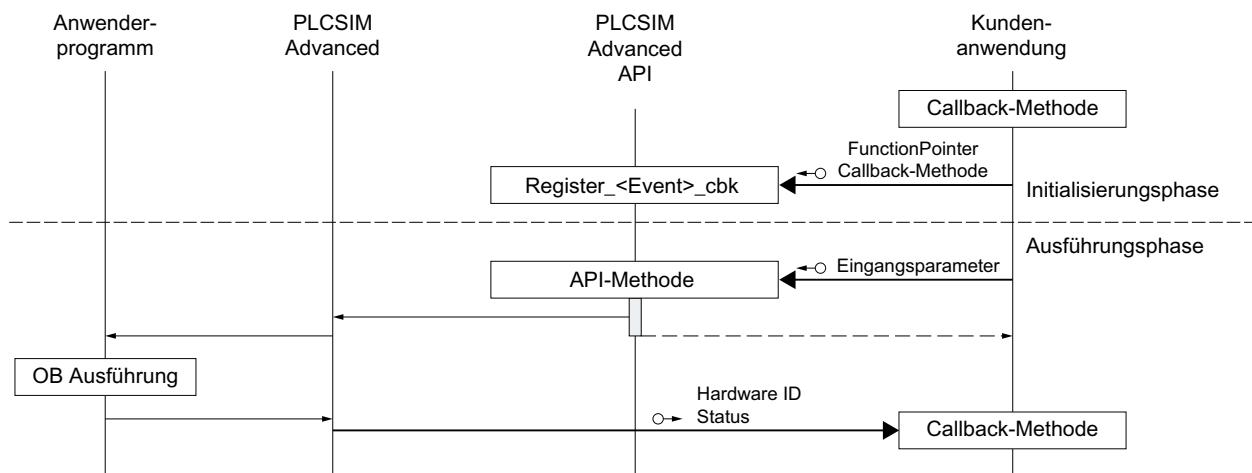


Bild 7-4 Ablaufschema bei der Simulation von Ereignissen

7.11.2 ReadRecordDone/WriteRecordDone

ReadRecordDone()

Mit dieser API-Methode meldet die Simulation eines E/A-Moduls an die CPU, dass das asynchrone Lesen eines Datensatzes abgeschlossen ist. Die Simulation stellt dabei die gelesenen Informationen zur Verfügung.

Tabelle 7-177 ReadRecordDone() - Native C++

Syntax	<code>ERuntimeErrorCode ReadRecordDone(SDataRecordInfo in_RecordInfo, BYTE* in_Data, UINT32 in_Status);</code>	
Parameter	<code>SDataRecordInfo in_RecordInfo</code>: Struktur, die die Datensatz-Information enthält. Siehe <code>SDataRecordInfo</code> (Seite 234). <code>BYTE* in_Data</code>: Byte-Array des gelesenen Datensatzes mit der Länge, die definiert ist über <code>DataSize</code> in der Struktur <code>SDataRecordInfo</code>. <code>UINT32 in_Status</code>: Status der Auftragsausführung	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_DOES_NOT_EXIST</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Das Byte-Array des gelesenen Datensatzes überschreitet die Länge <code>DDATARECORD_MAX_SIZE = 64000</code> .
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-178 ReadRecordDone() - .NET (C#)

Syntax	<code>void ReadRecordDone(SDataRecordInfo in_RecordInfo, BYTE[] in_Data, UInt32 in_Status);</code>	
Parameter	<code>SDataRecordInfo in_RecordInfo</code>: Struktur, die die Datensatz-Information enthält. Siehe <code>SDataRecordInfo</code> (Seite 234). <code>BYTE[] in_Data</code>: Byte-Array des gelesenen Datensatzes mit der Länge, die definiert ist über <code>DataSize</code> in der Struktur <code>SDataRecordInfo</code>. <code>UInt32 in_Status</code>: Status der Auftragsausführung	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.IndexOutOfRangeException</code>	Das Byte-Array des gelesenen Datensatzes überschreitet die Länge <code>DataRecordMaxSize = 64000</code> .
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

WriteRecordDone()

Mit dieser API-Methode meldet die Simulation eines E/A-Moduls an die CPU, dass das asynchrone Schreiben eines Datensatzes abgeschlossen ist.

Tabelle 7-179 WriteRecordDone() - Native C++

Syntax	<code>ERuntimeErrorCode WriteRecordDone(SDataRecordInfo in_RecordInfo, UINT32 in_Status);</code>	
Parameter	<code>SDataRecordInfo in_RecordInfo</code>: Struktur, die die Datensatz-Information enthält. Siehe <code>SDataRecordInfo</code> (Seite 234). <code>UINT32 in_Status</code>: Status der Auftragsausführung	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_DOES_NOT_EXIST</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-180 WriteRecordDone() - .NET (C#)

Syntax	<code>void WriteRecordDone(SDataRecordInfo in_RecordInfo, UInt32 in_Status);</code>	
Parameter	<code>SDataRecordInfo in_RecordInfo</code>: Struktur, die die Datensatz-Information enthält. Siehe <code>SDataRecordInfo</code> (Seite 234). <code>UInt32 in_Status</code>: Status der Auftragsausführung	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

HINWEIS**Rückgabewerte eines geschriebenen Datensatzes**

Die Rückgabewerte des Status eines geschriebenen Datensatzes können bei S7-PLCSIM Advanced von den Rückgabewerten einer realen Hardware-CPU abweichen.

7.11.3 AlarmNotification

AlarmNotification()

Diese Funktion löst Diagnosemeldungen nach PROFINET-Standard aus.

Jeder Aufruf dieser Funktion ruft den Diagnosealarm-OB (OB 82) einmal auf, unabhängig von der Anzahl und vom Schweregrad der übermittelten Diagnoseeinträge.

Tabelle 7-181 AlarmNotification() - Native C++

Syntax	ERuntimeErrorCode AlarmNotification(UINT16 in_HardwareIdentifizier, UINT16 in_ModuleState, UINT16 in_NumberOfDiagnosisEvents, SDiagExtChannelDescription* in_ArrayOfDiagnosisEvents, UINT16* out_SequenceNumber);	
Parameter	• UINT16 in_HardwareIdentifizier: Die HW-Kennung des Moduls oder Submoduls, das den Diagnoseeintrag sendet. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. • UINT16 in_ModuleState: Modulzustand. Gültig sind folgende Zustände: DMODULE_STATE_OK = 0, DMODULE_STATE_ERROR = 1, DMODULE_STATE_MAINT_DEMANDED = 2, DMODULE_STATE_MAINT_REQUIRED = 4 Der Parameter in_ModuleState ergibt sich aus der Summe (Veroderung) der Schweregrade im Feld SDiagExtChannelDescription. Wenn z. B. für ein Modul sowohl bei "Wartungsanforderung" als auch bei "Wartungsbedarf" ein Diagnosealarm generiert werden soll, dann wählen Sie als Modulstatus "6". • UINT16 in_NumberOfDiagnosisEvents: Mehrere Diagnoseeinträge können mit einem einzigen API-Aufruf an die CPU gesendet werden. Gültiger Bereich: 0 bis 16. 0 bedeutet, dass kein Diagnoseeintrag für das Submodul oder den Kanal erscheinen soll. • SDiagExtChannelDescription* in_ArrayOfDiagnosisEvents: Zeiger auf ein Feld mit Diagnoseeinträgen. Das Feld muss mit der Anzahl der Diagnoseeinträge übereinstimmen. Es kann auch ein Null-Zeiger sein. Definitionen siehe SDiagExtChannelDescription (Seite 236). • UINT16* out_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_STATE	Das Modul ist momentan gezogen.
	SREC_DOES_NOT_EXIST	Die HW-Kennung des Moduls existiert nicht.
	SREC_WRONG_MODULE_TYPE	Die Kanalnummer existiert für das Modul nicht.
	SREC_WRONG_ARGUMENT	Der Wert für den Modulzustand ist ungültig.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-182 AlarmNotification() - .NET (C#)

Syntax	<pre>void AlarmNotification(ushort in_HardwareIdentifier, ushort in_ModuleState, ushort in_NumberOfDiagnosisEvents, SDiagExtChannelDescription [] in_ArrayOfDiagnosisEvents, Out ushort out_SequenceNumber);</pre>	
Parameter	• <code>ushort in_HardwareIdentifier</code>: Die HW-Kennung des Moduls oder Submoduls, das den Diagnoseeintrag sendet. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. • <code>ushort in_ModuleState</code>: Modulzustand. Gültig sind folgende Zustände: <code>ModuleState.Ok = 0</code>, <code>ModuleState.Error = 1</code>, <code>ModuleState.MaintenanceDemanded = 2</code>, <code>ModuleState.MaintenanceRequired = 4</code> Der Parameter <code>in_ModuleState</code> ergibt sich aus der Summe (Veroderung) der Schweregrade im Feld <code>SDiagExtChannelDescription</code>. Wenn z. B. für ein Modul sowohl bei "Wartungsanforderung" als auch bei "Wartungsbedarf" ein Diagnosealarm generiert werden soll, dann wählen Sie als Modulstatus "6". • <code>ushort in_NumberOfDiagnosisEvents</code>: Mehrere Diagnoseeinträge können mit einem einzigen API-Aufruf an die CPU gesendet werden. Gültiger Bereich: 0 bis 16. 0 bedeutet, dass kein Diagnoseeintrag für das Submodul oder den Kanal erscheinen soll. • <code>SDiagExtChannelDescription [] in_ArrayOfDiagnosisEvents</code>: Zeiger auf ein Feld mit Diagnoseeinträgen. Das Feld muss mit der Anzahl der Diagnoseeinträge übereinstimmen. Es kann auch ein Null-Zeiger sein. Definitionen siehe <code>SDiagExtChannelDescription</code> (Seite 236). • <code>Out ushort out_SequenceNumber</code>: S7-PLCSIM Advanced weist jedem Alarm-Ereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_h). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Modul ist momentan gezogen.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.WrongArgument</code>	Der Wert für den Modulzustand ist ungültig.
	<code>ERuntimeErrorCode.WrongModuleType</code>	Die Kanalnummer existiert für das Modul nicht.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.
Beispiel	<pre>ushort seqNumber; var In_ArrayOfDiagnosisEvent = new SDiagExtChannelDescription[] { new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0001, ExtErrorType = 0, Direction = EDiagProperty.Appear,Severity =EDiagSeverity.MaintDemanded}, new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0002, ExtErrorType = 0, Direction = EDiagProperty.Appear,Severity =EDiagSeverity.Failure}, new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0003, ExtErrorType = 0, Direction = EDiagProperty.Appear,Severity =EDiagSeverity.MaintRequired}, Instance.AlarmNotification(269, 7, 3, In_ArrayOfDiagnosisEvent, out seqNumber);</pre>	

```
//Parameter ModuleState ist Summe der Schweregrade im Array
S DiagExtChannelDescription oben: 4+2+1
```

7.11.4 ProcessEvent

ProcessEvent()

Mit dieser Funktion können Prozessalarme von zentralen und dezentralen Eingangsmodulen simuliert werden.

Tabelle 7-183 ProcessEvent() - Native C++

Syntax	ERuntimeErrorCode ProcessEvent(UINT16 in_HardwareIdentifier, UINT16 in_Channel, EProcessEvent in_ProcessEventType, UINT16* out_SequenceNumber);	
Parameter	- UINT16 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Prozessereignis sendet. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. - UINT16 in_Channel: Der Kanal des E/A-Moduls, das das Prozessereignis sendet. - EProcessEvent in_ProcessEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module; siehe EProcessEvent (Seite 255). - UINT16* out_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_STATE	Das Modul ist momentan gezogen.
	SREC_DOES_NOT_EXIST	Die HW-Kennung des Moduls existiert nicht.
	SREC_NOT_SUPPORTED_BY_MODULE	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-184 ProcessEvent() - .NET (C#)

Syntax	void ProcessEvent(ushort in_HardwareIdentifier, ushort in_Channel, EProcessEvent in_ProcessEventType, Out ushort out_SequenceNumber);	
Parameter	- ushort in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Prozessereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. - ushort in_Channel: Der Kanal des E/A-Moduls, das das Prozessereignis generiert. - EProcessEvent in_ProcessEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module; siehe EProcessEvent (Seite 255). - Out ushort out_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.

Ausnahmen	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Modul ist momentan gezogen.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Priorität beim Prozessalarm-OB

Die Prozessalarme, die über die S7-PLCSIM Advanced API ausgelöst werden, werden sequenziell in das Anwenderprogramm übertragen.

Nur wenn Ereignisse gleichzeitig eintreffen, entscheidet die Priorität des zugeordneten Prozessalarm-OBs über die Reihenfolge der Ausführung.

7.11.5 PullOrPlugEvent

PullOrPlugEvent()

Diese Funktion löst Ziehen/Stecken-Ereignisse aus. Für diese Ereignisse wird der Alarm-OB (OB 83) "Pull or plug of modules" ausgeführt.

HINWEIS

Ziehen und Stecken von Kopfbaugruppen dezentraler Peripheriesysteme und von R1-Geräten

Eine reale PLC kann kein Ziehen oder Stecken einer Kopfbaugruppe in einem dezentralen Peripheriesystem erkennen, außer bei R1-Geräten.

Im Unterschied dazu lässt eine simulierte PLC in S7-PLCSIM Advanced das Ziehen und Stecken von Kopfbaugruppen in dezentralen Peripheriesystemen und bei R1-Geräten zu. Ihr Programm kann dieses Ereignis für einen derartigen Fall nutzen, es entspricht aber keinem Ereignis, das bei einer realen PLC vorkommen würde.

HINWEIS

PN/PC Coupler-Geräte

Bei einem physischen PN/PN Coupler-Gerät kann das Modul nicht abgezogen oder angeschlossen werden. S7-PLCSIM Advanced ermöglicht das Abziehen und Anschließen von PN/PN Couplern, wenn Proxy HwID als Eingabe für Ereignisse des Abziehens und Anschließens verwendet wird.

Tabelle 7-185 PullOrPlugEvent() - Native C++

Syntax	<code>ERuntimeErrorCode PullOrPlugEvent(UINT16 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UINT16* out_SequenceNumber);</code>
Parameter	<code>UINT16 in_HardwareIdentifier</code>: Die HW-Kennung des Moduls oder Submoduls, das das Ziehen/Stecken-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören.

7.11 Azyklische Dienste

	- EPullOrPlugEventType in PullOrPlugEventType: Ein Wert aus der Liste der vordefinierten Typen von Ziehen/Stecken-Ereignissen; siehe EPullOrPlugEventType (Seite 255). - UINT16* out_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_STATE	Das Modul ist momentan gezogen.
	SREC_WRONG_MODULE_TYPE	Der falsche Modultyp wurde gewählt, z. B. wenn ein Onboard-E/A einer Kompakt-CPU gezogen werden soll.
	SREC_NOT_SUPPORTED_BY_MODULE	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	SREC_DOES_NOT_EXIST	Die HW-Kennung des Moduls existiert nicht.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-186 PullOrPlugEvent() - .NET (C#)

Syntax	void PullOrPlugEvent(ushort in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, Out ushort out_SequenceNumber);	
Parameter	- ushort in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Ziehen/Stecken-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. - EPullOrPlugEventType in_PullOrPlugEventType: Ein Wert aus der Liste der vordefinierten Typen von Ziehen/Stecken-Ereignissen; siehe EPullOrPlugEventType (Seite 255). - Out ushort out_SequenceNumber S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1.	
Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Instanz ist im Runtime Manager nicht registriert.
	ERuntimeErrorCode.WrongModuleState	Das Modul ist momentan gezogen.
	ERuntimeErrorCode.WrongModuleType	Der falsche Modultyp wurde gewählt, z. B. wenn ein Onboard-E/A einer Kompakt-CPU gezogen werden soll.
	ERuntimeErrorCode.DoesNotExist	Die HW-Kennung des Moduls existiert nicht.
	ERuntimeErrorCode.NotSupportedByModule	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

7.11.6 StatusEvent

StatusEvent()

Diese Funktion dient dazu, den Status-Ereignis-OB (OB 55) auszulösen. Statusereignisse werden nur für Module in einem dezentralen Peripheriesystem unterstützt.

Tabelle 7-187 StatusEvent() - Native C++

Syntax	<code>ERuntimeErrorCode StatusEvent(UINT16 in_HardwareIdentifizier, UINT16 in_Specifizier);</code>	
Parameter	UINT16 in_HardwareIdentifizier: Die HW-Kennung des Moduls, das das Status-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. UINT16 in_Specifizier: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 55-Aufrufs verfügbar.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_STATE	Das Modul ist momentan gezogen.
	SREC_NOT_SUPPORTED_BY_MODULE	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	SREC_DOES_NOT_EXIST	Die HW-Kennung des Moduls existiert nicht.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-188 StatusEvent() - .NET (C#)

Syntax	<code>void StatusEvent(ushort in_HardwareIdentifizier, ushort in_Specifizier);</code>	
Parameter	ushort in_HardwareIdentifizier: Die HW-Kennung des Moduls, das das Status-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. ushort in_Specifizier: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 55-Aufrufs verfügbar.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Modul ist momentan gezogen.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.11.7 ProfileEvent

ProfileEvent()

Diese Funktion wird benutzt, um den Profile-Ereignis OB (OB 57) auszulösen. Profile-Ereignisse werden nur für Module in einem dezentralen E/A-System unterstützt.

Tabelle 7-189 ProfileEvent() - Native C++

Syntax	<code>ERuntimeErrorCode ProfileEvent(UINT16 in_HardwareIdentifizier, UINT16 in_Specifizier);</code>	
Parameter	<code>UINT16 in_HardwareIdentifizier</code>: Die HW-Kennung des Moduls, das das Profile-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. <code>UINT16 in_Specifizier</code>: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 57-Aufrufs verfügbar.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_WRONG_MODULE_STATE</code>	Das Modul ist momentan gezogen.
	<code>SREC_NOT_SUPPORTED_BY_MODULE</code>	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	<code>SREC_DOES_NOT_EXIST</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-190 ProfileEvent() - .NET (C#)

Syntax	<code>void ProfileEvent(ushort in_HardwareIdentifizier, ushort in_Specifizier);</code>	
Parameter	<code>ushort in_HardwareIdentifizier</code>: Die HW-Kennung des Moduls, das das Profile-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. <code>ushort in_Specifizier</code>: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 57-Aufrufs verfügbar.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Modul ist momentan gezogen.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.11.8 UpdateEvent

UpdateEvent()

Diese Funktion wird benutzt, um den Update-Ereignis-OB (OB 56) auszulösen. Update-Ereignisse werden nur für Module in einem dezentralen Peripheriesystem unterstützt.

Tabelle 7-191 UpdateEvent() - Native C++

Syntax	<code>ERuntimeErrorCode UpdateEvent(UINT16 in_HardwareIdentifizier, UINT16 in_Specifizier);</code>	
Parameter	UINT16 in_HardwareIdentifizier: Die HW-Kennung des Moduls, das das Update-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. UINT16 in_Specifizier: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 56-Aufrufs verfügbar.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_STATE	Das Modul ist momentan gezogen.
	SREC_NOT_SUPPORTED_BY_MODULE	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	SREC_DOES_NOT_EXIST	Die HW-Kennung des Moduls existiert nicht.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-192 UpdateEvent() - .NET (C#)

Syntax	<code>void UpdateEvent(ushort in_HardwareIdentifizier, ushort in_Specifizier);</code>	
Parameter	ushort in_HardwareIdentifizier: Die HW-Kennung des Moduls, das das Update-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören. ushort in_Specifizier: Der Parameter wird als Alarm-Specifizier an das Alarm-Telegramm übergeben. Er ist als Eingangsparameter des OB 56-Aufrufs verfügbar.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime-Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Modul ist momentan gezogen.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die HW-Kennung des Moduls existiert nicht.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	Das Modul wird bei dieser Anwenderaktion nicht unterstützt.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.11.9 GetConfiguredProcessEvent

GetConfiguredProcessEvents()

Mit dieser API-Methode können zur Laufzeit die im TIA Portal konfigurierten Prozessalarme ausgelesen werden.

Wenn keine Prozessereignisse vorliegen, wird `SREC_OK` zurückgegeben. Der Wert für `EventsCount` ist dann 0.

Tabelle 7-193 GetConfiguredProcessEvents() - Native C++

Syntax	<code>ERuntimeErrorCode GetConfiguredProcessEvents(UINT16* out_EventsCount,);</code>	
Parameter	<code>SConfiguredProcessEvents* inout_ProcessEvents:</code> Zeiger oder Referenz auf einen benutzerdefinierten Speicher, der das Feld mit den heruntergeladenen konfigurierten Prozessereignissen enthält. Die Struktur <code>SConfiguredProcessEvents</code> (Seite 235) enthält Informationen über diese Prozessereignisse. <code>UINT16* out_EventsCount:</code> Zeiger oder Referenz auf eine Variable, die die Anzahl der konfigurierten Prozessereignisse enthält.	
Rückgabewerte	Runtime Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.
	<code>SREC_INTERFACE_REMOVED</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>SREC_TIMEOUT</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-194 GetConfiguredProcessEvents() - .NET (C#)

Syntax	<code>SConfiguredProcessEvents [] GetConfiguredProcessEvents();</code>	
Parameter	Keine	
Rückgabewerte	Feld mit konfigurierten Prozessereignissen und Feldgröße ergeben die Anzahl der konfigurierten Prozessereignisse.	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

7.11.10 RackOrStationFaultEvent

Beschreibung

Diese Funktion wird benutzt, um den RackOrStationFault-Ereignis OB (OB 86) auszulösen. Diese Ereignisse werden nur bei dezentralen Geräten unterstützt.

Tabelle 7-195 RackOrStationFaultEvent() - Native C++

Syntax	<code>ERuntimeErrorCode RackOrStationFaultEvent(UINT16 in_HardwareIdentifizier, ERackOrStationFaultType in_EventType);</code>	
Parameter	<code>UINT16 in_HardwareIdentifizier:</code> Die HW-Kennung des Geräts, das das Ereignis sendet. Verwenden Sie die HW-Kennung vom Typ <code>Hw_Device</code>. <code>ERackOrStationFaultType in_EventType:</code> Ein Wert aus der Liste der vordefinierten Typen von Ereignissen; siehe <code>ERackOrStationFaultType</code> (Seite 257).	
Rückgabewerte	Runtime Fehlercode	Beschreibung
	<code>SREC_OK</code>	Die Funktion ist erfolgreich.

Rückgabewerte	SREC_INTERFACE_REMOVED	Die Instanz ist im Runtime Manager nicht registriert.
	SREC_WRONG_MODULE_TYPE	Die spezifizierte HW-Kennung ist nicht die eines dezentralen Geräts.
	SREC_WRONG_MODULE_STATE	Das Gerät mit der spezifizierten HW-Kennung meldet bereits den Zustand Fault/Return.
	SREC_DOES_NOT_EXIST	Die spezifizierte HW-Kennung des Geräts existiert nicht.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 7-196 RackOrStationFaultEvent() - .NET (C#)

Syntax	<code>void RackOrStationFaultEvent(ushort in_HardwareIdentifizier, ERackOrStationFaultType in_EventType);</code>	
Parameter	<code>ushort in_HardwareIdentifizier</code>: Die HW-Kennung des Geräts, das das Ereignis sendet. Verwenden Sie die HW-Kennung vom Typ <code>Hw_Device</code>. <code>ERackOrStationFaultType in_EventType</code>: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen; siehe <code>ERackOrStationFaultType</code> (Seite 257).	
Ausnahmen	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime Fehlercode	Beschreibung
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	Die Instanz ist im Runtime Manager nicht registriert.
	<code>ERuntimeErrorCode.DoesNotExist</code>	Die spezifizierte HW-Kennung des Geräts existiert nicht.
	<code>ERuntimeErrorCode.WrongModuleType</code>	Die spezifizierte HW-Kennung ist nicht die eines dezentralen Geräts.
	<code>ERuntimeErrorCode.WrongModuleState</code>	Das Gerät mit der spezifizierten HW-Kennung meldet bereits den Zustand Fault/Return.
	<code>ERuntimeErrorCode.Timeout</code>	Die Funktion kehrt nicht rechtzeitig wieder.

Weitere Informationen

Weitere Informationen zur HW-Kennung finden Sie in der Onlinehilfe STEP 7.

7.12 Ereignisse für Instances

7.12.1 Ereignisse für Betriebszustand und Zykluskontrolle

Für die Schnittstelle Instances werden folgende Ereignisse ausgelöst:

Tabelle 7-197 Ereignisse für Instances

Ereignis	Ursache
OnOperatingStateChanged (Seite 175)	Der Betriebszustand der simulierten PLC-Instanz hat sich geändert.
OnLedChanged (Seite 178)	Die LED-Anzeige der simulierten PLC-Instanz hat sich geändert.
OnSoftwareConfigurationChanged (Seite 181)	Die Konfiguration der simulierten PLC-Instanz ändert sich: • Beim Hochlauf aus der Virtual SIMATIC Memory Card • Zu Beginn eines Downloads Wenn dieses Ereignis ausgelöst wird, wird die gespeicherte Variablentabelle zurückgesetzt.
	Die Konfiguration der simulierten PLC-Instanz hat sich geändert: • Nach dem Hochlauf aus der Virtual SIMATIC Memory Card • Am Ende eines Downloads • Wenn sich die IP-Adresse ändert
OnSyncPointReached (Seite 184)	Die simulierte PLC-Instanz hat einen Synchronisationspunkt erreicht. Wenn die simulierte PLC-Instanz in der Betriebsart <code>Default</code> betrieben wird, muss das Flag <code>SendSyncEventInDefaultMode</code> gesetzt werden, um das Ereignis zu empfangen. Siehe <code>SendSyncEventInDefaultMode</code> (Seite 148).
OnHardwareConfigChanged (Seite 187)	Wenn sich die Hardware-Konfiguration der simulierten PLC geändert hat, wird das Ereignis zu folgenden Zeitpunkten ausgelöst: • Nach dem Hochlauf aus der Virtual SIMATIC Memory Card • Während des Downloads der Hardwarekonfiguration

7.12.1.1 Simulations-Partner synchronisieren

SingleStep-Betriebsarten

Mit den SingleStep-Betriebsarten der simulierten PLC-Instanz werden mehrere Simulationspartner (Clients) mit Hilfe von Synchronisationspunkten synchronisiert. Die Betriebsarten definieren den Synchronisationspunkt, an dem die Instanz in den Freeze-Zustand wechselt und das Ereignis `OnSyncPointReached` sendet.

Tabelle 7-198 Zyklusgesteuerte Betriebsarten (SingleStep)

Betriebsart ¹	Synchronisationspunkt		Minimale Zykluszeit ²	Sendetakt "Bus" ³
	Zykluskontrollpunkt	Vor dem Lesen des Teilprozessabbilds		
	"C"	"P"		
SingleStep_C ⁴	✓			
SingleStep_P		✓		
SingleStep_CP ⁴	✓	✓		
SingleStep_CT ⁴	✓		✓	
SingleStep_CPT ⁴	✓	✓	✓	
SingleStep_Bus	✓			✓

¹ Ein Wechsel von einer SingleStep-Betriebsart zu einer anderen während des Betriebs würde dazu führen, dass der Eingangsparameter `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns` im ersten Callback-Ereignis `SyncPointReached` in der neuen Betriebsart auf 0 gesetzt würde. Wir empfehlen, die SingleStep-Betriebsart nicht während des laufenden Betriebs zu wechseln.

² Außerdem wird die Mindestzykluszeit von OB 1 in dieser Betriebsart überschrieben. Wenn Sie eine Mindestzykluszeit von 200 ms definieren, beträgt der Mindestabstand zwischen zwei Zykluskontrollpunkten 200 virtuelle Millisekunden. Die Voreinstellung ist 100 ms.

³ Sendetakt des Peripheriesystems (PROFIBUS oder PROFINET), der für die zyklusgesteuerte Synchronisation der simulierten PLC-Instanz verwendet werden soll. Sie legen den Sendetakt des entsprechenden Peripheriesystems in den STEP 7-Eigenschaften fest, z. B. in den Eigenschaften der PROFINET-Schnittstelle der CPU (Erweiterte Optionen > Echtzeit-Einstellungen > E/A-Kommunikation > Sendetakt).

⁴ Während der Ausführung dieser Betriebsarten gibt der Eingangsparameter des ersten Callback-Ereignisses für `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns` den Wert 0 zurück. Nachfolgende Callback-Ereignisse für `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns` geben dann Werte ungleich 0 zurück.

API-Funktionen / Ereignisse

- `GetOverwrittenMinimalCycleTime_ns()` (Seite 148)
- `SetOverwrittenMinimalCycleTime_ns()` (Seite 148)
- `OverwrittenMinimalCycleTime_ns { get; set; }` (Seite 148)
- `RunToNextSyncPoint()` (Seite 184)
- `OnSyncPointReached` (Seite 184)
- `EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32` (Seite 211) / `Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32` (Seite 216)

Bedingungen für die Betriebsart `SingleStep_Bus`

Für die Betriebsart `SingleStep_Bus` müssen die folgenden Bedingungen erfüllt sein:

- Sie müssen Ausgänge am Controller projiziert haben.
- Sie müssen im Taktsynchronalarm-OB die Anweisung `SYNC_PO` zur Synchronisation des Prozessabbilds der Ausgänge aufrufen.

HINWEIS

Zusätzlich zum Sendetakt des Busses wird auch der Zykluskontrollpunkt als Synchronisationspunkt verwendet. Im Zykluskontrollpunkt hat die übergebene ID den Wert `0xFFFF`. Die in_Parameter "in_TimeSinceSameSyncPoint_ns" und "in_TimeSinceAnySyncPoint_ns" haben den Wert 0.

Eine Synchronisation von OBs und Teilprozessabbildern ist nur möglich, wenn die OBs die Anweisungen "SYNC_PI" und "SYNC_PO" aufrufen. Damit Sie Teilprozessabbild 1 (TPA 1) für eine F-Ablaufgruppe verwenden können, konfigurieren Sie die "Vor-/Nachverarbeitung der F-Ablaufgruppe" im Safety Administration Editor.

Für die Konfiguration der Vor- und Nachverarbeitung der F-Ablaufgruppe benötigen Sie neue Standard-FCs für "SYNC_PI" und "SYNC_PO" mit den folgenden Eigenschaften:

- Name des FC: `SYNC_PI`
`#RET_VAL_PI := SYNC_PI(PART := "TPA 1", FLADDR => #FLADDR_PI);`
- Name des FC: `SYNC_PO`
`#RET_VAL_PO := SYNC_PO(PART := "TPA 1", FLADDR => #FLADDR_PO);`

Um das Teilprozessabbild 1 (TPA 1) zu synchronisieren, fügen Sie die FCs im Safety Administration Editor unter "Vor-/Nachverarbeitung der F-Ablaufgruppe" hinzu.

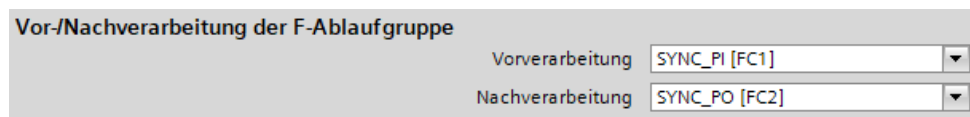


Bild 7-5 Safety Administration Editor

Nach dem Übersetzen dieser Änderungen steht das Teilprozessabbild 1 (TPA 1) auch für den fehlersicheren Organisationsbaustein (FOB) zur Verfügung.

Hinweise zu den Betriebsarten `SingleStep_P`, `SingleStep_Bus` und `SingleStep_CP`

HINWEIS

Verlust eines `OnSyncPointReached` Ereignisses beim Download eines taktsynchronen OBs im Betriebszustand RUN

Wenn Sie in den Betriebsarten `SingleStep_P` und `SingleStep_Bus` auf das Ereignis `OnSyncPointReached` registriert sind, sendet die simulierte PLC-Instanz beim Download eines taktsynchronen OBs dieses Ereignis nicht an den API-Client.

Abhilfe:

1. Verwenden Sie das Ereignis `OnSoftwareConfigurationChanged`, um den Beginn und das Ende eines Downloads zu ermitteln.
2. Registrieren und melden Sie das Ereignis `OnSyncPointReached` ab oder gehen Sie mit dem Verlust des `OnSyncPointReached` Ereignisses entsprechend um.

HINWEIS**Hinweis zur Betriebsart SingleStep_CP**

Herunterladen von Software-Änderungen im Betriebszustand RUN.

Wenn Sie im Betriebszustand RUN in der Betriebsart SingleStep_CP einen Software-Download durchführen, kann es passieren, dass der Download nicht möglich ist und die Simulation stehen bleibt. Grund hierfür kann eine Überlastsituation während des Downloads in RUN sein.

Abhilfe:

1. Verwenden Sie die Funktion "Virtual Time Scaling" und wählen Sie einen niedrigen Skalierungsfaktor (z. B. Skalierungsfaktor 0,25).
2. Stellen Sie ausreichend Ressourcen bereit, um die Leistung der Laufzeit und der Kommunikation von S7-PLCSIM Advanced zu erhöhen.

Freeze-Zustand beenden

Die Funktion `RunToNextSyncPoint()` hebt den Freeze-Zustand auf und veranlasst die simulierte PLC-Instanz, bis zum nächsten Synchronisationspunkt weiterzulaufen.

Auch ein Wechsel in den Betriebszustand Default beendet den Freeze-Zustand.

Beispiel

Die Abbildung zeigt schematisch den Ablauf in der Betriebsart SingleStep_CP.

Neben dem Ereignis `OnSyncPointReached`, sendet die simulierte PLC-Instanz auch die virtuelle Zeit, seit der letzte Synchronisationspunkt derselben oder einer beliebigen Teilprozessabbild-ID erreicht wurde (`TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns`).

Die Funktion `RunToNextSyncPoint()` hebt den Freeze-Zustand auf.

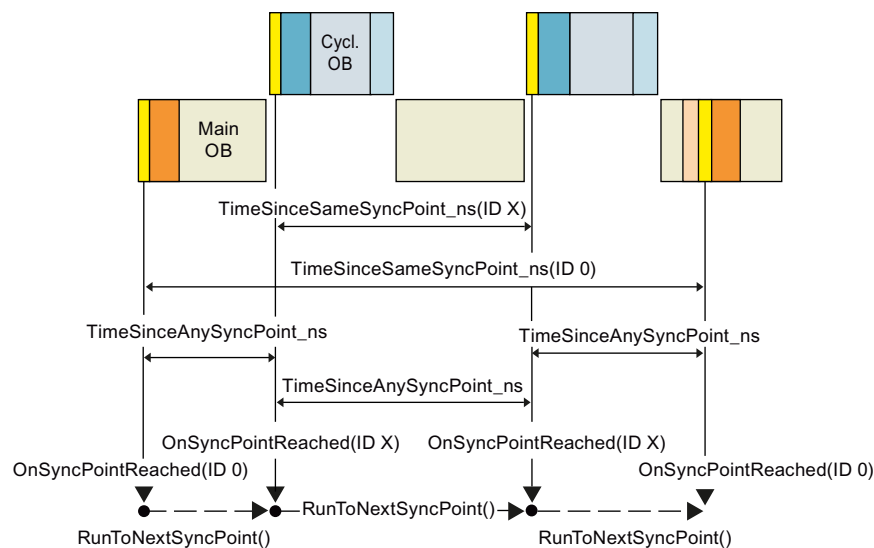


Bild 7-6 Beispiel: Ablauf in der Betriebsart SingleStep_CP

Einstellungen in der Beobachtungstabelle ändern

HINWEIS

Trigger wählen für das Beobachten von Variablen in den SingleStep-Betriebsarten

Im TIA Portal zeigt die Beobachtungstabelle im Basismodus die Werte für Ausgänge und Merker **vor** der Verarbeitung.

Um die Variablenwerte **nach** der Verarbeitung anzuzeigen, gehen Sie folgendermaßen vor:

1. Wählen Sie für die Beobachtungstabelle den erweiterten Modus

2. Wählen Sie in der Spalte "Beobachten mit Trigger" "Zyklusende, permanent".

TimespanSynchronized-Betriebsarten

Mit den TimespanSynchronized-Betriebsarten der simulierten PLC-Instanz werden mehrere Simulations-Partner (Clients) zeitgesteuert synchronisiert. Die Betriebsarten definieren den Synchronisationspunkt, an dem die Instanz in den Freeze-Zustand wechselt und das Ereignis OnSyncPointReached sendet.

Tabelle 7-199 Zeitgesteuerte Betriebsarten (TimespanSynchronized)

Betriebsart	Synchronisationspunkt	
	Zykluskontrollpunkt	Vor dem Lesen des Teilprozessab-bilds
	"C"	"P"
TimespanSynchronized_C	✓	
TimespanSynchronized_CP	✓	✓
TimespanSynchronized_P		✓

- API-Funktionen / Ereignisse
- Einstellungen zur Zykluskontrolle (Seite 148)
 - StartProcessing() (Seite 148)
 - OnSyncPointReached (Seite 184)

Freeze-Zustand beenden

Die Funktion StartProcessing(t) hebt den Freeze-Zustand auf und veranlasst die simulierte PLC-Instanz, mindestens so lange wie angefordert weiterzulaufen (auf Basis der virtuellen Zeit), bevor sie am nächsten Synchronisationspunkt wieder in den Freeze-Zustand geht.

Auch ein Wechsel in den Betriebszustand Default beendet den Freeze-Zustand.

Beispiel

Die Abbildung zeigt schematisch den Ablauf in der Betriebsart TimespanSynchronized_CP.

Neben dem Ereignis OnSyncPointReached sendet die simulierte PLC-Instanz auch die Laufzeit seit dem letzten Aufruf der Funktion StartProcessing(t) (TimeSinceSameSyncPoint_ns / TimeSinceAnySyncPoint_ns).

Die Funktion `StartProcessing()` hebt den Freeze-Zustand auf.

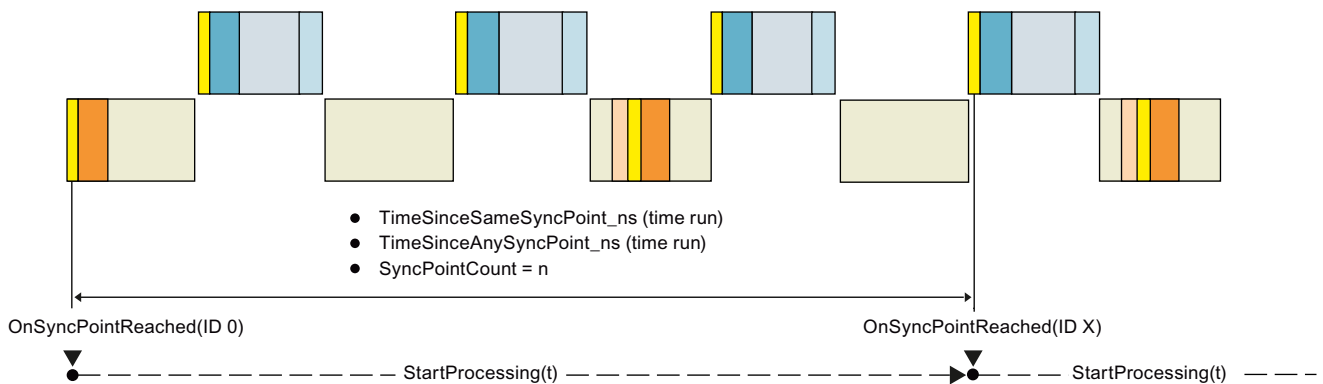


Bild 7-7 Beispiel: Ablauf in der Betriebsart TimespanSynchronized_CP

Beschreibung

Für die zeitgesteuerten Betriebsarten werden mindestens zwei Clients auf Basis einer virtuellen Zeitspanne synchronisiert. Ein Client kann eine Instanz einer simulierten PLC-Instanz sein oder eine Anwendung, die die Runtime API nutzt (API-Client). Die Synchronisierung muss von einem Synchronisations-Master durchgeführt werden. Der Synchronisations-Master beauftragt einen Client, eine bestimmte Zeitspanne zu laufen. Die Zeitspanne gibt der Master in Nanosekunden vor. Der Client läuft dann für die erwartete Zeitspanne, bevor der Client am nächsten Synchronisationspunkt in den Freeze-Zustand geht. Vor dem Wechsel in den Freeze-Zustand sendet der Client an den Master die genaue Zeitspanne, die er aktuell benötigt hat. Danach signalisiert der Master dem nächsten Client, aufzuholen.

API-Client als Master

Der API-Client als Master signalisiert jedem Client, wann er starten soll. Der Master erhält von jedem Client Ereignisse, wenn sie eingetreten sind.

Ein API-Client kann nur Instanzen einer simulierten PLC "zeitlich verwalten". Der API-Client erhält keine Ereignisse von anderen API-Clients. Er kann keine Meldungen an andere API-Clients senden.

7.12.1.2 Ereignisse OnOperatingStateChanged

OnOperatingStateChanged

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-200 OnOperatingStateChanged - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_EOS_EOS OnOperatingStateChanged;</code>
Parameter	Keine. Siehe <code>Delegate_II_EREC_DT_EOS_EOS</code> (Seite 215).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnOperatingStateChangedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-201 RegisterOnOperatingStateChangedCallback() - Native C++

Syntax	<code>void RegisterOnOperatingStateChangedCallback (EventCallback_II_SREC_ST_SROS_SROS in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_SROS_SROS in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_SROS_SROS (Seite 207).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

RegisterOnOperatingStateChangedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Das Registrieren eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 7-202 RegisterOnOperatingStateChangedEvent() - Native C++

Syntax	<code>void RegisterOnOperatingStateChangedEvent(); void RegisterOnOperatingStateChangedEvent(HANDLE* in_Event);</code>
Parameter	- None: Ein internes Event-Objekt wird registriert. - HANDLE* in_Event: Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine
Beispiel C++	<pre>// Thread 1 ----- ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } // Register the internal event object psa->RegisterOnOperatingStateChangedEvent(); // Thread 2 ----- while (condition) { // Wait for the event to be set (timeout after 10s) bool isEventSet = psa->WaitForOnOperatingStateChangedEvent(10000); if (isEventSet) { // Do Something ... } }</pre>

Beispiel C++	<pre> // Thread 1 ----- ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } // Create an event object HANDLE eventHandle = CreateEvent(NULL, FALSE, FALSE, NULL); // Register the user created event object psa->RegisterOnOperatingStateChangedEvent(&eventHandle); // Do Something ... // Clean up the handle CloseHandle(eventHandle); // Thread 2 ----- while (condition) { // Wait for the event to be set //OR: WaitForSingleObject(eventHandle, INFINITE); //psa- >WaitForOnOperatingStateChangedEvent(); // Do Something ... } </pre>
--------------	--

Tabelle 7-203 RegisterOnOperatingStateChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnOperatingStateChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnOperatingStateChangedCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-204 UnregisterOnOperatingStateChangedCallback() - Native C++

Syntax	<pre> void UnregisterOnOperatingStateChangedCallback (EventCallback_II_SREC_ST_SROS_SROS in_CallbackFunction = nullptr); </pre>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

UnregisterOnOperatingStateChangedEvent()

Meldet das Event-Objekt ab.

Tabelle 7-205 UnregisterOnOperatingStateChangedEvent() - Native C++

Syntax	<code>void UnregisterOnOperatingStateChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

7.12 Ereignisse für Instances

Tabelle 7-206 UnregisterOnOperatingStateChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnOperatingStateChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnOperatingStateChangedEvent()

Die Funktion blockiert das Programm solange, bis das registrierte Event-Objekt im signalisierten Zustand ist oder bis das Timeout-Intervall überschritten ist.

Tabelle 7-207 WaitForOnOperatingStateChangedEvent() - Native C++

Syntax	<code>bool WaitForOnOperatingStateChangedEvent();</code> <code>bool WaitForOnOperatingStateChangedEvent(UINT32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

Tabelle 7-208 WaitForOnOperatingStateChangedEvent() - .NET (C#)

Syntax	<code>bool WaitForOnOperatingStateChangedEvent();</code> <code>bool WaitForOnOperatingStateChangedEvent(UInt32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UInt32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

7.12.1.3 Ereignisse OnLedChanged

OnLedChanged

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-209 OnLedChanged - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_ELT_ELM OnLedChanged;</code>
Parameter	Keine. Siehe Delegate_II_EREC_DT_ELT_ELM (Seite 215) .
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnLedChangedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-210 RegisterOnLedChangedCallback() - Native C++

Syntax	<code>void RegisterOnLedChangedCallback(EventCallback_II_SREC_ST_SRLT_SRLM in_CallbackFunction);</code>
Parameter	<code>EventCallback_II_SREC_ST_SRLT_SRLM in_CallbackFunction</code>: Eine Callback-Funktion, die ein Ereignis abonniert. Siehe EventCallback_II_SREC_ST_SRLT_SRLM (Seite 209).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

RegisterOnLedChangedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Das Registrieren eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 7-211 RegisterOnLedChangedEvent() - Native C++

Syntax	<code>void RegisterOnLedChangedEvent();</code> <code>void RegisterOnLedChangedEvent(HANDLE* in_Event);</code>
Parameter	<code>None</code>: Ein internes Event-Objekt wird registriert. <code>HANDLE* in_Event</code>: Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

Tabelle 7-212 RegisterOnLedChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnLedChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnLedChangedCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-213 UnregisterOnLedChangedCallback() - Native C++

Syntax	<code>void UnregisterOnLedChangedCallback();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion <code>Register*()</code> Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

UnregisterOnLedChangedEvent()

Meldet das Event-Objekt ab.

Tabelle 7-214 UnregisterOnLedChangedEvent() - Native C++

Syntax	<code>void UnregisterOnLedChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 7-215 UnregisterOnLedChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnLedChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnLedChangedEvent()

Die Funktion blockiert das Programm solange, bis das registrierte Event-Objekt im signalisierten Zustand ist oder bis das Timeout-Intervall überschritten ist.

Tabelle 7-216 WaitForOnLedChangedEvent() - Native C++

Syntax	<code>bool WaitForOnLedChangedEvent();</code> <code>bool WaitForOnLedChangedEvent(UINT32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

Tabelle 7-217 WaitForOnLedChangedEvent() - .NET (C#)

Syntax	<code>bool WaitForOnLedChangedEvent();</code> <code>bool WaitForOnLedChangedEvent(UInt32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UInt32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

7.12.1.4 Ereignisse OnSoftwareConfigurationChanged

OnSoftwareConfigurationChanged()

Meldet Software-Änderungen in den Betriebszuständen STOP und RUN.

Tabelle 7-218 OnSoftwareConfigurationChanged

Syntax	<code>event Delegate_SRSCC OnSoftwareConfigurationChanged();</code>
Parameter	Instance, ERuntimeErrorCode, SYSTEMTIME, ESoftwareConfigChanged
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnSoftwareConfigurationChangedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es kann nur eine Callback-Funktion für das Ereignis registriert sein. Die Registrierung einer neuen Callback-Funktion führt zum Löschen der vorhergehenden.

Tabelle 7-219 RegisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedCallback();</code>
Parameter	CallbackFunction
Rückgabewerte	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnSoftwareConfigurationChangedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Die Registrierung eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 7-220 RegisterOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	• None Ein internes Event-Objekt wird registriert. • HANDLE* Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

UnregisterOnSoftwareConfigurationChangedCallback()

Meldet die Callback-Funktion ab. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 7-221 UnregisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedCallback();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnConfigurationChangedEvent()

Meldet das Event-Objekt ab.

Tabelle 7-222 UnregisterOnConfigurationChangedEvent() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 7-223 UnregisterOnConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnConfigurationChangedEvent()

Die Funktion blockiert das Programm solange, bis das registrierte Event-Objekt in den signalisierten Zustand gesetzt ist oder bis das Timeout-Intervall überschritten wird.

Tabelle 7-224 WaitForOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void WaitForOnSoftwareConfigurationChangedEvent();</code>
Parameter	• None: Das Zeitlimit ist auf <code>INFINITE</code> gesetzt. • UINT32 in Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.
Hinweis	Die Funktion blockiert das CoSimulation-Programm solange, bis ein Software-Download in den signalisierten Zustand gesetzt oder bis das Timeout-Intervall überschritten wird.

Tabelle 7-225 WaitForOnSoftwareConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void WaitForOnSoftwareConfigurationChangedEvent();</code>
Parameter	• None: Das Zeitlimit ist auf <code>INFINITE</code> gesetzt. • UINT32 in Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.
Hinweis	Die Funktion blockiert das CoSimulation-Programm solange, bis ein Software-Download in den signalisierten Zustand gesetzt oder bis das Timeout-Intervall überschritten wird.

7.12.1.5 Ereignisse OnIPAddressChanged

RegisterOnIPAddressChangedCallback()

Anmeldefunktion für den Empfang von Callback-Funktionen, wenn sich eine IP-Adresse geändert hat.

Tabelle 7-226 RegisterOnIPAddressChangedCallback() - Native C++

Syntax	<code>RegisterOnIPAddressChangedCallback();</code>
Parameter <code>in_CallbackFunction</code>	<code>IInstance* in_Sender</code> Instanz-Objekt <code>ERuntimeErrorCode in_ErrorCode</code> Ergebnis der Operation <code>SYSTEMTIME in_SystemTime</code> CPU-Systemzeit des Ereignisses <code>UINT8 in_InterfaceId</code> ID der betroffenen CPU-Schnittstelle <code>const SIPSuite4& in_SIP</code> Referenz der SIPSuite4-Struktur
Rückgabewerte	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

UnregisterOnIPAddressChangedCallback()

Abmeldefunktion für den Empfang von Callback-Funktionen, wenn sich eine IP-Adresse geändert hat.

Tabelle 7-227 UnregisterOnIPAddressChangedCallback() - Native C++

Syntax	<code>void UnregisterOnIPAddressChangedCallback();</code>
Parameter <code>in_CallbackFunction</code>	Adresse der spezifischen Funktion oder nullptr, um die zuletzt angemeldete Callback-Funktion zu entfernen
Rückgabewerte	Keine

RegisterOnIPAddressChangedEvent()

Anmeldefunktion für den Empfang von Ereignissen, wenn sich eine IP-Adresse geändert hat.

Tabelle 7-228 RegisterOnIPAddressChangedEvent() - Native C++

Syntax	<code>void IInstance::RegisterOnIPAddressChangedEvent();</code> <code>void IInstance::RegisterOnIPAddressChangedEvent(HANDLE* in_Event);</code>
Parameter <code>in_Event</code>	HANDLE* (ein benutzerspezifisches Event-Objekt wird angemeldet)
Rückgabewerte	Keine

UnregisterOnIPAddressChangedEvent()

Abmeldefunktion für den Empfang von Ereignissen, wenn sich eine IP-Adresse geändert hat.

Tabelle 7-229 UnregisterOnIPAddressChangedEvent() - Native C++

Syntax	<code>void IInstance::UnregisterOnIPAddressChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

OnIPAddressChanged()

Das Ereignis wird aufgerufen, wenn sich die IP-Konfiguration der CPU-Schnittstelle geändert hat.

Tabelle 7-230 OnIPAddressChanged() - .NET (C#)

Syntax	<code>void IInstance::OnIPAddressChanged();</code>
Parameter	<code>IInstance in_Sender</code> Instanz-Objekt <code>ERuntimeErrorCode in_ErrorCode</code> Ergebnis der Operation <code>DateTime in_SystemTime</code> CPU-Systemzeit des Ereignisses <code>UINT8 in_InterfaceId</code> ID der betroffenen CPU-Schnittstelle <code>SIPSuite4 in_SIP</code> IP-Adresse der Schnittstelle
Rückgabewerte	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

WaitForOnIPAddressChangedEvent()

Blockierfunktion bis zur Änderung der IP-Adresse.

Tabelle 7-231 WaitForOnIPAddressChangedEvent() - Native C++

Syntax	<code>bool IInstance::WaitForOnIPAddressChangedEvent();</code> <code>bool IInstance::WaitForOnIPAddressChangedEvent(UINT32 in_Time_ms)</code>
Parameter <code>in_Time_ms</code>	Zeit in Millisekunden
Rückgabewerte	Liefert <code>True</code> zurück, wenn die Änderung innerhalb eines eingestellten Zeitraums durchgeführt wurde, ansonsten <code>False</code>
Hinweis	Keine

Tabelle 7-232 WaitForOnIPAddressChangedEvent() - .NET (C#)

Syntax	<code>bool IInstance::WaitForOnIPAddressChangedEvent();</code> <code>bool IInstance::WaitForOnIPAddressChangedEvent(uint in_Time_ms);</code>
Parameter <code>in_Time_ms</code>	Zeit in Millisekunden
Rückgabewerte	Liefert <code>True</code> zurück, wenn die Änderung innerhalb eines eingestellten Zeitraums durchgeführt wurde, ansonsten <code>False</code>
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

7.12.1.6 Ereignisse OnSyncPointReached**OnSyncPointReached**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-233 OnSyncPointReached - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 OnSyncPointReached;</code>
Parameter	Keine. Siehe <code>Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32</code> (Seite 216).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnSyncPointReachedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-234 RegisterOnSyncPointReachedCallback() - Native C++

Syntax	<pre>void RegisterOnSyncPointReachedCallback (EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 in_CallbackFunction);</pre>
Parameter	EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 in_CallbackFunction: Eine Callback-Funktion, die ein Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 (Seite 207).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

RegisterOnSyncPointReachedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Das Registrieren eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 7-235 RegisterOnSyncPointReachedEvent() - Native C++

Syntax	<pre>void RegisterOnSyncPointReachedEvent(); void RegisterOnSyncPointReachedEvent(HANDLE* in_Event);</pre>
Parameter	- None: Ein internes Event-Objekt wird registriert. - HANDLE* in_Event: Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

Tabelle 7-236 RegisterOnSyncPointReachedEvent() – .NET (C#)

Syntax	RegisterOnSyncPointReachedEvent() – .NET (C#)
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnSyncPointReachedCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-237 UnregisterOnSyncPointReachedCallback() - Native C++

Syntax	<pre>void UnregisterOnSyncPointReachedCallback();</pre>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

UnregisterOnSyncPointReachedEvent()

Meldet das Event-Objekt ab.

Tabelle 7-238 UnregisterOnSyncPointReachedEvent() - Native C++

Syntax	<code>void UnregisterOnSyncPointReachedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 7-239 UnregisterOnSyncPointReachedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnSyncPointReachedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnSyncPointReachedEvent()

Die Funktion blockiert das Programm solange, bis das registrierte Event-Objekt im signalisierten Zustand ist oder bis das Timeout-Intervall überschritten ist.

Tabelle 7-240 WaitForOnSyncPointReachedEvent() - Native C++

Syntax	<code>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent();</code> <code>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent(UINT32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	SOnSyncPointReachedResult: Eine Struktur, die Informationen zum Ereignis liefert. Siehe SOnSyncPointReachedResult (Seite 232).

Tabelle 7-241 WaitForOnSyncPointReachedEvent() - .NET (C#)

Syntax	<code>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent();</code> <code>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent(UInt32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UInt32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	SOnSyncPointReachedResult: Eine Struktur, die Informationen zum Ereignis liefert. Siehe SOnSyncPointReachedResult (Seite 232).

7.12.1.7 Ereignisse OnHardwareConfigChanged

OnHardwareConfigChanged

Meldet eine Event-Handler-Methode an oder ab.

Tabelle 7-242 OnHardwareConfigChanged - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT OnHardwareConfigChanged;</code>
Parameter	Keine; siehe Delegate_II_EREC_DT
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler-Methode läuft in einem separaten Thread.

RegisterOnHardwareConfigChangedCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-243 Tabelle: RegisterOnHardwareConfigChangedCallback() – Native C++

Syntax	<code>void RegisterOnHardwareConfigChangedCallback(EventCallback_II_SREC_ST in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST in_CallbackFunction Eine Callback-Funktion, die eine Ereignis abonniert; siehe EventCallback_II_SREC_ST()
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

RegisterOnHardwareConfigChangedEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Die Registrierung eines neuen Event-Objekts führt zum Löschen des vorhergehenden Event-Objekts.

Tabelle 7-244 RegisterOnHardwareConfigChangedEvent() – Native C++

Syntax	<code>void RegisterOnHardwareConfigChangedEvent();</code> <code>void RegisterOnHardwareConfigChangedEvent(Handle* in_Event);</code>
Parameter	- None: Ein internes Event-Objekt wird registriert. - Handle* in_Event Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

Tabelle 7-245 RegisterOnHardwareConfigChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnHardwareConfigChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnHardwareConfigChangedCallback()

UnregisterOnHardwareConfigChangedCallback() Meldet die Callback-Funktion ab.

Tabelle 7-246 UnregisterOnHardwareConfigChangedCallback() – Native C++

Syntax	<code>void UnregisterOnHardwareConfigChangedCallback();</code>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

UnregisterOnHardwareConfigChangedEvent()

Meldet das Event-Objekt ab.

Tabelle 7-247 UnregisterOnHardwareConfigurationChangedEvent() – Native C++

Syntax	<code>void UnregisterOnHardwareConfigChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 7-248 UnregisterOnHardwareConfigurationChangedEvent() – .NET (C#)

Syntax	<code>void UnregisterOnHardwareConfigChangedEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnHardwareConfigurationChangedEvent()

Die Funktion blockiert das Programm so lange bis das registrierte Event-Objekt im signalisierten Zustand ist oder bis das Timeout-Intervall überschritten ist.

Tabelle 7-249 WaitForOnHardwareConfigChangedEvent() – Native C++

Syntax	<code>bool WaitForOnHardwareConfigChangedEvent();</code> <code>bool WaitForOnHardwareConfigChangedEvent (in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

Tabelle 7-250 WaitForOnHardwareConfigChangedEvent() – .NET (C#)

Syntax	<code>bool WaitForOnHardwareConfigChangedEvent();</code> <code>bool WaitForOnHardwareConfigChangedEvent (UINT32 in_Time_ms);</code>
Parameter	- None: Das Zeitlimit ist auf INFINITE gesetzt. - UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	- true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. - false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

7.12.2 Ereignisse für Azyklische Dienste

OnDataRecordRead

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-251 OnDataRecordRead - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_SDRI OnDataRecordRead;</code>
Parameter	Keine. Siehe Delegate_II_EREC_DT_SDRI (Seite 218).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

OnDataRecordWrite

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-252 OnDataRecordWrite - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_SDR OnDataRecordWrite;</code>
Parameter	Keine. Siehe Delegate_II_EREC_DT_SDR (Seite 217).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnDataRecordReadCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-253 RegisterOnDataRecordReadCallback() - Native C++

Syntax	<code>void RegisterOnDataRecordReadCallback (Event Callback_II_SREC_ST_SDRI in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_SDRI in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_SDRI (Seite 209).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnDataRecordReadCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-254 UnregisterOnDataRecordReadCallback() - Native C++

Syntax	<code>void UnregisterOnDataRecordReadCallback();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

RegisterOnDataRecordWriteCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-255 RegisterOnDataRecordWriteCallback() - Native C++

Syntax	<code>void RegisterOnDataRecordWriteCallback (EventCallback_II_SREC_ST_SDRI_BYTE in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_SDRI_BYTE in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_SDRI_BYTE (Seite 210).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnDataRecordWriteCallback()

Meldet die Callback-Funktion ab. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 7-256 UnregisterOnDataRecordWriteCallback() - Native C++

Syntax	<code>void UnregisterOnDataRecordWriteCallback();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.1 Ereignisse OnAlarmNotification**OnAlarmNotificationDone()**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-257 OnAlarmNotificationDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_UINT32 OnAlarmNotificationDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32_UINT32 (Seite 220) .
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnAlarmNotificationDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-258 RegisterOnAlarmNotificationDoneCallback() - Native C++

Syntax	<code>void RegisterOnAlarmNotificationDoneCallback (Event Callback_II_SREC_ST_SDRI in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32_UINT32 in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32_UINT32 (Seite 210).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnAlarmNotificationDoneCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-259 UnregisterOnAlarmNotificationDoneCallback() - Native C++

Syntax	<code>void UnregisterOnAlarmNotificationDoneCallback ();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.2 Ereignisse OnProcessEvent

OnProcessEventDone()

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-260 OnProcessEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 OnProcessEventDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32. (Seite 219)
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnProcessEventDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-261 RegisterOnProcessEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnProcessEventDoneCallback (EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 (Seite 211)
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnProcessEventDoneCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-262 UnregisterOnProcessEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnProcessEventDoneCallback ();</code>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.3 Ereignisse OnPullOrPlugEvent**OnPullOrPlugEventDone()**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-263 OnPullOrPlugEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_EPPET_UINT32 OnPullOrPlugEventDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32_EPPET_UINT32 (Seite 218).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnPullOrPlugEventDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-264 RegisterOnPullOrPlugEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnPullOrPlugEventDoneCallback (EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 (Seite 211).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnPullOrPlugEventDoneCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-265 UnregisterOnPullOrPlugEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnPullOrPlugEventDoneCallback ();</code>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.4 Ereignisse OnStatusEvent**OnStatusEventDone()**

Meldet eine Event-Handler Methode an- oder ab.

Tabelle 7-266 OnStatusEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnStatusEventDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32 (Seite 219).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnStatusEventDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-267 RegisterOnStatusEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnStatusEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32 in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32 (Seite 212).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnStatusEventDoneCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-268 UnregisterOnStatusEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnStatusEventDoneCallback ();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.5 Ereignisse OnProfileEvent**OnProfileEventDone()**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-269 OnProfileEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnProfileEventDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32 (Seite 219).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnProfileEventDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-270 RegisterOnProfileEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnProfileEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameter	<code>EventCallback_II_SREC_ST_UINT32 in_CallbackFunction:</code> Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32 (Seite 212) .
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnProfileEventDoneCallback()

Meldet die Callback-Funktion ab. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 7-271 UnregisterOnProfileEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnProfileEventDoneCallback ();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion <code>Register*()</code> Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.6 Ereignisse OnUpdateEvent**OnUpdateEventDone()**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-272 OnUpdateEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnUpdateEventDone;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32 (Seite 219) .
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnUpdateEventDoneCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-273 RegisterOnUpdateEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnUpdateEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32 in_CallbackFunction: Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32 (Seite 212).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnUpdateEventDoneCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-274 UnregisterOnUpdateEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnUpdateEventDoneCallback ();</code>
Parameter	optional Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

7.12.2.7 Ereignisse RackOrStationFault**OnRackOrStationFaultEvent**

Meldet eine Event-Handler Methode an oder ab.

Tabelle 7-275 OnRackOrStationFaultEvent - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_ERSFET OnRackOrStationFault;</code>
Parameter	Keine. Siehe Delegate_SREC_ST_UINT32_ERSFET (Seite 220).
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnRackOrStationFaultEventCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 7-276 RegisterOnRackOrStationFaultEventCallback() - Native C++

Syntax	<code>void RegisterOnRackOrStationFaultEventCallback (EventCallback_II_SREC_ST_UINT32_ERSFET in_CallbackFunction);</code>
Parameter	EventCallback_II_SREC_ST_UINT32_ERSFET in_CallbackFunction. Eine Callback-Funktion, die das Ereignis abonniert. Siehe EventCallback_II_SREC_ST_UINT32_ERSFET (Seite 212)
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

UnregisterOnRackOrStationFaultEventCallback()

Meldet die Callback-Funktion ab.

Tabelle 7-277 UnregisterOnRackOrStationFaultEventCallback() - Native C++

Syntax	<code>void UnregisterOnRackOrStationFaultEventCallback ();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion Register*() Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet.
Rückgabewerte	Keine

API IRemoteRuntimeManager

8.1 Schnittstellen - Information und Einstellungen

Dispose()

Löscht die managed-Schnittstelle und entlädt die nativen Komponenten der Anwenderschnittstellen.

HINWEIS

Wenn die Schnittstelle des Remote Runtime Managers gelöscht wird, dann kann keine Instance-Schnittstelle, die von der IRemoteRuntimeManager-Schnittstelle erzeugt wurde, mehr genutzt werden.

Der .NET Garbage Collector bereinigt Ihre IRemoteRuntimeManager-Schnittstelle, wenn keine aktive Referenz mehr vorhanden ist.

Tabelle 8-1 Dispose() - .NET (C#)

Syntax	<code>void Dispose()</code>
Parameter	Keine
Rückgabewerte	Keine

GetIP() / IP { get; }

Gibt die IP-Adresse des PC zurück, auf dem der Remote Runtime Manager läuft. Wenn die Funktion fehlschlägt, ist der Rückgabewert 0.

Tabelle 8-2 GetIP() - Native C++

Syntax	<code>UIP GetIP();</code>
Parameter	Keine
Rückgabewerte	UIP: Gibt die IP-Adresse des PC zurück, auf dem der Runtime Manager läuft.

Tabelle 8-3 IP { get; } - .NET (C#)

Syntax	<code>SIP IP { get; }</code>
Parameter	Keine
Rückgabewerte	SIP: Gibt die IP-Adresse des PC zurück, auf dem der Runtime Manager läuft.

GetRemoteComputerName() / RemoteComputerName { get; }

Gibt den Namen des PC zurück, auf dem der Remote Runtime Manager läuft.

Wenn der Name des PC, auf dem der Remote Runtime Manager läuft, auf dem lokalen PC nicht identifiziert werden kann, wird die IP-Adresse zurückgegeben.

Tabelle 8-4 GetRemoteComputerName() - Native C++

Syntax	ERuntimeErrorCode GetRemoteComputerName (WCHAR* inout_Name, UINT32 in_ArrayLength);	
Parameter	- WCHAR* inout_Name: Ein vom Benutzer zugewiesenes Array für den Computernamen. - UINT32 in_ArrayLength: Die Arraygröße. Das Array sollte größer als MAX_COMPUTERNAME_LENGTH sein.	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	SREC_INDEX_OUT_OF_RANGE	Das Array ist zu klein, um den Computernamen aufzunehmen.

Tabelle 8-5 RemoteComputerName { get; } - .NET (C#)

Syntax	string RemoteComputerName { get; }	
Parameter	Keine	
Rückgabewerte	string: Name des PC, auf dem der Remote Runtime Manager läuft.	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.IndexOutOfRange	Das Array ist zu klein, um den Computernamen aufzunehmen.

Disconnect()

Schließt die Verbindung zum Remote Runtime Manager.

HINWEIS

Alle Anwendungen, die mit dem Remote Runtime Manager verbunden sind, verlieren diese Verbindung.

Tabelle 8-6 Disconnect() - Native C++

Syntax	ERuntimeErrorCode Disconnect();	
Parameter	Keine	
Rückgabewerte	Runtime-Fehlercode	Beschreibung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_INTERFACE_REMOVED	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	SREC_TIMEOUT	Die Funktion kehrt nicht rechtzeitig wieder.

Tabelle 8-7 Disconnect() - .NET (C#)

Syntax	void Disconnect();
Parameter	Keine

8.2 Ereignisse für IRemoteRuntimeManager

Rückgabewerte	Keine	
Ausnahmen	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime-Fehlercode	Beschreibung
	ERuntimeErrorCode.InterfaceRemoved	Die Schnittstelle ist vom Remote Runtime Manager getrennt.
	ERuntimeErrorCode.Timeout	Die Funktion kehrt nicht rechtzeitig wieder.

8.2 Ereignisse für IRemoteRuntimeManager

8.2.1 Ereignisse OnConnectionLost

Beschreibung

Das Ereignis wird ausgelöst, wenn die Verbindung zum Remote Runtime Manager gelöst wurde.

OnConnectionLost

Meldet eine Event-Handler Methode an oder ab.

Tabelle 8-8 OnConnectionLost - .NET (C#)

Syntax	event Delegate_IRRTM OnConnectionLost;
Parameter	Keine. Siehe Delegate_IRRTM (Seite 216)
Rückgabewerte	Keine
Ausnahmen	Keine
Hinweis	Die Event-Handler Methode läuft in einem separaten Thread.

RegisterOnConnectionLostCallback()

Wenn das Ereignis eintritt, wird die registrierte Callback-Funktion aufgerufen. Es können mehrere Callback-Funktionen für das Ereignis registriert sein.

Tabelle 8-9 RegisterOnConnectionLostCallback() - Native C++

Syntax	void RegisterOnConnectionLostCallback(EventCallback_IRRTM in_CallbackFunction);
Parameter	EventCallback_IRRTM in_CallbackFunction: Eine Callback-Funktion, die ein Ereignis abonniert. Siehe EventCallback_IRRTM (Seite 206).
Rückgabewerte	Keine
Hinweis	Die Callback-Funktion läuft in einem separaten Thread.

RegisterOnConnectionLostEvent()

Wenn das Ereignis eintritt, wird das registrierte Event-Objekt in den signalisierten Zustand gesetzt. Es kann nur ein Event-Objekt für das Ereignis registriert sein. Die Registrierung eines neuen Event-Objekts führt zum Löschen des vorhergehenden.

Tabelle 8-10 RegisterOnConnectionLostEvent() - Native C++

Syntax	<code>void RegisterOnConnectionLostEvent();</code> <code>void RegisterOnConnectionLostEvent(HANDLE* in_Event);</code>
Parameter	- None: Ein internes Event-Objekt wird registriert. <code>HANDLE* in_Event</code>: Ein Handle zu einem anwenderspezifischen Event-Objekt. Das Event-Objekt wird registriert.
Rückgabewerte	Keine

Tabelle 8-11 RegisterOnConnectionLostEvent() - .NET (C#)

Syntax	<code>void RegisterOnConnectionLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

UnregisterOnConnectionLostCallback()

Meldet die Callback-Funktion ab. Wenn das Ereignis eintritt, wird keine Callback-Funktion aufgerufen.

Tabelle 8-12 UnregisterOnConnectionLostCallback() - Native C++

Syntax	<code>void UnregisterOnConnectionLostCallback();</code>
Parameter	optionaler Parameter des gleichen Typs wie für die Funktion <code>Register*()</code> Wenn Sie den Parameter verwenden, deinstalliert er die spezifische Funktion, andernfalls wird die zuletzt registrierte Callback-Funktion abgemeldet
Rückgabewerte	Keine

UnregisterOnConnectionLostEvent()

Meldet das Event-Objekt ab.

Tabelle 8-13 UnregisterOnConnectionLostEvent() - Native C++

Syntax	<code>void UnregisterOnConnectionLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

Tabelle 8-14 UnregisterOnConnectionLostEvent() - .NET (C#)

Syntax	<code>void UnregisterOnConnectionLostEvent();</code>
Parameter	Keine
Rückgabewerte	Keine

WaitForOnConnectionLostEvent()

Die Funktion blockiert das Programm solange, bis das registrierte Event-Objekt im signalisierten Zustand ist oder bis das Timeout-Intervall überschritten ist.

Tabelle 8-15 WaitForOnConnectionLostEvent() - Native C++

Syntax	<code>bool WaitForOnConnectionLostEvent();</code> <code>bool WaitForOnConnectionLostEvent(UINT32 in_Time_ms);</code>
Parameter	• None: Das Zeitlimit ist auf INFINITE gesetzt. • UINT32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

Tabelle 8-16 WaitForOnConnectionLostEvent() - .NET (C#)

Syntax	<code>bool WaitForOnConnectionLostEvent();</code> <code>bool WaitForOnConnectionLostEvent(UInt32 in_Time_ms);</code>
Parameter	• None: Das Zeitlimit ist auf INFINITE gesetzt. • UInt32 in_Time_ms: Wert für das Zeitlimit in Millisekunden.
Rückgabewerte	• true: Wenn das Event-Objekt in den signalisierten Zustand gesetzt wurde. • false: Wenn während des definierten Zeitlimits kein Ereignis empfangen wurde.

Datentypen

Unterstützte Datentypen

In der S7-PLCSIM Advanced V6.0 unterstützt die Runtime API die Datentypen der S7-1500-CPU.

Datentypen konvertieren

Alle Datentypen werden beim Schreiben nicht BCD-codiert übergeben, sondern auf primitive Datentypen gemappt.

Die Datentypen Zähler, Datum und Uhrzeit müssen BCD-codiert an die API übergeben werden, damit die Werte in den Zähler geschrieben und beim Lesen keine falschen Werte zurückgegeben werden.

Führen Sie für diese Datentypen vor dem Schreiben eine BCD-Konvertierung und nach dem Lesen eine BCD-Rückkonvertierung durch.

Beispiel:

Wenn der Wert 999 als 2457_H an die API übergeben wird, dann modifiziert `Write` den Wert 2457_H zu 999. Ohne BCD-Konvertierung gibt es keinen `UInt16`-Wert und `Write` schreibt überhaupt keinen Wert.

Weitere Informationen

Informationen zu Datentypen und zur Konvertierung erhalten Sie im Kapitel "Datentypen" im Systemhandbuch SIMATIC STEP 7 Basic/Professional

(<https://support.industry.siemens.com/cs/ww/de/view/109755202>).

9.1 DLL-Importfunktionen (Native C++)

9.1.1 ApiEntry_Initialize

Beschreibung

Typ des zentralen Eintrittspunkts für die API-Bibliothek (DLL).

Tabelle 9-1 ApiEntry_Initialize - Native C++

Syntax	typedef ERuntimeErrorCode(*ApiEntry_Initialize)(ISimulationRuntimeManager** out_RuntimeManagerInterface);	
Parameter	- ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface: Zeiger auf einen Runtime Manager Schnittstellenzeiger. Der Zeiger muss mit NULL initialisiert werden. Die Schnittstelle wird innerhalb der Funktion erzeugt. - UINT32 in_InterfaceVersion: Die Version der API-Schnittstelle, die geladen werden soll: API_DLL_INTERFACE_VERSION.	
Rückgabewerte	Runtime Fehlercode	Bedingung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_WRONG_ARGUMENT	Der Zeiger auf die Runtime Manager Schnittstelle ist NULL.
	SREC_WRONG_VERSION	Die Version der genutzten Schnittstelle passt nicht zur Version der API-Bibliothek (DLL).
	SREC_CONNECTION_ERROR	Zum Runtime Manager kann keine Verbindung hergestellt werden.
	SREC_CONFIG_FILE_ERROR	Operation bezüglich der Konfigurationsdatei "UserInterfaceConfiguration.xml" ist fehlgeschlagen, z. B. Erzeugen, Lesen, Schreiben.

9.1.2 ApiEntry_DestroyInterface

Beschreibung

Typ des Eintrittspunkts für DestroyInterface [\(Seite 30\)](#).

Tabelle 9-2 ApiEntry_DestroyInterface - Native C++

Syntax	typedef ERuntimeErrorCode(*ApiEntry_DestroyInterface)(IBaseInterface* in_Interface);	
Parameter	IBaseInterface* in_Interface: Die Schnittstelle, die gelöscht werden soll.	
Rückgabewerte	Runtime Fehlercode	Bedingung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_WRONG_ARGUMENT	Der Zeiger auf die Schnittstelle ist NULL.

9.2 Event Callback-Funktionen (Native C++)

9.2.1 EventCallback_VOID

Beschreibung

Tabelle 9-3 EventCallback_VOID - Native C++

Syntax	<code>typedef void (*EventCallback_VOID)();</code>
Parameter	Keine
Rückgabewerte	Keine

9.2.2 EventCallback_SRCC_UINT32_UINT32_INT32

Beschreibung

Tabelle 9-4 EventCallback_SRCC_UINT32_UINT32_INT32 - Native C++

Syntax	<code>ERuntimeConfigChanged in_RuntimeConfigChanged, UINT32 in_Param1, UINT32 in_Param2, INT32 in_Param3);</code>			
Parameter	<code>ERuntimeConfigChanged in_RuntimeConfigChanged</code>	<code>UInt32 in_Param1</code>	<code>UInt32 in_Param2</code>	<code>Int32 in_Param3</code>
	<code>SRCC_INSTANCE_REGISTERED</code>	-	-	ID der registrierten Instanz
	<code>SRCC_INSTANCE_UNREGISTERED</code>	-	-	ID der nicht registrierten Instanz
	<code>SRCC_CONNECTION_OPENED</code>	IP des Remote Runtime Managers	Port des Remote Runtime Managers	-
	<code>SRCC_CONNECTION_CLOSED</code>	IP des Remote Runtime Managers	Port des Remote Runtime Managers	-
	<code>SRCC_PORT_OPENED</code>	Der offene Port	-	-
	<code>SRCC_PORT_CLOSED</code>	-	-	-
Rückgabewerte	Keine			

9.2.3 EventCallback_SRRSI_AD

Beschreibung

Tabelle 9-5 EventCallback_SRRSI_AD - Native C++

Syntax	<code>typedef void (*EventCallback_SRRSI_AD) (EAutodiscoverType in_AutodiscoverMsg, SAutodiscoverData in_AutodiscoverData);</code>
Parameter	in_AutodiscoverMsg: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen; siehe EAutodiscoverType (Seite 258). - SRRSI_DISCOVER_STARTED, wenn der Identifizierungs-Vorgang durch einen erfolgreichen Aufruf der Funktion RunAutodiscover() gestartet wurde. - SRRSI_DISCOVER_DATA, wenn ein Runtime Manager im Netzwerk durch den Identifizierungs-Vorgang ermittelt werden konnte. Für genaue Informationen zu dem gefundenen Runtime Manager siehe Parameter in_AutodiscoverData. - SRRSI_DISCOVER_FINISHED wenn der Identifizierungs-Vorgang nach Ablauf der über den Parameter „in_Timeout“ definierten Zeit abgeschlossen wurde. - SRRSI_DISCOVER_STARTED und SRRSI_DISCOVER_FINISHED werden immer getriggert, auch wenn keine Daten empfangen werden. in_AutodiscoverData: Daten vom Remote Runtime Manager. Der Parameter enthält nur dann gültige Daten, wenn in_AutodiscoverMsg = SRRSI_DISCOVER_DATA. Ansonsten wird er mit 0 initialisiert. Siehe SAutodiscoverData (Seite 238).
Rückgabewerte	Keine

9.2.4 EventCallback_IRRTM

Beschreibung

Tabelle 9-6 EventCallback_IRRTM - Native C++

Syntax	<code>typedef void (*EventCallback_IRRTM) (IRemoteRuntimeManager* in_Sender);</code>
Parameter	IRemoteRuntimeManager* in_Sender: Eine Schnittstelle des Remote Runtime Managers, die dieses Ereignis empfängt.
Rückgabewerte	Keine

9.2.5 EventCallback_II_SREC_ST_SROS_SROS

Beschreibung

Tabelle 9-7 EventCallback_II_SREC_ST_SROS_SROS - Native C++

Syntax	typedef void (*EventCallback_II_SREC_ST_SROS_SROS)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, EOperatingState in_PrevState, EOperatingState in_OperatingState);	
Parameter	- IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. - ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode - SYSTEMTIME in_SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. - EOperatingState in_PrevState: Der Betriebszustand vor dem Wechsel - EOperatingState in_OperatingState: Der aktuelle Betriebszustand	
Rückgabewerte	Keine	
Fehlercodes	Runtime-Fehlercode	Bedingung
	SREC_OK	Die Funktion ist erfolgreich.
	SREC_WARNING_TRIAL_MODE_ACTIVE	Keine Lizenz verfügbar. Sie können die Instanz ohne Einschränkung mit der Trial License nutzen. Danach wird die Instanz abgeschaltet.
	SREC_LICENSE_NOT_FOUND	Der Testmodus ist abgelaufen.
	SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE	Ein Problem mit der ausgewählten Kommunikationsschnittstelle ist aufgetreten. Überprüfen Sie Ihre Einstellungen.

9.2.6 EventCallback_II_SREC_ST_UINT32_INT64_UINT32

Beschreibung

Tabelle 9-8 EventCallback_II_SREC_ST_UINT32_INT64_UINT32 - Native C++

Syntax	typedef void (*EventCallback_II_SREC_ST_UINT32_INT64_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_Id, INT64 in_TimeSinceSameSyncPoint_ns, INT64 in_TimeSinceAnySyncPoint_ns, UINT32 in_SyncPointCount);	
Parameter	- IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. - ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode - SYSTEMTIME in_SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. - UINT32 in_Id: Für die Betriebsart SingleStep_Bus: Die ID des Peripheriesystems, das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet wird. Für alle anderen Betriebsarten gilt: Die ID des Teilprozessabbilds (TPA), das dieses Ereignis auslöst. 0 für den Zykluskontrollpunkt (End of cycle).	

	• <code>INT64 in_TimeSinceSameSyncPoint_ns</code>: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt derselben Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>. • <code>INT64 in_TimeSinceAnySyncPoint_ns</code>: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt einer beliebigen Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>. • <code>UINT32 in_SyncPointCount</code>: Die Anzahl der Synchronisationspunkte seit dem letzten Ereignis. Wenn die Ereignisse schneller ausgelöst als empfangen werden, dann werden mehrere Ereignisse zu einem zusammengefasst. In diesem Fall enthält dieser Wert die Anzahl der Zyklen, seit das letzte Ereignis empfangen wurde.
Rückgabewerte	Keine

9.2.7 EventCallback_II_SREC_ST

Beschreibung

Tabelle 9-9 EventCallback_II_SREC_ST - Native C++

Syntax	<code>typedef void (*EventCallback_II_SREC_ST)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime);</code>
Parameter	• <code>IInstance* in_Sender</code>: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. • <code>ERuntimeErrorCode in_ErrorCode</code>: Ein möglicher Fehlercode • <code>SYSTEMTIME in_SystemTime</code>: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde.
Rückgabewerte	Keine

9.2.8 EventCallback_II_SREC_ST_SRICC_UINT32_UINT32_UINT32_UINT32

Beschreibung

Tabelle 9-10 EventCallback_II_SREC_ST_SRICC_UINT32_UINT32_UINT32_UINT32 - Native C++

Syntax	<code>typedef void (*EventCallback_II_SREC_ST_SRICC_UINT32_UINT32_UINT32_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, EInstanceConfigChanged in_InstanceConfigChanged, UINT32 in_Param1, UINT32 in_Param2, UINT32 in_Param3, UINT32 in_Param4);</code>				
Parameter	• <code>IInstance in_Sender</code>: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. • <code>ERuntimeErrorCode in_ErrorCode</code>: Ein möglicher Fehlercode • <code>SYSTEMTIME in_SystemTime</code>: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde.				
	<code>EInstanceConfigChanged in_InstanceConfigChanged</code>	<code>UINT32 in_Param1</code>	<code>UINT32 in_Param2</code>	<code>UINT32 in_Param3</code>	<code>UINT32 in_Param4</code>

Parameter	SRICC_HARDWARE_SOFTWARE_CHANGED	–	–	–	–
	SRICC_IP_CHANGED	Die ID der Schnittstelle	Die neue IP	Die neue Subnetzmaske	Das neue Standard-Gateway
Rückgabewerte	Keine				

9.2.9 EventCallback_II_SREC_ST_SRLT_SRLM

Beschreibung

Tabelle 9-11 EventCallback_II_SREC_ST_SRLT_SRLM - Native C++

Syntax	<code>typedef void (*EventCallback_II_SREC_ST_SRLT_SRLM)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, ELEDType in_LEDType, ELEDMode in_LEDMode);</code>
Parameter	IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode SYSTEMTIME in_SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. ELEDType in_LEDType: Der LED-Typ, der seinen Zustand gewechselt hat. ELEDMode in_LEDMode: Der neue Zustand der LED-Anzeige.
Rückgabewerte	Keine

9.2.10 EventCallback_II_SREC_ST_SDRI

Beschreibung

Tabelle 9-12 EventCallback_II_SREC_ST_SDRI - Native C++

Syntax	<code>typedef void (*EventCallback_II_SREC_ST_SDRI)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, SDataRecordInfo in_DataRecordInfo);</code>
Parameter	IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode SYSTEMTIME in_SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. SDataRecordInfo in_DataRecordInfo: Die Struktur SDataRecordInfo enthält folgende Informationen: – Die HW-Kennung, von der die CPU den Datensatz lesen soll – Den Index des abgeholten Datensatzes – Die maximale Größe des Datensatzes, die das IO-Device übertragen kann
Rückgabewerte	Keine

9.2.11 EventCallback_II_SREC_ST_SDRI_BYTE

Beschreibung

Tabelle 9-13 EventCallback_II_SREC_ST_SDRI_BYTE - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_SDRI_BYTE)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, SDataRecordInfo in_DataRecordInfo const BYTE* in_Data);</pre>
Parameter	<code>IInstance* in_Sender</code>: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. <code>ERuntimeErrorCode in_ErrorCode</code>: Ein möglicher Fehlercode <code>SYSTEMTIME in_SystemTime</code>: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. <code>SDataRecordInfo in_DataRecordInfo</code>: Die Struktur <code>SDataRecordInfo</code> enthält folgende Informationen: - Die HW-Kennung, an die die CPU den Datensatz schreiben möchte - Den Index des gelieferten Datensatzes - Die Größe des Datensatzes <code>const BYTE* in_Data</code>: Der Datensatz. Dieser Zeiger wird ungültig, nachdem die Callback-Funktion zurückkehrt.
Rückgabewerte	Keine

9.2.12 EventCallback_II_SREC_ST_UINT32_UINT32

Beschreibung

Tabelle 9-14 EventCallback_II_SREC_ST_UINT32_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier), UINT32 in_SequenceNumber);</pre>
Parameter	<code>IInstance* in_Sender</code>: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. <code>ERuntimeErrorCode in_ErrorCode</code>: Ein möglicher Fehlercode <code>SYSTEMTIME in_SYSTEMTIME</code>: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. <code>UINT32 in_HardwareIdentifier</code>: Die HW-Kennung des Moduls oder Submoduls, das das Diagnoseereignis sendet. <code>UINT32 in_SequenceNumber</code>: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardware-System verwendet der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.2.13 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32

Beschreibung

Tabelle 9-15 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier, UINT32 in_Channel, EProcessEventType in_ProcessEventType, UINT32 in_SequenceNumber);</pre>
Parameter	IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode SYSTEMTIME in_SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UINT32 in_HardwareIdentifier: Die HW-Kennung des Peripheriemoduls, das das Prozessereignis sendet. UINT32 in_Channel: Der Kanal des E/A-Moduls, das das Prozessereignis sendet. EProcessEventType in_ProcessEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module; siehe EProcessEventType (Seite 255). UINT32 in_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardwaresystem nutzt der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.2.14 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32

Beschreibung

Tabelle 9-16 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_EPPET_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SYSTEMTIME, UINT32 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UINT32 in_SequenceNumber);</pre>
Parameter	IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode SYSTEMTIME in_SYSTEMTIME: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UINT32 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Ziehen/Stecken-Ereignis sendet. EPullOrPlugEventType in_PullOrPlugEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module; siehe EPullOrPlugEventType (Seite 255).

9.2 Event Callback-Funktionen (Native C++)

	• UINT32 in SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardware-System verwendet der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.2.15 EventCallback_II_SREC_ST_UINT32_ERSFET

Beschreibung

Tabelle 9-17 EventCallback_II_SREC_ST_UINT32_ERSFET - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_ERSFET) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifizier, ERackOrStationFaultType in_EventType);</pre>
Parameter	• IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. • ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode • SYSTEMTIME in_SYSTEMTIME: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. • UINT32 in_HardwareIdentifizier: Die HW-Kennung des Moduls oder Submoduls, das das Diagnoseereignis sendet. • ERackOrStationFaultType in_EventType: Ein Wert aus der Liste der vordefinierten RackOrStationFault-Ereignis-Typen. Siehe ERackOrStationFaultType (Seite 257).
Rückgabewerte	Keine

9.2.16 EventCallback_II_SREC_ST_UINT32

Beschreibung

Tabelle 9-18 EventCallback_II_SREC_ST_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifizier);</pre>
Parameter	• IInstance* in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. • ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode • SYSTEMTIME in_SYSTEMTIME: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. • UINT32 in_HardwareIdentifizier: Die HW-Kennung des Moduls oder Submoduls, das das Status-, Update- oder Profile-Ereignis generiert. Die Kennung muss zu einer Hardware-Komponente im aktuell geladenen Projekt gehören.
Rückgabewerte	Keine

9.3 Delegat Definitionen (Managed Code)

9.3.1 Delegate_Void

Beschreibung

Tabelle 9-19 Delegate_Void - .NET (C#)

Syntax	<code>delegate void Delegate_Void();</code>
Parameter	Keine
Rückgabewerte	Keine

9.3.2 Delegate_SRCC_UINT32_UINT32_INT32

Beschreibung

Tabelle 9-20 Delegate_SRCC_UINT32_UINT32_INT32 - .NET (C#)

Syntax	<code>delegate void Delegate_SRCC_UINT32_UINT32_INT32(ERuntimeConfigChanged in_RuntimeConfigChanged, UInt32 in_Param1, UInt32 in_Param2, Int32 in_Param3);</code>			
Parameter	ERuntimeConfigChanged	UInt32 in_Param1	UInt32 in_Param2	Int32 in_Param3
	InstanceRegistered	-	-	ID der registrierten Instanz
	InstanceUnregistered	-	-	ID der nicht registrierten Instanz
	ConnectionOpened	IP des Remote Runtime Managers	Port des Remote Runtime Managers	-
	ConnectionClosed	IP des Remote Runtime Managers	Port des Remote Runtime Managers	-
	PortOpened	Der offene Port	-	-
	PortClosed	-	-	-
Rückgabewerte	Keine			

9.3.3 Delegate_SRRSI_AD

Beschreibung

Tabelle 9-21 Delegate_SRRSI_AD - .NET (C#)

Syntax	<code>delegate void Delegate_SRRSI_AD(EAutodiscoverType in_AutodiscoverType, SAutodiscoverData in_AutodiscoverData);</code>
Parameter	<code>in_AutodiscoverType</code> Ein Wert aus der Liste der vordefinierten Typen von Ereignissen; siehe <code>EAutodiscoverType</code> (Seite 258). <code>AutodiscoverStarted</code>, wenn der Identifizierungs-Vorgang durch einen erfolgreichen Aufruf der Funktion <code>RunAutodiscover()</code> gestartet wurde. <code>AutodiscoverData</code>, wenn ein Runtime Manager im Netzwerk durch den Identifizierungs-Vorgang ermittelt werden konnte. Für genaue Informationen zu dem gefundenen Runtime Manager siehe Parameter <code>in_AutodiscoverData</code>. <code>AutodiscoverFinished</code>, wenn der Identifizierungs-Vorgang nach Ablauf der über den Parameter „<code>in_Timeout</code>“ definierten Zeit abgeschlossen wurde. <code>AutodiscoverStarted</code> und <code>AutodiscoverFinished</code> werden immer getriggert, auch wenn keine Daten empfangen werden. <code>in_AutodiscoverData</code> Daten vom Remote Runtime Manager. Der Parameter enthält nur dann gültige Daten, wenn <code>in_AutodiscoverType = AutodiscoverData</code>. Ansonsten wird er mit 0 initialisiert. Siehe <code>SAutodiscoverData</code> (Seite 238).
Rückgabewerte	Keine

9.3.4 Delegate_II_EREC_DT

Beschreibung

Tabelle 9-22 Delegate_II_EREC_DT - .NET (C#)

Syntax	<code>delegate void Delegate_II_EREC_DT (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime);</code>
Parameter	<code>IInstance in_Sender</code>: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. <code>ERuntimeErrorCode in_ErrorCode</code>: Ein möglicher Fehlercode <code>DateTime in_DateTime</code>: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde.
Rückgabewerte	Keine

9.3.5 Delegate_II_EREC_DT_EOS_EOS

Beschreibung

Tabelle 9-23 Delegate_II_EREC_DT_EOS_EOS - .NET (C#)

Syntax	delegate void Delegate_II_EREC_DT_EOS_EOS(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, EOperatingState in_PrevState, EOperatingState in_OperatingState);	
Parameter	- IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. - ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode - DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. - EOperatingState in_PrevState: Der Betriebszustand vor dem Wechsel - EOperatingState in_OperatingState: Der aktuelle Betriebszustand	
Rückgabewerte	Keine	
Fehlercodes	Runtime-Fehlercode	Bedingung
	ERuntimeErrorCode.OK	Die Funktion ist erfolgreich.
	ERuntimeErrorCode.WarningTrialModeActive	Keine Lizenz verfügbar. Sie können die Instanz ohne Einschränkung mit der Trial License nutzen. Danach wird die Instanz abgeschaltet.
	ERuntimeErrorCode.LicenseNotFound	Der Testmodus ist abgelaufen.
	ERuntimeErrorCode.CommunicationInterfaceNotAvailable	Ein Problem mit der ausgewählten Kommunikationsschnittstelle ist aufgetreten. Überprüfen Sie Ihre Einstellungen.

9.3.6 Delegate_II_EREC_DT_ELT_ELM

Beschreibung

Tabelle 9-24 Delegate_II_EREC_DT_ELT_ELM - .NET (C#)

Syntax	delegate void Delegate_II_EREC_DT_ELT_ELM(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, ELEDType in_LEDType, ELEDMode in_LEDMode);	
Parameter	- IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. - ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode - DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. - ELEDType in_LEDType: Der LED-Typ, der seinen Zustand gewechselt hat. - ELEDMode in_LEDMode: Der neue Zustand der LED-Anzeige.	
Rückgabewerte	Keine	

9.3.7 Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32

Beschreibung

Tabelle 9-25 Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 - .NET (C#)

Syntax	<pre>delegate void Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_Id, Int64 in_TimeSinceSameSyncPoint_ns, Int64 in_TimeSinceAnySyncPoint_ns, UInt32 in_SyncPointCount);</pre>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_Id: Für die Betriebsart <code>SingleStep_Bus</code>: Die ID des Peripheriesystems, das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet wird. Für alle anderen Betriebsarten gilt: Die ID des Teilprozessabbilds (TPA), das dieses Ereignis auslöst. 0 für den Zykluskontrollpunkt (End of cycle). Hinweis: In den <code>SingleStep</code>-Betriebsarten hat die übergebene ID "in_Id" für den Zykluskontrollpunkt den Wert 0xFFFF. In diesem Fall werden die Time-Werte auf 0 gesetzt. Int64 in_TimeSinceSameSyncPoint_ns: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt derselben Teilprozessabbild-ID erreicht wurde. Oder die Prozesszeit für die zeitgesteuerten Betriebsarten. Int64 in_TimeSinceAnySyncPoint_ns: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt einer beliebigen Teilprozessabbild-ID erreicht wurde. Oder die Prozesszeit für die zeitgesteuerten Betriebsarten. UInt32 in_SyncPointCount: Die Anzahl der Synchronisationspunkte seit dem letzten Ereignis. Wenn die Ereignisse schneller ausgelöst als empfangen werden, dann werden mehrere Ereignisse zu einem zusammengefasst. In diesem Fall enthält dieser Wert die Anzahl der Zyklen, seit das letzte Ereignis empfangen wurde.
Rückgabewerte	Keine

9.3.8 Delegate_IRRTM

Beschreibung

Tabelle 9-26 Delegate_IRRTM - .NET (C#)

Syntax	<pre>delegate void Delegate_IRRTM(IRemoteRuntimeManager in_Sender);</pre>
Parameter	IRemoteRuntimeManager in_Sender: Eine Schnittstelle des Remote Runtime Managers, die dieses Ereignis empfängt.
Rückgabewerte	Keine

9.3.9 Delegate_II_EREC_DT_SRICC_UINT32_UINT32_UINT32_UINT32

Beschreibung

Tabelle 9-27 Delegate_II_EREC_DT_SRICC_UINT32_UINT32_UINT32_UINT32 - .NET (C#)

Syntax	<pre>delegate void Delegate_II_EREC_DT_SRICC_UINT32_UINT32_UINT32_UINT32(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, EInstanceConfigChanged in_InstanceConfigChanged, UInt32 in_Param1, UInt32 in_Param2, UInt32 in_Param3, UInt32 in_Param4);</pre>				
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde.				
	EInstanceCon- figChanged in_InstanceC- onfigChanged	UInt32 in_Param1	UInt32 in_Param2	UInt32 in_Param3	UInt32 in_Param4
	HardwareSoft- wareChanged	-	-	-	-
	IPChanged	Die ID der Schnitt- stelle	Die neue IP	Die neue Subnetz- maske	Das neue Stan- dard-Gateway
Rückgabewerte	Keine				

9.3.10 Delegate_II_EREC_DT_SDRI

Beschreibung

Tabelle 9-28 Delegate_II_EREC_DT_SDRI - .NET (C#)

Syntax	<pre>delegate void Delegate_II_EREC_DT_SDRI (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, SDataRecordInfo in_DataRecordInfo);</pre>				
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. SDataRecordInfo in_DataRecordInfo: Die Struktur SDataRecordInfo enthält folgende Informationen: - Die HW-Kennung, an die die CPU den Datensatz schreiben möchte - Den Index des gelieferten Datensatzes - Die Größe des Datensatzes - Den Datensatz				
Rückgabewerte	Keine				

9.3.11 Delegate_II_EREC_DT_SDR

Beschreibung

Tabelle 9-29 Delegate_II_EREC_DT_SDR - .NET (C#)

Syntax	<pre>delegate void Delegate_II_EREC_DT_SDR (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, SDataRecord in_DataRecord);</pre>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. SDataRecord in_DataRecord: Die Struktur SDataRecord enthält folgende Informationen: – Die HW-Kennung, an die die CPU den Datensatz schreibt – Den Index des gelieferten Datensatzes – Größe des Datensatzes – Den Datensatz
Rückgabewerte	Keine

9.3.12 Delegate_SREC_ST_UINT32_EPET_UINT32

Beschreibung

Tabelle 9-30 Delegate_SREC_ST_UINT32_EPET_UINT32 - .NET (C#)

Syntax	<pre>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UInt32 in_SequenceNumber);</pre>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Ziehen/Stecken-Ereignis generiert hat. EPullOrPlugEventType in_PullOrPlugEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module (Seite 255). UInt32 in_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Entsprechend den PROFINET-Standards ist die Sequenznummer nur 10 Bits weit, sie geht von 1 bis 0x7FF. Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardware-System verwendet der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.3.13 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32

Beschreibung

Tabelle 9-31 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 - Native C++

Syntax	<pre>delegate void Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, UInt32 in_Channel, EProcessEventType in_ProcessEventType, UInt32 in_SequenceNumber);</pre>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das das Prozessereignis generiert. UInt32 in_Channel: Der Kanal des E/A-Moduls, das das Prozessereignis sendet. EProcessEventType in_ProcessEventType: Ein Wert aus der Liste der vordefinierten Typen von Ereignissen für S7-Module; siehe EProcessEventType (Seite 255). UInt32 in_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardwaresystem nutzt der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.3.14 Delegate_SREC_ST_UINT32

Beschreibung

Tabelle 9-32 Delegate_SREC_ST_UINT32 - .NET (C#)

Syntax	<pre>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier);</pre>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_HardwareIdentifier: Die ID des Moduls, das das Status-, Update- oder Profile-Ereignis generiert.
Rückgabewerte	Keine

9.3.15 Delegate_SREC_ST_UINT32_UINT32

Beschreibung

Tabelle 9-33 Delegate_SREC_ST_UINT32_UINT32 - .NET (C#)

Syntax	<code>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, UInt32 in_SequenceNumber);</code>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das den Diagnoseeintrag sendet. UInt32 in_SequenceNumber: S7-PLCSIM Advanced weist jedem Alarmereignis eine eindeutige Folgenummer zu. Nach PROFINET-Standard ist die Sequenznummer 10 Bits weit (1 bis 7FF_H). Wenn die höchste Nummer erreicht ist, beginnt die Nummerierung wieder bei 1. Hinweis In einem realen Hardware-System verwendet der IO-Controller die Sequenznummer, um zu überprüfen, ob er einen Prozessalarm verloren hat. Bei der Simulation stellt die Sequenznummer den Bezug zwischen Alarmanforderung und der zugehörigen azyklischen Meldung her.
Rückgabewerte	Keine

9.3.16 Delegate_SREC_ST_UINT32_ERSFET

Beschreibung

Tabelle 9-34 Delegate_SREC_ST_UINT32_ERSFET - .NET (C#)

Syntax	<code>delegate void Delegate_SREC_ST_UINT32_ERSFET(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, ERackOrStationFaultType in_EventType);</code>
Parameter	IInstance in_Sender: Eine Schnittstelle der Instanz, die dieses Ereignis empfängt. ERuntimeErrorCode in_ErrorCode: Ein möglicher Fehlercode DateTime in_DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. UInt32 in_HardwareIdentifier: Die HW-Kennung des Moduls oder Submoduls, das den Diagnoseeintrag sendet. ERackOrStationFaultType in_EventType: Ein Wert aus der Liste der vordefinierten RackOrStationFault-Ereignis-Typen (Seite 257).
Rückgabewerte	Keine

9.4 Definitionen und Konstanten

Folgende Bezeichner werden in der API verwendet:

Tabelle 9-35 Definitionen - Native C++

Bezeichner	Wert	Beschreibung
DINSTANCE_NAME_MAX_LENGTH	64	Der eindeutige Name einer Instanz muss kleiner sein als dieser Wert.
DSTORAGE_PATH_MAX_LENGTH	130	Die maximale Pfadlänge zur virtuellen Memory Card. Einschließlich der NULL-Terminierung.
DTAG_NAME_MAX_LENGTH	300	Die maximale Länge des Namens einer PLC-Variablen. Einschließlich der NULL-Terminierung.
DTAG_ARRAY_DIMENSION	6	Die maximale Anzahl der Dimension eines multi-dimensionalen Feldes.
DCONTROLLER_NAME_MAX_LENGTH	128	Die maximale Länge des Controller-Namens. Einschließlich der NULL-Terminierung.
DCONTROLLER_SHORT_DESIGNATION_MAX_LENGTH	32	Die maximale Länge der Kurzbezeichnung des Controllers (CPU-Typ). Einschließlich der NULL-Terminierung.
DALARM_NOTIFICATION_MAX_DIAG_EVENTS	100	Die maximale Anzahl der Diagnoseereignisse, die in einer Diagnosemeldung gesendet werden.
DPROCESS_EVENT_NAME_MAX_LENGTH	123	Die maximale Länge des Namens für das Prozessereignis. Einschließlich der NULL-Terminierung.
DPROCESS_EVENTS_MAX_ITEMS	256	Die maximale Anzahl konfigurierbaren Prozessereignisse.
DMODULE_STATE_OK	0	AlarmNotification: Modulstatus OK
DMODULE_STATE_ERROR	1	AlarmNotification: Modulstatus fehlerhaft
DMODULE_STATE_MAINT_DEMANDED	2	AlarmNotification: Wartungsanforderung
DMODULE_STATE_MAINT_REQUIRED	4	AlarmNotification: Wartungsbedarf

Tabelle 9-36 Konstanten - .NET (C#)

Bezeichner	Wert	Beschreibung
RuntimeConstants.InstanceNameLength	64	Der eindeutige Name einer Instanz muss kleiner sein als dieser Wert.
RuntimeConstants.StoragePathMaxLength	130	Die maximale Pfadlänge zur virtuellen Memory Card. Einschließlich der NULL-Terminierung.
RuntimeConstants.TagNameMaxLength	300	Die maximale Länge des Namens einer PLC-Variablen. Einschließlich der NULL-Terminierung.
RuntimeConstants.TagArrayDimension	6	Die maximale Anzahl der Dimension eines multi-dimensionalen Feldes.
RuntimeConstants.ControllerNameMaxLength	128	Die maximale Länge des Controller-Namens. Einschließlich der NULL-Terminierung.
RuntimeConstants.ControllerShortDesignationMaxLength	32	Die maximale Länge der Kurzbezeichnung des Controllers (CPU-Typ). Einschließlich der NULL-Terminierung.
ModuleState.Ok	0	AlarmNotification: Modulstatus OK
ModuleState.Error	1	AlarmNotification: Modulstatus fehlerhaft
ModuleState.MaintenanceDemanded	2	AlarmNotification: Wartungsanforderung
ModuleState.MaintenanceRequired	4	AlarmNotification: Wartungsbedarf

9.5 Unions (Native C++)

9.5.1 UIP

Beschreibung

Enthält eine IPv4-Adresse.

Tabelle 9-37 UIP - Native C++

Syntax	<pre>union UIP { DWORD IP; BYTE IPs[4]; };</pre>
Member	• DWORD IP: Die IP-Adresse in einem einzelnen DWORD • BYTE IPs[4]: Die vier Elemente der IP in absteigender Reihenfolge
Beispiel	Beispiel für eine IP-Adresse: 192.168.0.1 UIP.IP = 0xCOA80001 UIP.IPs[3] = 192, UIP.IPs[2] = 168, UIP.IPs[1] = 0, UIP.IPs[0] = 1

9.5.2 UDataValue

Beschreibung

Enthält den Wert einer PLC-Variable.

Tabelle 9-38 UDataValue - Native C++

Syntax	<pre>union UDataValue { bool Bool; INT8 Int8; INT16 Int16; INT32 Int32; INT64 Int64; UINT8 UInt8; UINT16 UInt16; UINT32 UInt32; UINT64 UInt64; float Float; double Double; CHAR Char; WCHAR WChar; };</pre>
Member	• bool Bool: 1-Byte boolscher Wert • INT8 Int8: 1-Byte-Ganzzahl mit Vorzeichen • INT16 Int16: 2-Byte-Ganzzahl mit Vorzeichen • INT32 Int32: 4-Byte-Ganzzahl mit Vorzeichen • INT64 Int64: 8-Byte-Ganzzahl mit Vorzeichen • UINT8 UInt8: 1-Byte-Ganzzahl ohne Vorzeichen • UINT16 UInt16: 2-Byte-Ganzzahl ohne Vorzeichen

	• <code>UINT32 UInt32:</code> 4-Byte-Ganzzahl ohne Vorzeichen • <code>UINT64 UInt64:</code> 8-Byte-Ganzzahl ohne Vorzeichen • <code>float Float:</code> 4-Byte Gleitkomma Wert • <code>double Double:</code> 8-Byte Gleitkomma Wert • <code>CHAR Char:</code> 1-Byte Wert Zeichen • <code>WCHAR WChar:</code> 2-Byte Wert Zeichen
--	---

9.6 Strukturen

Folgende Strukturen stehen zur Verfügung:

- `SDataValue` (Seite 224)
- `SDVBNI` (Seite 225)
- `SDataValueByAddress` (Seite 225)
- `SDataValueByAddressWithCheck` (Seite 226)
- `SDataValueByName` (Seite 227)
- `SDataValueByNameWithCheck` (Seite 228)
- `SConnectionInfo` (Seite 228)
- `SInstanceInfo` (Seite 229)
- `SDimension` (Seite 229)
- `STagInfo` (Seite 230)
- `SIP` (Seite 231)
- `SIPSuite4` (Seite 232)
- `SOnSyncPointReachedResult` (Seite 232)
- `SDataRecordInfo` (Seite 234)
- `SDataRecord` (Seite 235)
- `SConfiguredProcessEvents` (Seite 235)
- `S DiagExtChannelDescription` (Seite 236)
- `SAutodiscoverData` (Seite 238)

9.6.1 SDataValue

Beschreibung

Die Struktur enthält den Wert und den Typ einer PLC-Variablen.

Tabelle 9-39 SDataValue - Native C++

Syntax	<pre>struct SDataValue { UDataValue Value; EPrimitiveDataType Type; };</pre>
Member	• UDataValue Value: Der Wert der PLC-Variablen • EPrimitiveDataType Type: Typ der PLC-Variablen

Tabelle 9-40 SDataValue - .NET (C#)

Syntax	<pre>struct SDataValue { bool Bool { get; set; } Int8 Int8 { get; set; } Int16 Int16 { get; set; } Int32 Int32 { get; set; } Int64 Int64 { get; set; } UInt8 UInt8 { get; set; } UInt16 UInt16 { get; set; } UInt32 UInt32 { get; set; } UInt64 UInt64 { get; set; } float Float { get; set; } double Double { get; set; } sbyte Char { get; set; } char WChar { get; set; } EPrimitiveDataType Type { get; set; } }</pre>
Member	• bool Bool: 1-Byte boolscher Wert • Int8 Int8: 1-Byte-Ganzzahl mit Vorzeichen • Int16 Int16: 2-Byte-Ganzzahl mit Vorzeichen • Int32 Int32: 4-Byte-Ganzzahl mit Vorzeichen • Int64 Int64: 8-Byte-Ganzzahl mit Vorzeichen • UInt8 UInt8: 1-Byte-Ganzzahl ohne Vorzeichen • UInt16 UInt16: 2-Byte-Ganzzahl ohne Vorzeichen • UInt32 UInt32: 4-Byte-Ganzzahl ohne Vorzeichen • UInt64 UInt64: 8-Byte-Ganzzahl ohne Vorzeichen • float Float: 4-Byte Gleitkomma Wert • double Double: 8-Byte Gleitkomma Wert • sbyte Char: 1-Byte Wert Zeichen

	• char WChar: 2-Byte Wert Zeichen • EPrimitiveDataType Type: Typ der PLC-Variablen
--	---

9.6.2 SDVBNI

Beschreibung

Diese Struktur ist nur für den internen Gebrauch. Ändern Sie diese Struktur nicht.

Tabelle 9-41 SDVBNI - Native C++

Syntax	<code>struct SDVBNI</code>
--------	----------------------------

Tabelle 9-42 SDVBNI - .NET (C#)

Syntax	<code>struct SDVBNI</code>
--------	----------------------------

9.6.3 SDataValueByAddress

Beschreibung

Diese Struktur repräsentiert eine PLC-Variable, die über ihre Adresse aufgerufen wird.

Tabelle 9-43 SDataValueByAddress - Native C++

Syntax	<pre>struct SDataValueByAddress { UINT32 Offset; UINT8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; }</pre>
Member	• UINT32 Offset Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • UINT8 Bit Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • SDataValue DataValue Der Wert der PLC-Variablen • ERuntimeErrorCode ErrorCode – SREC_TIMEOUT, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – SREC_WARNING_INVALID_CALL, wenn zuvor keine Funktion RegisterOnSyncPointReachedEvent aufgerufen wurde, siehe ERuntimeErrorCode

Tabelle 9-44 SDataValueByAddress - .NET (C#)

Syntax	<pre>struct SDataValueByAddress { UInt32 Offset; UInt8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; }</pre>
--------	--

Member	• <code>UINT32 Offset</code> Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>UINT8 Bit</code> Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>SDataValue DataValue</code> Der Wert der PLC-Variablen • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – <code>SREC_WARNING_INVALID_CALL</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde, siehe <code>ERuntimeErrorCode</code>
--------	--

9.6.4 SDataValueByAddressWithCheck

Beschreibung

Diese Struktur repräsentiert eine PLC-Variable, die über ihre Adresse aufgerufen wird.

Tabelle 9-45 SDataValueByAddressWithCheck - Native C++

Syntax	<pre>struct SDataValueByAddressWithCheck { UINT32 Offset; UINT8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; bool ValueHasChanged; }</pre>
Member	• <code>UINT32 Offset</code> Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>UINT8 Bit</code> Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>SDataValue DataValue</code> Der Wert der PLC-Variablen • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – <code>SREC_WARNING_INVALID_CALL</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde, siehe <code>ERuntimeErrorCode</code> • <code>bool ValueHasChanged</code>

Tabelle 9-46 SDataValueByAddressWithCheck - .NET (C#)

Syntax	<pre>struct SDataValueByAddressWithCheck { UInt32 Offset; UInt8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; bool ValueHasChanged; }</pre>
Member	• <code>UINT32 Offset</code> Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>UINT8 Bit</code> Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet • <code>SDataValue DataValue</code> Der Wert der PLC-Variablen

	• <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – <code>SREC_WARNING_INVALID_CALL</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde, siehe <code>ERuntimeErrorCode</code> • <code>bool ValueHasChanged</code>
--	--

9.6.5 SDataValueByName

Beschreibung

Diese Struktur repräsentiert eine PLC-Variable, die über ihren Namen aufgerufen wird.

Tabelle 9-47 SDataValueByName - Native C++

Syntax	<pre>struct SDataValueByName { WCHAR Name[DTAG_NAME_MAX_LENGTH]; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; }</pre>
Member	• <code>WCHAR Name[DTAG_NAME_MAX_LENGTH]</code> Der Name der Variablen • <code>SDataValue DataValue</code> Der Wert der PLC-Variablen • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – <code>SREC_WARNING_INVALID_CALL</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde, siehe <code>ERuntimeErrorCode</code> • <code>SDVBNI Internal</code>

Tabelle 9-48 SDataValueByName - .NET (C#)

Syntax	<pre>struct SDataValueByName { String Name; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; }</pre>
Member	• <code>String Name</code> Der Name der Variablen • <code>SDataValue DataValue</code> Der Wert der PLC-Variablen • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde – <code>SREC_WARNING_INVALID_CALL</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde, siehe <code>ERuntimeErrorCode</code> • <code>SDVBNI Internal</code>

9.6.6 SDataValueByNameWithCheck

Beschreibung

Diese Struktur repräsentiert eine PLC-Variable, die über ihren Namen aufgerufen wird.

Tabelle 9-49 SDataValueByNameWithCheck - Native C++

Syntax	<pre>struct SDataValueByNameWithCheck { WCHAR Name[DTAG_NAME_MAX_LENGTH]; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; bool ValueHasChanged; };</pre>
--------	--

Tabelle 9-50 SDataValueByNameWithCheck - .NET (C#)

Syntax	<pre>struct SDataValueByNameWithCheck { String Name; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; bool ValueHasChanged; }</pre>
--------	--

9.6.7 SConnectionInfo

Beschreibung

Diese Struktur enthält die IP-Adresse und den Port einer TCP/IP-Verbindung.

HINWEIS

Bei einer Software/Open Controller-Instanz muss die IP-Adresse die gleiche wie im TIA Portal-Projekt sein.

Wählen Sie in der Gerätekonfiguration im TIA Portal "Set IP address in the project" (IP-Adresse im Projekt festlegen) aus und konfigurieren Sie eine statische IP-Adresse zur PLC-Schnittstelle. Weisen Sie keine IP-Adressen mit den Werten 0.0.0.0 oder 192.168.0.1 zu.

Beim Herunterladen Ihrer Instanz wählen Sie die jeweilige Variante "Direct at slot 1 Xn" (Zu Slot 1 Xn leiten) in der Ausklappoption Connection to interface/subnet (Verbindung mit Interface/Subnet) im Fenster Extended download to device (erweiterter Download zu Gerät). Das Ladevorschauenfenster des TIA Portals zeigt eine Fehlermeldung an, wenn die IP-Adressen nicht übereinstimmen. In diesem Fall muss der Download abgebrochen werden und die Instanz erneut erstellt werden.

Tabelle 9-51 SConnectionInfo - Native C++

Syntax	<pre>struct SConnectionInfo { UIP IP; UINT16 Port; };</pre>
--------	---

Tabelle 9-52 SConnectionInfo - .NET (C#)

Syntax	<pre>struct SConnectionInfo { SIP IP; UInt16 Port; }</pre>
--------	--

9.6.8 SInstanceInfo

Beschreibung

Diese Struktur enthält eine IPv4-Adresse.

Tabelle 9-53 SInstanceInfo - Native C++

Syntax	<pre>struct SInstanceInfo { INT32 ID; WCHAR Name[DINSTANCE_NAME_MAX_LENGTH]; };</pre>
Member	• INT32 ID: Die ID der Instanz • WCHAR Name[DINSTANCE_NAME_MAX_LENGTH]: Der Name der Instanz

Tabelle 9-54 SInstanceInfo - .NET (C#)

Syntax	<pre>struct SInstanceInfo { Int32 ID; String Name; }</pre>
Member	• Int32 ID: Die ID der Instanz • String Name: Der Name der Instanz

9.6.9 SDimension

Beschreibung

Diese Struktur enthält Informationen zur Dimension eines Feldes.

Tabelle 9-55 SDimension - Native C++

Syntax	<pre>struct SDimension { INT32 StartIndex; UINT32 Count; };</pre>
--------	---

Tabelle 9-56 SDimension - .NET (C#)

Syntax	<pre>struct SDimension { Int32 StartIndex; UInt32 Count; }</pre>
--------	--

9.6.10 STagInfo

Beschreibung

Diese Struktur enthält Informationen zu einer PLC-Variablen.

Tabelle 9-57 STagInfo - Native C++

Syntax	<pre>struct STagInfo { WCHAR Name[DTAG_NAME_MAX_LENGTH]; EArea Area; EDataType DataType; EPrimitiveDataType PrimitiveDataType; UINT16 Size; UINT32 Offset; UINT8 Bit; UINT8 DimensionCount; UINT32 Index; UINT32 ParentIndex; SDimension Dimension[DTAG_ARRAY_DIMENSION]; };</pre>
Member	• WCHAR Name[DTAG_NAME_MAX_LENGTH]: Der Name der Variablen • EArea Area: Der CPU-Bereich, in dem sich die Variable befindet. • EDataType DataType: Der CPU-Datentyp der Variablen • EPrimitiveDataType PrimitiveDataType: Der primitive Datentyp der Variablen • UINT16 Size: Die Größe der Variablen in Byte • UINT32 Offset: Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet. • UINT8 Bit: Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet. • UINT8 DimensionCount: Die Anzahl der Dimensionen des Feldes. 0, wenn es sich bei der Variablen nicht um ein Feld handelt. • UINT32 Index: Der Index der Variablen • UINT32 ParentIndex: Wenn diese Variable in eine weitere Variable eingebettet ist (wie ein Element einer Struktur), dann zeigt dieser Wert den Index der übergeordneten Variablen. Der Wert ist 0, wenn die Variable keine übergeordnete Variable hat. • SDimension Dimension[DTAG_ARRAY_DIMENSION]: Informationen zu jeder Dimension des Feldes

Tabelle 9-58 STagInfo - .NET (C#)

Syntax	<pre>public struct STagInfo { String Name; EArea Area; EDataType DataType; EPrimitiveDataType PrimitiveDataType; UInt16 Size; UInt32 Offset; UInt8 Bit; UInt32 Index; UInt32 ParentIndex; SDimension[] Dimension; }</pre>
--------	---

Member	• <code>String Name</code>: Der Name der Variablen • <code>EArea Area</code>: Der CPU-Bereich, in dem sich die Variable befindet. • <code>EDataType DataType</code>: Der CPU-Datentyp der Variablen • <code>EPrimitiveDataType PrimitiveDataType</code>: Der primitive Datentyp der Variablen • <code>UInt16 Size</code>: Die Größe der Variablen in Byte. • <code>UInt32 Offset</code>: Der Byte-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet. • <code>UInt8 Bit</code>: Der Bit-Offset der Variablen, wenn sie sich nicht in einem Datenbaustein befindet. • <code>UInt32 Index</code>: Der Index der Variablen • <code>UInt32 ParentIndex</code>: Wenn diese Variable in eine weitere Variable eingebettet ist (wie ein Element einer Struktur), dann zeigt dieser Wert den Index der übergeordneten Variablen. Der Wert ist 0, wenn die Variable keine übergeordnete Variable hat. • <code>SDimension[] Dimension</code>: Informationen zu jeder Dimension des Feldes. Leer, wenn die Variable kein Feld ist.
--------	---

9.6.11 SIP

Beschreibung

Diese Struktur enthält eine IPv4-Adresse.

Tabelle 9-59 SIP - .NET (C#)

Syntax	<pre>struct SIP { byte[] IPArray { get; set; } UInt32 IPDWord { get; set; } string IPString { get; set; } }</pre>
Member	• <code>UInt32 IPDWord</code>: Die IP-Adresse in einem einzelnen DWORD • <code>byte[] IPArray</code>: Die vier Elemente der IP in absteigender Reihenfolge • <code>string IPString</code>: Die IPv4-Adresse als String
Beispiel	Beispiel für eine IP-Adresse: 192.168.0.1 <code>SIP.IPDWord = 0xC0A80001</code> <code>SIP.IPArray[3] = 192, SIP.IPArray[2] = 168, SIP.IPArray[1] = 0, SIP.IPArray[0] = 1</code> <code>SIP.IPString = "192.168.0.1"</code>

9.6.12 SIPSuite4

Beschreibung

Diese Struktur enthält eine IPv4-Suite.

Tabelle 9-60 SIPSuite4 - Native C++

Syntax	<pre>struct SIPSuite4 { UIP IPAddress; UIP SubnetMask; UIP DefaultGateway; };</pre>
Member	• UIP IPAddress: Die IP-Adresse • UIP SubnetMask: Die Subnetzmaske • UIP DefaultGateway: Das Standard-Gateway

Tabelle 9-61 SIPSuite4 - .NET (C#)

Syntax	<pre>struct SIPSuite4 { SIP IPAddress; SIP SubnetMask; SIP DefaultGateway; }</pre>
Member	• SIP IPAddress: Die IP-Adresse • SIP SubnetMask: Die Subnetzmaske • SIP DefaultGateway: Das Standard-Gateway

9.6.13 SOnSyncPointReachedResult

Beschreibung

Diese Struktur enthält die Ergebnisse des OnSyncPointReached-Ereignisses.

Tabelle 9-62 SOnSyncPointReachedResult - Native C++

Syntax	<pre>struct SOnSyncPointReachedResult { ERuntimeErrorCode ErrorCode; SYSTEMTIME SystemTime; UINT32 Id; INT64 TimeSinceSameSyncPoint_ns; INT64 TimeSinceAnySyncPoint_ns; UINT32 SyncPointCount; };</pre>
Member	• ERuntimeErrorCode ErrorCode: – SREC_TIMEOUT, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde. In einem solchen Fall werden die struct-Werte nicht verwendet. – SREC_WARNING_INVALID_CALL, wenn zuvor keine Funktion RegisterOnSyncPointReachedEvent aufgerufen wurde. Siehe ERuntimeErrorCode. • SYSTEMTIME SystemTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde.

	• UINT32 Id: Für die Betriebsart <code>SingleStep_Bus</code>: Die ID des Peripheriesystems, das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet wird. Für alle anderen Betriebsarten gilt: Die ID des Teilprozessabbilds (TPA), das dieses Ereignis auslöst. 0 für den Zykluskontrollpunkt (End of cycle). Hinweis: In den SingleStep-Betriebsarten hat die übergebene ID für den Zykluskontrollpunkt den Wert 0xFFFF. In diesem Fall werden die Time-Werte auf 0 gesetzt. • INT64 TimeSinceSameSyncPoint_ns: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt derselben Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>. • INT64 TimeSinceAnySyncPoint_ns: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt einer beliebigen Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>. • UINT32 SyncPointCount: Die Anzahl der Synchronisationspunkte seit dem letzten Ereignis. Wenn die Ereignisse schneller ausgelöst als empfangen werden, dann werden mehrere Ereignisse zu einem zusammengefasst. In diesem Fall enthält dieser Wert die Anzahl der Zyklen, seit das letzte Ereignis empfangen wurde.
--	--

Tabelle 9-63 SOnSyncPointReachedResult - .NET (C#)

Syntax	<pre>struct SOnSyncPointReachedResult { ERuntimeErrorCode ErrorCode; DateTime SystemTime; UInt32 Id; Int64 TimeSinceSameSyncPoint_ns; Int64 TimeSinceAnySyncPoint_ns; UInt32 SyncPointCount; }</pre>
Member	• ERuntimeErrorCode ErrorCode: – <code>ERuntimeErrorCode.Timeout</code>, wenn während der definierten Zeitspanne kein Ereignis ausgelöst wurde. – <code>WarningInvalidCall</code>, wenn zuvor keine Funktion <code>RegisterOnSyncPointReachedEvent</code> aufgerufen wurde. Siehe <code>ERuntimeErrorCode</code>. • DateTime DateTime: Die virtuelle Systemzeit der simulierten PLC-Instanz zum Zeitpunkt, als dieses Ereignis ausgelöst wurde. • UInt32 Id: Für die Betriebsart <code>SingleStep_Bus</code>: Die ID des Peripheriesystems, das zur zyklusgesteuerten Synchronisation der simulierten PLC-Instanz verwendet wird. Für alle anderen Betriebsarten gilt: Die ID des Teilprozessabbilds (TPA), das dieses Ereignis auslöst. 0 für den Zykluskontrollpunkt (End of cycle). • Int64 TimeSinceSameSyncPoint_ns: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt derselben Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>.

	• <code>Int64 TimeSinceAnySyncPoint_ns</code>: Die virtuelle Zeit (in Nanosekunden), seit der letzte Synchronisationspunkt einer beliebigen Teilprozessabbild-ID erreicht wurde. Für die zeitgesteuerten Betriebsarten: Laufzeit seit dem letzten Aufruf der Funktion <code>StartProcessing()</code>. • <code>UInt32 SyncPointCount</code>: Die Anzahl der Synchronisationspunkte seit dem letzten Ereignis. Wenn die Ereignisse schneller ausgelöst als empfangen werden, dann werden mehrere Ereignisse zu einem zusammengefasst. In diesem Fall enthält dieser Wert die Anzahl der Zyklen, seit das letzte Ereignis empfangen wurde.
--	---

9.6.14 SDataRecordInfo

Beschreibung

Diese Struktur enthält Lesen/Schreiben Datensatz-Informationen.

Tabelle 9-64 SDataRecordInfo - Native C++

Syntax	<pre>struct SDataRecordInfo { UINT32 HardwareId; UINT32 RecordIdx; UINT32 DataSize; };</pre>
Member	• <code>UINT32 HardwareId</code>: Die ID des Hardware-Moduls (Hardware-Identifizier) • <code>UINT32 RecordIdx</code>: Die Datensatznummer • <code>UINT32 DataSize</code>: Die Datensatzgröße

Tabelle 9-65 SDataRecordInfo - .NET (C#)

Syntax	<pre>struct SDataRecordInfo { UInt32 HardwareId; UInt32 RecordIdx; UInt32 DataSize; }</pre>
Member	• <code>UInt32 ID</code>: Die ID des Hardware-Moduls • <code>UInt32 RecordIdx</code>: Die Datensatznummer • <code>UInt32 DataSize</code>: Die Datensatzgröße

9.6.15 SDataRecord

Beschreibung

Diese Struktur enthält Lesen/Schreiben Datensatz-Informationen und Datensätze.

Tabelle 9-66 SDataRecord - .NET (C#)

Syntax	<pre>struct SDataRecord { UInt32 HardwareId; byte[] Data }</pre>
Member	• SDataRecordInfo Info: Die Datensatz-Information; siehe SDataRecordInfo (Seite 234) • byte[] Data: Die Feldgröße

9.6.16 SConfiguredProcessEvents

Beschreibung

Diese Struktur enthält Informationen über die konfigurierten Prozessereignisse.

Tabelle 9-67 SConfiguredProcessEvents - Native C++

Syntax	<pre>struct SConfiguredProcessEvents { UINT16 HardwareIdentifier; UINT16 Channel; EProcessEventType ProcessEventType; WCHAR Name[DPROCESS_EVENT_NAME_MAX_LENGTH]; };</pre>
Member	• UINT16 HardwareIdentifier: Die HW-Kennung • UINT16 Channel: Der Kanal des I/O-Moduls, das das Prozessereignis generiert. • EProcessEventType ProcessEventType: Der Typ des konfigurierten Prozessereignisses • WCHAR Name[DPROCESS_EVENT_NAME_MAX_LENGTH]: Der Name des Prozessereignisses

Tabelle 9-68 SConfiguredProcessEvents - .NET (C#)

Syntax	<pre>public struct SConfiguredProcessEvents { ushort HardwareIdentifier; ushort Channel; EProcessEventType ProcessEventType; string Name; }</pre>
Member	• ushort HardwareIdentifier: Die HW-Kennung • ushort Channel: Der Kanal des I/O-Moduls, das das Prozessereignis generiert. • EProcessEventType ProcessEventType: Der Typ des konfigurierten Prozessereignisses • string Name: Der Name des Prozessereignisses

9.6.17 SDiagExtChannelDescription

Beschreibung

Diese Struktur enthält Lesen/Schreiben Datensatz-Informationen und Datensätze.

Tabelle 9-69 SDiagExtChannelDescription - Native C++

Syntax	<pre>struct SDiagExtChannelDescription { UINT16 ChannelNumber; UINT16 ErrorType; UINT16 ExtErrorType; EDiagSeverity Severity; EDiagProperty Direction; };</pre>
Member	• UINT16 ChannelNumber: Wenn sich der Alarm auf einen bestimmten Kanal des IO-Device bezieht (z. B. Kurzschluss), dann muss dieser Parameter die Nummer des gestörten Kanals enthalten. Wenn der Alarm von einem Modul oder Submodul generiert wird, dann muss die Nummer des Kanals auf 0x8000 gesetzt werden. • UINT16 ErrorType: Der Parameter definiert Fehlertypen nach PROFINET-Standard, siehe Absatz "Fehlertypen". • EDiagSeverity Severity: Der Wert des Schweregrads für die Diagnose; siehe EDiagSeverity (Seite 257). • EDiagProperty Direction: Der Wert für die Kommend/Gehend-Information; siehe EDiagProperty (Seite 256). • UINT16 ExtErrorType: Dieser Parameter bietet die Möglichkeit, mehr Details zum Diagnosealarm zu definieren. Dies ist hilfreich in Kombination mit PDEV-Fehlertypen, die für CPU-interne Module generiert werden. Der Standardwert sollte 0 sein.

Tabelle 9-70 SDiagExtChannelDescription - .NET (C#)

Syntax	<pre>struct SDiagExtChannelDescription { UInt16 ChannelNumber; UInt16 ErrorType; UInt16 ExtErrorType; EDiagSeverity Severity; EDiagProperty Direction; };</pre>
Member	• UInt16 ChannelNumber: Wenn sich der Alarm auf einen bestimmten Kanal des IO-Device bezieht (z. B. Kurzschluss), dann muss dieser Parameter die Nummer des gestörten Kanals enthalten. Wenn der Alarm von einem Modul oder Submodul generiert wird, dann muss die Nummer des Kanals auf 0x8000 gesetzt werden. • UInt16 ErrorType: Der Parameter definiert Fehlertypen nach PROFINET-Standard, siehe Absatz "Fehlertypen". • EDiagSeverity Severity: Der Wert des Schweregrads für die Diagnose; siehe EDiagSeverity (Seite 257). • EDiagProperty Direction: Der Wert für die Kommend/Gehend-Information; siehe EDiagProperty (Seite 256). • UInt16 ExtErrorType: Dieser Parameter bietet die Möglichkeit, mehr Details zum Diagnosealarm zu definieren. Dies ist hilfreich in Kombination mit PDEV-Fehlertypen, die für CPU-interne Module generiert werden. Der Standardwert sollte 0 sein.

Fehlertypen

Die folgende Tabelle enthält wichtige Fehlertypen (`ErrorType`) nach PROFINET-Standard:

Tabelle 9-71 Fehlertypen nach PROFINET-Standard

Wert	Bedeutung
0x0000	Reserviert / Unbekannter Fehler
0x0001	Kurzschluss
0x0002	Unterspannung
0x0003	Überspannung
0x0004	Überlast
0x0005	Übertemperatur
0x0006	Drahtbruch
0x0007	Obergrenze überschritten
0x0008	Untergrenze unterschritten
0x0009	Fehler

Die folgende Tabelle enthält Fehlertypen `ExtChannelErrorType` für `ChannelErrorType` "Remote mismatch":

Tabelle 9-72 Fehlertypen `ExtChannelErrorType`

Wert	Bedeutung	Gebrauch
0x0000	Reserviert	-
0x0001 bis 0x7FFF	Herstellerkennung	Alarm/Diagnose
0x8000	Peer name of station mismatch	Alarm/Diagnose
0x8001	Peer name of port mismatch	Alarm/Diagnose
0x8002	Peer RT_CLASS_3 mismatch	Alarm/Diagnose
0x8003	Peer MAU Type mismatch	Alarm/Diagnose

9.6.18 SAutodiscoverData

Beschreibung

Diese Struktur enthält die IP-Adresse, den Port, die Runtime-Version und den Namen des Computers, auf dem sich ein Runtime Manager befindet, der bereit ist, eine Remote-Verbindung herzustellen.

Tabelle 9-73 SAutodiscoverData - Native C++

Syntax	<pre>public struct SAutodiscoverData { UIP IP; UINT16 Port; DWORD RuntimeVersion; WCHAR ComputerName[MAX_COMPUTERNAME_LENGTH + 1]; };</pre>
--------	---

Tabelle 9-74 SAutodiscoverData - .NET (C#)

Syntax	<pre>public struct SAutodiscoverData { public SIP IP; public ushort Port; public uint RuntimeVersion; public string ComputerName; }</pre>
--------	---

9.6.19 SMACAddress

Beschreibung

Definiert die Struktur einer MAC-Adresse.

Tabelle 9-75 SMACAddress - Native C++

Syntax	<pre>#define DMAC_ADDRESS_LEN 6 struct SMACAddress { BYTE Octets[DMAC_ADDRESS_LEN]; };</pre>
--------	---

9.6.20 SNetInterfaceInfo

Beschreibung

Beschreibt die Struktur einer Netzwerkschnittstelle.

Tabelle 9-76 SNetInterfaceInfo - Native C++

Syntax	<pre>struct SNetInterfaceInfo { DWORD interfaceIndex; UMACAddress MACAddress; WCHAR interfaceName[64]; WCHAR interfaceDescription[64]; };</pre>
--------	---

9.7 Aufzählungen

Folgende Aufzählungen stehen zur Verfügung:

- ERuntimeErrorCode (Seite 239)
- EArea (Seite 242)
- EOperatingState (Seite 242)
- EOperatingMode (Seite 243)
- ECPUType (Seite 244)
- ECommunicationInterface (Seite 246)
- ENetworkMode (Seite 246)
- EPLCInterface (Seite 247)
- ELEDType (Seite 247)
- ELEDMode (Seite 248)
- EPrimitiveDataType (Seite 248)
- EDataType (Seite 250)
- ETagListDetails (Seite 253)
- ERuntimeConfigChanged (Seite 254)
- EInstanceConfigChanged (Seite 254)
- EPullOrPlugEventType (Seite 255)
- EProcessEventTypes (Seite 255)
- ESoftwareConfigChanged (Seite 256)
- EDirection (Seite 256)
- EDiagProperty (Seite 256)
- EDiagSeverity (Seite 257)
- ERackOrStationFaultType (Seite 257)
- ECycleTimeMonitoringMode (Seite 258)
- EAutodiscoverType (Seite 258)

9.7.1 ERuntimeErrorCode

Beschreibung

Diese Aufzählung enthält alle Fehlercodes, die von der Simulation Runtime API genutzt werden. Die meisten API-Funktionen geben einen dieser Fehlercodes zurück. Wenn die Funktionen erfolgreich sind, ist der Rückgabewert immer SREC_OK / OK. Fehler werden mit negativen Werten zurückgegeben, Warnungen mit positiven.

Tabelle 9-77 ERuntimeErrorCode - Native C++

Syntax	<pre>enum ERuntimeErrorCode { SREC_OK = 0, SREC_INVALID_ERROR_CODE = -1, SREC_NOT_IMPLEMENTED = -2, SREC_INDEX_OUT_OF_RANGE = -3, SREC_DOES_NOT_EXIST = -4, SREC_ALREADY_EXISTS = -5, SREC_UNKNOWN_MESSAGE_TYPE = -6, SREC_INVALID_MESSAGE_ID = -7, SREC_WRONG_ARGUMENT = -8, SREC_WRONG_PIPE = -9, SREC_CONNECTION_ERROR = -10, SREC_TIMEOUT = -11, }</pre>
--------	--

```

SREC_MESSAGE_CORRUPT = -12,
SREC_WRONG_VERSION = -13,
SREC_INSTANCE_NOT_RUNNING = -14,
SREC_INTERFACE_REMOVED = -15,
SREC_SHARED_MEMORY_NOT_INITIALIZED = -16,
SREC_API_NOT_INITIALIZED = -17,
SREC_WARNING_ALREADY_EXISTS = 18,
SREC_NOT_SUPPORTED = -19,
SREC_WARNING_INVALID_CALL = 20,
SREC_ERROR_LOADING_DLL = -21,
SREC_SIGNAL_NAME_DOES_NOT_EXIST = -22,
SREC_SIGNAL_TYPE_MISMATCH = -23,
SREC_SIGNAL_CONFIGURATION_ERROR = -24,
SREC_NO_SIGNAL_CONFIGURATION_LOADED = -25,
SREC_CONFIGURED_CONNECTION_NOT_FOUND = -26,
SREC_CONFIGURED_DEVICE_NOT_FOUND = -27,
SREC_INVALID_CONFIGURATION = -28,
SREC_TYPE_MISMATCH = -29,
SREC_LICENSE_NOT_FOUND = -30,
SREC_NO_LICENSE_AVAILABLE = -31,
SREC_WRONG_COMMUNICATION_INTERFACE = -32,
SREC_LIMIT_REACHED = -33,
SREC_NO_STORAGE_PATH_SET = -34,
SREC_STORAGE_PATH_ALREADY_IN_USE = -35,
SREC_MESSAGE_INCOMPLETE = -36,
SREC_ARCHIVE_STORAGE_NOT_CREATED = -37,
SREC_RETRIEVE_STORAGE_FAILURE = -38,
SREC_INVALID_OPERATING_STATE = -39,
SREC_INVALID_ARCHIVE_PATH = -40,
SREC_DELETE_EXISTING_STORAGE_FAILED = -41,
SREC_CREATE_DIRECTORIES_FAILED = -42,
SREC_NOT_ENOUGH_MEMORY = -43,
SREC_WARNING_TRIAL_MODE_ACTIVE = 44,
SREC_NOT_RUNNING = -45,
SREC_NOT_EMPTY = -46,
SREC_NOT_UP_TO_DATE = -47,
SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE = -48,
SREC_WARNING_NOT_COMPLETE = 49,
SREC_VIRTUAL_SWITCH_MISCONFIGURED = -50,
SREC_RUNTIME_NOT_AVAILABLE = -51,
SREC_IS_EMPTY = -52,
SREC_WRONG_MODULE_STATE = -53,
SREC_WRONG_MODULE_TYPE = -54,
SREC_NOT_SUPPORTED_BY_MODULE = -55,
SREC_INTERNAL_ERROR = -56,
SREC_STORAGE_TRANSFER_ERROR = -57,
SREC_ANOTHER_VARIANT_OF_PLCSIM_RUNNING = -58,
SREC_ACCESS_DENIED = -59,
SREC_NOT_ALLOWED_DURING_DOWNLOAD = -60,
SREC_AUTODISCOVER_ALREADY_RUNNING = -61,
SREC_INVALID_STORAGE = -62,
SREC_WARNING_UNSUPPORTED_PCAP_DRIVER = 63, // (Abgekündigt seit V4)
SREC_WARNING_RUNNING_ON_TIA_PORTAL_TEST_SUITE = 64,
SREC_PCAP_DRIVER_NOT_RUNNING = -65,
SREC_UNSUPPORTED_PCAP_DRIVER = -66,
SREC_CONFIG_FILE_ERROR = -67,
SREC_WARNING_PASSWORD_PROTECTION_ERROR = 68,
SREC_BUFFER_OVERFLOW = -69,
SREC_NOT_ALLOWED = -70,
SREC_WARNING_NO_EXTERNAL_COMMUNICATION = 71
SREC_INSTANCE_ALREADY_RUNNING = -72
SREC_OUT_OF_MEMORY = -73
};

```

Tabelle 9-78 ERuntimeErrorCode - .NET (C#)

Syntax	<pre> enum ERuntimeErrorCode { OK = 0, InvalidErrorCode = -1, NotImplemented = -2, IndexOutOfRange = -3, DoesNotExist = -4, AlreadyExists = -5, UnknownMessageType = -6, InvalidMessageId = -7, WrongArgument = -8, WrongPipe = -9, ConnectionError = -10, Timeout = -11, MessageCorrupt = -12, WrongVersion = -13, InstanceNotRunning = -14, InterfaceRemoved = -15, SharedMemoryNotInitialized = -16, ApiNotInitialized = -17, WarningAlreadyExists = 18, NotSupported = -19, WarningInvalidCall = 20, ErrorLoadingDll = -21, SignalNameDoesNotExist = -22, SignalTypeMismatch = -23, SignalConfigurationError = -24, NoSignalConfigurationLoaded = -25, ConfiguredConnectionNotFound = -26, ConfiguredDeviceNotFound = -27, InvalidConfiguration = -28, TypeMismatch = -29, LicenseNotFound = -30, NoLicenseAvailable = -31, WrongCommunicationInterface = -32, LimitReached = -33, NoStartupPathSet = -34, StartupPathAlreadyInUse = -35, MessageIncomplete = -36, ArchiveStorageNotCreated = -37, RetrieveStorageFailure = -38, InvalidOperatingState = -39, InvalidArchivePath = -40, DeleteExistingStorageFailed = -41, CreateDirectoriesFailed = -42, NotEnoughMemory = -43, WarningTrialModeActive = 44, NotRunning = -45, NotEmpty = -46, NotUpToDate = -47, CommunicationInterfaceNotAvailable = -48, WarningNotComplete = 49, VirtualSwitchMisconfigured = -50, RuntimeNotAvailable = -51, IsEmpty = -52, WrongModuleState = -53, WrongModuleType = -54, NotSupportedByModule = -55, InternalError = -56, StorageTransferError = -57, AnotherVariantOfPlcsimRunning = -58, AccessDenied = -59, NotAllowedDuringDownload = -60, AutodiscoverAlreadyRunning = -61, InvalidStorage = -62, WarningUnsupportedPcapDriver = 63, //(Abgekündigt seit V4) WarningRunningOnTiaPortalTestSuite = 64, PCAPDriverNotRunning = -65, UnsupportedPcapDriver = -66, </pre>
--------	--

	<pre>ConfigFileError = -67, WarningPasswordForProtectionError = 68, BufferOverflow = - 69, NotAllowed = -70, WarningNoExternalCommunication = 71 InstanceAlreadyRunning = -72 OutOfMemory = -73 }</pre>
--	---

9.7.2 EArea

Beschreibung

Diese Aufzählung enthält alle PLC-Areas, die verfügbare PLC-Variablen beinhalten.

Tabelle 9-79 EArea - Native C++

Syntax	<pre>enum EArea { SRA_INVALID_AREA = 0, SRA_INPUT = 1, SRA_MARKER = 2, SRA_OUTPUT = 3, SRA_COUNTER = 4, SRA_TIMER = 5, SRA_DATABLOCK = 6, SRA_ENUMERATION_SIZE = 7 };</pre>
--------	---

Tabelle 9-80 EArea - .NET (C#)

Syntax	<pre>public enum EArea { InvalidArea = 0, Input = 1, Marker = 2, Output = 3, Counter = 4, Timer = 5, DataBlock = 6, }</pre>
--------	---

9.7.3 EOperatingState

Beschreibung

Diese Aufzählung enthält alle Betriebszustände einer simulierten PLC-Instanz.

Tabelle 9-81 EOperatingState - Native C++

Syntax	<pre>enum EOperatingState { SROS_INVALID_OPERATING_STATE = 0, SROS_OFF = 1, SROS_BOOTING = 2, SROS_STOP = 3, SROS_STARTUP = 4, SROS_RUN = 5, SROS_FREEZE = 6, SROS_SHUTTING_DOWN = 7, SROS_HOLD = 8, SROS_ENUMERATION_SIZE = 9 };</pre>
--------	--

Tabelle 9-82 EOperatingState - .NET (C#)

Syntax	<pre>enum EOperatingState { InvalidOperatingState = 0, Off = 1, Booting = 2, Stop = 3, Startup = 4, Run = 5, Freeze = 6, ShuttingDown = 7, Hold = 8 }</pre>
--------	---

9.7.4 EOperatingMode

Beschreibung

Diese Aufzählung enthält alle Betriebsarten einer simulierten PLC-Instanz.

Tabelle 9-83 EOperatingMode - Native C++

Syntax	<pre>enum EOperatingMode { SROM_DEFAULT = 0, SROM_SINGLE_STEP_C = 1, SROM_SINGLE_STEP_CT = 2, SROM_TIMESPAN_SYNCHNRONIZED_C = 3, SROM_SINGLE_STEP_P = 4, SROM_TIMESPAN_SYNCHNRONIZED_P = 5, SROM_SINGLE_STEP_CP = 6, SROM_SINGLE_STEP_CPT = 7, SROM_TIMESPAN_SYNCHNRONIZED_CP = 8, SROM_SINGLE_STEP_BUS = 9 };</pre>
--------	--

Tabelle 9-84 EOperatingMode - .NET (C#)

Syntax	<pre>public enum EOperatingMode { Default = 0, SingleStep_C = 1, SingleStep_CT = 2, TimespanSynchronized_C = 3, SingleStep_P = 4, TimespanSynchronized_P = 5, SingleStep_CP = 6, SingleStep_CPT = 7, TimespanSynchronized_CP = 8, SingleStep_Bus = 9 }</pre>
--------	--

9.7.5 ECPUType

Beschreibung

Diese Aufzählung enthält alle CPU-Typen, die in eine simulierte PLC-Instanz geladen werden können.

Tabelle 9-85 ECPUType - Native C++

Syntax	<pre>enum ECPUType { SRCT_1500_Unspecified = 0x000005DC, SRCT_1511 = 0x000005E7, SRCT_1513 = 0x000005E9, SRCT_1515 = 0x000005EB, SRCT_1516 = 0x000005EC, SRCT_1517 = 0x000005ED, SRCT_1518 = 0x000005EE, SRCT_1511C = 0x000405E7, SRCT_1512C = 0x000405E8, SRCT_1511F = 0x000105E7, SRCT_1513F = 0x000105E9, SRCT_1515F = 0x000105EB, SRCT_1516F = 0x000105EC, SRCT_1517F = 0x000105ED, SRCT_1518F = 0x000105EE, SRCT_1511T = 0x000805E7, SRCT_1515T = 0x000805EB, SRCT_1516T = 0x000805EC, SRCT_1517T = 0x000805ED, SRCT_1511TF = 0x000905E7, SRCT_1515TF = 0x000905EB, SRCT_1516TF = 0x000905EC, SRCT_1517TF = 0x000905ED, SRCT_1518ODK = 0x001005EE, SRCT_1518FODK = 0x001105EE, SRCT_1518MFP = 0x004005EE, SRCT_1518FMFP = 0x004105EE, SRCT_1518T = 0x000805EE, SRCT_1518TF = 0x000905EE, SRCT_1504DTF = 0x000905E0, SRCT_1507DTF = 0x000905E3, SRCT_ET200SP_Unspecified = 0x000205DC, SRCT_1510SP = 0x000205E6, SRCT_1512SP = 0x000205E8, SRCT_1510SPF = 0x000305E6, SRCT_1512SPF = 0x000305E8, SRCT_1514SP = 0x000205E7, SRCT_1514SPT = 0x000205E9, SRCT_1514SPF = 0x000305E7, SRCT_1514SPTF = 0x000305E9, SRCT_1514PA = 0x000305EB, SRCT_1500_RH_Unspecified = 0x008005DC, SRCT_1513R = 0x008005E9, SRCT_1515R = 0x008005EB, SRCT_1517H = 0x008005ED, SRCT_1518HF = 0x008105EE, SRCT_ET200PRO_Unspecified = 0x002005DC, SRCT_1513PRO = 0x002005E9, SRCT_1513PROF = 0x002105E9, SRCT_1516PRO = 0x002005EC, SRCT_1516PROF = 0x002105EC, SRCT_1500_SW_OC_Unspecified = 0x010005DC, SRCT_1507S = 0x010005E3, SRCT_1507SF = 0x010105E3, SRCT_1508S = 0x010005E4, SRCT_1508SF = 0x010105E4, SRCT_1508ST = 0x010805E4, SRCT_1508STF = 0x010905E4, }</pre>
--------	---

	<pre> SRCT_1505SP = 0x010005E1, SRCT_1505SPF = 0x010105E1, SRCT_1505SPT = 0x010805E1, SRCT_1505SPTF = 0x010905E1 } </pre>
--	---

Tabelle 9-86 ECPUType - .NET (C#)

Syntax	<pre> enum ECPUType { CPU1500_Unspecified = 0x000005DC, CPU1511_ = 0x000005E7, CPU1513 = 0x000005E9, CPU1515 = 0x000005EB, CPU1516 = 0x000005EC, CPU1517 = 0x000005ED, CPU1518 = 0x000005EE, CPU1511C = 0x000405E7, CPU1512C = 0x000405E8, CPU1511F = 0x000105E7, CPU1513F = 0x000105E9, CPU1515F = 0x000105EB, CPU1516F = 0x000105EC, CPU1517F = 0x000105ED, CPU1518F = 0x000105EE, CPU1511T = 0x000805E7, CPU1515T = 0x000805EB, CPU1516T = 0x000805EC, CPU1517T = 0x000805ED, CPU1511TF = 0x000905E7, CPU1515TF = 0x000905EB, CPU1516TF = 0x000905EC, CPU1517TF = 0x000905ED, CPU1518ODK = 0x001005EE, CPU1518FODK = 0x001105EE, CPU1518MFP = 0x004005EE, CPU1518FMFP = 0x004105EE, CPU1518T = 0x000805EE, CPU1518TF = 0x000905EE, CPU1504DTF = 0x000905E0, CPU1507DTF = 0x000905E3, CPUET200SP_Unspecified = 0x000205DC, CPU1510SP = 0x000205E6, CPU1512SP = 0x000205E8, CPU1510SPF = 0x000305E6, CPU1512SPF = 0x000305E8, CPU1514SP = 0x000205E7, CPU1514SPT = 0x000205E9, CPU1514SPF = 0x000305E7, CPU1514SPTF = 0x000305E9, SRCT_1514PA = 0x000305EB, CPU1500_RH_Unspecified = 0x008005DC, CPU1513R_ = 0x008005E9, CPU1515R = 0x008005EB, CPU1517H = 0x008005ED, CPU1518HF = 0x008105EE, CPUET200PRO_Unspecified = 0x002005DC, CPU1513PRO = 0x002005E9, CPU1513PROF = 0x002105E9, CPU1516PRO = 0x002005EC, CPU1516PROF = 0x002105EC, SRCT_1500_SW_OC_Unspecified = 0x010005DC, SRCT_1507S_ = 0x010005E3, SRCT_1507SF = 0x010105E3, SRCT_1508S = 0x010005E4, SRCT_1508SF = 0x010105E4, SRCT_1508ST = 0x010805E4, SRCT_1508STF = 0x010905E4, SRCT_1505SP = 0x010005E1, SRCT_1505SPF = 0x010105E1, </pre>
--------	---

	<pre>SRCT_1505SPT = 0x010805E1, SRCT_1505SPTF = 0x010905E1 }</pre>
--	--

9.7.6 ECommunicationInterface

Beschreibung

Diese Aufzählung enthält die verfügbaren Kommunikationsschnittstellen eines simulierten Controllers.

Tabelle 9-87 ECommunicationInterface - Native C++

Syntax	<pre>enum ECommunicationInterface { SRCI_NONE = 0, SRCI_SOFTBUS = 1, SRCI_TCPIP = 2, SRCI_ENUMERATION_SIZE = 3 };</pre>
--------	---

Tabelle 9-88 ECommunicationInterface - .NET (C#)

Syntax	<pre>enum ECommunicationInterface { None = 0, Softbus = 1, TCPIP = 2, }</pre>
--------	---

9.7.7 ENetworkMode

Beschreibung

Diese Aufzählung enthält alle Kommunikationsmodi einer simulierten PLC-Instanz.

Tabelle 9-89 ENetworkMode - Native C++

Syntax	<pre>enum class ENetworkMode { STATE_UNKNOWN = -1, TCPIP_MULTIPLE_ADAPTER = 0, TCPIP_SINGLE_ADAPTER = 1, SOFTBUS = 2 };</pre>
--------	---

Tabelle 9-90 ENetworkMode - .NET (C#)

Syntax	<pre>public enum class ENetworkMode { UnknownState TCIPIMultipleAdapter TCIPISingleAdapter Softbus };</pre>
--------	---

9.7.8 EPLCInterface

Beschreibung

Diese Aufzählung enthält die verfügbaren Schnittstellen einer simulierten PLC-Instanz.

Tabelle 9-91 EPLCInterface - Native C++

Syntax	<pre>enum class EPLCInterface { IE1 = 0, IE2, IE3 };</pre>
--------	--

9.7.9 ELEDType

Beschreibung

Diese Aufzählung enthält alle LED-Typen für eine simulierte PLC-Instanz.

Tabelle 9-92 ELEDType - Native C++

Syntax	<pre>enum ELEDType { SRLT_STOP = 0, SRLT_RUN = 1, SRLT_ERROR = 2, SRLT_MAINT = 3, SRLT_REDUND = 4, SRLT_FORCE = 5, SRLT_BUSF1 = 6, SRLT_BUSF2 = 7, SRLT_BUSF3 = 8, SRLT_BUSF4 = 9, SRLT_ENUMERATION_SIZE = 10 };</pre>
--------	--

Tabelle 9-93 ELEDType - .NET (C#)

Syntax	<pre>enum ELEDType { Stop = 0, Run = 1, Error = 2, Maint = 3, Redund = 4, Force = 5, Busf1 = 6, Busf2 = 7, Busf3 = 8, Busf4 = 9 };</pre>
--------	--

9.7.10 ELEDMode

Beschreibung

Diese Aufzählung enthält alle LEDs einer simulierten PLC-Instanz.

Tabelle 9-94 ELEDMode - Native C++

Syntax	<pre>enum ELEDMode { SRLM_OFF = 0, SRLM_ON = 1, SRLM_FLASH_FAST = 2, SRLM_FLASH_SLOW = 3, SRLM_INVALID = 4 };</pre>
--------	---

Tabelle 9-95 ELEDMode - .NET (C#)

Syntax	<pre>enum ELEDMode { Off = 0, On = 1, FlashFast = 2, FlashSlow = 3, Invalid = 4 }</pre>
--------	---

9.7.11 EPrimitiveDataType

Beschreibung

Diese Aufzählung enthält alle primitiven Datentypen, die von den E/A-Zugriffsfunktionen genutzt werden.

Tabelle 9-96 EPrimitiveDataType - Native C++

Syntax	<pre>enum EPrimitiveDataType { SRPDT_UNSPECIFIC = 0, SRPDT_STRUCT = 1, SRPDT_BOOL = 2, SRPDT_INT8 = 3, SRPDT_INT16 = 4, SRPDT_INT32 = 5, SRPDT_INT64 = 6, SRPDT_UINT8 = 7, SRPDT_UINT16 = 8, SRPDT_UINT32 = 9, SRPDT_UINT64 = 10, SRPDT_FLOAT = 11, SRPDT_DOUBLE = 12, SRPDT_CHAR = 13, SRPDT_WCHAR = 14 };</pre>
--------	---

Tabelle 9-97 EPrimitiveDataType - .NET (C#)

Syntax	<pre>enum EPrimitiveDataType { Unspecific = 0, Struct = 1, Bool = 2, Int8 = 3, Int16 = 4, Int32 = 5, Int64 = 6, UInt8 = 7, UInt16 = 8, UInt32 = 9, UInt64 = 10, Float = 11, Double = 12, Char = 13, WChar = 14 }</pre>
--------	--

Kompatible primitive Datentypen

Die folgenden Tabellen zeigen die primitiven Datentypen der Anwenderschnittstelle (API) und die Datentypen der S7-PLCSIM Advanced-Instanz, die in der gespeicherten Variablentabelle konfiguriert sind. Die kompatiblen Datentypen sind mit "X" markiert.

Tabelle 9-98 Kompatible primitive Datentypen - Lesen

API	S7-PLCSIM Advanced-Instanz												
	Bool	INT				UINT				Float	Double	Char	WChar
		8	16	32	64	8	16	32	64				
Bool	X												
INT8		X											
INT16		X	X			X							
INT32		X	X	X		X	X						
INT64		X	X	X	X	X	X	X					
UINT8						X							
UINT16						X	X						
UINT32						X	X	X					
UINT64						X	X	X	X				
Float										X			
Double											X		
Char												X	
WChar													X

Tabelle 9-99 Kompatible primitive Datentypen - Schreiben

API	S7-PLCSIM Advanced-Instanz												
	Bool	INT				UINT				Float	Double	Char	WChar
		8	16	32	64	8	16	32	64				
Bool	X												
INT8		X	X	X	X								
INT16			X	X	X								

9.7 Aufzählungen

INT32				X	X								
INT64					X								
UINT8			X	X	X	X	X	X	X				
UINT16				X	X		X	X	X				
UINT32					X			X	X				
UINT64									X				
Float										X			
Double											X		
Char												X	
WChar													X

9.7.12 EDataType

Beschreibung

Diese Aufzählung enthält alle PLC-Datentypen (STEP 7).

Tabelle 9-100 EDataType - Native C++

Syntax	<pre>enum EDataType { SRDT_UNKNOWN = 0, SRDT_BOOL = 1, SRDT_BYTE = 2, SRDT_CHAR = 3, SRDT_WORD = 4, SRDT_INT = 5, SRDT_DWORD = 6, SRDT_DINT = 7, SRDT_REAL = 8, SRDT_DATE = 9, SRDT_TIME_OF_DAY = 10, SRDT_TIME = 11, SRDT_S5TIME = 12, SRDT_DATE_AND_TIME = 14, SRDT_STRUCT = 17, SRDT_STRING = 19, SRDT_COUNTER = 28, SRDT_TIMER = 29, SRDT_IEC_Counter = 30, SRDT_IEC_Timer = 31, SRDT_LREAL = 48, SRDT_ULINT = 49, SRDT_LINT = 50, SRDT_LWORD = 51, SRDT_USINT = 52, SRDT_UINT = 53, SRDT_UDINT = 54, SRDT_SINT = 55, SRDT_WCHAR = 61, SRDT_WSTRING = 62, SRDT_LTIME = 64, SRDT_LTIME_OF_DAY = 65, SRDT_LDT = 66, SRDT_DTL = 67, SRDT_IEC_LTimer = 68, SRDT_IEC_SCounter = 69, SRDT_IEC_DCounter = 70, SRDT_IEC_LCounter = 71, SRDT_IEC_UCounter = 72, SRDT_IEC_USCounter = 73,</pre>
--------	---

```

SRDT_IEC_UDCounter = 74,
SRDT_IEC_ULCounter = 75,
SRDT_ERROR_STRUCT = 97,
SRDT_NREF = 98,
SRDT_CREF = 101,
SRDT_AOM_IDENT = 128,
SRDT_EVENT_ANY = 129,
SRDT_EVENT_ATT = 130,
SRDT_EVENT_HWINT = 131,
SRDT_HW_ANY = 144,
SRDT_HW_IOSYSTEM = 145,
SRDT_HW_DPMaster = 146,
SRDT_HW_DEVICE = 147,
SRDT_HW_DPSLAVE = 148,
SRDT_HW_IO = 149,
SRDT_HW_MODULE = 150,
SRDT_HW_SUBMODULE = 151,
SRDT_HW_HSC = 152,
SRDT_HW_PWM = 153,
SRDT_HW_PTO = 154,
SRDT_HW_INTERFACE = 155,
SRDT_HW_IEPORT = 156,
SRDT_OB_ANY = 160,
SRDT_OB_DELAY = 161,
SRDT_OB_TOD = 162,
SRDT_OB_CYCLIC = 163,
SRDT_OB_ATT = 164,
SRDT_CONN_ANY = 168,
SRDT_CONN_PRG = 169,
SRDT_CONN_OUC = 170,
SRDT_CONN_R_ID = 171,
SRDT_PORT = 173,
SRDT_RTM = 174,
SRDT_PIP = 175,
SRDT_OB_PCycle = 192,
SRDT_OB_HWINT = 193,
SRDT_OB_DIAG = 195,
SRDT_OB_TIMEERROR = 196,
SRDT_OB_STARTUP = 197,
SRDT_DB_ANY = 208,
SRDT_DB_WWW = 209,
SRDT_DB_DYN = 210,
SRDT_DB = 257
};

```

Tabelle 9-101 EDataType - .NET (C#)

Syntax	<pre> public enum EDataType { Unknown = 0, Bool = 1, Byte = 2, Char = 3, Word = 4, Int = 5, DWord = 6, DInt = 7, Real = 8, Date = 9, TimeOfDay = 10, Time = 11, S5Time = 12, DateAndTime = 14, Struct = 17, String = 19, Counter = 28, Timer = 29, IEC_Counter = 30, IEC_Timer = 31, LReal = 48, </pre>
--------	---

```

ULInt = 49,
LInt = 50,
LWord = 51,
USInt = 52,
UInt = 53,
UDInt = 54,
SInt = 55,
WChar = 61,
WString = 62,
LTime = 64,
LTimeOfDay = 65,
LDT = 66,
DTL = 67,
IEC_LTimer = 68,
IEC_SCounter = 69,
IEC_DCounter = 70,
IEC_LCounter = 71,
IEC_UCounter = 72,
IEC_USCounter = 73,
IEC_UDCounte = 74,
IEC_ULCounter = 75,
ErrorStruct = 97,
NREF = 98,
CREF = 101,
Aom_Ident = 128,
Event_Any = 129,
Event_Att = 130,
Event_HwInt = 131,
Hw_Any = 144,
Hw_IoSystem = 145,
Hw_DpMaster = 146,
Hw_Device = 147,
Hw_DpSlave = 148,
Hw_Io = 149,
Hw_Module = 150,
Hw_SubModule = 151,
Hw_Hsc = 152,
Hw_Pwm = 153,
Hw_Pto = 154,
Hw_Interface = 155,
Hw_IEPort = 156,
OB_Any = 160,
OB_Delay = 161,
OB_Tod = 162,
OB_Cyclic = 163,
OB_Att = 164,
Conn_Any = 168,
Conn_Prg = 169,
Conn_Ouc = 170,
Conn_R_ID = 171,
Port = 173,
Rtm = 174,
Pip = 175,
OB_PCycle = 192,
OB_HwInt = 193,
OB_Diag = 195,
OB_TimeError = 196,
OB_Startup = 197,
DB_Any = 208,
DB_WWW = 209,
DB_Dyn = 210,
DB = 257
}

```

9.7.13 ETagListDetails

Beschreibung

Diese Aufzählung enthält alle PLC-Areas, die beim Aktualisieren der Variablentabelle als Filter genutzt werden können.

Tabelle 9-102 ETagListDetails - Native C++

Syntax	<pre>enum ETagListDetails { SRTLD_NONE = 0, SRTLD_IO = 1, SRTLD_M = 2, SRTLD_IOM = 3, SRTLD_CT = 4, SRTLD_IOCT = 5, SRTLD_MCT = 6, SRTLD_IOMCT = 7, SRTLD_DB = 8, SRTLD_IODB = 9, SRTLD_MDB = 10, SRTLD_IOMDB = 11, SRTLD_CTDB = 12, SRTLD_IOCTDB = 13, SRTLD_MCTDB = 14, SRTLD_IOMCTDB = 15 };</pre>
--------	---

Tabelle 9-103 ETagListDetails - .NET (C#)

Syntax	<pre>enum ETagListDetails { None = 0, IO = 1, M = 2, IOM = 3, CT = 4, IOCT = 5, MCT = 6, IOMCT = 7, DB = 8, IODB = 9, MDB = 10, IOMDB = 11, CTDB = 12, IOCTDB = 13, MCTDB = 14, IOMCTDB = 15 };</pre>
--------	---

9.7.14 ERuntimeConfigChanged

Beschreibung

Diese Aufzählung enthält alle möglichen Ursachen für ein OnSoftwareConfigurationChanged-Ereignis, das der Runtime Manager sendet.

Tabelle 9-104 ERuntimeConfigChanged - Native C++

Syntax	<pre>enum ERuntimeConfigChanged { SRCC_INSTANCE_REGISTERED = 0, SRCC_INSTANCE_UNREGISTERED = 1, SRCC_CONNECTION_OPENED = 2, SRCC_CONNECTION_CLOSED = 3, SRCC_PORT_OPENED = 4, SRCC_PORT_CLOSED = 5 };</pre>
--------	---

Tabelle 9-105 ERuntimeConfigChanged - .NET (C#)

Syntax	<pre>enum ERuntimeConfigChanged { InstanceRegistered = 0, InstanceUnregistered = 1, ConnectionOpened = 2, ConnectionClosed = 3, PortOpened = 4, PortClosed = 5 };</pre>
--------	---

9.7.15 EInstanceConfigChanged

Beschreibung

Diese Aufzählung enthält alle möglichen Ursachen dafür, dass ein Ereignis OnSoftwareConfigurationChanged von einer simulierten PLC-Instanz gesendet wurde.

Tabelle 9-106 EInstanceConfigChanged - Native C++

Syntax	<pre>enum EInstanceConfigChanged { SRICC_HARDWARE_SOFTWARE_CHANGED = 0, SRICC_IP_CHANGED = 1 };</pre>
--------	---

Tabelle 9-107 EInstanceConfigChanged - .NET (C#)

Syntax	<pre>enum EInstanceConfigChanged { HardwareSoftwareChanged = 0, IPChanged = 1 };</pre>
--------	--

9.7.16 EPullOrPlugEventType

Beschreibung

Diese Aufzählung enthält vordefinierte Typen von Ziehen/Stecken-Ereignissen für S7-Module.

Tabelle 9-108 EPullOrPlugEventType - Native C++

Syntax	<pre>enum EPullOrPlugEventType { SR_PPE_UNDEFINED = 0, SR_PPE_PULL_EVENT = 1, SR_PPE_PLUG_EVENT = 2, SR_PPE_PLUG_EVENT_ERROR_REMAINS = 3, SR_PPE_PLUG_WRONG_MODULE_EVENT = 4 };</pre>
--------	---

Tabelle 9-109 EPullOrPlugEventType - .NET (C#)

Syntax	<pre>enum EPullOrPlugEventType { Undefined = 0, Pull = 1, Plug = 2, PlugErrorRemains = 3, PlugWrongModule = 4 }</pre>
--------	---

9.7.17 EProcessEventType

Beschreibung

Diese Aufzählung enthält vordefinierte Typen von Prozessereignissen für S7-Module.

Tabelle 9-110 EProcessEventType - Native C++

Syntax	<pre>enum EProcessEventType { SR_PET_UNDEFINED = 0, SR_PET_RISING_EDGE = 1, SR_PET_FALLING_EDGE = 2, SR_PET_LIMIT1_UNDERRUN = 3, SR_PET_LIMIT1_OVERRUN = 4, SR_PET_LIMIT2_UNDERRUN = 5, SR_PET_LIMIT2_OVERRUN = 6 };</pre>
--------	--

Tabelle 9-111 EProcessEventType - .NET (C#)

Syntax	<pre>enum EProcessEventType { Undefined = 0, RisingEdge = 1, FallingEdge = 2, Limit_1_Underrun = 3, Limit_1_Overrun = 4, Limit_2_Underrun = 5, Limit_2_Overrun = 6 }</pre>
--------	--

9.7.18 ESoftwareConfigChanged

Beschreibung

Diese Aufzählung enthält Indikatoren für Downloadfälle bei einem Ereignis OnSoftwareConfigurationChanged.

Tabelle 9-112 ESoftwareConfigChanged - Native C++

Syntax	<pre>enum ESoftwareConfigChanged { SRSCC_SOFTWARE_CHANGED_IN_STOP, SRSCC_SOFTWARE_CHANGED_IN_RUN };</pre>
--------	---

9.7.19 EDirection

Beschreibung

Diese Aufzählung enthält Eigenschaften der Diagnosemeldung.

Tabelle 9-113 EDirection - Native C++

Syntax	<pre>enum EDirection { SRD_DIRECTION_INPUT = 0, SRD_DIRECTION_OUTPUT = 1 };</pre>
--------	---

Tabelle 9-114 EDirection - .NET (C#)

Syntax	<pre>enum EDirection { Input = 0, Output = 1 }</pre>
--------	--

9.7.20 EDiagProperty

Beschreibung

Diese Aufzählung enthält die Kommend-/Gehend-Information der Diagnosemeldung.

Tabelle 9-115 EDiagProperty - Native C++

Syntax	<pre>enum EDiagProperty { SRP_DIAG_APPEAR = 1, SRP_DIAG_DISAPPEAR = 2 };</pre>
--------	--

Tabelle 9-116 EDiagProperty - .NET (C#)

Syntax	<pre>enum EDiagProperty { Appear = 1, Disappear = 2 }</pre>
--------	---

9.7.21 EDiagSeverity

Beschreibung

Diese Aufzählung enthält den Schweregrad der Diagnosemeldung (Fehler, Wartungsanforderung, Wartungsbedarf).

Tabelle 9-117 EDiagSeverity - Native C++

Syntax	<pre>enum EDiagSeverity { SRDS_SEVERITY_FAILURE = 0, SRDS_SEVERITY_MAINTENANCE_DEMANDED = 1, SRDS_SEVERITY_MAINTENANCE_REQUIRED = 2 };</pre>
--------	--

Tabelle 9-118 EDiagSeverity - .NET (C#)

Syntax	<pre>enum EDiagSeverity { Failure = 0, MaintDemaned = 1, MaintRequired = 2 }</pre>
--------	--

9.7.22 ERackOrStationFaultType

Beschreibung

Diese Aufzählung enthält die Typen des RackOrStationFault-Ereignisses.

Tabelle 9-119 ERackOrStationFaultType - Native C++

Syntax	<pre>enum ERackOrStationFaultType { SR_RS_FFAULT = 0, SR_RS_FRETURN = 1 };</pre>
--------	--

Tabelle 9-120 ERackOrStationFaultType - .NET (C#)

Syntax	<pre>enum ERackOrStationFaultType { Fault = 0, Return = 1 }</pre>
--------	---

9.7.23 ECycleTimeMonitoringMode

Beschreibung

Diese Aufzählung enthält die Quellen für den Timer zur maximalen Zykluszeitüberwachung.

Tabelle 9-121 ECycleTimeMonitoringMode - Native C++

Syntax	<pre>enum ECycleTimeMonitoringMode { SRCTMM_DOWNLOADED = 0, SRCTMM_IGNORED = 1, SRCTMM_SPECIFIED = 2 };</pre>
--------	---

Tabelle 9-122 ECycleTimeMonitoringMode - .NET (C#)

Syntax	<pre>enum ECycleTimeMonitoringMode { Downloaded = 0, Ignored = 1, Specified = 2 }</pre>
--------	---

9.7.24 EAutodiscoverType

Beschreibung

Diese Aufzählung wird bei der Callback-Funktion "Autodiscover" verwendet.

Tabelle 9-123 EAutodiscoverType - Native C++

Syntax	<pre>enum EAutodiscoverType { SRRSI_DISCOVER_STARTED = 0, SRRSI_DISCOVER_DATA = 1, SRRSI_DISCOVER_FINISHED = 2 };</pre>
--------	---

Tabelle 9-124 EAutodiscoverType - .NET (C#)

Syntax	<pre>public enum EAutodiscoverType { AutodiscoverStarted = 0, AutodiscoverData = 1, AutodiscoverFinished = 2 }</pre>
--------	--

Beispiele/Anwendungsfälle

10.1 Beispiel: Kommunikation über TCP/IP im "Single Adapter"-Netzwerkmodus (Promiscuous-Modus)

Im Single Adapter Network Mode (Promiscuous-Modus) liest der Netzwerkkarte alle ankommenden Telegramme. Der Netzwerkkarte liest auch Telegramme, die nicht für den Netzwerkkarte bestimmt sind. Anschließend gibt der Netzwerkkarte die Daten zur Verarbeitung an das Betriebssystem weiter.

Auf Virtualisierungsplattformen oder Firmennetzwerken ist es jedoch aus Sicherheitsgründen oftmals nicht erlaubt, die Netzwerkschnittstelle des PC im Single Adapter Network Mode (Promiscuous-Modus) zu betreiben. Ein Beispiel für den Multi-Adapter Mode (Nicht-Promiscuous-Modus) finden Sie unter dem nächsten Thema.

Die folgende Grafik gibt Ihnen einen Überblick über die Möglichkeiten und Unterschiede zwischen den folgenden Kommunikationsmöglichkeiten:

- Lokale TPC/IP-Kommunikation
- Single Adapter Network Mode (Promiscuous-Modus)
- Multi-Adapter Network Mode (Non-Promiscuous-Modus)

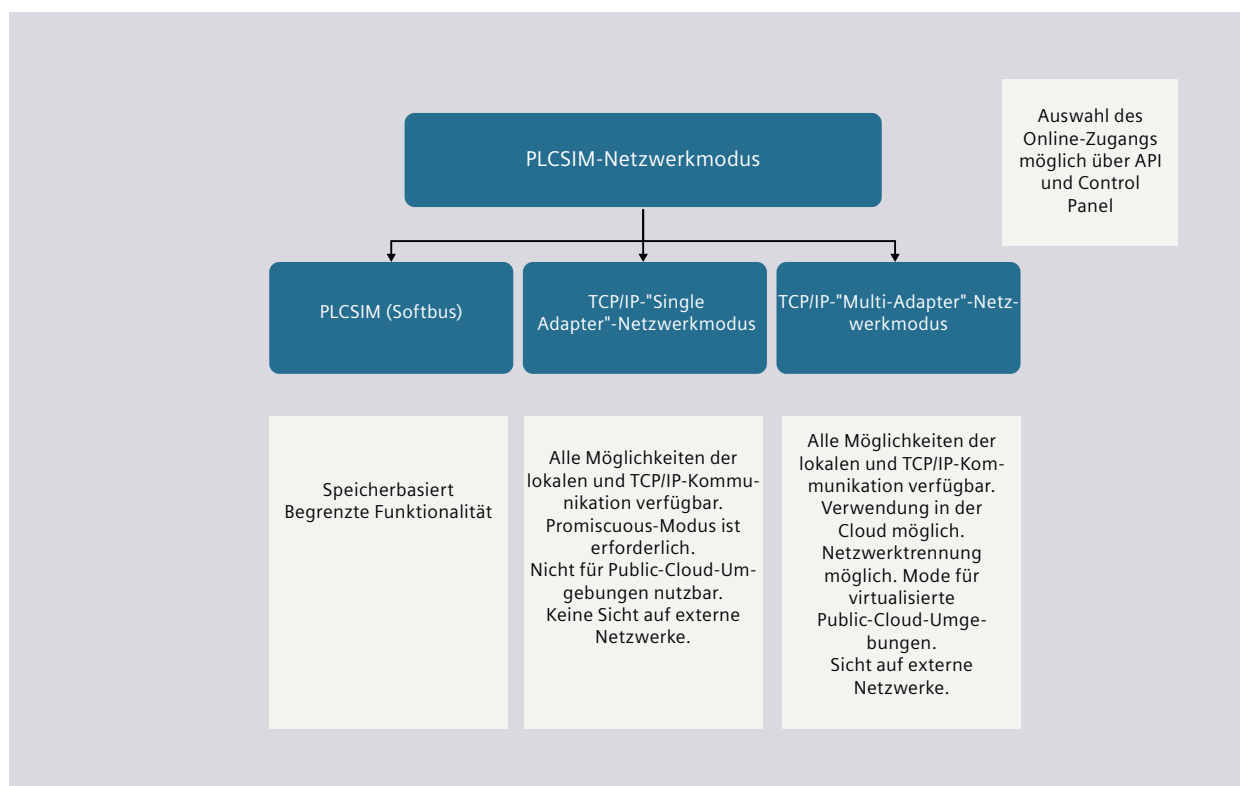


Bild 10-1 PLCSIM_networking

Beispielcode

In diesem Beispiel ist die physische Schnittstelle auf den Promiscuous-Modus eingestellt und empfängt alle Pakete; die PCAP filtert den Datenverkehr. Jede der PLC-Schnittstellen verfügt über eine eigene, eindeutige MAC-Adresse. Mit einem PLCSIM Virtual Switch können maximal zwei Netzwerkadapter verbunden werden.

```
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

//Verbindung initialisieren
InitializeApi(&rtm);

//TCP/IP-"Single Adapter"-Modus einstellen (voreingestellter Modus
ist Softbus)
rtm->SetNetworkMode(ENetworkMode::TCPIP_SINGLE_ADAPTER);

rtm->RegisterInstance(&instance0);
rtm->RegisterInstance(&instance1);
instance0->PowerOn(60000);
instance1->PowerOn(60000);

ShutdownAndFreeApi(rtm);
```

10.2 Beispiel: Kommunikation über TCP/IP im Multi-Adapter Network Mode (Non-Promiscuous-Modus)

Im Multi-Adapter Network Mode (Non-Promiscuous-Modus) stellt die Auswertung von MAC-Adressen sicher, dass nur Daten das Betriebssystem erreichen, die für den Netzwerkadapter bestimmt sind.

Übersicht

Die folgende Grafik gibt Ihnen einen Überblick über die Möglichkeiten und Unterschiede zwischen den folgenden Kommunikationsmöglichkeiten:

- Lokale TPC/P-Kommunikation
- Single Adapter Network Mode (Promiscuous-Modus)
- Multi-Adapter Network Mode (Non-Promiscuous-Modus)

10.2 Beispiel: Kommunikation über TCP/IP im Multi-Adapter Network Mode (Non-Promiscuous-Modus)

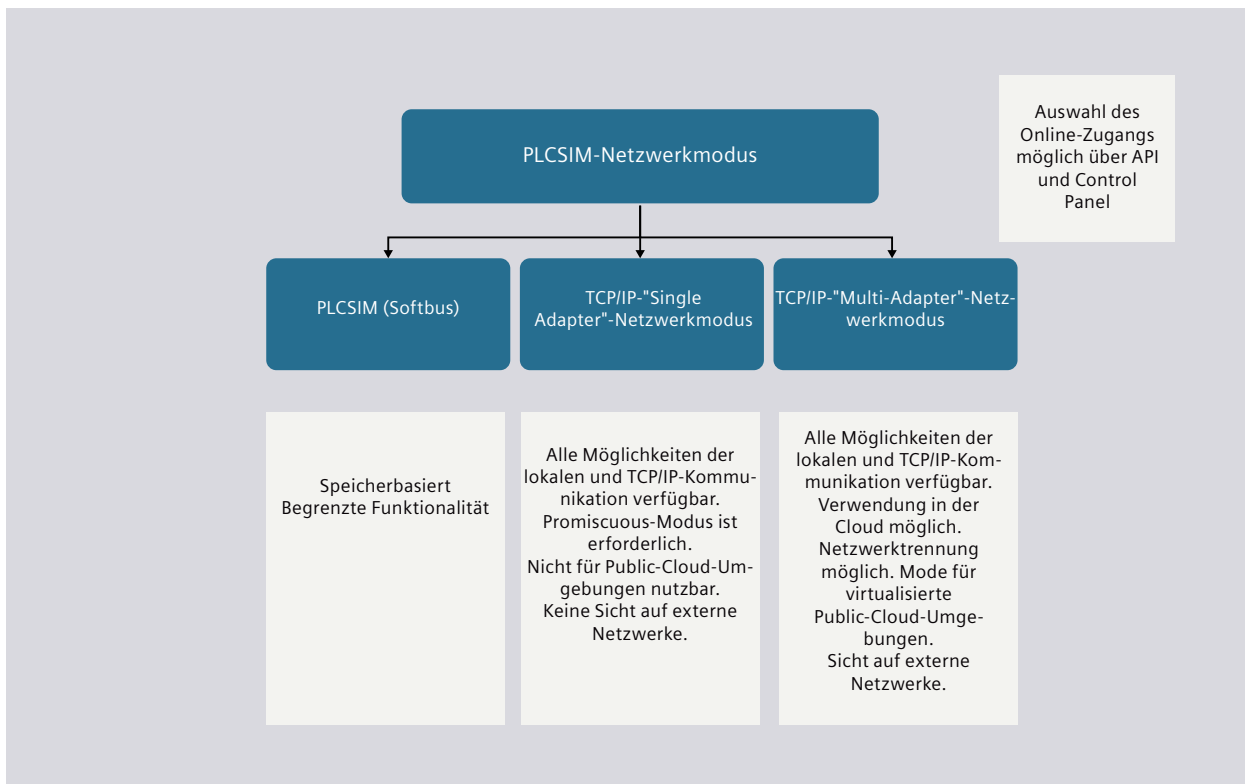


Bild 10-2 PLCSIM-Networking

Anwendungsfälle

Die folgenden Anwendungsfälle beschreiben, wie Sie auch im Multi-Adapter Network Mode (Non-Promiscuous-Modus) kommunizieren können. Sie kommunizieren in diesem Modus, indem Sie eine Eins-zu-eins-Zuordnung (Mapping) der MAC-Adressen zwischen den CPU-Netzwerkschnittstellen und den Netzwerkadaptern des PC vornehmen.

Die folgenden Anwendungsfälle gehen davon aus, dass Sie in einer Umgebung arbeiten, in der Sicherheitseinschränkungen gelten und Sie deshalb in den Multi-Adapter Network Mode (Non-Promiscuous-Modus) gewechselt haben.

Anwendungsfall 1:

Sie starten mehrere Instanzen. Jede CPU-Schnittstelle ist der MAC-Adresse des jeweiligen physischen oder virtuellen PC-Adapters zugeordnet.

Vorbedingung: Jede Schnittstelle einer laufenden Instanz ist einer eigenen physikalischen oder virtuellen PC-Schnittstelle zugeordnet (Eins-zu-eins-Zuordnung).

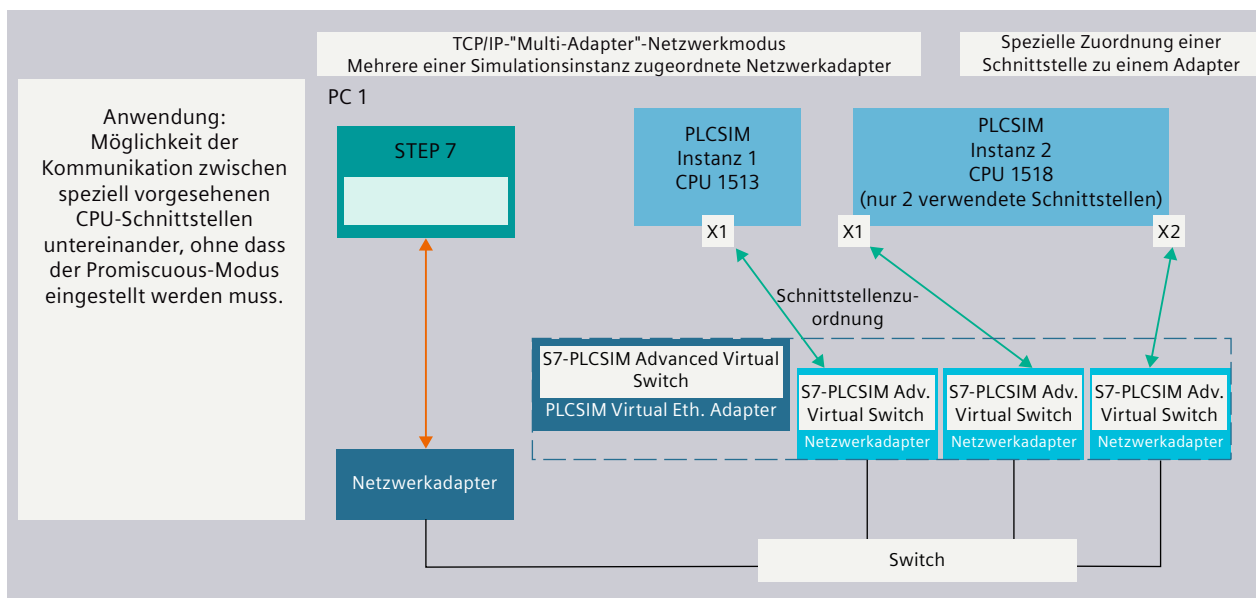


Bild 10-3 Anwendungsfall 1

Anwendungsfall 2:

Der folgende Anwendungsfall zeigt das Verhalten bei mehreren Instanzen, die dieselben MAC-Adressen besitzen. Jede CPU-Schnittstelle ist dem entsprechenden physischen oder virtuellen PC-Adapter zugeordnet.

10.2 Beispiel: Kommunikation über TCP/IP im Multi-Adapter Network Mode (Non-Promiscuous-Modus)

Das Szenario ist nur gültig, wenn jeweils nur eine Instanz läuft.

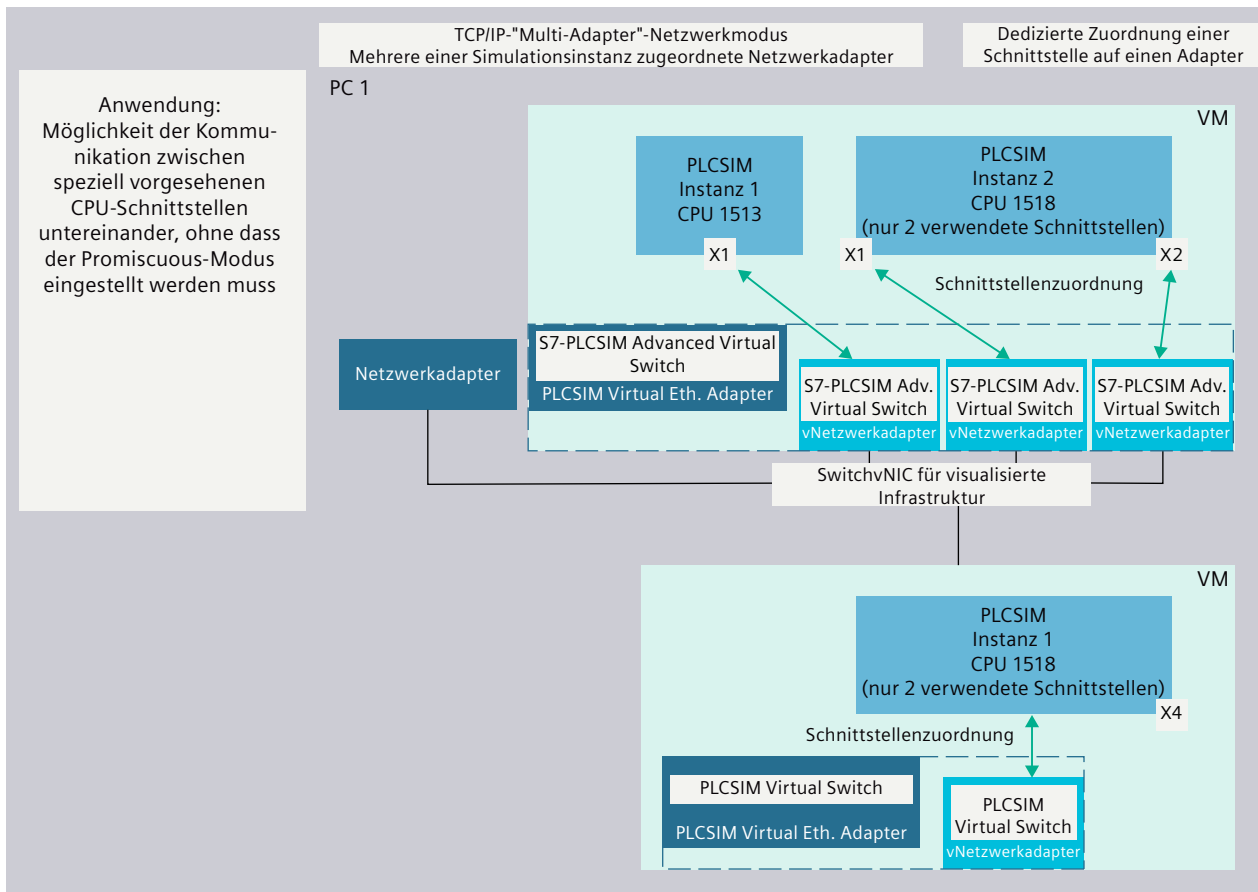


Bild 10-4 Anwendungsfall 2

Vorgehensweise

Für eine externe Kommunikation im Multi-Adapter Network Mode (Non-Promiscuous-Modus) sind in Abhängigkeit von Ihrem Anwendungsfall die folgenden Schritte nötig.

1. Wechseln Sie mit der folgenden Anweisung in den Multi-Adapter Network Mode (Non-Promiscuous-Modus):
`SetNetworkMode(SRCM_VSWITCH_NON_PROMISCUOUS)`
2. Um eine neue Instanz zu registrieren, rufen Sie `RegisterInstance()` auf.
3. Um anhand einer Liste von PC-Schnittstellen auszuwählen, welche PC-Schnittstelle welcher CPU-Schnittstelle zuordenbar ist, rufen Sie `GetNetInterfaces()` auf.
4. Um eine Schnittstellen-Zuordnung (Mapping) zwischen CPU- und PC-Schnittstelle durchzuführen, rufen Sie `SetNetInterfaceMapping(0, 23)` auf.
5. Um zu überprüfen, ob die Konfiguration korrekt ist, rufen Sie `CheckNetInterfaceMapping()` auf.
6. Um die Instanz zu starten, rufen Sie `PowerOn()` auf.
Vor dem Start führt die Instanz eine interne Überprüfung durch.

HINWEIS

Einstellen eines Speicherpfads für eine neue Instanz

Wenn Sie über die Funktion `SetStoragePath()` einen Speicherpfad für eine neue Instanz einstellen, rufen Sie diese Funktion auf, bevor die MAC-Adresse eingestellt wird. Andernfalls wird die MAC-Adresse verworfen.

Diese Vorgehensweise gilt auch für den gespeicherten CPU-Typ und die CPU-Seriennummer.

Die Funktion `CleanupStoragePath()` löscht neben dem Speicherpfad auch die Zuordnung der Netzwerkschnittstellen.

Beispiele in C++

Die folgenden Beispiele zeigen mögliche Netzwerkkonfigurationen in C++.

Achten Sie bei Szenarien mit mehr als einem Adapter darauf, die Instanz mit einer CPU zu registrieren, die die erforderliche Anzahl von Schnittstellen besitzt. In den folgenden Beispielen nutzen wir 3 Schnittstellen. Deswegen registrieren wir die Instanz mit einer CPU mit 3 PROFINET-Schnittstellen, wie z. B. der CPU 1518-4 PN/DP.

Beispiel 1: Vereinfachte Einstellungen

```
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

//Verbindung initialisieren
InitializeApi(&rtm);

// NON-P -Modus einstellen
rtm->SetNetworkMode(ENetworkMode::TCPIP_MULTIPLE_ADAPTER);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance0);
instance0->SetNetInterfaceMapping(EPLCInterface::IE1, L"Ethernet");
```

10.2 Beispiel: Kommunikation über TCP/IP im Multi-Adapter Network Mode (Non-Promiscuous-Modus)

```

instance0->SetNetInterfaceMapping(EPLCInterface::IE2, L"Ethernet
2");
instance0->SetNetInterfaceMapping(EPLCInterface::IE3, L"Ethernet
3");
instance0->PowerOn(60000);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance1);
instance1->SetNetInterfaceMapping(EPLCInterface::IE1, L"Ethernet
4");
instance1->SetNetInterfaceMapping(EPLCInterface::IE2, L"Ethernet
5");
instance1->SetNetInterfaceMapping(EPLCInterface::IE3, L"Ethernet
6");
instance1->PowerOn(60000);

ShutdownAndFreeApi(rtm);

```

Beispiel 2: Vollständige Einstellungen

```

#include <iostream>
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

// Verbindung initialisieren
InitializeApi(&rtm);

// Benutzer kann aktuellen Netzwerkmodus mit GetNetworkMode.
abrufenVoreingestellter Modus ist Softbus.
ENetworkMode netMode;
rtm->GetNetworkMode(&netMode);

// Wenn TCPIP_MULTIPLE_ADAPTER gesetzt ist, wird er von
SetNetworkMode nicht erneut gesetzt.
rtm->SetNetworkMode(ENetworkMode::TCPIP_MULTIPLE_ADAPTER);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance0);
rtm->RegisterInstance(ECPUType::SRCT_1518, &instance1);

// Netzwerkschnittstellen im Computer sammeln
// Anzahl Netzwerkschnittstellen abrufen
UINT count;
result = rtm->GetNetInterfaces(nullptr, &count);

// Netzwerkschnittstellen abrufen
// GetNetInterfaces gibt Adapter mit deaktiviertem vSwitch und/oder
nicht verbundene Adapter zurück
std::unique_ptr<SNetInterfaceInfo[]> netInterfaces =
std::make_unique<SNetInterfaceInfo[]>(count);
result = rtm->GetNetInterfaces(netInterfaces.get(), &count);

```

```

// gewünschte Schnittstelle einer Instanzschnittstelle zuordnen
// jede der Instanzschnittstellen muss eine eindeutige oder keine
// Zuordnung (Mapping) aufweisen
// wir können Schnittstelle nach diesen drei Überlastsituationen
// zuordnen
//
// interfaceIndex
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE1,
netInterfaces.get()[0].interfaceIndex);
// Friendly Name
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE2,
netInterfaces.get()[1].interfaceName);
// MAC address
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE3,
&netInterfaces.get()[2].MACAddress);

// Mapping zu zweiter Instanz vornehmen
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE1,
&netInterfaces.get()[3].MACAddress);
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE2,
&netInterfaces.get()[4].MACAddress);
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE3,
&netInterfaces.get()[5].MACAddress);

// wir können auch prüfen, was der Schnittstelle zugeordnet ist
INT32 mappedInterfaceIndex =
instance0->GetNetInterfaceMapping(EPLCInterface::IE1);
if (mappedInterfaceIndex == 0)
{
std::cout << "No mapping is set";
}

if (mappedInterfaceIndex == netInterfaces.get()[0].interfaceIndex)
{
std::cout << netInterfaces.get()[0].interfaceName << "is mapped on
EPLCInterface::IE1";
}

// S7-PLCSIM Advanced Virtual Switch entsprechend festgelegter
// Zuordnung (Mapping) einstellen
// !! erfordert übergeordneten (Admin-)Modus!! !!
rtm->SetNetInterfaceBindings();

// prüfen, ob Einstellung gültig ist - optional -
// CheckNetInterfaceMapping() function is also called inside PowerOn()
result = instance0->CheckNetInterfaceMapping();
if (result != SREC_OK)
{
return result;
}

instance0->PowerOn(60000);
instance1->PowerOn(60000);

ShutdownAndFreeApi(rtm);

```